Комп'ютерне моделювання задач прикладної математики

Дифузія невзаємодіючих частинок. Рівняння Ланжевена

Лабораторна робота 1

__Студентка Пороскун Олена. Група ПМ.м-21__

Task 1

In [1]:
```
#pip install matplotlib
```

In [2]:
```
from matplotlib import pyplot as plt
```

In [3]:
```
import random
import numpy as np
import math
import time
```

In [4]:
```
def ksi():
    max_num = 2147483647
    p1 = random.randint(0, max_num) / (max_num+1)
    p2 = random.randint(0, max_num) / (max_num+1)
    if (p1==0):
        p1 = 1e-10
    if (p2==0):
        p2 = 1e-10
    return math.sqrt(-2*math.log(p1)) * math.sin(2*math.pi*p2)

#ksi()
```

Запишемо функції для побудови графіків: для одного значення D і для двох одночасно.

In [5]:
```
def fun_plot1(times, delt, D = 2, t_max=11):

    plt.scatter(times, delt, label="times/delt", s=20, c='m')
    plt.plot([0, t_max], [D*0, D*t_max], label= f'{D}'"t",linewidth = 1)

    plt.xlabel("time")
    plt.ylabel("Delta")
    plt.title("Графік дельта в залежності від часу D=" f'{D}')
    plt.legend()


def fun_plot2(times,delt, times2,delt2, D1 = 2, D2 = 0.2, t_max=11):

    plt.scatter(times, delt, label="times/delt",s=15)
    plt.plot([0, t_max], [D1*0, D1*t_max], label= f'{D1}'"t",linewidth = 1)

    plt.scatter(times2, delt2, label="times2/delt2",s=15)
    plt.plot([0, t_max], [D2*0, D2*t_max], label= f'{D2}'"t",linewidth = 1)
    plt.legend()
```

In [6]:
```
def funct1(D):

    times = []
    delt =  []

    t = 0                   # початковий момент часу
    dt = 0.002              # крок по часу 2*(1e-3)
    t_fin = 10              # кінцевий момент часу
    t_write = 0.1           # початковий омент часу для запису
    dt_write = 0.2          # крок для запису даних

    N = 10000               # кількість точок(частинок)

    x = np.zeros(N)         # зануляємо початкові координати точок

    while(t < t_fin):
        for i in range(x.size):          # for(i = 0..N)
            x[i] += math.sqrt(D*dt)*ksi()

            if(t >= t_write):
                x_mean = 0
                x2_mean = 0

                for j in range(x.size):      # for(i=0..N):
                    x_mean  += x[j]
                    x2_mean += x[j]*x[j]

                x_mean = x_mean / N
                x2_mean = x2_mean / N
                Delta = x2_mean - x_mean*x_mean

                times.append(t)
                delt.append(Delta)
                # ін спосіб розрахунку станд. відхилення
                #Delta2 = np.std(x)
                #delt2.append(Delta2)

                #print(t, Delta)

                t_write += dt_write
        t += dt
        t = round(t, 3)

    #fun_plot1(times, delt, D)

    return times, delt
```

Код для розрахунку часу виконання програми: import time start_time = time.time() main()... print("--- %s seconds ---" % (time.time() - start_time))

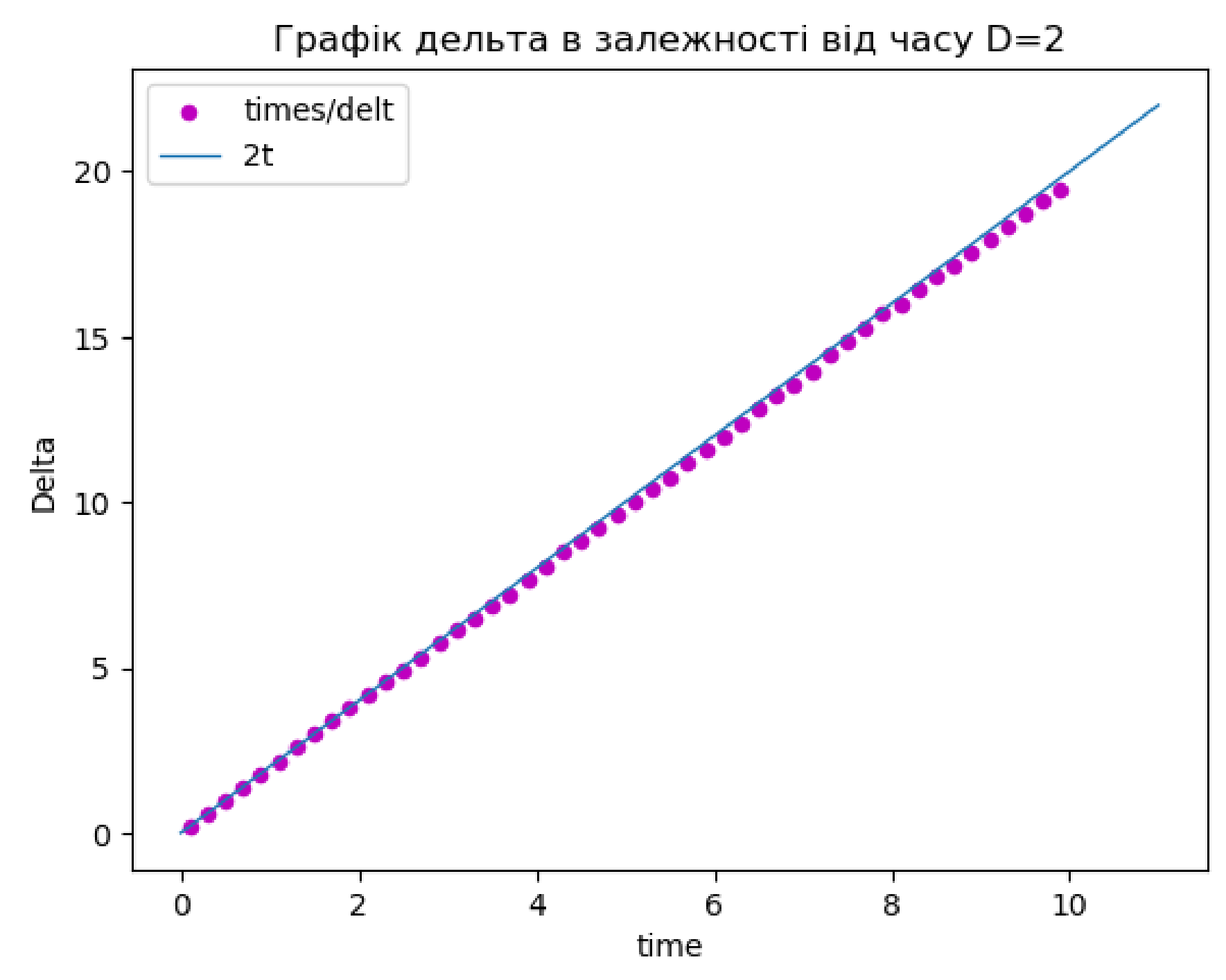
In [7]:
```
start_time = time.time()

D1 = 2
t1, delt1 = funct1(D1)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
--- 113.26215958595276 seconds ---
```

In [8]:
```
print(len(t1))
```

```
50
```

In [9]:
```
fun_plot1(t1, delt1, D1)
```



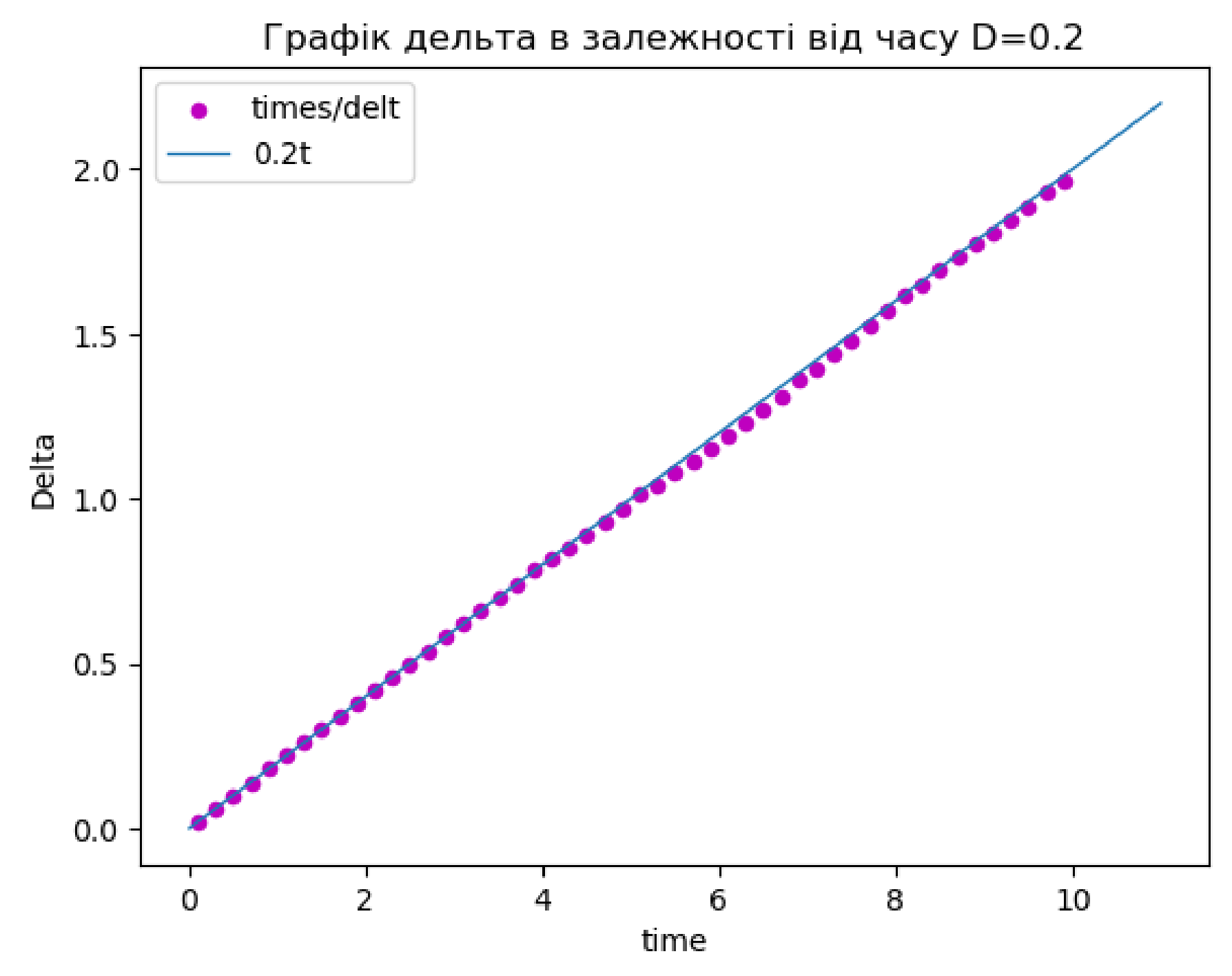
In [10]:
```
start_time = time.time()

D2 = 0.2
t2, delt2 = funct1(D2)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
--- 113.33723187446594 seconds ---
```

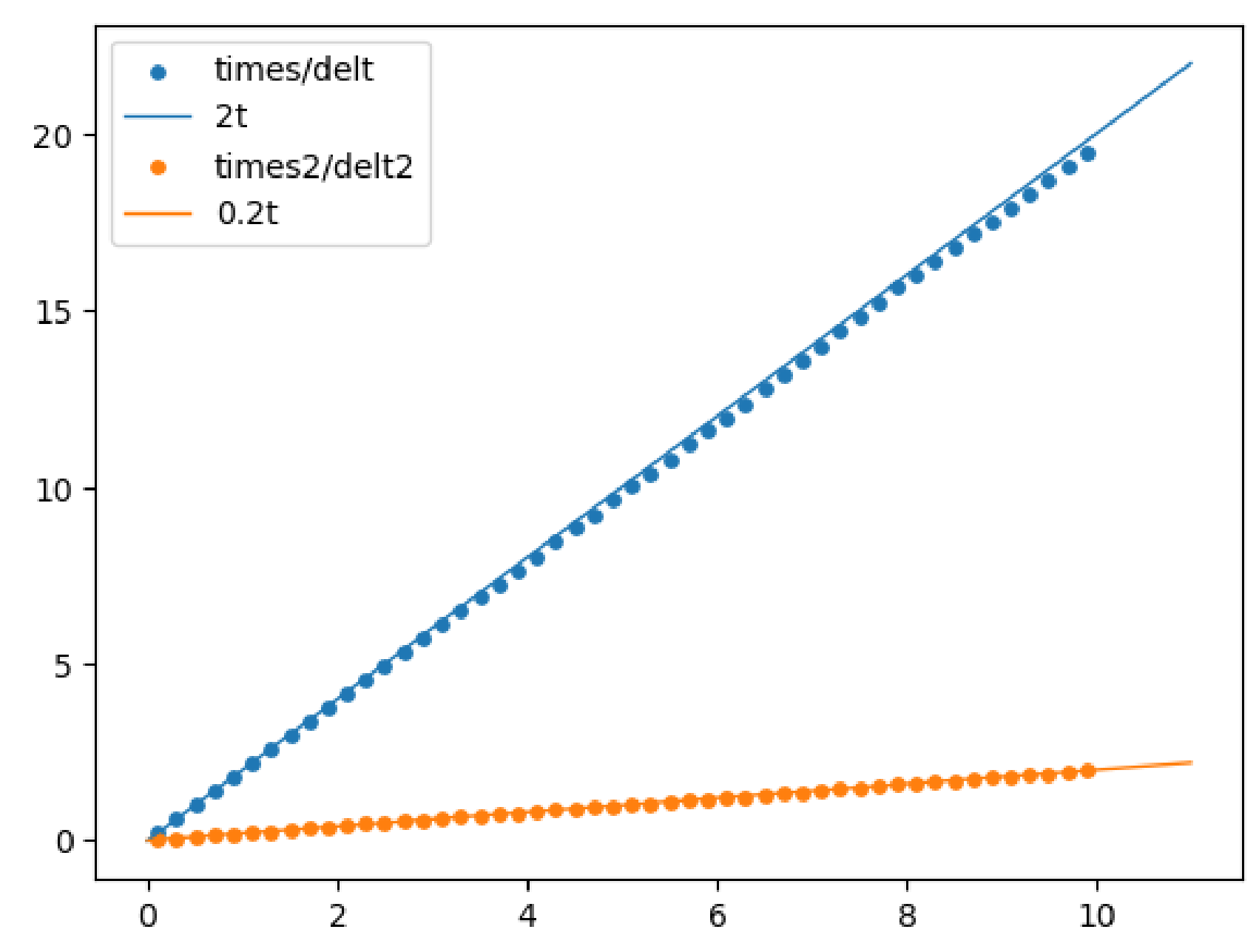
In [11]:
```
print(len(t2))
```

```
50
```

In [12]:
```
fun_plot1(t2, delt2, D2)
```

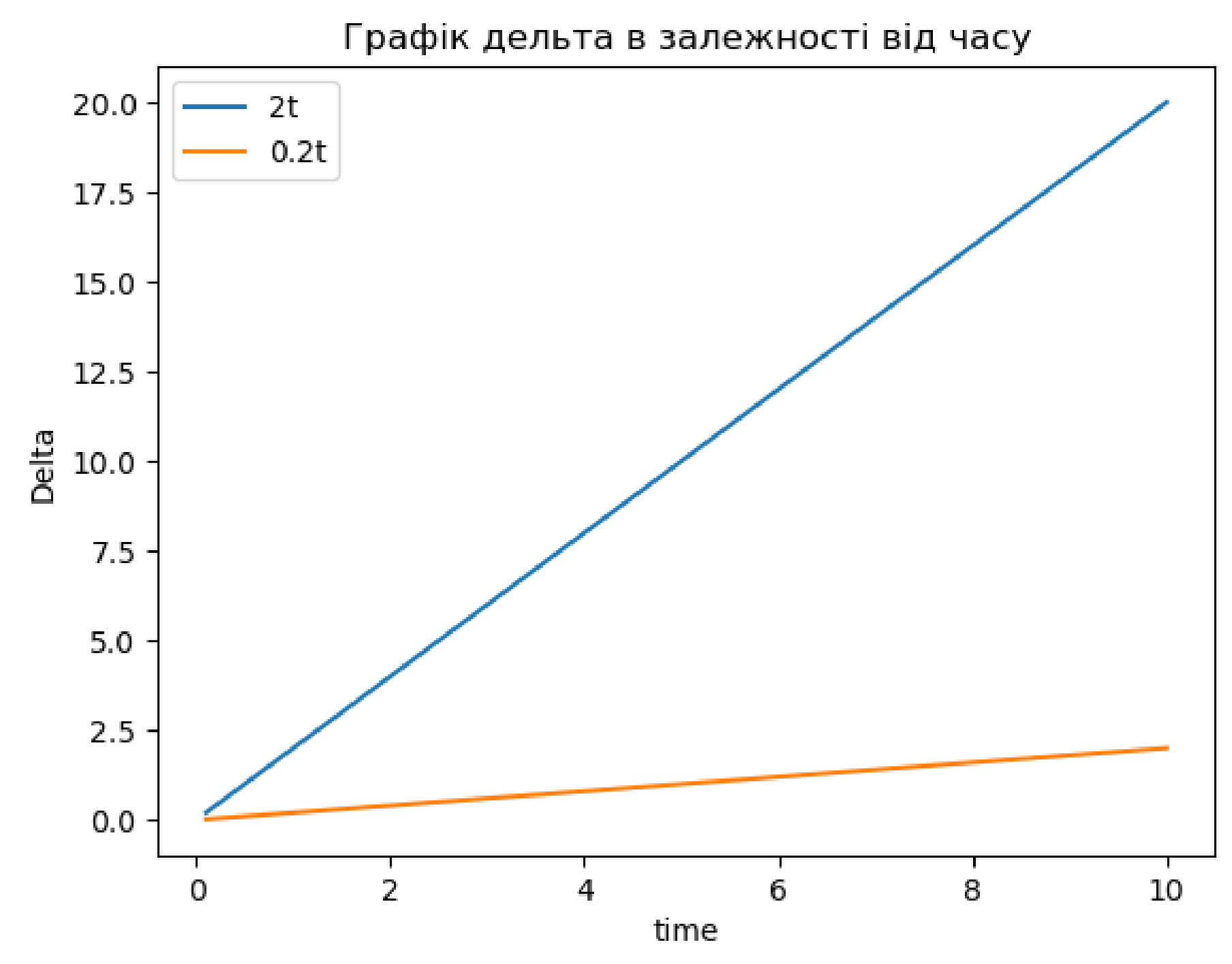


In [13]:
```
fun_plot2(t1, delt1, t2,delt2)
```



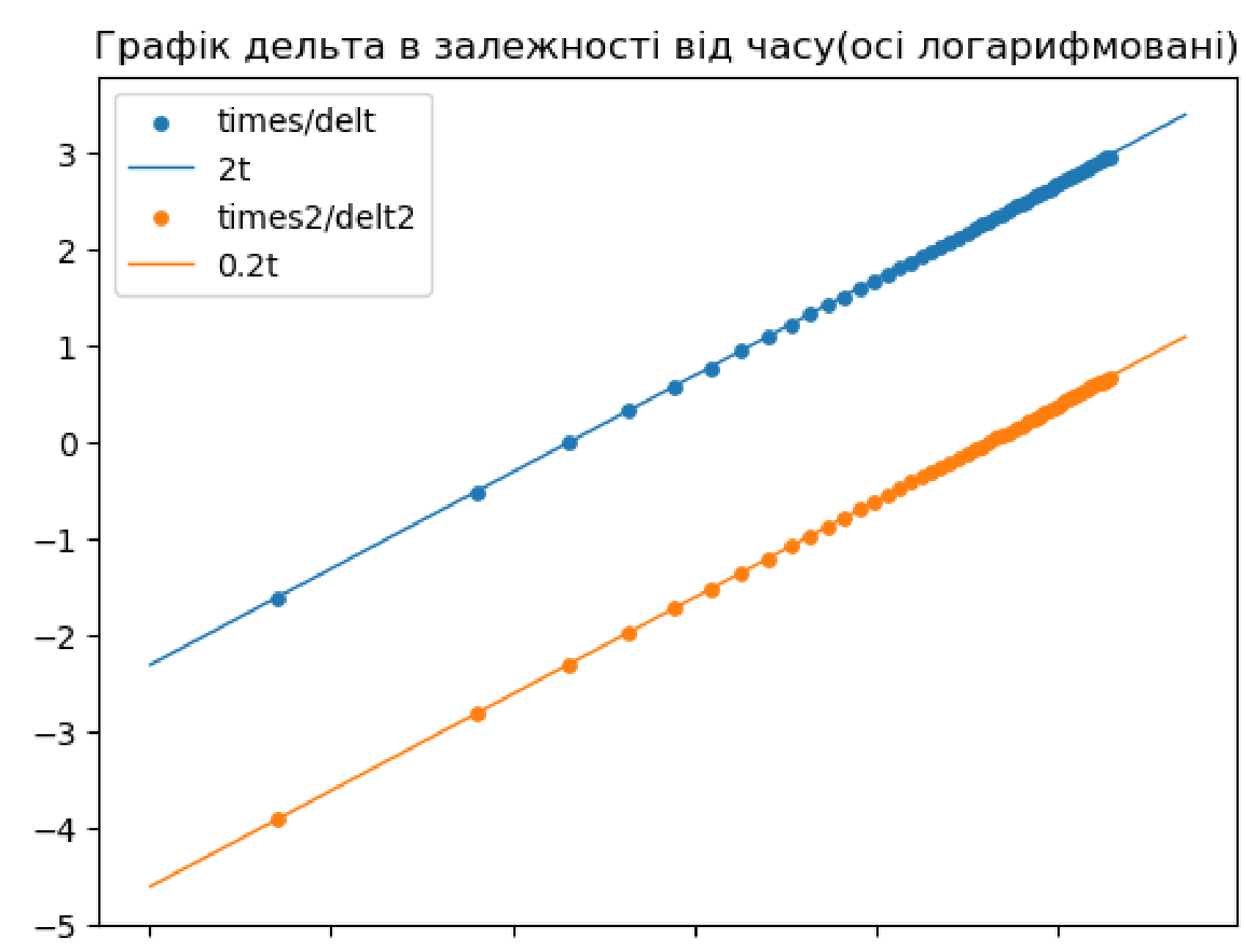
Вигляд прямих в залежності від рівняння.

In [14]:
```
def plotLinesForD12(t_start=0.1, t_end=10, D1=2,D2=0.2):

    plt.plot([t_start, t_end], [D1*t_start, D1*t_end], label= f'{D1}'"t")
    plt.plot([t_start, t_end], [D2*t_start, D2*t_end], label= f'{D2}'"t")
    plt.xlabel("time")
    plt.ylabel("Delta")
    plt.title("Графік дельта в залежності від часу")
    plt.legend()

plotLinesForD12()
```



Побудуємо графік дельта в залежності від часу з логарифмованими осями.

In [15]:
```
def fun_plot_log(times,delt, times2,delt2, D1 = 2, D2 = 0.2, t_max=15):

    times = np.log(times)
    delt  = np.log(delt)
    plt.scatter(times, delt, label="times/delt",s=15) # точки для розрахованих точок з D = 2

    x = [0.05, t_max]
    x = np.log(x)

    y1 = [0.05*D1, D1*t_max]
    y1 = np.log(y1)
    plt.plot(x, y1, label= f'{D1}'"t",linewidth = 1) # логарифмована пряма y=2t

    times2 = np.log(times2)
    delt2  = np.log(delt2)
    plt.scatter(times2, delt2, label="times2/delt2",s=15) # точки для розрахованих точок з D = 0.2

    y2 = [0.05*D2, D2*t_max]
    y2 = np.log(y2)
    plt.plot(x, y2, label= f'{D2}'"t",linewidth = 1) # логарифмована пряма y=0.2t

    plt.legend()
    plt.title("Графік дельта в залежності від часу(осі логарифмовані)")

fun_plot_log(t1, delt1, t2, delt2)
```



Комп'ютерне моделювання задач прикладної математики

Дифузія невзаємодіючих частинок. Рівняння Ланжевена

Лабораторна робота 1

Студентка Пороскун Олена. Група ПМ.м-21

Task 2

In [1]:
```python
from matplotlib import pyplot as plt
```

In [2]:
```python
import random
import numpy as np
import math
import time
```

Функція випадкової величини.

In [3]:
```python
def ksi():
    max_num = 2147483647
    p1 = random.randint(0, max_num) / (max_num+1)
    p2 = random.randint(0, max_num) / (max_num+1)
    if (p1==0):
        p1 = 1e-10
    if (p2==0):
        p2 = 1e-10
    return math.sqrt(-2*math.log(p1)) * math.sin(2*math.pi*p2)

#ksi()
```

Функція, що розраховує координати частинок, час за який кожна з них вийде з інтервалу (-R,R) та середній час по всім частинкам.

In [4]:
```python
def funct2(R=5,D=1):

    t = 0                  # початковий момент часу
    dt = 0.002             # крок по часу 2*(1e-3)

    N = 1000              # кількість точок(частинок)

    x = np.zeros(N)        # зануляємо початкові координати точок
    times = np.zeros(N)    # час для кожної частинки коли вона вийшла за межі інтервалу (-R,R)

    # mean pasage time (середній час досягнення значення R)
    mpt = 0

    for i in range(x.size):
        t=0
        while(abs(x[i]) < R):
            x[i] += math.sqrt(D*dt)*ksi()
            if (abs(x[i]) >= R):
                times[i] = t
            t += dt

    mpt = np.mean(times)

    #print("R","\t D","\t mpt")
    print(R, "\t", D,"\t", mpt)

    return x, times, mpt
```

# Розрахунок одного виконання ф-ції funct2(): # dt = 0.2 N = 10000 ~ 36 seconds # dt = 0.02 N = 10000 ~ 345 seconds # dt = 0.02 N = 1000 ~ 35 seconds # dt = 0.002 N = 1000 ~ 625 seconds # dt = 0.002 N = 10000 ~ 8000 seconds # А зробити розрахунків з нею потрібно всього 4 рази. Тому час множимо на 4. # Код для розрахунку часу виконання програми: start_time = time.time() x1, t1, mpt1 = 0,0,0 x1, t1, mpt1 = funct2(R=5,D=0.1) print("--- %s seconds ---" % (time.time() - start_time))

- *Розрахуємо дані для D = [0.1, 0.2, 0.5, 1], R = 5.*

In [5]:
```python
start_time = time.time()

D = [0.1, 0.2, 0.5, 1]
x = []            # координати частинок
t = []            # час для кожної частинки коли вона вийшла за межі інтервалу (-R,R)
mpt = []          # mean pasage time (середній час досягнення значення R)

print("R","\t D","\t mpt")

for i in range(len(D)):
    x_, t_, mpt_ = funct2(R = 5, D = D[i])
    x.append(x_)
    t.append(t_)
    mpt.append(mpt_)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
R        D       mpt
5        0.1     250.7091780000322
5        0.2     124.41766800011091
5        0.5     51.393056000002964
5        1       25.78134800000055
--- 825.3829946517944 seconds ---
```

In [6]:
```python
print(np.shape(x))
print(np.shape(t))
print(np.shape(mpt))
```

```
(4, 1000)
(4, 1000)
(4,)
```

- *Розрахуємо дані для D = 1, R = [0.1, 0.2, 0.5, 1].*

In [7]:
```python
start_time = time.time()

D_new = 1
R_new = [0.1, 0.2, 0.5, 1]
x_new = []            # координати частинок
t_new = []            # час для кожної частинки коли вона вийшла за межі інтервалу (-R,R)
mpt_new = []          # mean pasage time (середній час досягнення значення R)

print("R","\t D","\t mpt")

for i in range(len(R_new)):
    x_, t_, mpt_ = funct2(R = R_new[i], D = D_new)
    x_new.append(x_)
    t_new.append(t_)
    mpt_new.append(mpt_)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
R        D       mpt
0.1      1       0.014758000000000007
0.2      1       0.04840000000000003
0.5      1       0.26764200000000016
1        1       1.05781199999999
--- 2.6031553745269775 seconds ---
```

In [8]:
```python
print(np.shape(x_new))
print(np.shape(t_new))
print(np.shape(mpt_new))
```

```
(4, 1000)
(4, 1000)
(4,)
```

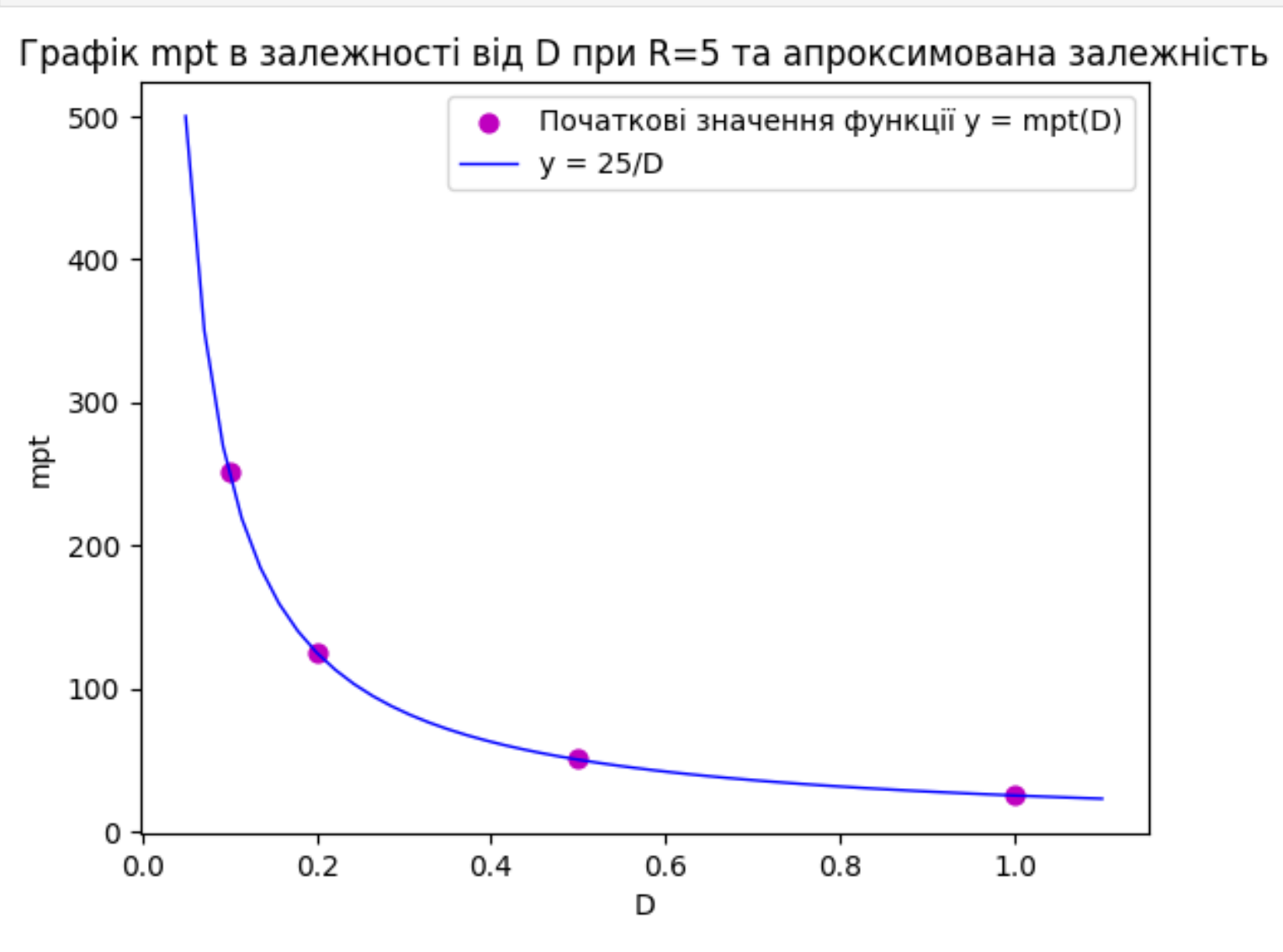
Функція, що априксимує дані, повертає: \ ~ coef - коефіцієнти рівняння, яке відповідає початковим даним, \ ~ polinom - рівняння з коефіцієнтами, \ ~ mpt_to_compare - значення функції для порівняння зі початковими

In [9]:
```python
def fun_aprox(X, mpt):

    k=1
    mpt_new = mpt*0
    eps = 1000

    bool_mpt = 0

    while ((bool_mpt < np.size(mpt)) & (eps >= 0.00001) & (k<10)):

        coef = np.polyfit(X, mpt,k)
        polinom = np.poly1d(coef)
        mpt_to_compare = polinom(X)

        mpt_old = np.round(mpt,5)
        mpt_to_compare = np.round(mpt_to_compare,5)

        eps = abs(mpt_old[0] - mpt_to_compare[0])

        bool_mpt = np.sum(mpt_old == mpt_to_compare)

        #print("\n k =",k)
        #print("coef:", coef)
        #print("y = aprox(x):\n",polinom)
        #print("Перевірка:\n", " X =", X, "\n mpt =    ", mpt_old,"\n mpt_to_compare =", mpt_to_compare)
        #print("bool_mpt =",bool_mpt)
        #print("eps =",eps)

        k += 1

    print("coef:", coef)
    print("y = aprox(x) :\n",polinom)
    print("Перевірка:\n", " X =", X, "\n", mpt_old, "= mpt\n",mpt_to_compare,"= mpt_to_compare")

    return coef, polinom, mpt_to_compare

#c0, p0,mpt_to_compare0 =  fun_aprox(D, mpt)
```

Побудуємо тепер початкові дані та знайдену криву (або криві y = 25/D та y = R^2 відповідно).

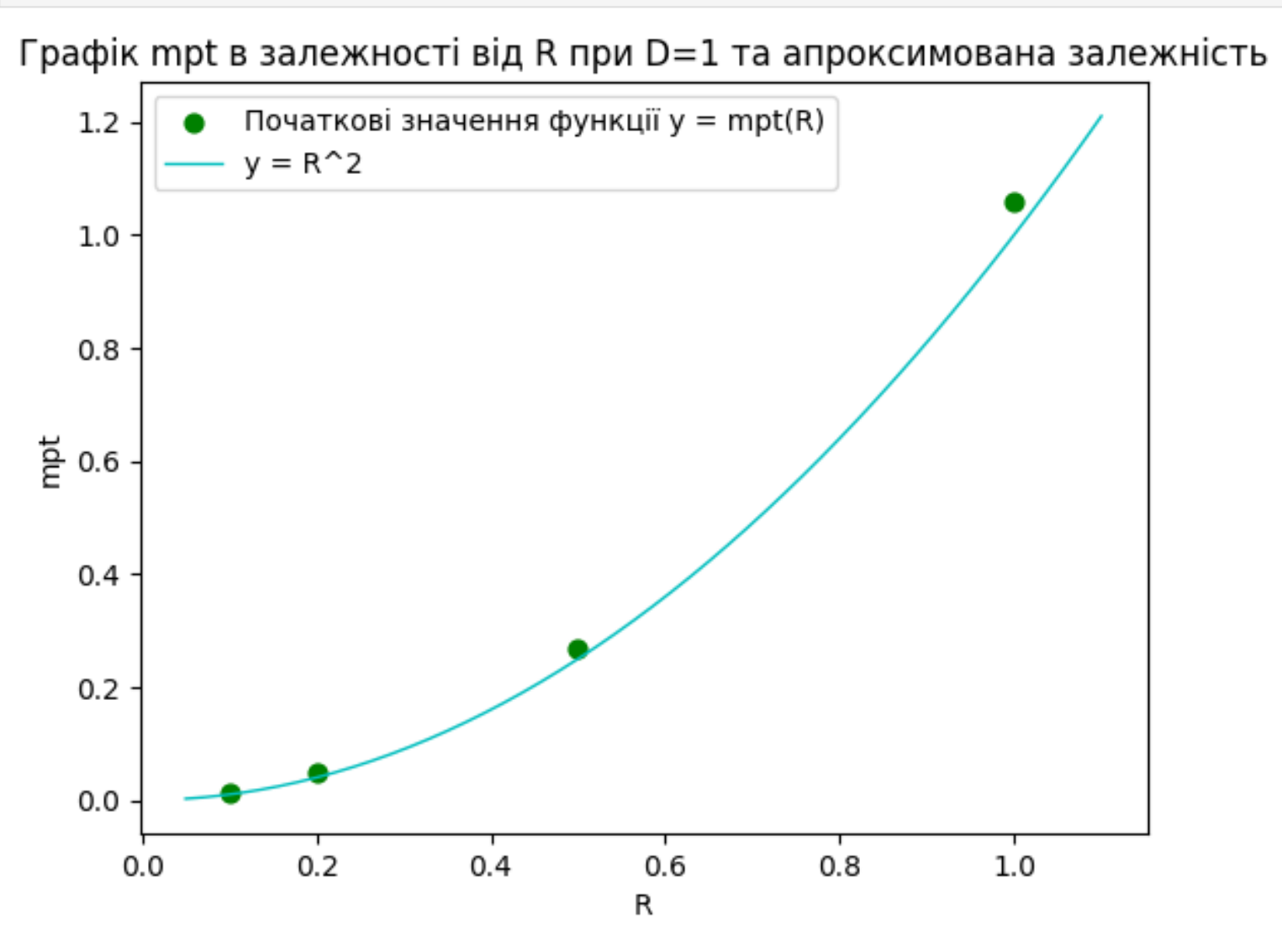
- Графік для D = [0.1, 0.2, 0.5, 1], R = 5.

In [10]:
```python
def fun_plot21_(mpt, D = [0.1, 0.2, 0.5, 1], R = 5):

    plt.scatter(D, mpt, label="Початкові значення функції y = mpt(D)", s=40, c='m')

    D_new = np.linspace(0.05, 1.1, 50)

    # порівняння з кривою y = 25/D
    mpt_new = 25 / D_new
    plt.plot(D_new, mpt_new, label= "y = 25/D", linewidth = 1, c='b')

    # порівняння з апроксимованою кривою деякого степеня
    # coef, polinom, mpt_to_compare = fun_aprox(D, mpt)
    # mpt_new = polinom(D_new)
    # plt.plot(D_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='c')

    plt.xlabel("D")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від D при R=" f'{R}' " та апроксимована залежність")
    plt.legend()

fun_plot21_(mpt)
```



- Графік для D = 1, R = [0.1, 0.2, 0.5, 1].

In [11]:
```python
def fun_plot22_(mpt, D = 1, R = [0.1, 0.2, 0.5, 1]):

    plt.scatter(R, mpt, label="Початкові значення функції y = mpt(R)", s=40, c='g')

    R_new = np.linspace(0.05, 1.1, 50)

    # порівняння з кривою y = R^2
    mpt_new = R_new**2
    plt.plot(R_new, mpt_new, label= "y = R^2",linewidth = 1,c='c')

    # порівняння з апроксимованою кривою деякого степеня
    # coef, polinom, mpt_to_compare = fun_aprox(R, mpt)
    # mpt_new = polinom(R_new)
    # plt.plot(R_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='b')

    plt.xlabel("R")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від R при D=" f'{D}' " та апроксимована залежність")
    plt.legend()

fun_plot22_(mpt_new)
```

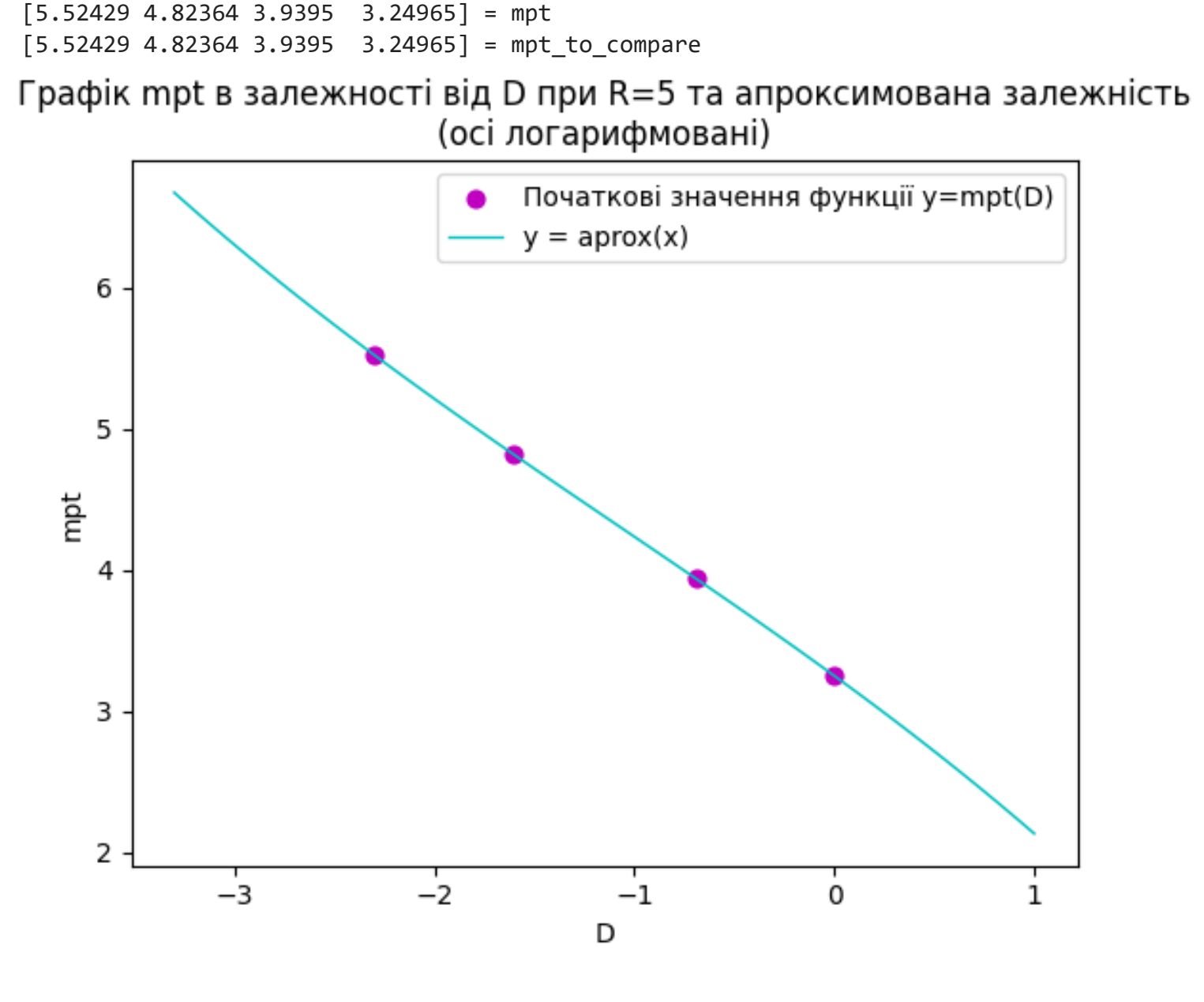


Побудуємо тепер дані у логарифмованих осях.

- Графік для D = [0.1, 0.2, 0.5, 1], R = 5.

In [12]:
```python
def fun_plot21_log(mpt, D = [0.1,0.2,0.5,1], R = 5):

    D = np.log(D)
    mpt = np.log(mpt)
    plt.scatter(D, mpt, label="Початкові значення функції y=mpt(D)", s=40, c='m')

    D_new = np.linspace(D[0]-1, D[len(D)-1]+1,50)

    coef, polinom, mpt_to_compare = fun_aprox(D, mpt)
    mpt_new = polinom(D_new)
    plt.plot(D_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='c')

    plt.xlabel("D")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від D при R=" f'{R}' " та апроксимована залежність\n(осі логарифмовані)")
    plt.legend()

fun_plot21_log(mpt)
```

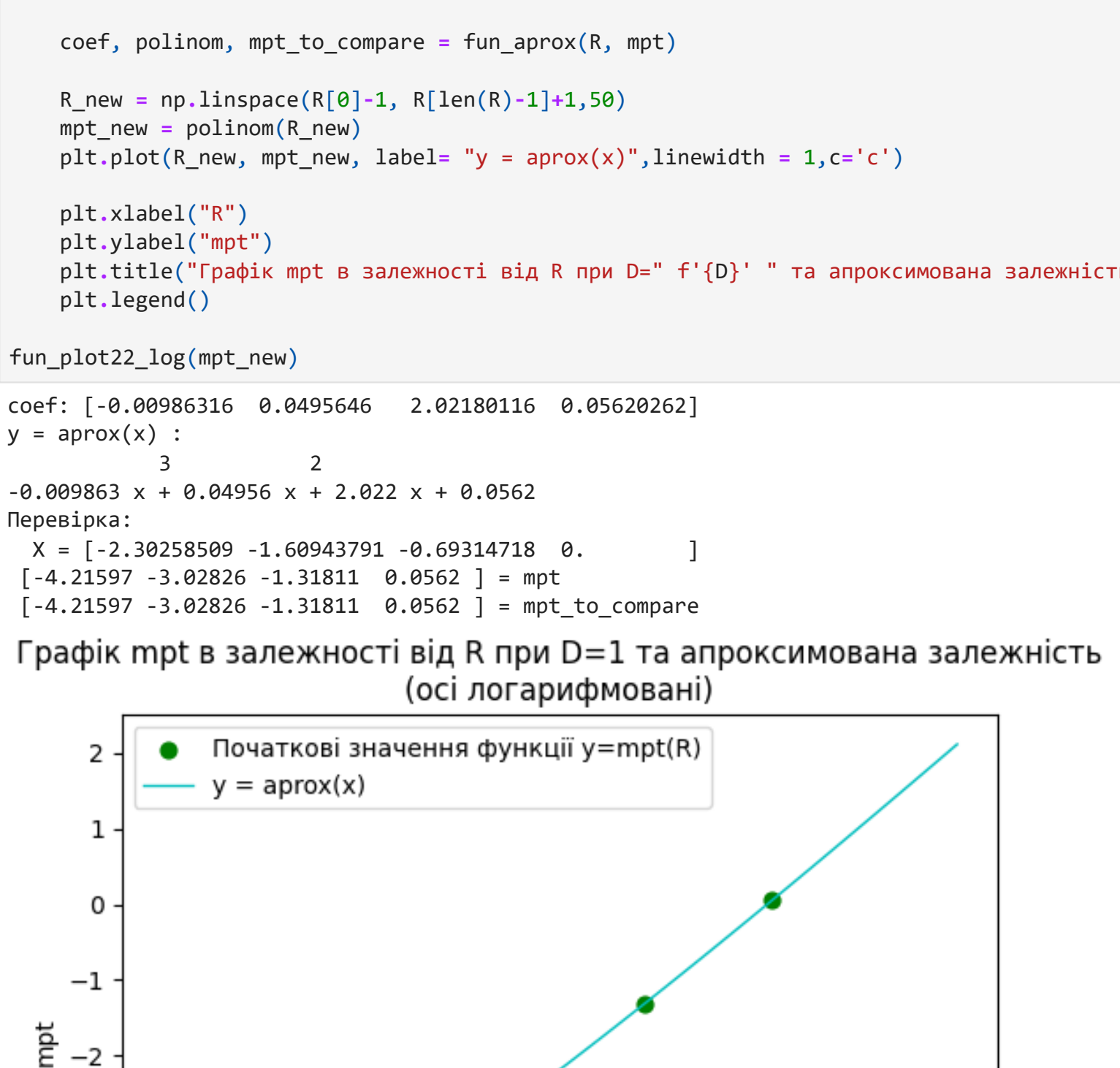
```
coef: [-0.02057347 -0.06621876 -1.0312605   3.24965128]
y = aprox(x) :
          3           2
-0.02057 x - 0.06622 x - 1.031 x + 3.25
Перевірка:
  X = [-2.30258509 -1.60943791 -0.69314718  0.        ]
 [5.52429 4.82364 3.9395  3.24965] = mpt
 [5.52429 4.82364 3.9395  3.24965] = mpt_to_compare
```



- Графік для D = 1, R = [0.1, 0.2, 0.5, 1].

In [13]:
```python
def fun_plot22_log(mpt, D = 1, R = [0.1, 0.2, 0.5, 1]):

    R = np.log(R)
    mpt = np.log(mpt)
    plt.scatter(R, mpt, label="Початкові значення функції y=mpt(R)", s=40, c='g')

    coef, polinom, mpt_to_compare = fun_aprox(R, mpt)

    R_new = np.linspace(R[0]-1, R[len(R)-1]+1,50)
    mpt_new = polinom(R_new)
    plt.plot(R_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='c')

    plt.xlabel("R")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від R при D=" f'{D}' " та апроксимована залежність\n(осі логарифмовані)")
    plt.legend()

fun_plot22_log(mpt_new)
```

```
coef: [-0.00986316  0.0495646   2.02180116  0.05620262]
y = aprox(x) :
           3           2
-0.009863 x + 0.04956 x + 2.022 x + 0.0562
Перевірка:
  X = [-2.30258509 -1.60943791 -0.69314718  0.        ]
 [-4.21597 -3.02826 -1.31811  0.0562 ] = mpt
 [-4.21597 -3.02826 -1.31811  0.0562 ] = mpt_to_compare
```



Побудуємо гістограму з осями часу та частоти.

In [14]:
```python
#!pip install seaborn
```

In [15]:
```python
import seaborn as sns
import pandas as pd
```

При визові функції 'distplot' у fun_histograma3() з'вляється попередження: C:\Users\Admin\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms). warnings.warn(msg, FutureWarning) Не будемо друкувати його завдяки наступному коду.

In [16]:
```python
import warnings
warnings.filterwarnings('ignore')
```

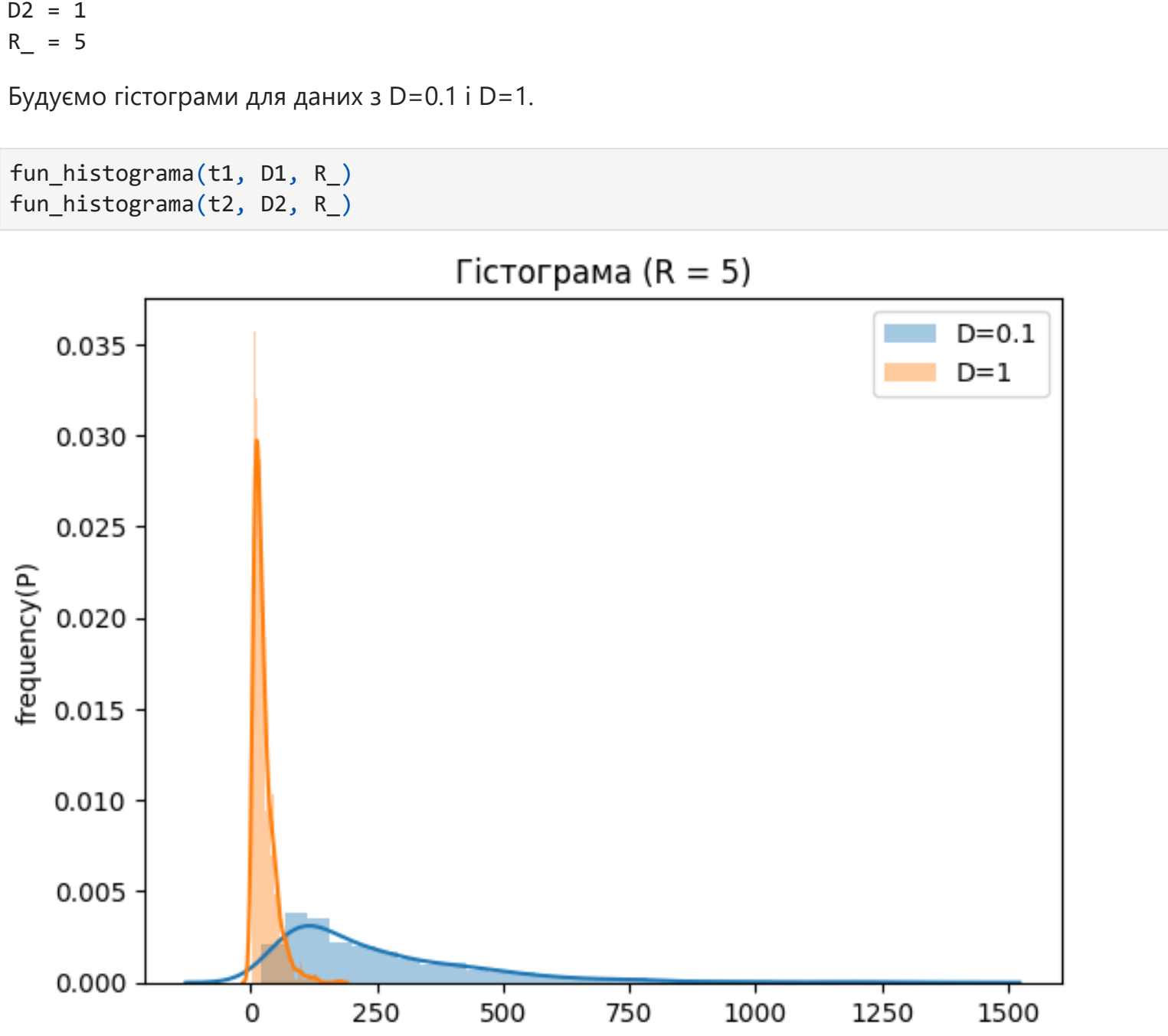
In [17]:
```python
def fun_histograma(time, D, R):

    # Графік частоти та функції розподілу
    sns_plot = sns.distplot(time,label="D=" f'{D}')
    #sns.distplot(time, rug=True, hist=True, label="D=" f'{D}')

    # Графік тільки функції розподілу
    #snsplot = sns.kdeplot(time, shade=True,label="D=" f'{D}')
    #fig = snsplot.get_figure()

    plt.xlabel('time')
    plt.ylabel('frequency(P)')
    plt.title("Гістограма (R = " f'{R}'")")
    plt.legend()
    #plt.show()

#t0, D0 = t[0], D[0]
#fun_histograma(t0, D0)
```

In [18]:
```python
t1, t2 = t[0], t[3]
D1, D2 = D[0], D[3]
R_ = 5
print("D1 =", D1, "\nD2 =", D2, "\nR_ =", R_)
```

```
D1 = 0.1
D2 = 1
R_ = 5
```

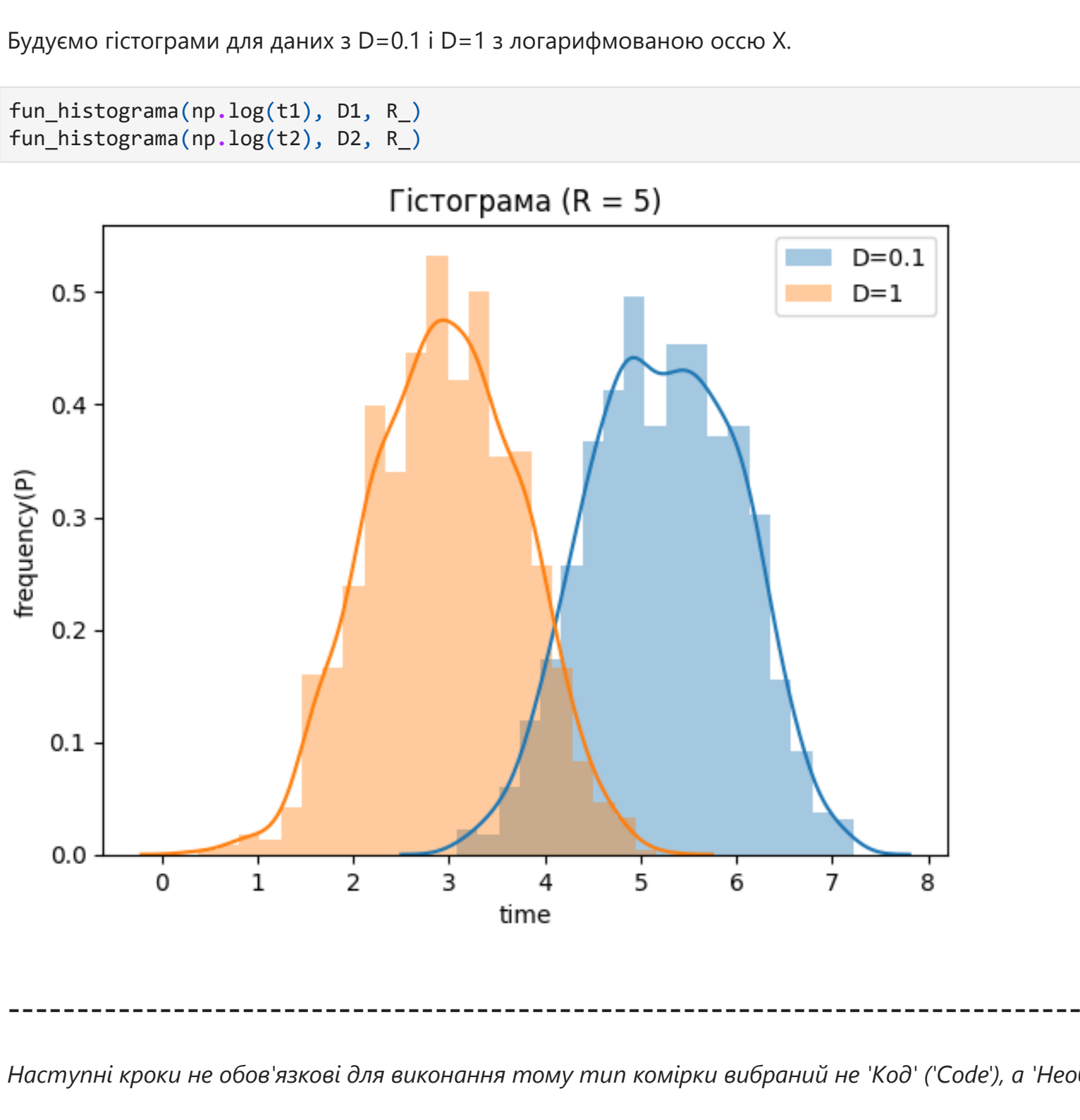
Будуємо гістограми для даних з D=0.1 і D=1.

In [19]:
```python
fun_histograma(t1, D1, R_)
fun_histograma(t2, D2, R_)
```



Будуємо гістограми для даних з D=0.1 і D=1 з логарифмованою оссю Х.

In [20]:
```python
fun_histograma(np.log(t1), D1, R_)
fun_histograma(np.log(t2), D2, R_)
```



Наступні кроки не обов'язкові для виконання тому тип комірки вибраний не 'Код' ('Code'), а 'Необроблений NBConvent' ('Raw NBConvent').

Тепер підкорегуємо дані та побудуємо гістограми знову.

print("t1: ",t1.mean(), t1.std()) print("t2: ",t2.mean(), t2.std())

- *1 варіант. Відмасштабуємо дані.*

mn1 = t1.mean() st1 = t1.std() t_new01 = (t1-mn1)/st1 print("t_new01: ",t_new01.mean(), t_new01.std()) mn2 = t2.mean() st2 = t2.std() t_new02 = (t2-mn2)/st2 print("t_new02: ",t_new02.mean(), t_new02.std()) fun_histograma(t_new01, D1) fun_histograma(t_new02, D2)

- *2 варіант. Нормалізуємо дані.*

print("t1: ",t1.mean(), t1.std()) print("t2: ",t2.mean(), t2.std()) t_new1_norm = (t1 - t1.min()) / (t1.max() - t1.min()) print("t_new1_norm: ", t_new1_norm.mean(), t_new1_norm.std()) t_new2_norm = (t2 - t2.min()) / (t2.max() - t2.min()) print("t_new2_norm: ", t_new2_norm.mean(), t_new2_norm.std()) fun_histograma(t_new1_norm, D1) fun_histograma(t_new2_norm, D2)

Або наступним чином:

def fun_histograma2(D1,time1,D2,time2): color_rectangle = np.random.rand(1, 3) # RGB plt.hist (time2, bins = 30, edgecolor='black', label="D=" f'{D1}',color = color_rectangle) color_rectangle = np.random.rand(1, 4) # RGB color_rectangle[:,3] = 0.4 plt.hist (time2, bins = 30, edgecolor='black', label="D=" f'{D2}',color = color_rectangle,width = 0.5) plt.xlabel('time') plt.ylabel('frequency(P)') plt.title('Гістограма') plt.legend() #plt.show() fun_histograma2(D1,t_new01,D2,t_new02)

In [ ]:
```python
```

Комп'ютерне моделювання задач прикладної математики

Дифузія невзаємодіючих частинок. Рівняння Ланжевена

Лабораторна робота 1

Студентка Пороскун Олена. Група ПМ.м-21

Task 3

In [1]:
```python
from matplotlib import pyplot as plt
```

In [2]:
```python
import random
import numpy as np
import math
import time
```

In [3]:
```python
def ksi():
    max_num = 2147483647
    p1 = random.randint(0, max_num) / (max_num+1)
    p2 = random.randint(0, max_num) / (max_num+1)
    if (p1==0):
        p1 = 1e-10
    if (p2==0):
        p2 = 1e-10
    return math.sqrt(-2*math.log(p1)) * math.sin(2*math.pi*p2)

#ksi()
```

Функція, що розраховує координати частинок, час за який кожна з них перетне другий локальний мінімум та середній час по всім частинкам.

In [4]:
```python
def funct3(a=1, D=1):

    t = 0                  # початковий момент часу
    dt = 0.002             # крок по часу

    N = 1000                # кількість точок(частинок)

    # потенціал V = a*x^4 - x^2
    # f(x) = - dV/dx = -(4a*x^3 - 2*x) = 2*x - 4a*x^3
    # x_n = x_n-1 + f(x_n-1)*dt + sqrt(D*dt)*ksi()

    # локальні мінімуми з р-ня f(x) = - dV/dx = 0: x_1 = 0, x_2,3 = +- 1/sqrt(2*a)
    minimum0 = 0
    minimum1 = -1/math.sqrt(2*a)
    minimum2 = -minimum1

    # початкові координати точок кладемо в локальний мінімум (той що зліва = -1/sqrt(2*a))
    x = [minimum1 for i in range(N)]

    # час для кожної частинки коли вона перетне другий мінімум
    times = np.zeros(N)

    # mean pasage time (середній час виходу за межі другого мінімуму)
    mpt = 0

    for i in range(np.size(x)):
        t=0
        while((x[i]) <= minimum2):
            f =  2*x[i] - 4*a*(x[i]**3)
            x[i] += f*dt + math.sqrt(D*dt)*ksi()
            if (x[i] > minimum2):
                times[i] = t
            t += dt
    mpt = np.mean(times)

    #print("a","\t D","\t mpt")
    print(a, "\t", D,"\t", mpt)

    return x, times, mpt

#x0, times0, mpt0 = funct3(a=0.5, D=1)
```

# Код для розрахунку часу виконання програми: start_time = time.time() {' main()... '} print("--- %s seconds ---" % (time.time() - start_time))

- *Розрахуємо дані для a = [0.5, 1, 1.5, 2], D = 1.*

In [5]:
```python
start_time = time.time()

a = [0.5, 1, 1.5, 2]
x = []             # координати частинок
t = []             # час для кожної частинки коли вона перетне другий локальний мінімум
mpt = []           # mean pasage time (середній час виходу за межі другого мінімуму)

print("a","\t D","\t mpt")

for i in range(len(a)):
    x_, t_, mpt_ = funct3(a = a[i], D = 1)
    x.append(x_)
    t.append(t_)
    mpt.append(mpt_)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
a        D       mpt
0.5      1       7.32068399999995
1        1       3.2956019999998842
1.5      1       2.342891999999927
2        1       1.7222939999999518
--- 26.304203510284424 seconds ---
```

In [6]:
```python
print(np.shape(x))
print(np.shape(t))
print(np.shape(mpt))
```

```
(4, 1000)
(4, 1000)
(4,)
```

- *Розрахуємо дані для a = 1, D = [0.5, 1, 1.5, 2].*

In [7]:
```python
start_time = time.time()

a_new = 1
D_new = [0.5, 1, 1.5, 2]
x_new = []             # координати частинок
t_new = []             # час для кожної частинки коли вона перетне другий локальний мінімум
mpt_new = []           # mean pasage time (середній час виходу за межі другого мінімуму)

print("a","\t D","\t mpt")

for i in range(len(a)):
    x_, t_, mpt_ = funct3(a = a_new, D = D_new[i])
    x_new.append(x_)
    t_new.append(t_)
    mpt_new.append(mpt_)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
a        D       mpt
1        0.5     7.152293999999871
1        1       3.4728659999998817
1        1.5     2.31681399999992
1        2       1.7712099999999604
--- 25.963239669799805 seconds ---
```

In [8]:
```python
print(np.shape(x_new))
print(np.shape(t_new))
print(np.shape(mpt_new))
```

```
(4, 1000)
(4, 1000)
(4,)
```

- *Розрахуємо дані для a = 1, D = [0.75, 1.25, 1.75].*

In [9]:
```python
start_time = time.time()

a_new2 = 1
D_new2 = [0.75, 1.25, 1.75]
x_new2 = []             # координати частинок
t_new2 = []             # час для кожної частинки коли вона перетне другий локальний мінімум
mpt_new2 = []           # mean pasage time (середній час виходу за межі другого мінімуму)

print("a","\t D","\t mpt")

for i in range(len(D_new2)):
    x_, t_, mpt_ = funct3(a = a_new2, D = D_new2[i])
    x_new2.append(x_)
    t_new2.append(t_)
    mpt_new2.append(mpt_)

print("--- %s seconds ---" % (time.time() - start_time))
```

```
a        D       mpt
1        0.75    4.433153999999897
1        1.25    2.9295719999999026
1        1.75    2.052847999999939
--- 16.0100359916687 seconds ---
```

In [10]:
```python
print(np.shape(x_new2))
print(np.shape(t_new2))
print(np.shape(mpt_new2))
```

```
(3, 1000)
(3, 1000)
(3,)
```

Функція, що априксимує дані, повертає: \ ~ coef - коефіцієнти рівняння, яке відповідає початковим даним, \ ~ polinom - рівняння з коефіцієнтами, \ ~ mpt_to_compare - значення функції для порівняння зі початковими

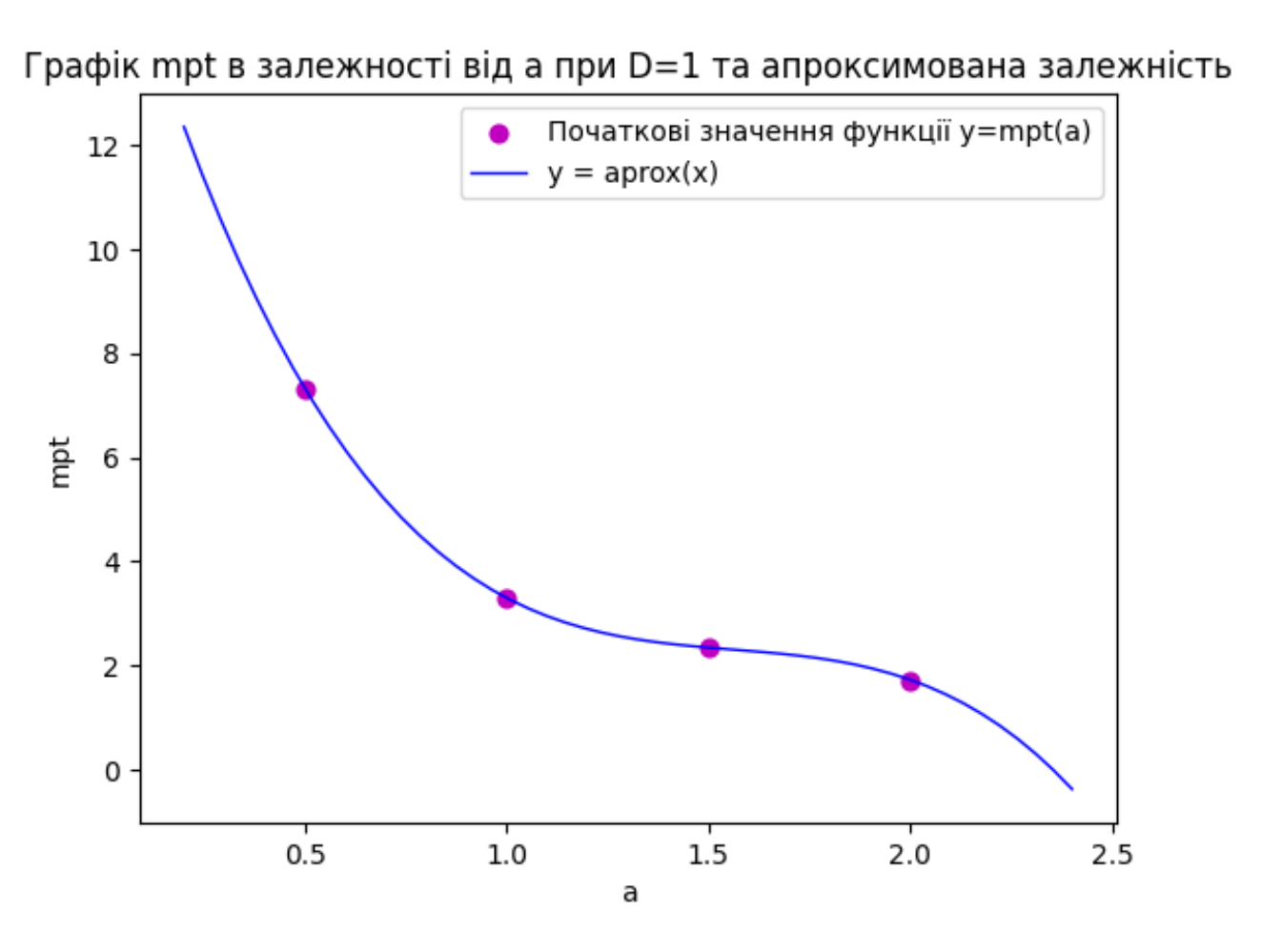
In [11]:
```python
def fun_aprox(X, mpt):

    k=1
    mpt_new = mpt*0
    eps = 1000

    bool_mpt = 0

    while ((bool_mpt < np.size(mpt)) & (eps >= 0.00001) & (k<10)):

        coef = np.polyfit(X, mpt,k)
        polinom = np.poly1d(coef)
        mpt_to_compare = polinom(X)

        mpt_old = np.round(mpt,5)
        mpt_to_compare = np.round(mpt_to_compare,5)

        eps = abs(mpt_old[0] - mpt_to_compare[0])

        bool_mpt = np.sum(mpt_old == mpt_to_compare)

        #print("\n k =",k)
        #print("coef:", coef)
        #print("y = aprox(x):\n",polinom)
        #print("Перевірка:\n", " X =", X, "\n mpt =    ", mpt_old,"\n mpt_to_compare =", mpt_to_compare)
        #print("bool_mpt =",bool_mpt)
        #print("eps =",eps)

        k += 1

    print("X:", X)
    print("coef:", coef)
    print("y = aprox(x) :\n",polinom)
    print("Перевірка:\n", " X =", X, "\n", mpt_old, "= mpt\n",mpt_to_compare,"= mpt_to_compare", "\n")

    return coef, polinom, mpt_to_compare

#c0, p0,mpt_to_compare0 =  fun_aprox(D, mpt)
```

Побудуємо тепер початкові дані та знайдену криву.

- Графік для a = [0.5, 1, 1.5, 2], D = 1.

In [12]:
```python
def fun_plot31(mpt, a = [0.5, 1, 1.5, 2], D = 1):

    plt.scatter(a, mpt, label="Початкові значення функції y=mpt(a)", s=40, c='m')

    coef, polinom, mpt_to_compare = fun_aprox(a, mpt)

    a_new = np.linspace(0.2, 2.4,50)
    mpt_new = polinom(a_new)
    plt.plot(a_new, mpt_new, label = "y = aprox(x)", linewidth = 1, c='b')

    plt.xlabel("a")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від а при D=" f'{D}' " та апроксимована залежність")
    plt.legend()

fun_plot31(mpt)
```

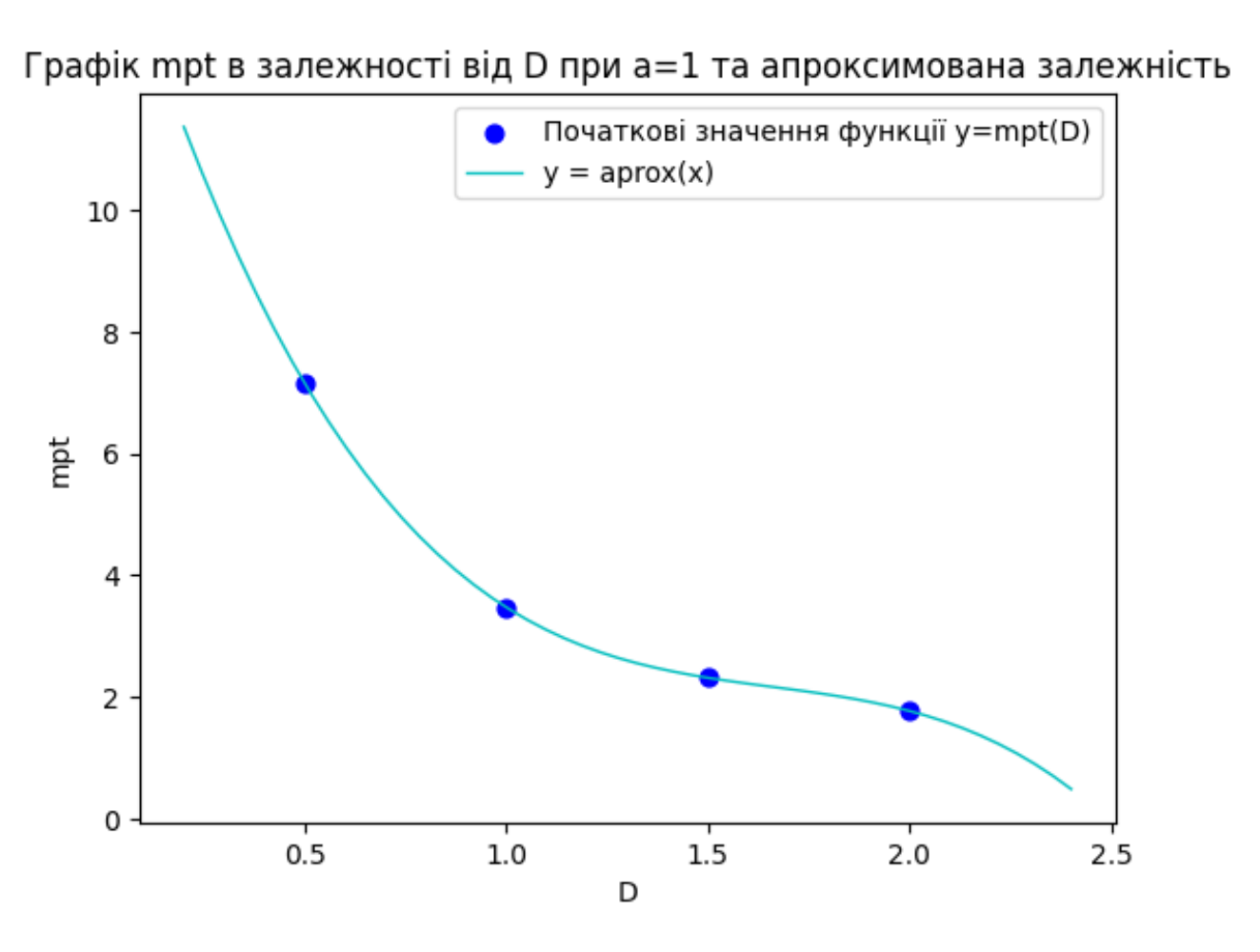
```
X: [0.5, 1, 1.5, 2]
coef: [ -3.65368   17.105784 -27.3149    17.158398]
y = aprox(x) :
         3         2
-3.654 x + 17.11 x - 27.31 x + 17.16
Перевірка:
  X = [0.5, 1, 1.5, 2]
 [7.32068 3.2956  2.34289 1.72229] = mpt
 [7.32068 3.2956  2.34289 1.72229] = mpt_to_compare
```



- Графік для a = 1, D = [0.5, 1, 1.5, 2].

In [13]:
```python
def fun_plot32(mpt, a = 1, D = [0.5, 1, 1.5, 2]):

    plt.scatter(D, mpt, label="Початкові значення функції y=mpt(D)", s=40, c='b')

    coef, polinom, mpt_to_compare = fun_aprox(D, mpt)

    D_new = np.linspace(0.2, 2.4,50)
    mpt_new = polinom(D_new)
    plt.plot(D_new, mpt_new, label = "y = aprox(x)", linewidth = 1, c='c')

    plt.xlabel("D")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від D при a=" f'{a}' " та апроксимована залежність")
    plt.legend()

fun_plot32(mpt_new)
```

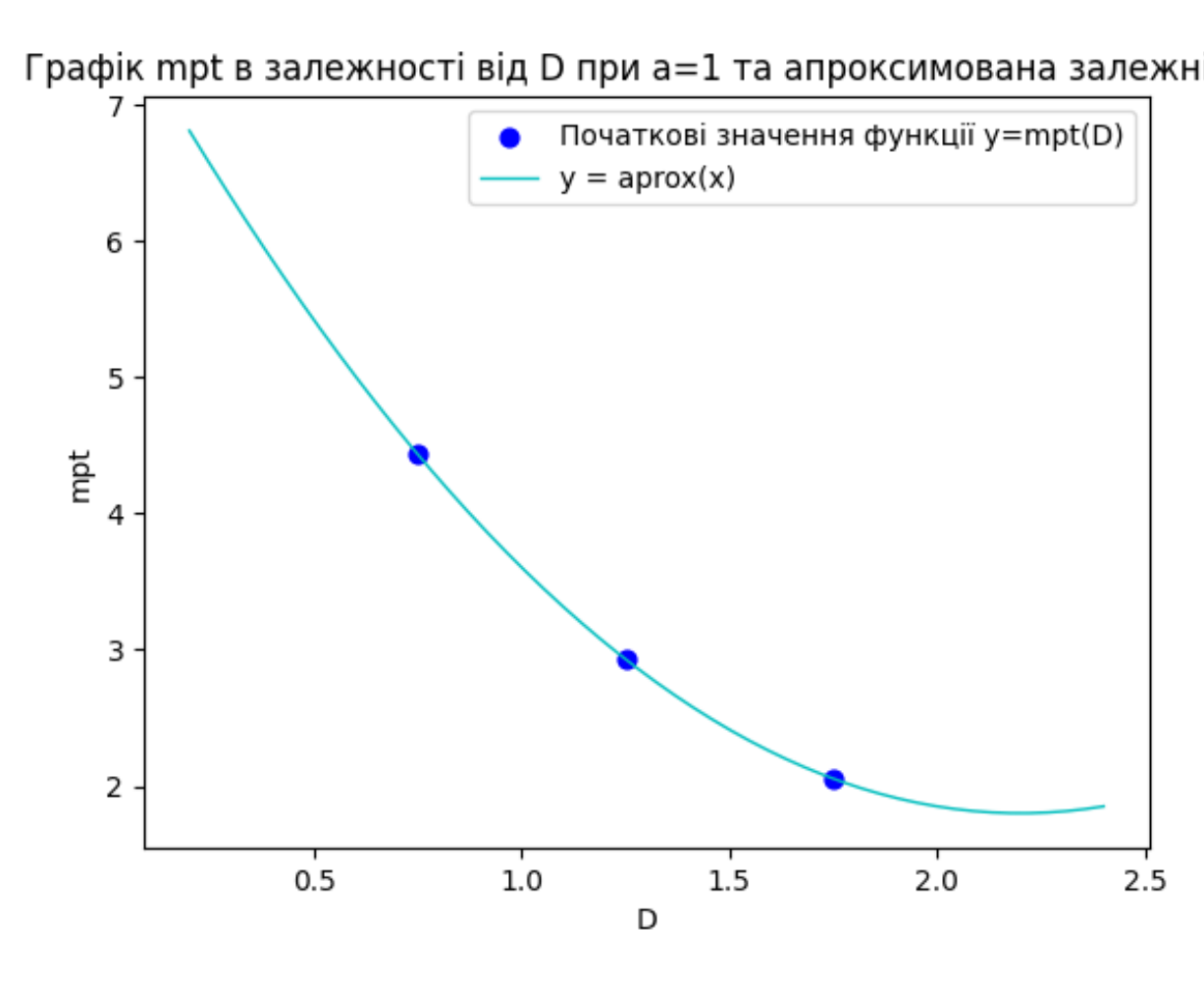
```
X: [0.5, 1, 1.5, 2]
coef: [ -2.55057067  12.698464   -21.94305333  15.268026  ]
y = aprox(x) :
         3        2
-2.551 x + 12.7 x - 21.94 x + 15.27
Перевірка:
  X = [0.5, 1, 1.5, 2]
 [7.15229 3.47287 2.31681 1.77121] = mpt
 [7.15229 3.47287 2.31681 1.77121] = mpt_to_compare
```



- Графік для a = 1, D = [0.75, 1.25, 1.75].

In [14]:
```python
fun_plot32(mpt_new2, D=D_new2)
```

```
X: [0.75, 1.25, 1.75]
coef: [ 1.253716   -5.514596    7.86388575]
y = aprox(x) :
        2
1.254 x - 5.515 x + 7.864
Перевірка:
  X = [0.75, 1.25, 1.75]
 [4.43315 2.92957 2.05285] = mpt
 [4.43315 2.92957 2.05285] = mpt_to_compare
```

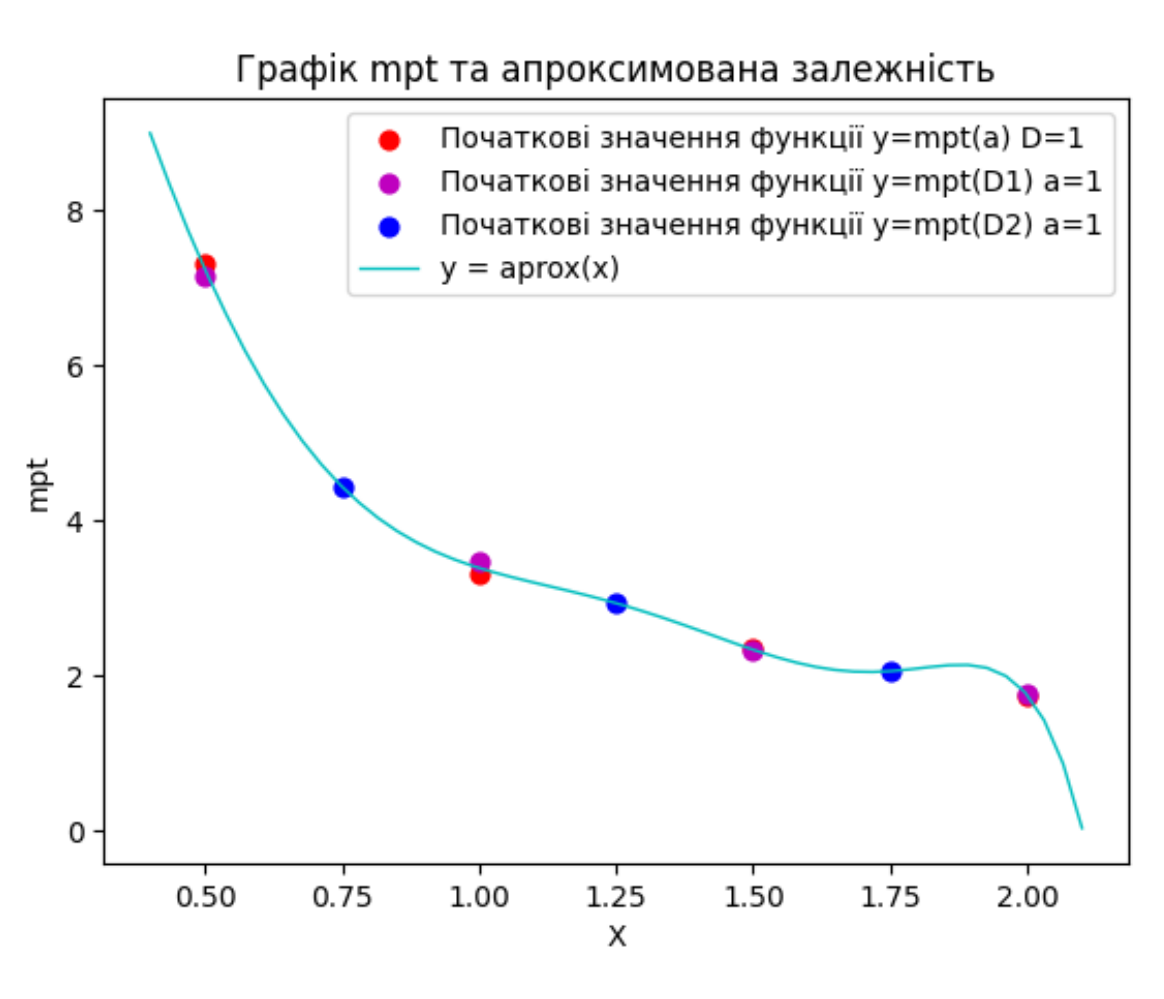


Побудуємо дані всіх 3 випадків на одному графіку. Потім спробуємо апроксимувати ці дані. На цьому ж графіку побудуємо апроксимовану криву.

In [15]:
```python
import warnings
warnings.filterwarnings('ignore')
```

In [16]:
```python
def fun_plot33(mpt, mpt1, mpt2):

    # 1 частина
    D_const = 1
    a = [0.5, 1, 1.5, 2]
    plt.scatter(a, mpt, label="Початкові значення функції y=mpt(a) D="f'{D_const}', s=40, c='r')

    # 2 частина
    a_const = 1
    D1 = [0.5, 1, 1.5, 2]
    plt.scatter(D1, mpt1, label="Початкові значення функції y=mpt(D1) a="f'{a_const}', s=40, c='m')

    # 3 частина
    D2 = [0.75, 1.25, 1.75]
    plt.scatter(D2, mpt2, label="Початкові значення функції y=mpt(D2) a="f'{a_const}', s=40, c='b')

    X = np.hstack((a, D1, D2))
    y = np.hstack((mpt, mpt1, mpt2))

    x_apr = np.linspace(0.4, 2.1, 50)
    coef, polinom, mpt_to_compare = fun_aprox(X, y)
    mpt_new = polinom(x_apr)

    #plt.scatter(X, y, label="Початкові значення функції", s=200, c=[0.5, 0.5, 0.5, 0.3])
    plt.plot(x_apr, mpt_new, label = "y = aprox(x)", linewidth = 1, c='c')

    plt.xlabel("X")
    plt.ylabel("mpt")
    plt.title("Графік mpt та апроксимована залежність")
    plt.legend()

fun_plot33(mpt, mpt_new, mpt_new2)
```

```
X: [0.5  1.   1.5  2.   0.5  1.   1.5  2.   0.75 1.25 1.75]
coef: [ -0.35730522   0.83303976   1.17417409  -1.31312152  -5.69733574
  -1.58847052  18.17375186   1.02523442 -27.72060756  18.85487443]
y = aprox(x) :
         9         8         7         6         5         4         3
-0.3573 x + 0.833 x + 1.174 x - 1.313 x - 5.697 x - 1.588 x + 18.17 x
          2
 + 1.025 x - 27.72 x + 18.85
Перевірка:
  X = [0.5  1.   1.5  2.   0.5  1.   1.5  2.   0.75 1.25 1.75]
 [7.32068 3.2956  2.34289 1.72229 7.15229 3.47287 2.31681 1.77121 4.43315
 2.92957 2.05285] = mpt
 [7.23649 3.38423 2.32985 1.74675 7.23649 3.38423 2.32985 1.74675 4.43315
 2.92957 2.05285] = mpt_to_compare
```

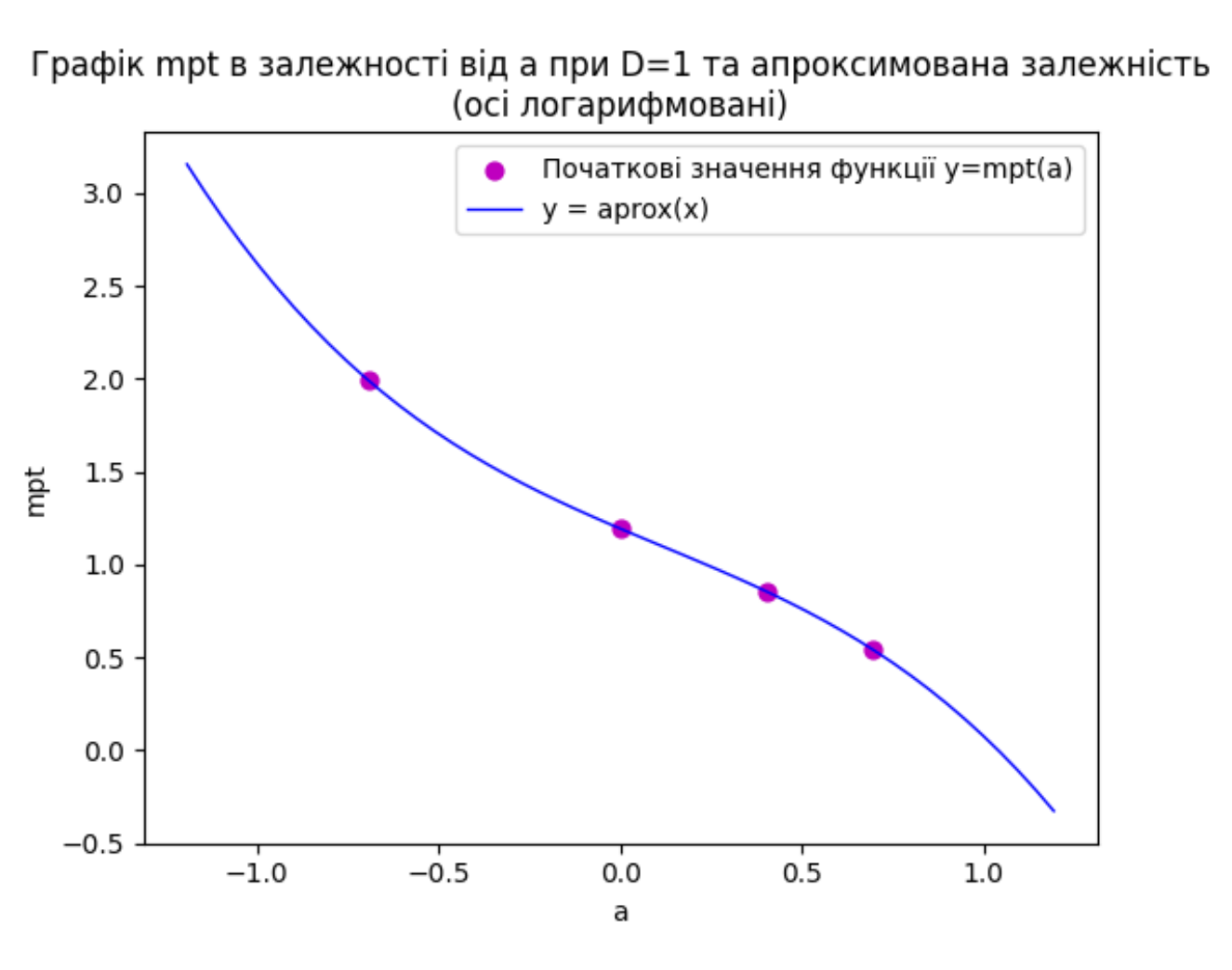


Побудуємо тепер дані у логарифмованих осях.

- Графік для a = [0.5, 1, 1.5, 2], D = 1.

In [17]:
```python
def fun_plot31_log(mpt, a = [0.5, 1, 1.5, 2], D = 1):

    a = np.log(a)
    mpt = np.log(mpt)
    plt.scatter(a, mpt, label="Початкові значення функції y=mpt(a)", s=40, c='m')

    coef, polinom, mpt_to_compare = fun_aprox(a, mpt)

    a_new = np.linspace(a[0]-0.5, a[len(a)-1]+0.5,50)
    mpt_new = polinom(a_new)
    plt.plot(a_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='b')

    plt.xlabel("a")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від а при D=" f'{D}' " та апроксимована залежність\n(осі логарифмовані)")
    plt.legend()

fun_plot31_log(mpt)
```

```
X: [-0.69314718  0.          0.40546511  0.69314718]
coef: [-0.44095522  0.15525263 -0.83196524  1.19258885]
y = aprox(x) :
         3          2
-0.441 x + 0.1553 x - 0.832 x + 1.193
Перевірка:
  X = [-0.69314718  0.          0.40546511  0.69314718]
 [1.9907  1.19259 0.85139 0.54366] = mpt
 [1.9907  1.19259 0.85139 0.54366] = mpt_to_compare
```

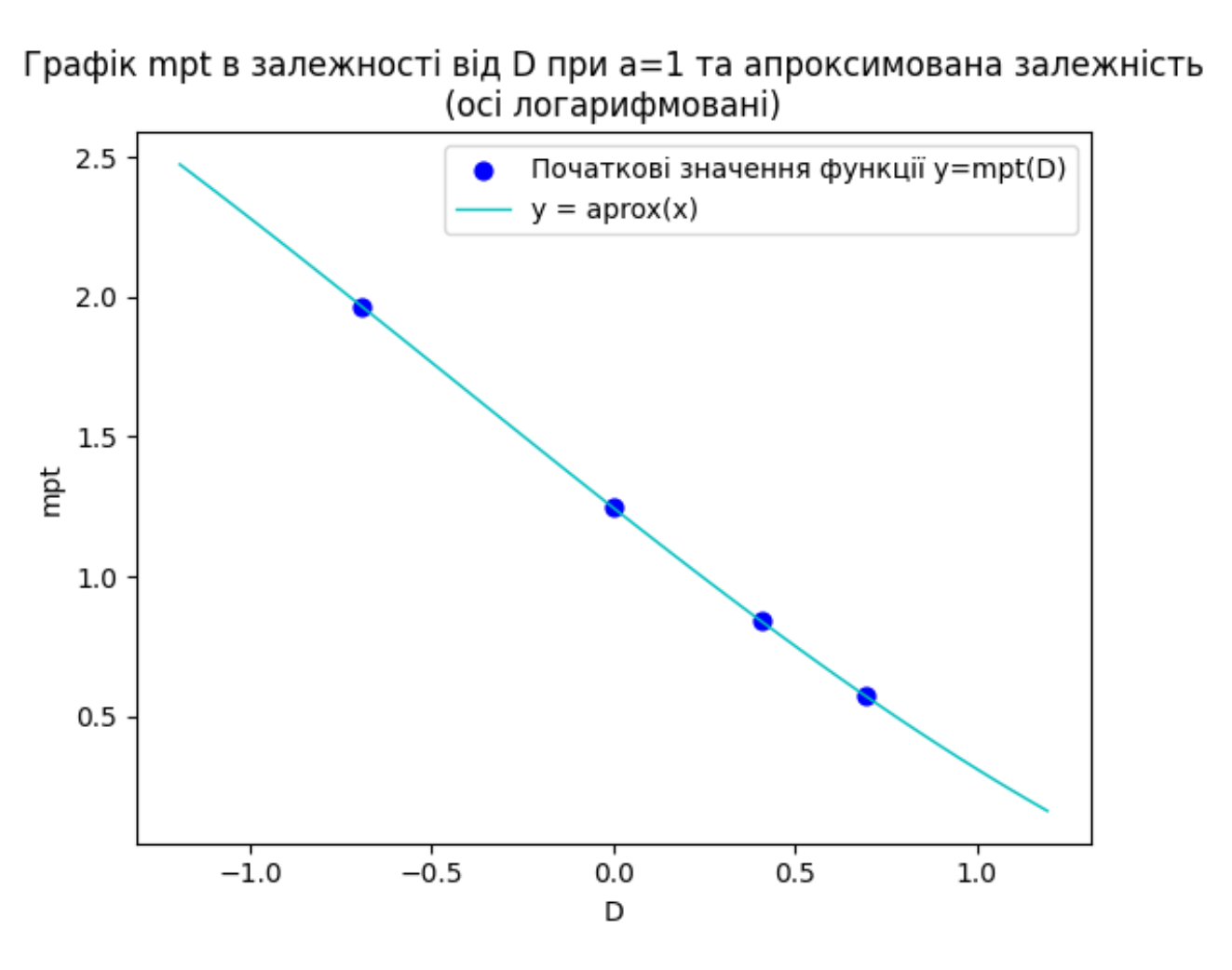


- Графік для a = 1, D = [0.5, 1, 1.5, 2].

In [18]:
```python
def fun_plot32_log(mpt, a = 1, D = [0.5, 1, 1.5, 2]):

    D = np.log(D)
    mpt = np.log(mpt)
    plt.scatter(D, mpt, label="Початкові значення функції y=mpt(D)", s=40, c='b')

    coef, polinom, mpt_to_compare = fun_aprox(D, mpt)

    D_new = np.linspace(D[0]-0.5, D[len(D)-1]+0.5,50)
    mpt_new = polinom(D_new)
    plt.plot(D_new, mpt_new, label= "y = aprox(x)",linewidth = 1,c='c')

    plt.xlabel("D")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від D при a=" f'{a}' " та апроксимована залежність\n(осі логарифмовані)")
    plt.legend()

#fun_plot32_log(mpt_new)
```

In [19]:
```python
fun_plot32_log(mpt_new,  D = D_new)
```

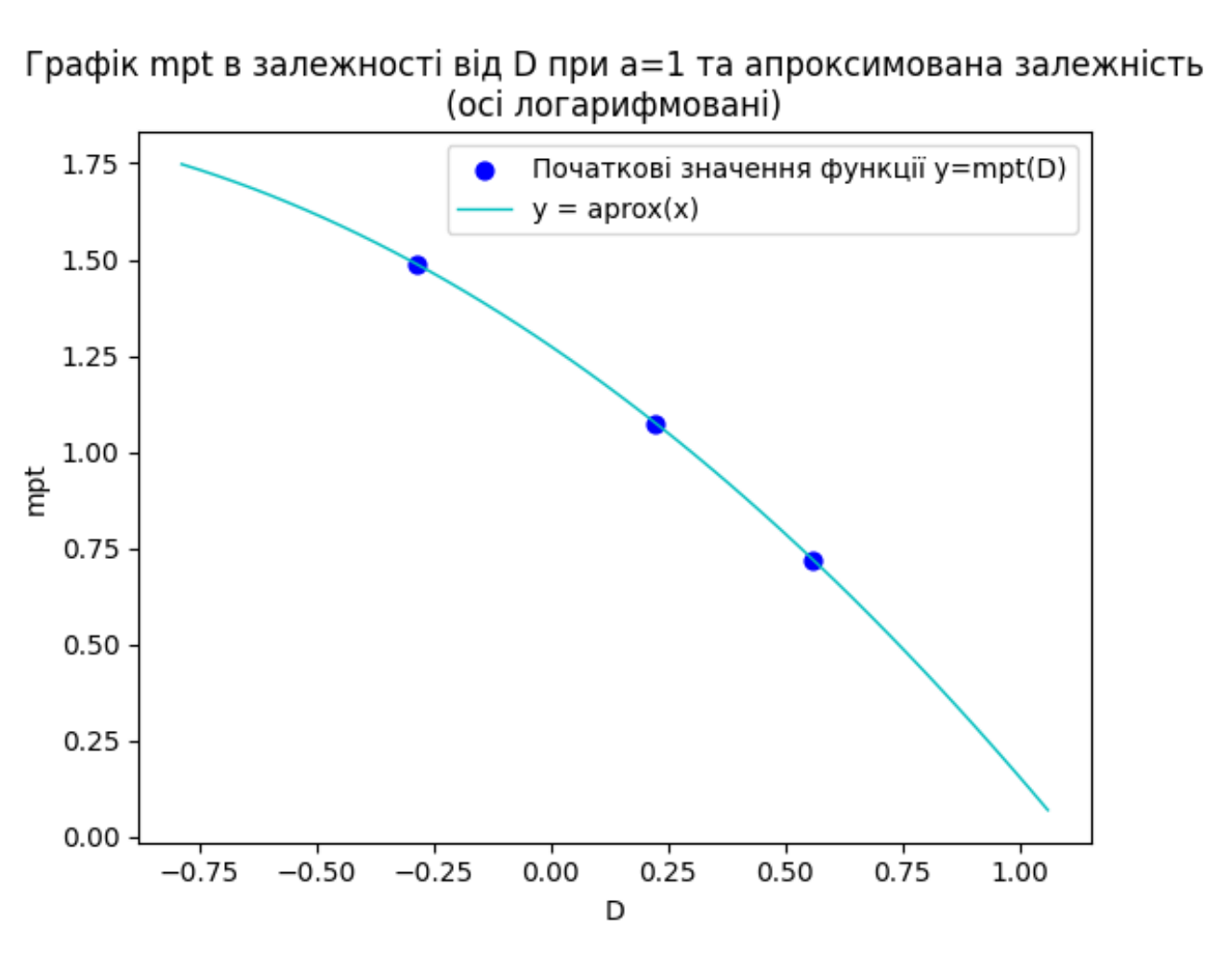
```
X: [-0.69314718  0.          0.40546511  0.69314718]
coef: [ 0.03868396  0.05113475 -1.02542121  1.24498019]
y = aprox(x) :
         3           2
0.03868 x + 0.05113 x - 1.025 x + 1.245
Перевірка:
  X = [-0.69314718  0.          0.40546511  0.69314718]
 [1.96743 1.24498 0.84019 0.57166] = mpt
 [1.96743 1.24498 0.84019 0.57166] = mpt_to_compare
```



- Графік для a = 1, D = [0.75, 1.25, 1.75].

In [20]:
```python
fun_plot32_log(mpt_new2, D = D_new2)
```

```
X: [-0.28768207  0.22314355  0.55961579]
coef: [-0.29031125 -0.82968806  1.27445136]
y = aprox(x) :
         2
-0.2903 x - 0.8297 x + 1.274
Перевірка:
  X = [-0.28768207  0.22314355  0.55961579]
 [1.48911 1.07486 0.71923] = mpt
 [1.48911 1.07486 0.71923] = mpt_to_compare
```



Побудуємо гістограму з осями часу та частоти.

In [21]:
```python
import seaborn as sns
import pandas as pd
```

При визові функції 'distplot' у fun_histograma3() з'являється попередження: C:\Users\Admin\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms). warnings.warn(msg, FutureWarning) Не будемо друкувати його завдяки наступному коду.

In [22]:
```python
import warnings
warnings.filterwarnings('ignore')
```

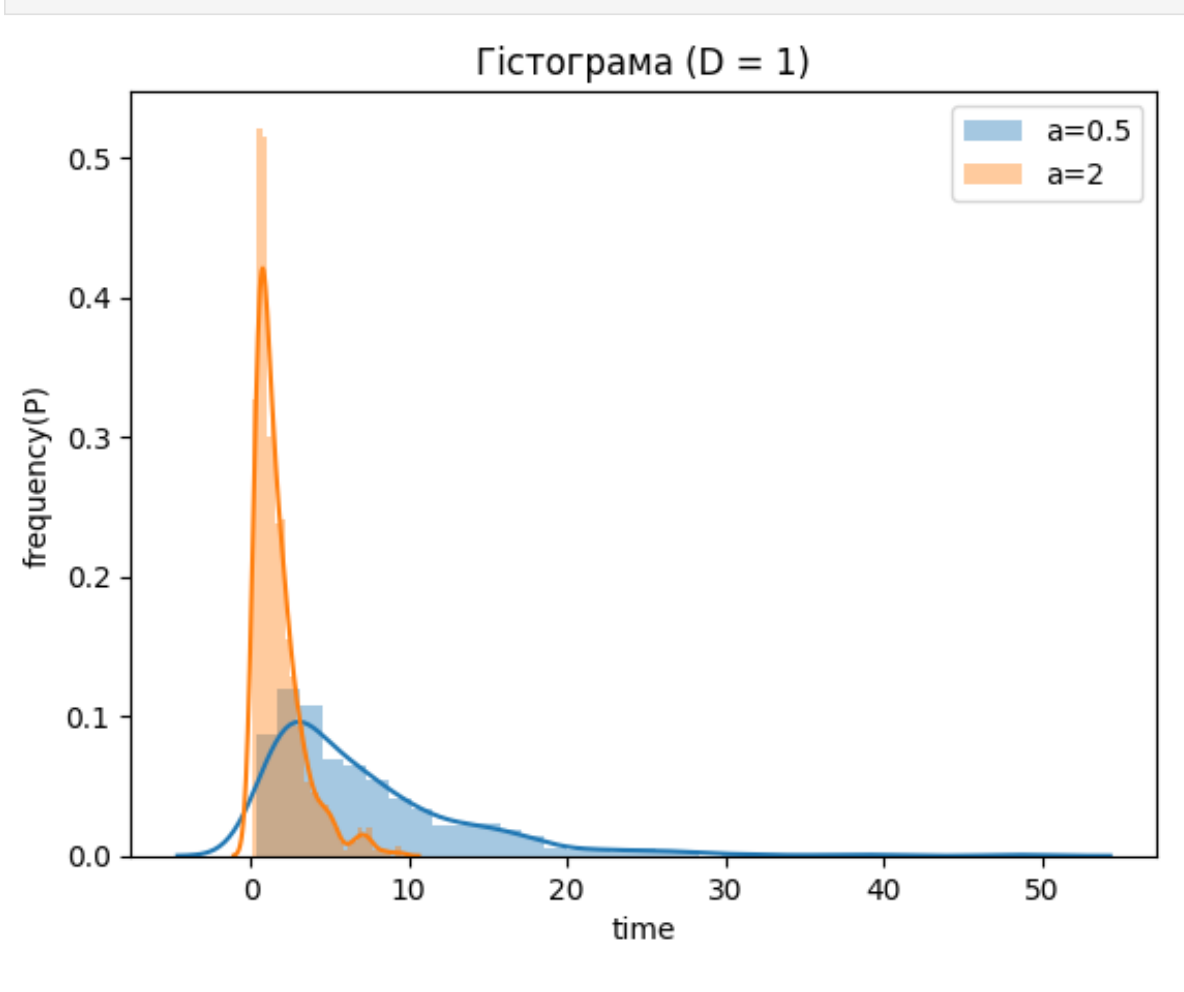
In [23]:
```python
def fun_histograma(time, a = 1, D = 1):

    # Графік частоти та функції розподілу
    sns_plot = sns.distplot(time,label="a=" f'{a}')
    #sns.distplot(time, rug=True, hist=True, label="a=" f'{a}')

    # Графік тільки функції розподілу
    #snsplot = sns.kdeplot(time, shade=True,label="a=" f'{a}')
    #fig = snsplot.get_figure()

    plt.xlabel('time')
    plt.ylabel('frequency(P)')
    plt.title("Гістограма (D = " f'{D}'")")
    plt.legend()
    #plt.show()

#t0, a0 = t[0], a[0]
#fun_histograma(t0, a0)
```

In [24]:
```python
t1, t2 = t[0], t[3]
a1, a2 = a[0], a[3]
D_ = 1
print("a1 =", a1, "\na2 =", a2, "\nD_ =", D_)
```

```
a1 = 0.5
a2 = 2
D_ = 1
```

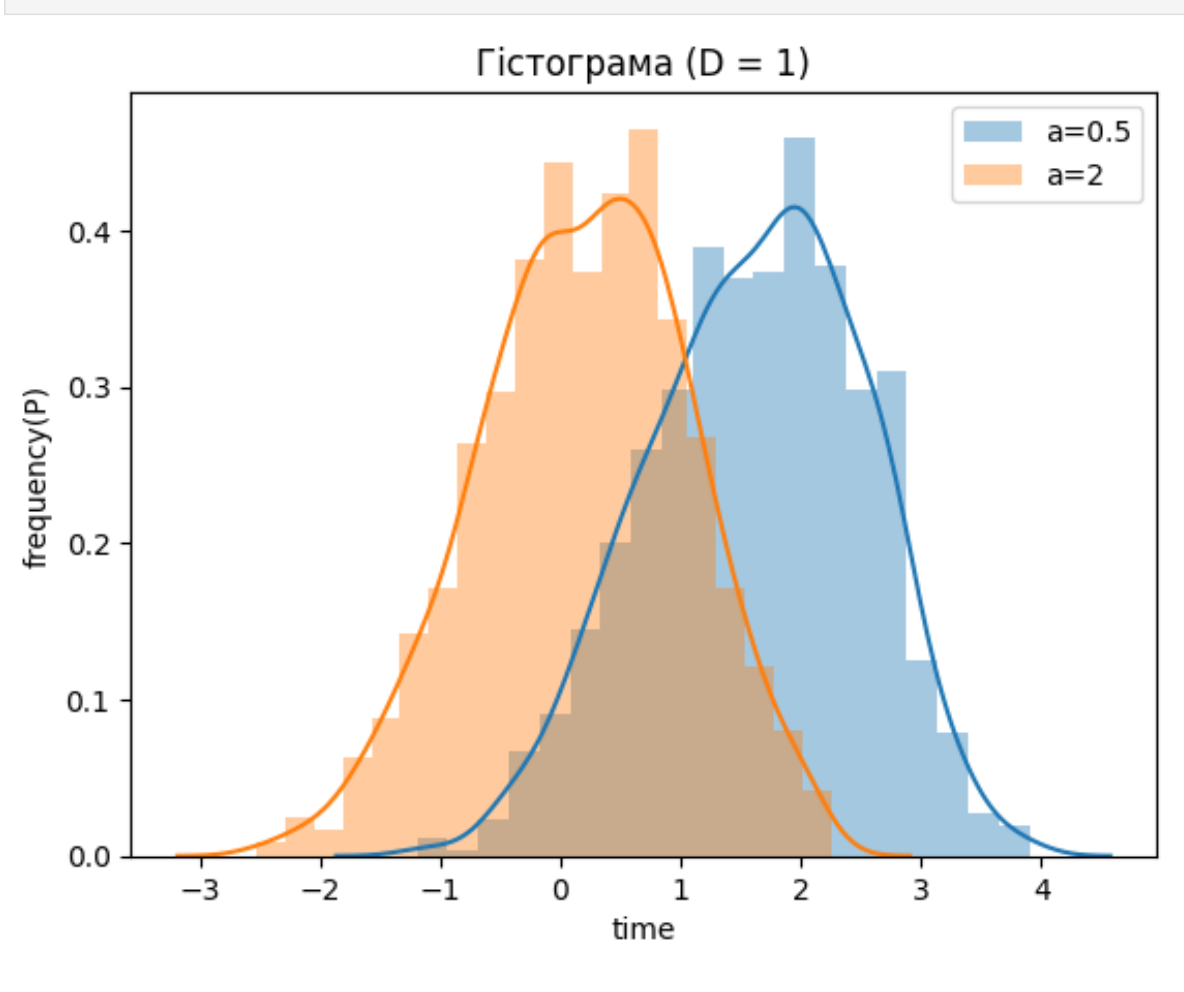
Будуємо гістограми для даних з a=0.5 і a=2.

In [25]:
```python
fun_histograma(t1, a1, D_)
fun_histograma(t2, a2, D_)
```



Будуємо гістограми для даних з a=0.5 і a=2 з логарифмованою оссю X.

In [26]:
```python
fun_histograma(np.log(t1), a1, D_)
fun_histograma(np.log(t2), a2, D_)
```



In [ ]:

Комп'ютерне моделювання задач прикладної математики

Основи класичної молекулярної динаміки.

Лабораторна робота 2

Студентка Пороскун Олена. Група ПМ.м-21

Task 1

In [1]:
```python
from matplotlib import pyplot as plt
```

In [2]:
```python
import random
import numpy as np
import math
import time
```

In [3]:
```python
count = 10 # кількість частинок
```

Розраховуємо випадкову величину.

In [4]:
```python
def randV():
    #random.seed(s)
    max_num = 2147483647
    ksi = random.randint(0, max_num) / (max_num+1) # ksi = rand_r(&seedp)/(RAND_MAX+1.0);
    return ksi
#randV()
```

Розраховуємо початкові значення координат (x_0, y_0) та швидкостей частинок (vx_0, vy_0), що знаходяться у резервуарі розміром (Lx, Ly).

In [5]:
```python
def Inital_XY(N=100, Lx = 20, Ly = 20, vmax = 20):
    x0,  y0   = np.zeros(N), np.zeros(N)
    vx0, vy0  = np.zeros(N), np.zeros(N)
    for i in range(N):
        x0[i] = randV()*Lx;
        y0[i] = randV()*Ly;
        vx0[i] = vmax*(2*randV() - 1);
        vy0[i] = vmax*(2*randV() - 1);
    return x0, y0, vx0, vy0
#x0_, y0_, vx0_, vy0_ = Inital_XY()
```

In [6]:
```python
x0, y0, vx0, vy0 = Inital_XY(N=count)
```

Перевіряємо межі початкових значень,

In [7]:
```python
print(min(x0),min(y0))
print(max(vx0), max(vy0))
```

```
0.45563272200524807 2.332343077287078
8.696286790072918 16.450089812278748
```

Перевіряємо розмірність масивів.

In [8]:
```python
print(np.shape(x0),np.shape(y0), np.shape(vx0), np.shape(vy0))
```

```
(10,) (10,) (10,) (10,)
```

Виводимо перші 5 значень з кожного масиву.

In [9]:
```python
list(zip(x0[:5], y0[:5]))
```

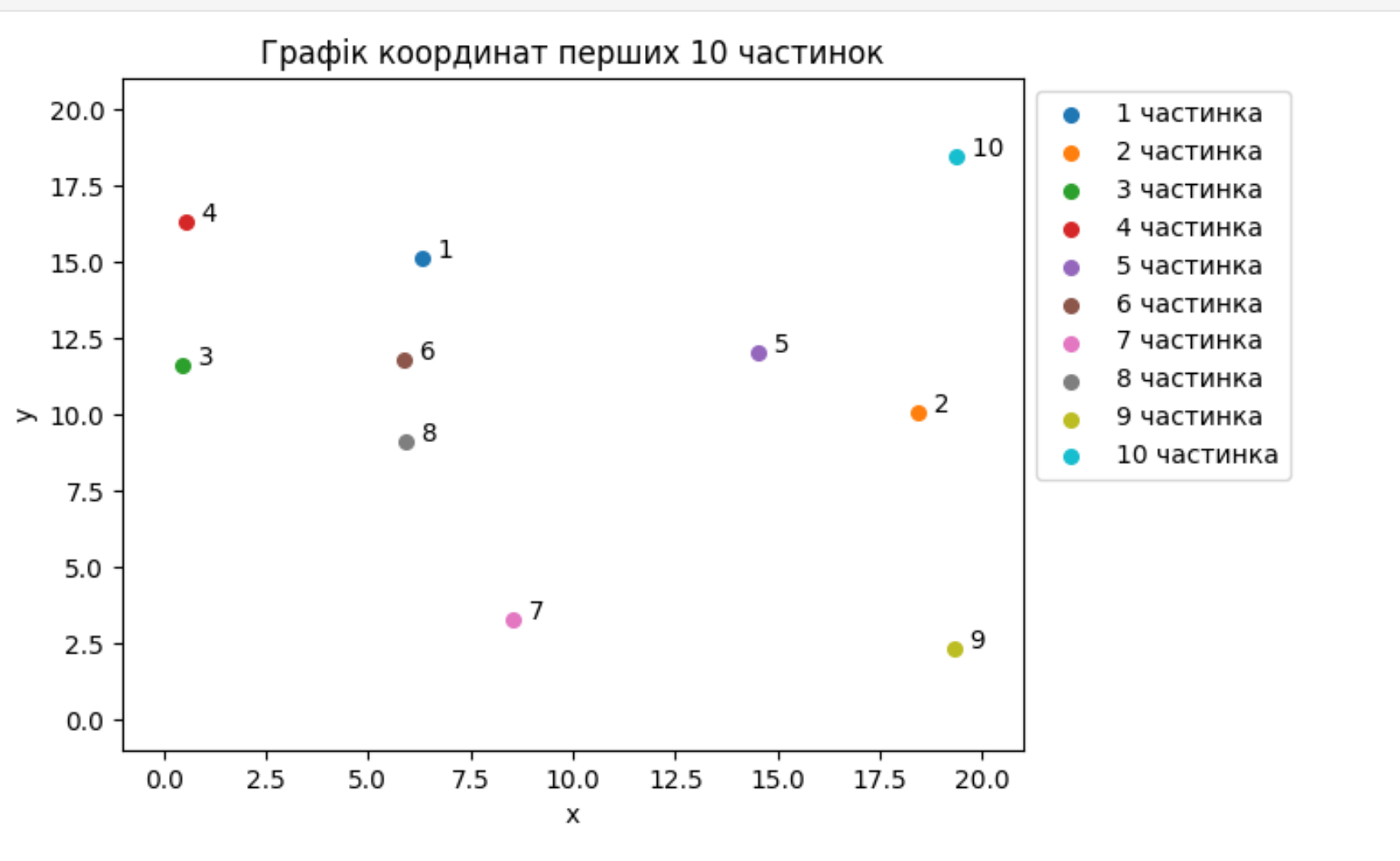
Out[9]:
```
[(6.331618744879961, 15.141642168164253),
 (18.424385534599423, 10.052028484642506),
 (0.45563272200524807, 11.603333950042725),
 (0.5433681514114141, 16.333590550348163),
 (14.517122879624367, 12.030019732192159)]
```

In [10]:
```python
list(zip(vx0[:5], vy0[:5]))
```

Out[10]:
```
[(-11.858040168881416, 16.175101436674595),
 (-10.006225500255823, -11.834053508937359),
 (-14.596377462148666, 13.971778433769941),
 (-2.8078273870050907, 6.42348563298583),
 (-6.485968232154846, 6.841713711619377)]
```

Будуємо графік координат декількох перших частинок (за замовчуванням їх буде 10 або всі).

In [11]:
```python
def fun_plot1(x, y, n=10):

    if (n < 2):
        n = 1
    if (n > len(x)):
        n = len(x)

    # 1 спосіб
    for i in range(n):
        plt.scatter(x[i], y[i], s=30, label=f'{i+1}' " частинка")
        plt.text(x[i], y[i], "  "f'{i+1}')

    # 2 спосіб
    #plt.scatter(x[:n], y[:n], s=20, c='m')

    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Графік координат перших " f'{n}' " частинок" )
    if (n < 20):
        plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    ax = [0, 20, 0, 20]
    ax2=[-1, 21, -1, 21]
    ax3=[-1, 32, -1, 21]
    plt.axis(ax2)

fun_plot1(x0, y0)
```



Розглянемо основні ідеї методу молекулярної динаміки. Припустимо, що між N(N − 1)/2 парами частинок діє двочастинковий потенціал.

Як придатний приклад такого потенціалу можна розглянути потенціал Леннарда-Джонса

$$U(r_{12}) = 4\varepsilon\{(\frac{\sigma}{r_{12}})^{12} - (\frac{\sigma}{r_{12}})^{6}\}$$

де r_{12} — відстань між частинками 1 і 2,

а ε ("відстань" взаємодії частинок 1 та 2) і σ (глибина потенціальної ями або "енергія") — сталі.

In [12]:
```python
# (двочастинковий) потенціал Леннарда-Джонса, що діє між N(N − 1)/2 парами частинками
def phi(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    U_r = 4*eps*(sr6*sr6 - sr6)
    return U_r
```

Такий потенціал добре описує притягання у випадку, коли частинки віддалені на значну відстань, і відштовхування, коли вони зближені. У такому разі еволюція системи у часі відбувається відповідно до детерміністичних законів руху кожної частинки, причому сила взаємодії F_{ij} спрощується і подається так:

$$\overrightarrow{F}(|\overrightarrow{x_i} - \overrightarrow{x_j}|) = -\frac{\partial}{\partial \overrightarrow{x_i}} U(|\overrightarrow{x_i} - \overrightarrow{x_j}|)$$

In [13]:
```python
# сила взаємодії між частинкамии
def F(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    F_r = 24*eps*sigma/r*sr6*(2*sr6 - 1)
    return F_r
```

Граничні умови (коли частинка виходить за межі резервуара)

In [14]:
```python
# жорсткі граничні умови
def Test11(X, Y, Vx, Vy, Lx=20, Ly=20):
    k=0
    while ((X < 0) | (X > Lx) | (Y < 0) | (Y > Ly)):
        if (X < 0):
            X = -X
            Vx = -Vx

        if (X > Lx):
            X = Lx-(X-Lx)
            Vx = -Vx

        if (Y < 0):
            Y = -Y
            Vy = -Vy

        if (Y > Ly):
            Y = Ly-(Y-Ly)
            Vy = -Vy
        k+=1

    return X, Y, Vx, Vy, k

#X_, Y_, Vx_, Vy_, k_ = Test11(X=-4, Y=250, Vx=9, Vy=8, Lx=20, Ly=20)
#print(X_, Y_, Vx_, Vy_) #4 10 -9 8
```

Використовуємо алгоритм Ейлера і розраховуємо координати (x, y) частинок з часом.

Алгоритм Ейлера

$$a_{xi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial x_i}U(x_i, y_i)$$

$$a_{yi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial y_i}U(x_i, y_i)$$

$$x_i(t + \Delta t) = x_i(t) + v_{xi}(t)\Delta t$$

$$y_i(t + \Delta t) = y_i(t) + v_{yi}(t)\Delta t$$

$$v_{xi}(t + \Delta t) = v_{xi}(t) + a_{xi}(t)\Delta t$$

$$v_{yi}(t + \Delta t) = v_{yi}(t) + a_{yi}(t)\Delta t$$

$$a = \frac{dv}{dt} = \frac{d^2x}{dt^2}$$

$$\frac{dx}{dt} = v$$

$$\frac{dv}{dt} = a$$

In [15]:
```python
def funct1(x0, y0, vx0, vy0, mas=1, Lx=20, Ly=20, test_ = "Test11"):

    N = len(x0) # к-сть частинок

    t = 0       # початковий час
    t_fin = 20  # кінцевий час
    dt = 0.0002 # крок по часу

    # тимчасовий масив координат та швидкостей
    x_n, y_n = np.zeros(N), np.zeros(N)
    vx_n, vy_n = np.zeros(N), np.zeros(N)

    size1 = t_fin/dt + 1
    size1 = int(size1)

    k = 0     # лічильник для масивів X_time, Y_time (координат кожної частинки для кожного проміжку часу)
    times = []
    X_time = np.zeros((size1, N))
    Y_time = np.zeros((size1, N))

    U, Temp, KinE = [], [], []
    u, T, kinE = 0, 0, 0

    while(t <= t_fin):
        t = round(t, 4)
        if (t % 5 == 0):
            print("time =", t)

        if (t == 0):
            x, y = x0, y0
            vx, vy = vx0, vy0

        for i in range(N):

            X, Y = x[i], y[i]
            Vx, Vy = vx[i], vy[i]

            X1 = X + Vx*dt
            Y1 = Y + Vy*dt

            fx, fy = 0, 0
            #u = 0
            for j in range(N):
                if (j!=i):
                    rx = X1 - x[j]
                    ry = Y1 - y[j]
                    r = math.sqrt(rx*rx + ry*ry)
                    if (r < math.sqrt(Lx*Lx + Ly*Ly)):
                        Fr = F(r)
                        Fr = Fr/mas
                        fx += (Fr * rx / r)
                        fy += (Fr * ry / r)
                        u += phi(r)

            Vx1 = Vx + fx*dt
            Vy1 = Vy + fy*dt

            if (test_ == "Test11"):
                X1, Y1, Vx1, Vy1, steps = Test11(X1, Y1, Vx1, Vy1)

            x_n[i], y_n[i] = X1, Y1
            vx_n[i], vy_n[i] = Vx1, Vy1

            #T += vx_n[i]*vx_n[i] + vy_n[i]*vy_n[i]

            kinE += 0.5*(mas*(Vx1*Vx1 + Vy1*Vy1))

        times.append(t)

        #T = T / N
        #Temp.append(T)

        kinE = kinE / N
        KinE.append(kinE)

        u = u / N
        U.append(u)

        X_time[k][:] = x_n
        Y_time[k][:] = y_n
        k += 1

        x, y = x_n, y_n
        vx, vy = vx_n, vy_n

        t += dt

    return times, X_time, Y_time, U, KinE
```

In [16]:
```python
start_time = time.time()
times1, X_time1, Y_time1, U1, K1 = funct1(x0, y0, vx0, vy0)
print("Ця частина коду виконувалася %s seconds." % round(time.time() - start_time))
```

```
time = 0
time = 5.0
time = 10.0
time = 15.0
time = 20.0
Ця частина коду виконувалася 35 seconds.
```

Перевіримо розмірність нових масивів.

In [17]:
```python
print("times1 \t", np.shape(times1))
print("X_time1\t", np.shape(X_time1))
print("Y_time1\t", np.shape(Y_time1))
print("U1 \t",   np.shape(U1))
print("K1 \t",   np.shape(K1))
```

```
times1   (100001,)
X_time1  (100001, 10)
Y_time1  (100001, 10)
U1       (100001,)
K1       (100001,)
```

Побудуємо графік.

plt.plot(times1, U1) plt.title("Залежність потенц. енергії від часу (усереднена по частинкам)") plt.xlabel("time") plt.ylabel("U")plt.plot(times1, K1) plt.title("Залежність кінет. енергії від часу (усереднена по частинкам)") plt.xlabel("time") plt.ylabel("K")

Порахуємо повну енергію $E = U + K$, де U - потенціальна енергія, а K - кінетична енергія.

$$U =< \sum\sum \varphi(|\overrightarrow{r_i}(t) - \overrightarrow{r_j}(t)|) >$$

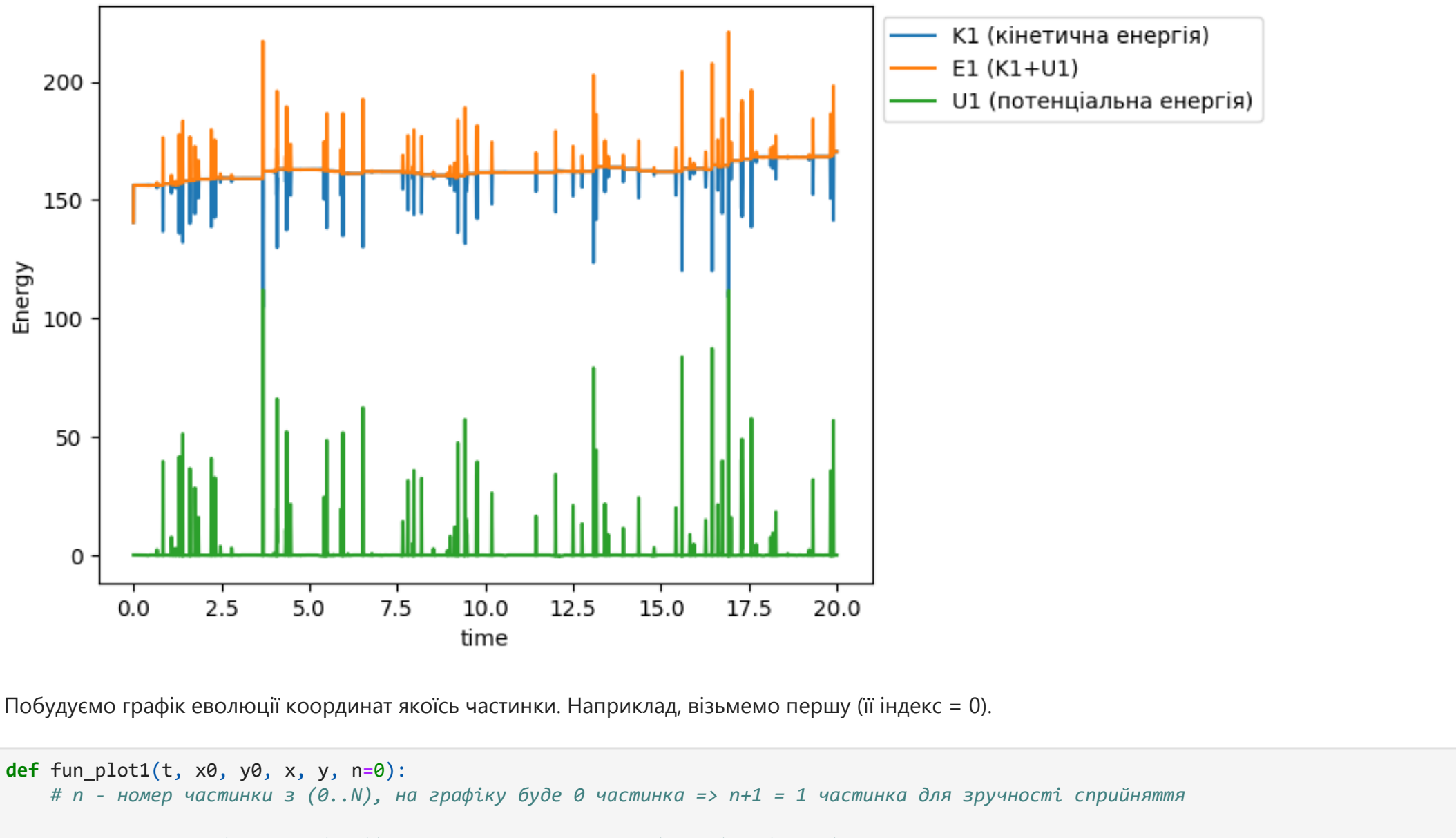
$$K =< \frac{1}{2}\sum m_i v_i^2(t) >$$

In [18]:
```python
#E1 = U1 + K1
E1 = []
if (len(U1) == len(K1)):
    for i in range(len(U1)):
        E1.append(U1[i] + K1[i])
```

Побудуємо графік залежності енергій від часу.

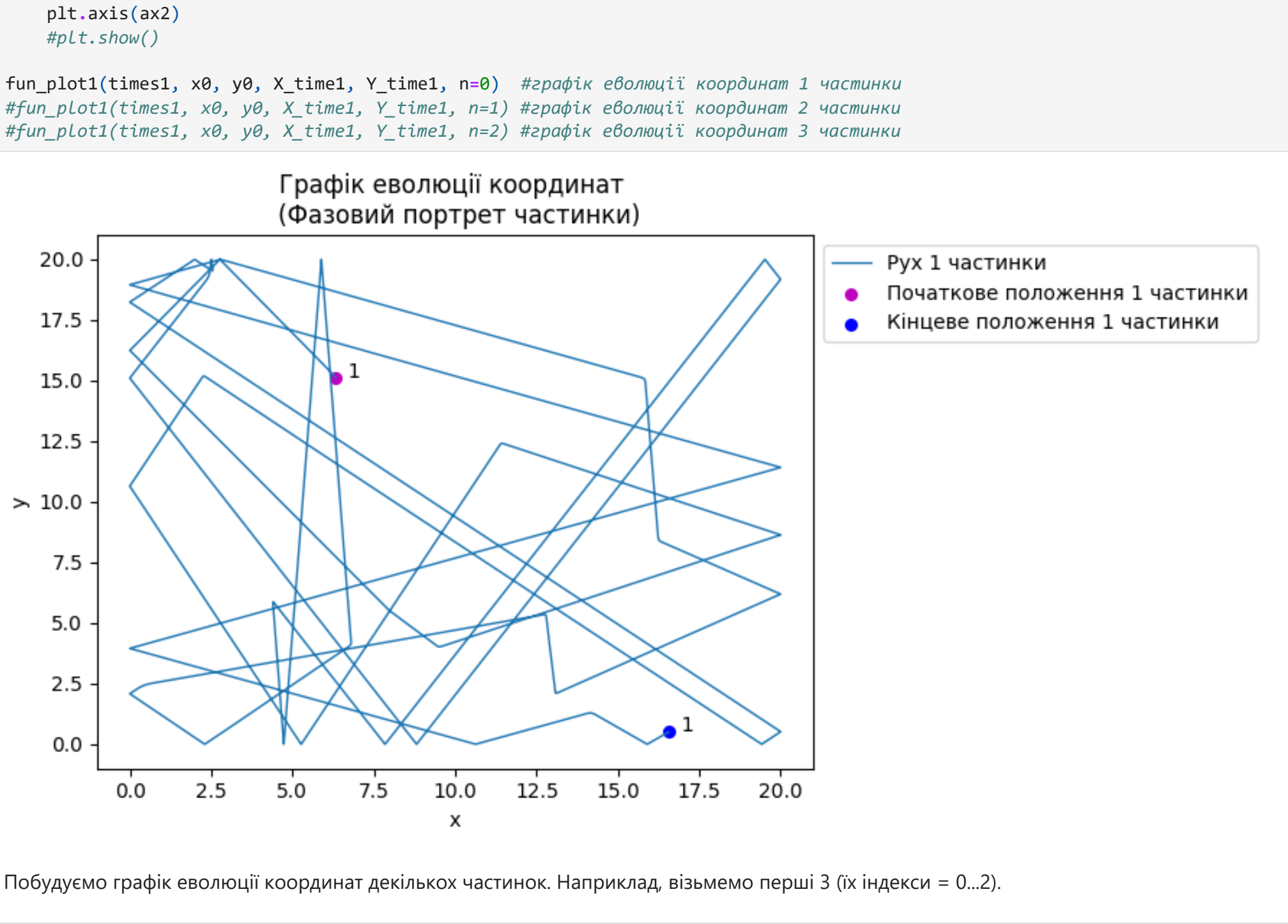
In [19]:
```python
tt = times1

plt.plot(tt, K1, label = "K1 (кінетична енергія)")
plt.plot(tt, E1, label = "E1 (K1+U1)")
plt.plot(tt, U1, label = "U1 (потенціальна енергія)")

plt.xlabel("time")
plt.ylabel("Energy")
plt.title("Графік енергій")
plt.legend(bbox_to_anchor=(1,1), loc="upper left")
```

Out[19]:
```
<matplotlib.legend.Legend at 0x22c7d37c790>
```



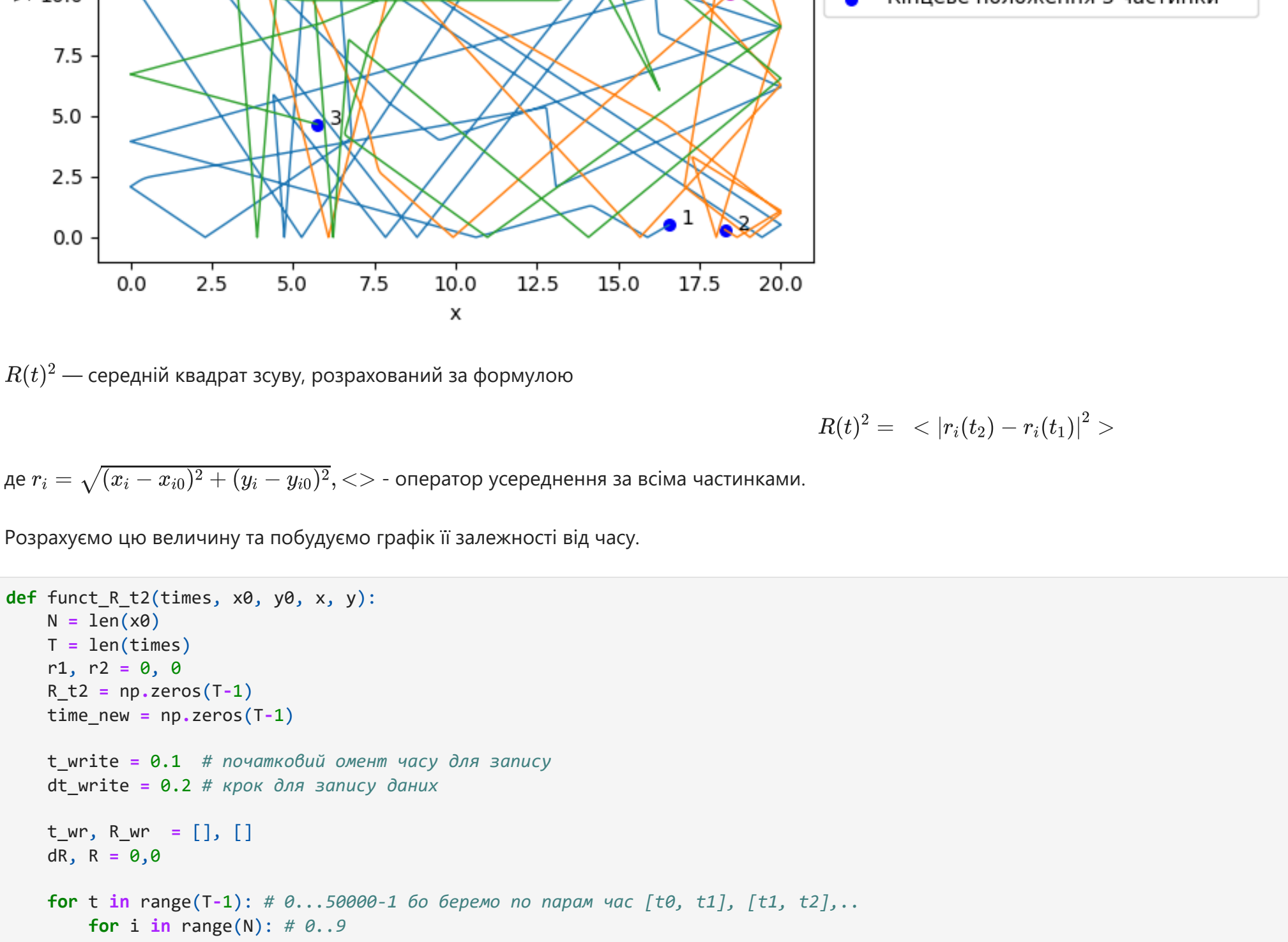
Побудуємо графік еволюції координат якоїсь частинки. Наприклад, візьмемо першу (її індекс = 0).

In [20]:
```python
def fun_plot1(t, x0, y0, x, y, n=0):
    # n - номер частинки з (0..N), на графіку буде 0 частинка => n+1 = 1 частинка для зручності сприйняття

    #p = plt.figure(figsize=(7,6))  #color=[1, 1, 0] #RGB (0,0,0)...(1.1.1)

    # створимо нові масиви координатзаписавши спочатку початкові координати частинки,
    # а потім ті які були розраховані в процесі моделювання
    xx1, yy1 = [], []
    xx1.append (x0[n])
    yy1.append (y0[n])
    for i in range(len(x)):
        xx1.append (x[i][n])
        yy1.append (y[i][n])

    lx = len(xx1)-1               # остан.індекс для нових масивів
    cond1 = (xx1[0] == x0[n])
    cond2 = (yy1[0] == y0[n])

    lt = len(t)-1                 # остан.індекс для масиву часу
    cond3 = (xx1[lx] == x[lt][n])
    cond4 = (yy1[lx] == y[lt][n])

    # перевірка відповідності елементів нових та старих масивів та побудова елементів графіку еволюціїі частинки
    if (cond1 & cond2 & cond3 & cond4):
        plt.plot(xx1, yy1, linewidth = 1, label="Рух "f'{n+1}' " частинки")
        plt.scatter(xx1[0],  yy1[0],  s=30, color='m', label="Початкове положення "f'{n+1}'" частинки")
        plt.scatter(xx1[lx], yy1[lx], s=30, color='b', label="Кінцеве положення "f'{n+1}'" частинки")
        plt.text(xx1[0],  yy1[0],  "  "f'{n+1}')
        plt.text(xx1[lx], yy1[lx], "  "f'{n+1}')

    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Графік еволюції координат \n (Фазовий портрет частинки)")
    plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    #ax1 = [0, 20, 0, 20]
    ax2 = [-1, 21, -1, 21]
    plt.axis(ax2)
    #plt.show()

fun_plot1(times1, x0, y0, X_time1, Y_time1, n=0)  #графік еволюції координат 1 частинки
#fun_plot1(times1, x0, y0, X_time1, Y_time1, n=1) #графік еволюції координат 2 частинки
#fun_plot1(times1, x0, y0, X_time1, Y_time1, n=2) #графік еволюції координат 3 частинки
```



Побудуємо графік еволюції координат декількох частинок. Наприклад, візьмемо перші 3 (їх індекси = 0...2).

In [21]:
```python
# count - к-сть частинок (10)
for i in range(count-7):
    fun_plot1(times1, x0, y0, X_time1, Y_time1, n=i)
```



$R(t)^2$ — середній квадрат зсуву, розрахований за формулою

$$R(t)^2 = \ < |r_i(t_2) - r_i(t_1)|^2 >$$

де $r_i = \sqrt{(x_i - x_{i0})^2 + (y_i - y_{i0})^2}, <>$ - оператор усереднення за всіма частинками.

Розрахуємо цю величину та побудуємо графік її залежності від часу.

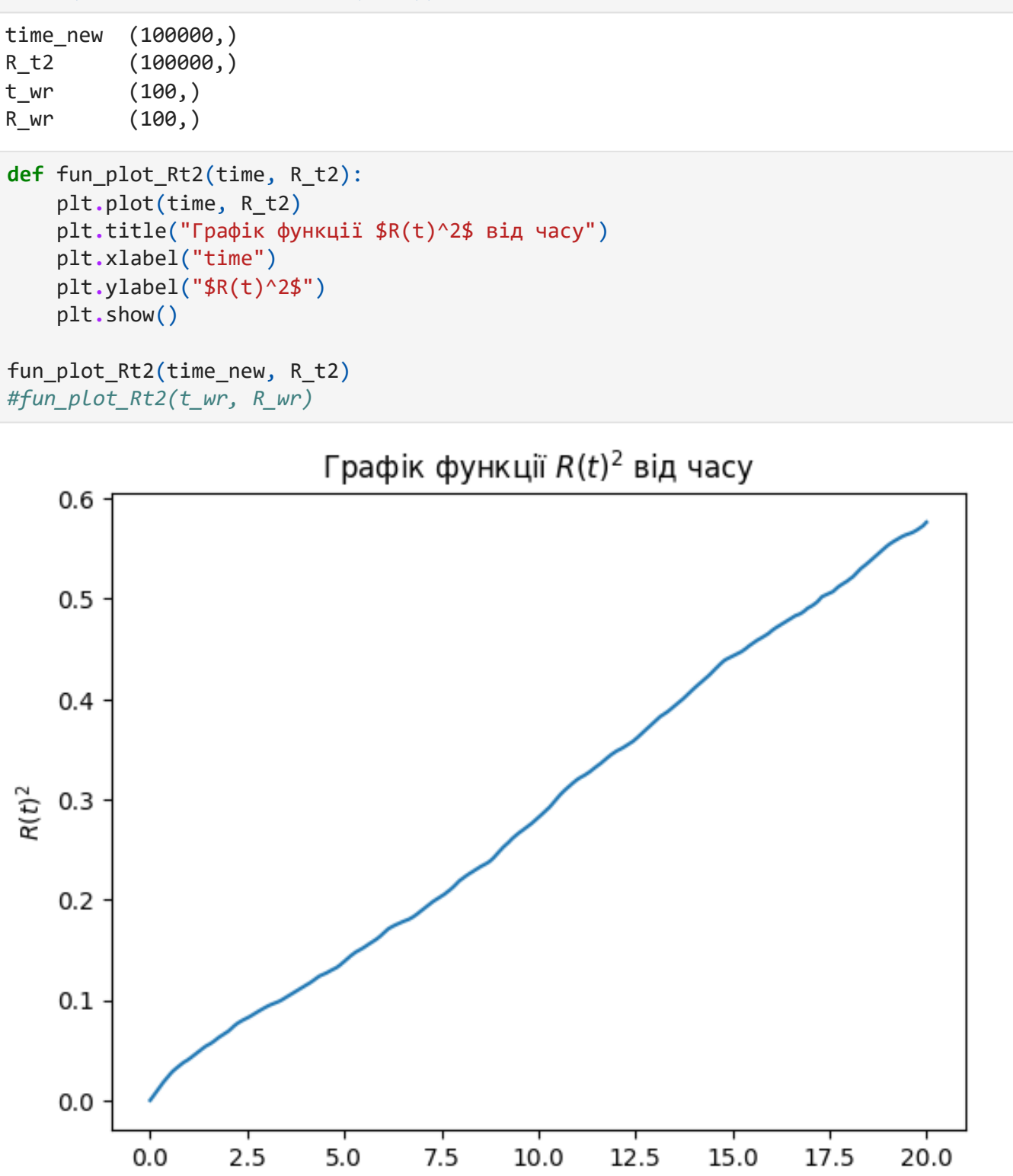
In [22]:
```python
def funct_R_t2(times, x0, y0, x, y):
    N = len(x0)
    T = len(times)
    r1, r2 = 0, 0
    R_t2 = np.zeros(T-1)
    time_new = np.zeros(T-1)

    t_write = 0.1  # початковий омент часу для запису
    dt_write = 0.2 # крок для запису даних

    t_wr, R_wr  = [], []
    dR, R = 0,0

    for t in range(T-1): # 0...50000-1 бо беремо по парам час [t0, t1], [t1, t2],..
        for i in range(N): # 0..9

            x1 = x[t][i]
            dx = x1 - x0[i]
            y1 = y[t][i]
            dy = y1 - y0[i]

            r1 = math.sqrt(dx*dx + dy*dy)

            x2 = x[t+1][i]
            dx2 = x2 - x0[i]
            y2 = y[t+1][i]
            dy2 = y2 - y0[i]

            r2 = math.sqrt(dx2*dx2 + dy2*dy2)

            dR = abs(r2 - r1)
            dR = dR**2
            R += dR

        R_t2[t] = R / N

        if(times[t] >= t_write):
            t_wr.append(times[t])
            R_wr.append(R_t2[t])
            t_write += dt_write

        time_new[t] = times[t]

    return time_new, R_t2, t_wr, R_wr
```

In [23]:
```python
time_new, R_t2, t_wr, R_wr = funct_R_t2(times1, x0, y0, X_time1, Y_time1)
```

In [24]:
```python
print("time_new ",   np.shape(time_new))
print("R_t2     ",   np.shape(R_t2))
print("t_wr     ",  np.shape(t_wr))
print("R_wr     ",  np.shape(R_wr))
```

```
time_new  (100000,)
R_t2      (100000,)
t_wr      (100,)
R_wr      (100,)
```

In [25]:
```python
def fun_plot_Rt2(time, R_t2):
    plt.plot(time, R_t2)
    plt.title("Графік функції $R(t)^2$ від часу")
    plt.xlabel("time")
    plt.ylabel("$R(t)^2$")
    plt.show()

fun_plot_Rt2(time_new, R_t2)
#fun_plot_Rt2(t_wr, R_wr)
```



---

help ('matplotlib.markers') help(plt.scatter)# marker="o", edgecolor="r", c="none", s=250 - пусті кружечки # marker="P", edgecolor="r", c="none", s=250 - пусті крестики # marker="o", edgecolor="r", c="none", s=250 - пусті зірочки

Комп'ютерне моделювання задач прикладної математики

Основи класичної молекулярної динаміки.

Лабораторна робота 2

Студентка Пороскун Олена. Група ПМ.м-21

Task 2

In [1]:
```
from matplotlib import pyplot as plt
```

In [2]:
```
import random
import numpy as np
import math
import time
```

In [3]:
```
count = 100 # кількість частинок
```

Розраховуємо випадкову величину.

In [4]:
```
def randV():
    #random.seed(s)
    max_num = 2147483647
    ksi = random.randint(0, max_num) / (max_num+1) # ksi = rand_r(&seedp)/(RAND_MAX+1.0);
    return ksi
#randV()
```

Розраховуємо початкові значення координат (x_0, y_0) та швидкостей частинок (vx_0, vy_0), що знаходяться у резервуарі розміром (Lx, Ly).

In [5]:
```
def Inital_XY(N=10, Lx = 20, Ly = 20, vmax = 20):
    x0,  y0   = np.zeros(N), np.zeros(N)
    vx0, vy0  = np.zeros(N), np.zeros(N)
    for i in range(N):
        x0[i] = randV()*Lx;
        y0[i] = randV()*Ly;
        vx0[i] = vmax*(2*randV() - 1);
        vy0[i] = vmax*(2*randV() - 1);
    return x0, y0, vx0, vy0
#x0_, y0_, vx0_, vy0_ = Inital_XY()
```

In [6]:
```
x0, y0, vx0, vy0 = Inital_XY(N=count)
```

Перевіряємо межі початкових значень,

In [7]:
```
print(min(x0),min(y0))
print(max(vx0), max(vy0))
```

```
0.26554856449365616 0.19104905426502228
19.873318672180176 19.519715066999197
```

Перевіряємо розмірність масивів.

In [8]:
```
print(np.shape(x0),np.shape(y0), np.shape(vx0), np.shape(vy0))
```

```
(100,) (100,) (100,) (100,)
```

Виводимо перші 5 значень з кожного масиву.

In [9]:
```
list(zip(x0[:5], y0[:5]))
```

Out[9]:
```
[(15.773125002160668, 8.313399041071534),
 (2.796621648594737, 10.571214957162738),
 (15.456303292885423, 14.710078490898013),
 (15.322227394208312, 1.2708038091659546),
 (17.543530883267522, 5.942685790359974)]
```

In [10]:
```
list(zip(vx0[:5], vy0[:5]))
```

Out[10]:
```
[(12.409982606768608, 11.300946287810802),
 (14.631292000412941, -14.609208796173334),
 (17.222262527793646, -8.845808207988739),
 (1.9811238907277584, -2.3282594233751297),
 (16.025061029940844, -15.055219419300556)]
```

Будуємо графік координат декількох перших частинок (за замовчуванням їх буде 10 або всі).

In [11]:
```
def fun_plot1(x, y, n=10):

    if (n < 2):
        n = 1
    if (n > len(x)):
        n = len(x)

    # 1 спосіб
    for i in range(n):
        plt.scatter(x[i], y[i], s=30, label=f'{i+1}' " частинка")
        plt.text(x[i], y[i], "  "f'{i+1}')

    # 2 спосіб
    #plt.scatter(x[:n], y[:n], s=20, c='m')

    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Графік координат перших " f'{n}' " частинок" )
    if (n < 20):
        plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    ax = [0, 20, 0, 20]
    ax2=[-1, 21, -1, 21]
    ax3=[-1, 32, -1, 21]
    plt.axis(ax2)

#fun_plot1(x0, y0)
```

Розглянемо основні ідеї методу молекулярної динаміки. Припустимо, що між N(N − 1)/2 парами частинок діє двочастинковий потенціал.

Як придатний приклад такого потенціалу можна розглянути потенціал Леннарда-Джонса

$$U(r_{12}) = 4\varepsilon\{(\frac{\sigma}{r_{12}})^{12} - (\frac{\sigma}{r_{12}})^{6}\}$$

де r_{12} — відстань між частинками 1 і 2,

а ε ("відстань" взаємодії частинок 1 та 2) і σ (глибина потенціальної ями або "енергія") — сталі.

In [12]:
```
# (двочастинковий) потенціал Леннарда-Джонса, що діє між N(N − 1)/2 парами частинками
def phi(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    U_r = 4*eps*(sr6*sr6 - sr6)
    return U_r
```

Такий потенціал добре описує притягання у випадку, коли частинки віддалені на значну відстань, і відштовхування, коли вони зближені. У такому разі еволюція системи у часі відбувається відповідно до детерміністичних законів руху кожної частинки, причому сила взаємодії F_{ij} спрощується і подається так:

$$\overrightarrow{F}(|\overrightarrow{x_i} - \overrightarrow{x_j}|) = -\frac{\partial}{\partial \overrightarrow{x_i}} U(|\overrightarrow{x_i} - \overrightarrow{x_j}|)$$

In [13]:
```
# сила взаємодії між частинкамии
def F(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    F_r = 24*eps*sigma/r*sr6*(2*sr6 - 1)
    return F_r
```

Граничні умови (коли частинка виходить за межі резервуара)

In [14]:
```
# жорсткі граничні умови
def Test11(X, Y, Vx, Vy, Lx=20, Ly=20):
    k=0
    while ((X < 0) | (X > Lx) | (Y < 0) | (Y > Ly)):
        if (X < 0):
            X = -X
            Vx = -Vx

        if (X > Lx):
            X = Lx-(X-Lx)
            Vx = -Vx

        if (Y < 0):
            Y = -Y
            Vy = -Vy

        if (Y > Ly):
            Y = Ly-(Y-Ly)
            Vy = -Vy
        k+=1

    return X, Y, Vx, Vy, k

#X_, Y_, Vx_, Vy_, k_ = Test11(X=-4, Y=250, Vx=9, Vy=8, Lx=20, Ly=20)
#print(X_, Y_, Vx_, Vy_) #4 10 -9 8
```

Використовуємо алгоритм Ейлера щоб розрахоувати координати (x, y) частинок з часом та з цими даними розрахувати значення середнього часу $mpt(R)$ досягнення значення R(відстані, що пройшла частинка).

Алгоритм Ейлера

$$a_{xi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial x_i}U(x_i, y_i)$$

$$a_{yi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial y_i}U(x_i, y_i)$$

$$x_i(t + \Delta t) = x_i(t) + v_{xi}(t)\Delta t$$

$$y_i(t + \Delta t) = y_i(t) + v_{yi}(t)\Delta t$$

$$v_{xi}(t + \Delta t) = v_{xi}(t) + a_{xi}(t)\Delta t$$

$$v_{yi}(t + \Delta t) = v_{yi}(t) + a_{yi}(t)\Delta t$$

$$a = \frac{dv}{dt} = \frac{d^2x}{dt^2}$$

$$\frac{dx}{dt} = v$$

$$\frac{dv}{dt} = a$$

In [15]:
```
def funct2(x0, y0, vx0, vy0, R=1, mas=1, Lx=20, Ly=20, test_ = "Test11"):

    N = len(x0) # к-сть частинок

    t = 0       # початковий час
    dt = 0.0002 # крок по часу

    # тимчасовий
    # масив координат та швидкостей
    x_n, y_n = np.zeros(N), np.zeros(N)
    vx_n, vy_n = np.zeros(N), np.zeros(N)

    times_R = np.zeros(N) # час для кожної частинки коли вона досягла R
    R_i = np.zeros(N)     # саме значення R_i(>=R), яке досягла кордината

    # mean pasage time (середній час досягнення значення R)
    mpt = 0

    for i in range(N):
        t = 0
        x, y = x0, y0
        vx, vy = vx0, vy0

        while (R_i[i] < R):

            t = round(t,5)

            X, Y = x[i], y[i]
            Vx, Vy = vx[i], vy[i]

            X1 = X + Vx*dt
            Y1 = Y + Vy*dt

            fx, fy = 0, 0
            for j in range(N):
                if (j!=i):
                    rx = X1 - x[j]
                    ry = Y1 - y[j]
                    r = math.sqrt(rx*rx + ry*ry)
                    if (r < math.sqrt(Lx*Lx + Ly*Ly)):
                        Fr = F(r)
                        Fr = Fr/mas
                        fx += (Fr * rx / r)
                        fy += (Fr * ry / r)

            Vx1 = Vx + fx*dt
            Vy1 = Vy + fy*dt

            if (test_ == "Test11"):
                X1, Y1, Vx1, Vy1, steps = Test11(X1, Y1, Vx1, Vy1)

            x_n[i], y_n[i] = X1, Y1
            vx_n[i], vy_n[i] = Vx1, Vy1

            dx = X1 - x0[i]
            dy = Y1 - y0[i]
            R_i[i] += math.sqrt(dx*dx + dy*dy)

            if (R_i[i] >= R):
                times_R[i] = t

            x, y = x_n, y_n
            vx, vy = vx_n, vy_n

            t += dt

    mpt = np.mean(times_R)
    mpt = round(mpt,5)

    #print("R","\t mpt")
    print(R, "\t", mpt)

    return times_R, R_i, mpt
```

In [16]:
```
#start_time = time.time()
R_ = [1, 2, 5, 10]
times = []   #
Ri    = []   #
mpt   = []   # mean pasage time (середній час досягнення значення R)
print("R","\t mpt")
for i in range(len(R_)):
    times_, Ri_, mpt_ = funct2(x0, y0, vx0, vy0, R = R_[i])
    times.append(times_)
    Ri.append(Ri_)
    mpt.append(mpt_)
#print("Ця частина коду виконувалася %s seconds." % round(time.time() - start_time, 2))
```

```
R        mpt
1        0.00463
2        0.00659
5        0.01034
10       0.01457
```

Перевіримо розмірність нових масивів.

In [17]:
```
print(np.shape(times))
print(np.shape(Ri))
print(np.shape(mpt))
```

```
(4, 100)
(4, 100)
(4,)
```

Функція, що априксимує дані, повертає:

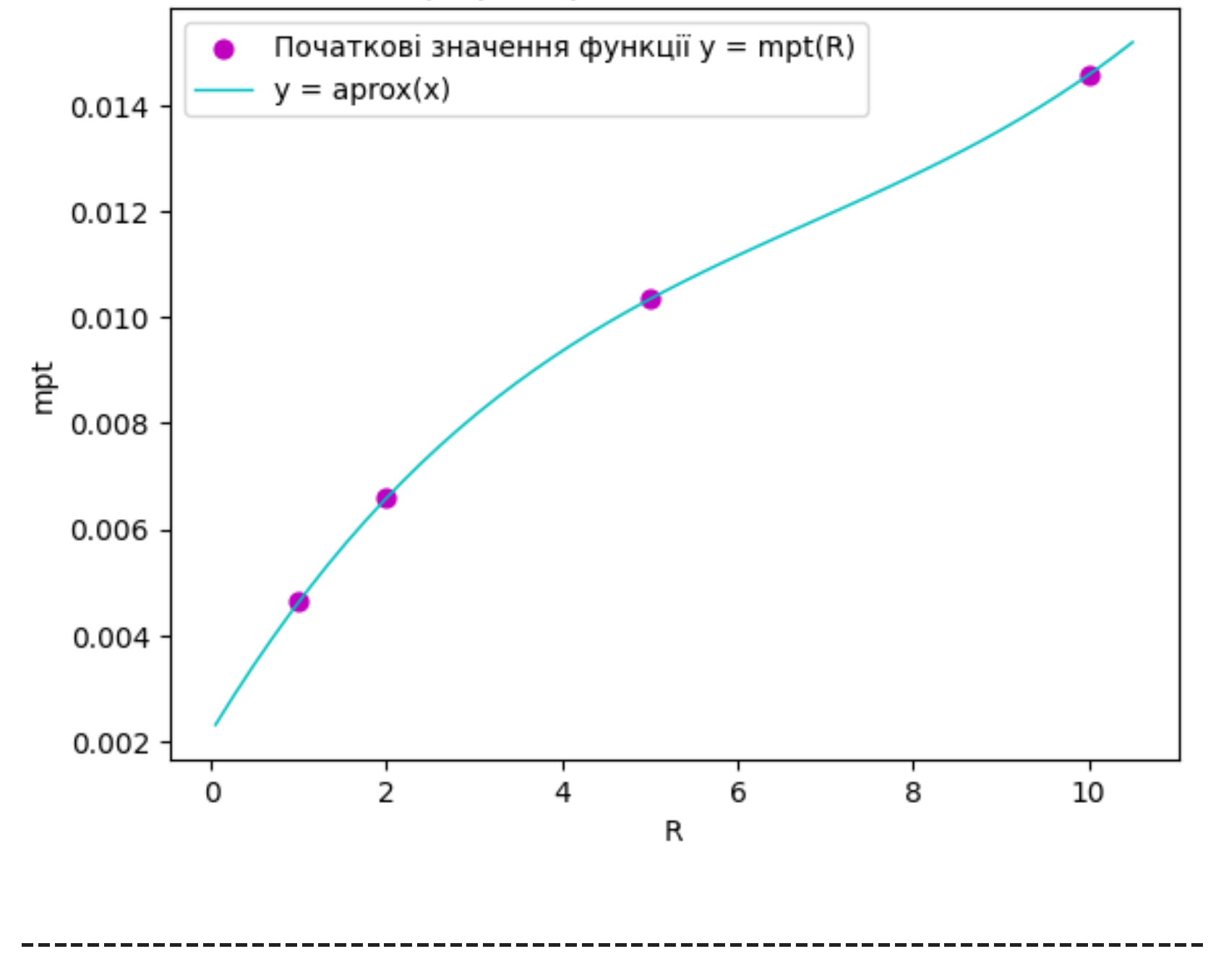
- coef - коефіцієнти рівняння, яке відповідає початковим даним,
- polinom - рівняння з коефіцієнтами,
- mpt_to_compare - значення функції для порівняння зі початковими.

In [18]:
```
def fun_aprox(X, mpt):
    k=1
    mpt_new = mpt*0
    eps = 1000
    bool_mpt = 0

    while ((eps >= 0.0001) & (k<10)):
        coef = np.polyfit(X, mpt,k)
        polinom = np.poly1d(coef)
        mpt_to_compare = polinom(X)

        mpt_old = np.round(mpt,5)
        mpt_to_compare = np.round(mpt_to_compare,5)

        eps = abs(mpt_old[0] - mpt_to_compare[0])
        bool_mpt = np.sum(mpt_old == mpt_to_compare)

        #print("\n k =",k)
        #print("coef:", coef)
        #print("y = aprox(x):\n",polinom)
        #print("Перевірка:\n", " X =", X, "\n mpt = ", mpt_old,"\n mpt_to_compare =", mpt_to_compare)
        #print("bool_mpt =",bool_mpt)
        #print("eps =",eps)

        k += 1

    print("coef:", coef)
    print("y = aprox(x) :\n",polinom)
    print("Перевірка:\n", " X =", X, "\n", mpt_old, "= mpt\n",mpt_to_compare,"= mpt_to_compare")

    return coef, polinom, mpt_to_compare

#c0, p0, mpt_to_compare0 = fun_aprox(R_, mpt)
```

Побудуємо тепер початкові дані та знайдену криву.

In [19]:
```
def fun_plot2(mpt, R = [1, 2, 5, 10]):

    plt.scatter(R, mpt, label="Початкові значення функції y = mpt(R)", s=40, c='m')

    # порівняння з апроксимованою кривою деякого степеня
    R_new = np.linspace(0.05, 10.5, 50)
    coef, polinom, mpt_to_compare = fun_aprox(R, mpt)
    mpt_new = polinom(R_new)
    plt.plot(R_new, mpt_new, label = "y = aprox(x)", linewidth = 1, c='c')

    plt.xlabel("R")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від R")
    plt.legend()

fun_plot2(mpt, R_)
```

```
coef: [ 1.41111111e-05 -2.90388889e-04  2.73238889e-03  2.17388889e-03]
y = aprox(x) :
            3             2
1.411e-05 x - 0.0002904 x + 0.002732 x + 0.002174
Перевірка:
  X = [1, 2, 5, 10]
 [0.00463 0.00659 0.01034 0.01457] = mpt
 [0.00463 0.00659 0.01034 0.01457] = mpt_to_compare
```



Комп'ютерне моделювання задач прикладної математики

Основи класичної молекулярної динаміки.

Лабораторна робота 2

Студентка Пороскун Олена. Група ПМ.м-21

Task 3

In [1]:
```python
from matplotlib import pyplot as plt
```

In [2]:
```python
import random
import numpy as np
import math
import time
```

In [3]:
```python
count = 50 # кількість частинок
```

Розраховуємо випадкову величину.

In [4]:
```python
def randV():
    #random.seed(s)
    max_num = 2147483647
    ksi = random.randint(0, max_num) / (max_num+1) # ksi = rand_r(&seedp)/(RAND_MAX+1.0);
    return ksi
#randV()
```

Розраховуємо початкові значення координат (x_0, y_0) та швидкостей частинок (vx_0, vy_0), що знаходяться у резервуарі розміром (Lx, Ly).

In [5]:
```python
def Inital_XY(N=10, Lx = 20, Ly = 20, vmax = 20):
    x0,  y0   = np.zeros(N), np.zeros(N)
    vx0, vy0  = np.zeros(N), np.zeros(N)
    for i in range(N):
        x0[i] = randV()*Lx;
        y0[i] = randV()*Ly;
        vx0[i] = vmax*(2*randV() - 1);
        vy0[i] = vmax*(2*randV() - 1);
    return x0, y0, vx0, vy0
#x0_, y0_, vx0_, vy0_ = Inital_XY()
```

In [6]:
```python
x0, y0, vx0, vy0 = Inital_XY(N=count)
```

Перевіряємо межі початкових значень,

In [7]:
```python
print(min(x0),min(y0))
print(max(vx0), max(vy0))
```

```
0.8172706048935652 0.47896682284772396
19.723131861537695 19.90949798375368
```

Перевіряємо розмірність масивів.

In [8]:
```python
print(np.shape(x0),np.shape(y0), np.shape(vx0), np.shape(vy0))
```

```
(50,) (50,) (50,) (50,)
```

Виводимо перші 5 значень з кожного масиву.

In [9]:
```python
list(zip(x0[:5], y0[:5]))
```

Out[9]:
```
[(5.344565445557237, 11.358099896460772),
 (1.207144232466817, 2.4572083726525307),
 (19.869896341115236, 14.526140419766307),
 (17.00371972285211, 4.76365159265697),
 (15.880661383271217, 8.696228200569749)]
```

In [10]:
```python
list(zip(vx0[:5], vy0[:5]))
```

Out[10]:
```
[(8.21661189198494, -16.45287286490202),
 (-14.350154343992472, 5.645126365125179),
 (-12.086515128612518, -11.710753459483385),
 (-0.6559620052576065, -1.2511676736176014),
 (-1.293065994977951, -3.0179922468960285)]
```

Будуємо графік координат декількох перших частинок (за замовчуванням їх буде 10 або всі).

In [11]:
```python
def fun_plot1(x, y, n=10):

    if (n < 2):
        n = 1
    if (n > len(x)):
        n = len(x)

    # 1 спосіб
    for i in range(n):
        plt.scatter(x[i], y[i], s=30, label=f'{i+1}' " частинка")
        plt.text(x[i], y[i], "  "f'{i+1}')

    # 2 спосіб
    #plt.scatter(x[:n], y[:n], s=20, c='m')

    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Графік координат перших " f'{n}' " частинок" )
    if (n < 20):
        plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    ax = [0, 20, 0, 20]
    ax2=[-1, 21, -1, 21]
    ax3=[-1, 32, -1, 21]
    plt.axis(ax2)

#fun_plot1(x0, y0)
```

Розглянемо основні ідеї методу молекулярної динаміки. Припустимо, що між N(N − 1)/2 парами частинок діє двочастинковий потенціал.

Як придатний приклад такого потенціалу можна розглянути потенціал Леннарда-Джонса

$$U(r_{12}) = 4\varepsilon\{(\frac{\sigma}{r_{12}})^{12} - (\frac{\sigma}{r_{12}})^{6}\}$$

де r_{12} — відстань між частинками 1 і 2,

а ε ("відстань" взаємодії частинок 1 та 2) і σ (глибина потенціальної ями або "енергія") — сталі.

In [12]:
```python
# (двочастинковий) потенціал Леннарда-Джонса, що діє між N(N − 1)/2 парами частинками
def phi(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    U_r = 4*eps*(sr6*sr6 - sr6)
    return U_r
```

Такий потенціал добре описує притягання у випадку, коли частинки віддалені на значну відстань, і відштовхування, коли вони зближені. У такому разі еволюція системи у часі відбувається відповідно до детерміністичних законів руху кожної частинки, причому сила взаємодії F_{ij} спрощується і подається так:

$$\overrightarrow{F}(|\overrightarrow{x_i} - \overrightarrow{x_j}|) = -\frac{\partial}{\partial\overrightarrow{x_i}}U(|\overrightarrow{x_i} - \overrightarrow{x_j}|)$$

In [13]:
```python
# сила взаємодії між частинкамии
def F(r, sigma=1, eps=1):
    sr6 = (sigma/r)**6
    F_r = 24*eps*sigma/r*sr6*(2*sr6 - 1)
    return F_r
```

Граничні умови (коли частинка виходить за межі резервуара)

In [14]:
```python
# жорсткі граничні умови
def Test11(X, Y, Vx, Vy, Lx=20, Ly=20):
    k=0
    while ((X < 0) | (X > Lx) | (Y < 0) | (Y > Ly)):
        if (X < 0):
            X = -X
            Vx = -Vx

        if (X > Lx):
            X = Lx-(X-Lx)
            Vx = -Vx

        if (Y < 0):
            Y = -Y
            Vy = -Vy

        if (Y > Ly):
            Y = Ly-(Y-Ly)
            Vy = -Vy
        k+=1

    return X, Y, Vx, Vy, k

#X_, Y_, Vx_, Vy_, k_ = Test11(X=-4, Y=250, Vx=9, Vy=8, Lx=20, Ly=20)
#print(X_, Y_, Vx_, Vy_) #4 10 -9 8
```

Використовуємо алгоритм Ейлера щоб розрахоувати координати (x, y) частинок з часом та з цими даними розрахувати значення середнього часу $mpt(R)$ досягнення значення R(відстані, що пройшла частинка).

$$R_i = \sqrt{(x_i - x_{i0})^2 + (y_i - y_{i0})^2}$$

Алгоритм Ейлера

$$a_{xi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial x_i}U(x_i, y_i)$$

$$a_{yi} = \frac{F(x_i, y_i)}{m_i} = -\frac{1}{m_i}\frac{\partial}{\partial y_i}U(x_i, y_i)$$

$$x_i(t + \Delta t) = x_i(t) + v_{xi}(t)\Delta t$$

$$y_i(t + \Delta t) = y_i(t) + v_{yi}(t)\Delta t$$

$$v_{xi}(t + \Delta t) = v_{xi}(t) + a_{xi}(t)\Delta t$$

$$v_{yi}(t + \Delta t) = v_{yi}(t) + a_{yi}(t)\Delta t$$

$$a = \frac{dv}{dt} = \frac{d^2x}{dt^2}$$

$$\frac{dx}{dt} = v$$

$$\frac{dv}{dt} = a$$

In [15]:
```python
def funct3(x0, y0, vx0, vy0, R=3, mas=1, sigm=1, epsilon=1, Lx=20, Ly=20, test_ = "Test11"):

    N = len(x0) # к-сть частинок

    t = 0       # початковий час
    dt = 0.0002 # крок по часу

    # тимчасовий масив координат та швидкостей
    x_n, y_n = np.zeros(N), np.zeros(N)
    vx_n, vy_n = np.zeros(N), np.zeros(N)

    times_R = np.zeros(N) # час для кожної частинки коли вона досягла R
    R_i = np.zeros(N)     # саме значення R_i(>=R), яке досягла кордината

    # mean pasage time (середній час досягнення значення R)
    mpt = 0

    for i in range(N):
        t = 0
        x, y = x0, y0
        vx, vy = vx0, vy0

        while (R_i[i] < R):

            t = round(t,5)

            X, Y = x[i], y[i]
            Vx, Vy = vx[i], vy[i]

            X1 = X + Vx*dt
            Y1 = Y + Vy*dt

            fx, fy = 0, 0
            for j in range(N):
                if (j!=i):
                    rx = X1 - x[j]
                    ry = Y1 - y[j]
                    r = math.sqrt(rx*rx + ry*ry)
                    if (r < math.sqrt(Lx*Lx + Ly*Ly)):
                        Fr = F(r, sigma = sigm, eps = epsilon)
                        Fr = Fr/mas
                        fx += (Fr * rx / r)
                        fy += (Fr * ry / r)

            Vx1 = Vx + fx*dt
            Vy1 = Vy + fy*dt

            if (test_ == "Test11"):
                X1, Y1, Vx1, Vy1, steps = Test11(X1, Y1, Vx1, Vy1)

            x_n[i], y_n[i] = X1, Y1
            vx_n[i], vy_n[i] = Vx1, Vy1

            dx = X1 - x0[i]
            dy = Y1 - y0[i]
            R_i[i] += math.sqrt(dx*dx + dy*dy)

            if (R_i[i] >= R):
                times_R[i] = t

            x, y = x_n, y_n
            vx, vy = vx_n, vy_n

            t += dt

    mpt = np.mean(times_R)
    mpt = round(mpt,5)

    #print("R","\teps"," \tsigma", "\tmpt ")
    print(R,"\t",epsilon,"\t",sigm,"\t", mpt)

    return times_R, R_i, mpt
```

Знайдемо середній час досягнення частинками $R = 3$.

- Розрахуємо дані для $R = 3,\ \sigma\ (sigma1) = [1, 3, 5, 7],\ \varepsilon\ (epsilon) = 1.$

In [16]:
```python
start_time = time.time()
sigma1 = [0.1, 0.4, 1, 1.5]
#sigma1 = [1, 1.5, 2, 2.5]
times1 = []   #
Ri1    = []   #
mpt1   = []   # mean pasage time (середній час досягнення значення R)
print("R","\t eps "," \t sigma", "\t mpt1")
for i in range(len(sigma1)):
    times_, Ri_, mpt_ = funct3(x0, y0, vx0, vy0, sigm = sigma1[i], epsilon = 1)
    times1.append(times_)
    Ri1.append(Ri_)
    mpt1.append(mpt_)
print("Ця частина коду виконувалася %s seconds." % round(time.time() - start_time, 2))
```

```
R        eps     sigma   mpt1
3        1       0.1     0.01048
3        1       0.4     0.01048
3        1       1       0.00984
3        1       1.5     0.00742
Ця частина коду виконувалася 1.86 seconds.
```

Перевіримо розмірність нових масивів.

In [17]:
```python
print(np.shape(times1))
print(np.shape(Ri1))
print(np.shape(mpt1))
```

```
(4, 50)
(4, 50)
(4,)
```

- Розрахуємо дані для $R = 3, \sigma\ (sigma) = 1, \varepsilon\ (epsilon2) = [0.5, 1, 1.5, 2].$

In [18]:
```python
start_time = time.time()
epsilon2 = [1, 3, 5, 10]
times2 = []   #
Ri2    = []   #
mpt2   = []   # mean pasage time (середній час досягнення значення R)
print("R","\t eps"," \t sigma", "\t mpt2")
for i in range(len(epsilon2)):
    times_, Ri_, mpt_ = funct3(x0, y0, vx0, vy0, sigm = 1, epsilon = epsilon2[i])
    times2.append(times_)
    Ri2.append(Ri_)
    mpt2.append(mpt_)
print("Ця частина коду виконувалася %s seconds." % round(time.time() - start_time, 2))
```

```
R        eps     sigma   mpt2
3        1       1       0.00984
3        3       1       0.00922
3        5       1       0.00903
3        10      1       0.00878
Ця частина коду виконувалася 1.03 seconds.
```

In [19]:
```python
print(np.shape(times2))
print(np.shape(Ri2))
print(np.shape(mpt2))
```

```
(4, 50)
(4, 50)
(4,)
```

Функція, що априксимує дані, повертає:

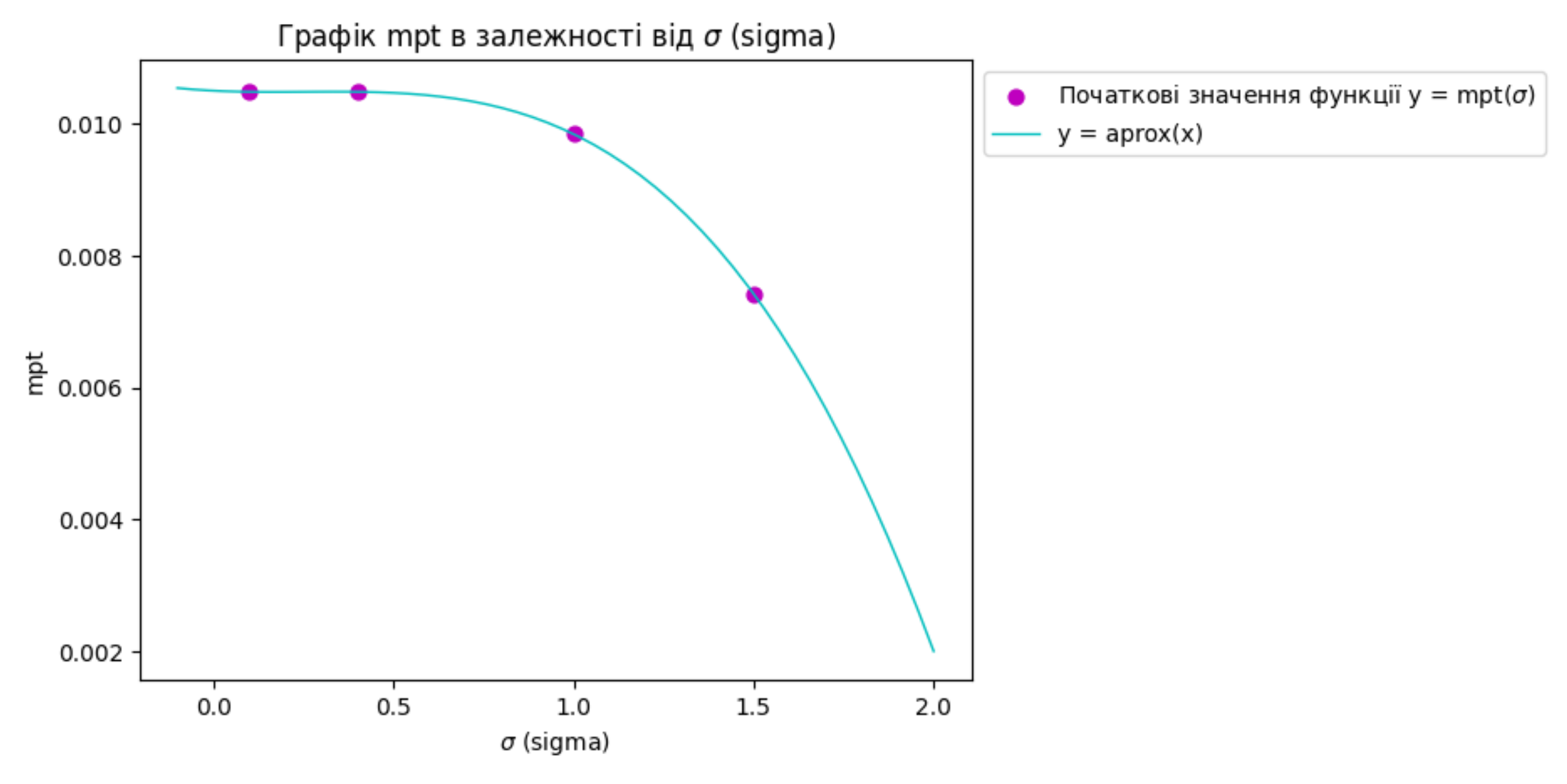
- coef - коефіцієнти рівняння, яке відповідає початковим даним,
- polinom - рівняння з коефіцієнтами,
- mpt_to_compare - значення функції для порівняння зі початковими.

In [20]:
```python
def fun_aprox(X, mpt):
    k=1
    mpt_new = mpt*0
    eps = 1000
    bool_mpt = 0

    while ((eps >= 0.00001) & (k<10)):
        coef = np.polyfit(X, mpt,k)
        polinom = np.poly1d(coef)
        mpt_to_compare = polinom(X)

        mpt_old = np.round(mpt,5)
        mpt_to_compare = np.round(mpt_to_compare,5)

        eps = abs(mpt_old[0] - mpt_to_compare[0])
        bool_mpt = np.sum(mpt_old == mpt_to_compare)

        #print("\n k =",k)
        #print("coef:", coef)
        #print("y = aprox(x):\n",polinom)
        #print("Перевірка:\n", " X =", X, "\n mpt = ", mpt_old,"\n mpt_to_compare =", mpt_to_compare)
        #print("bool_mpt =",bool_mpt)
        #print("eps =",eps)

        k += 1

    print("coef:", coef)
    print("y = aprox(x) :\n",polinom)
    print("Перевірка:\n", " X =", X, "\n", mpt_old, "= mpt\n",mpt_to_compare,"= mpt_to_compare")

    return coef, polinom, mpt_to_compare

#c0, p0, mpt_to_compare0 = fun_aprox(sigma, mpt)
```

Побудуємо тепер початкові дані та знайдену криву.

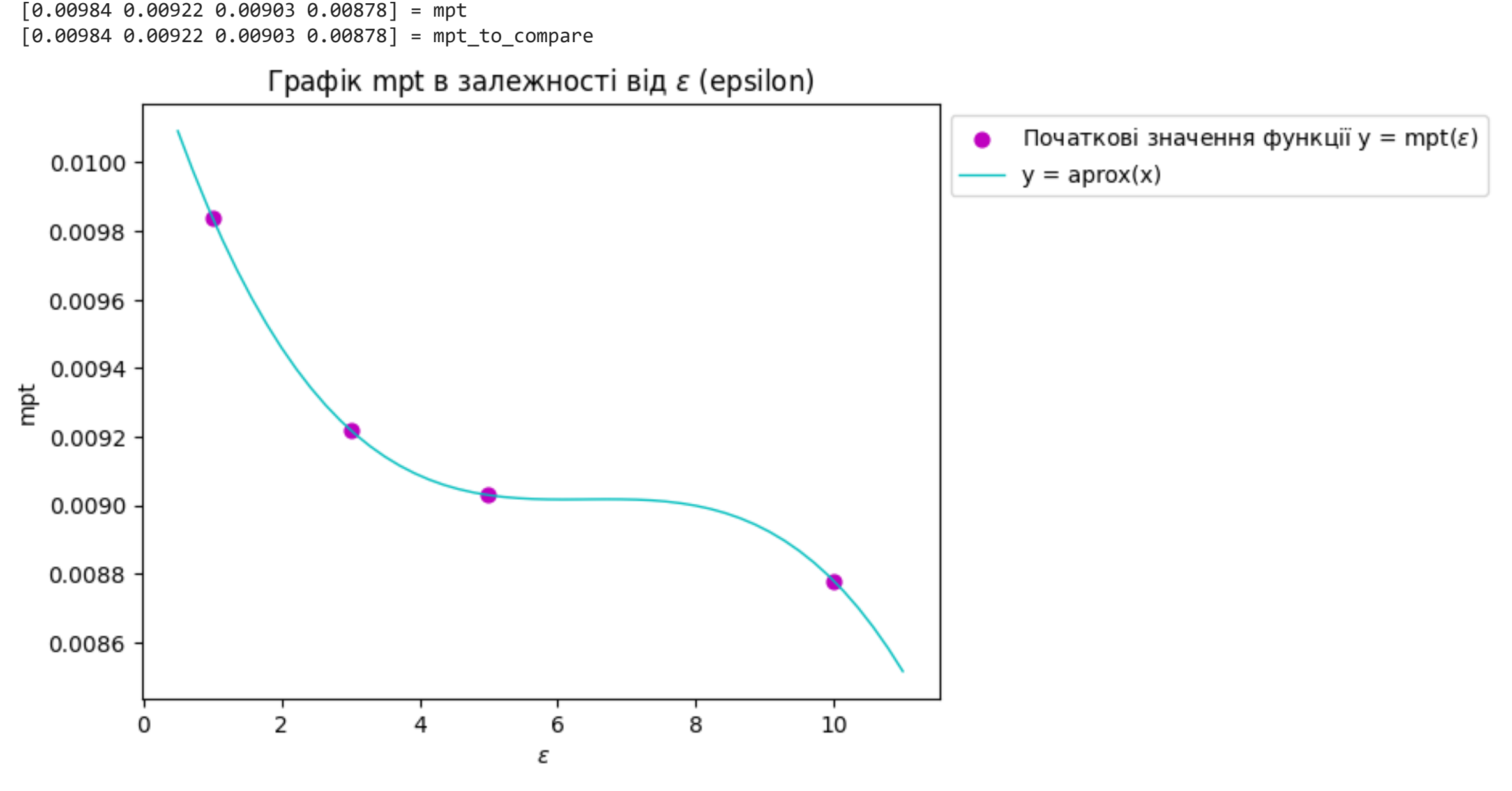
In [21]:
```python
def fun_plot31(mpt, sigm = [0.1, 0.4, 1, 1.5]):

    plt.scatter(sigm, mpt, label="Початкові значення функції y = mpt($\sigma$)", s=40, c='m')

    # порівняння з апроксимованою кривою деякого степеня
    #sigm_new = np.linspace(0.5, 8.5, 50)

    min_sigm = min(sigm)
    max_sigm  = max(sigm)
    sigm_new = np.linspace(min_sigm-0.2, max_sigm+0.5, 50)

    coef, polinom, mpt_to_compare = fun_aprox(sigm, mpt)
    mpt_new = polinom(sigm_new)
    plt.plot(sigm_new, mpt_new, label = "y = aprox(x)", linewidth = 1, c='c')

    plt.xlabel("$\sigma$ (sigma)")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від $\sigma$ (sigma)")
    plt.legend(bbox_to_anchor=(1,1), loc="upper left")

fun_plot31(mpt1, sigma1)
```

```
coef: [-0.00160366  0.0012203  -0.00027338  0.01049674]
y = aprox(x) :
            3           2
-0.001604 x + 0.00122 x - 0.0002734 x + 0.0105
Перевірка:
  X = [0.1, 0.4, 1, 1.5]
 [0.01048 0.01048 0.00984 0.00742] = mpt
 [0.01048 0.01048 0.00984 0.00742] = mpt_to_compare
```



In [22]:
```python
def fun_plot32(mpt, epsilon = [1, 3, 5, 10]):

    plt.scatter(epsilon, mpt, label="Початкові значення функції y = mpt($\epsilon$)", s=40, c='m')

    # порівняння з апроксимованою кривою деякого степеня
    min_eps = min(epsilon)
    max_eps = max(epsilon)
    epsilon_new = np.linspace(min_eps-0.5, max_eps+1, 50)
    coef, polinom, mpt_to_compare = fun_aprox(epsilon, mpt)
    mpt_new = polinom(epsilon_new)
    plt.plot(epsilon_new, mpt_new, label = "y = aprox(x)", linewidth = 1, c='c')

    plt.xlabel(" $ \epsilon $")
    plt.ylabel("mpt")
    plt.title("Графік mpt в залежності від $ \epsilon $ (epsilon)")
    plt.legend(bbox_to_anchor=(1,1), loc="upper left")

fun_plot32(mpt2, epsilon2)
```

```
coef: [-5.25793651e-06  1.01071429e-04 -6.45932540e-04  1.03901190e-02]
y = aprox(x) :
              3             2
-5.258e-06 x + 0.0001011 x - 0.0006459 x + 0.01039
Перевірка:
  X = [1, 3, 5, 10]
 [0.00984 0.00922 0.00903 0.00878] = mpt
 [0.00984 0.00922 0.00903 0.00878] = mpt_to_compare
```



Комп'ютерне моделювання задач прикладної математики

Динамічні системи.

Лабораторна робота 3

Студентка Пороскун Олена. Група ПМ.м-21

Варіант 8

Task 1 Одне рівняння.

Стаціонарні стани

Динамічне рівняння

$$\frac{dx}{dt} = f(x,a,b)$$

Стаціонарні стани

$$f(x) = 0$$

Стійкість стаціонарних станів

$$\lambda = \frac{df}{dx}|_{x=x_i}$$

$$(\lambda < 0 - \text{стійкий стан}, \lambda > 0 - \text{нестійкий стан})$$

Умова

5-8: система 1) при $\tau_y = \tau_z = 0$, $sigma(\sigma) = 1$

1)

$$\begin{cases} \tau_x \frac{dx}{dt} & = & \sigma(y-x) \\ \tau_y \frac{dy}{dt} & = & x(r-z)-y \\ \tau_z \frac{dz}{dt} & = & xy-bz \end{cases}$$

Р-ня

Знайдемо динамічне рівняння $\frac{dx}{dt}$.

- $\tau_z = 0:$

  $\tau_z \frac{dz}{dt} = xy - bz = 0$
  $z = \frac{xy}{b}$

- $\tau_y = 0:$

  $\tau_y \frac{dy}{dt} = x(r-z) - y = x(r - (\frac{xy}{b})) - y = xr - \frac{x^2y}{b} - y = 0$
  $y(1+\frac{x^2}{b}) = xr$
  $y = \frac{xr}{1+\frac{x^2}{b}} = \frac{bxr}{b+x^2}$

- $\tau_x:$

  $\tau_x \frac{dx}{dt} = \sigma(y-x) = \frac{bxr}{b+x^2} - x = \frac{bxr-xb-x^3}{b+x^2} = \frac{x(br-b-x^2)}{b+x^2}$

  Так як $\tau_x = 1$, отримуємо динам. рівняння:

$$\frac{dx}{dt} = \frac{x(br-b-x^2)}{b+x^2} = f(x,r,b)$$

Знайдемо стаціонарні стани
$f(x) = 0$

$f(x,r,b) = \frac{x(br-b-x^2)}{b+x^2} = 0$
$D(f): b + x^2 \neq 0 \to x^2 \neq -b \to x \neq \pm i\sqrt{b}$

$x(br-b-x^2) = 0$
$x_1 = 0$
$або$
$br - b - x^2 = 0$
$x^2 = b(r-1)$
$x_{2,3} = \pm\sqrt{b(r-1)}$

Розглянемо різні значення параметру r. Параметр $b > 0$.

- $0 < r < 1:$
  $x_1 = 0$
  Маємо тільки 1 стаціонарний стан, бо розглядаємо тільки дійсні розв'язки $(b(r-1)$ має бути $> 0)$.

- $r > 1:$
  $x_1 = 0$
  $x_{2,3} = \pm\sqrt{b(r-1)}$
  Маємо 3 стаціонарні стани.

Далі побудуємо графік залежності динамічної величини x від параметра r. При цьому параметр $b = const$.

In [1]:
```
from matplotlib import pyplot as plt
import numpy as np
```

In [2]:
```
b = 5
```

- $0 < r < 1:$

In [3]:
```
r11 = np.linspace(0.05, 1, 25, endpoint=False)
print(r11)
r11 = [round(ri,3) for ri in r11]
x11 = [0 for i in range(len(r11))]
print(np.shape(r11),np.shape(x11))
```

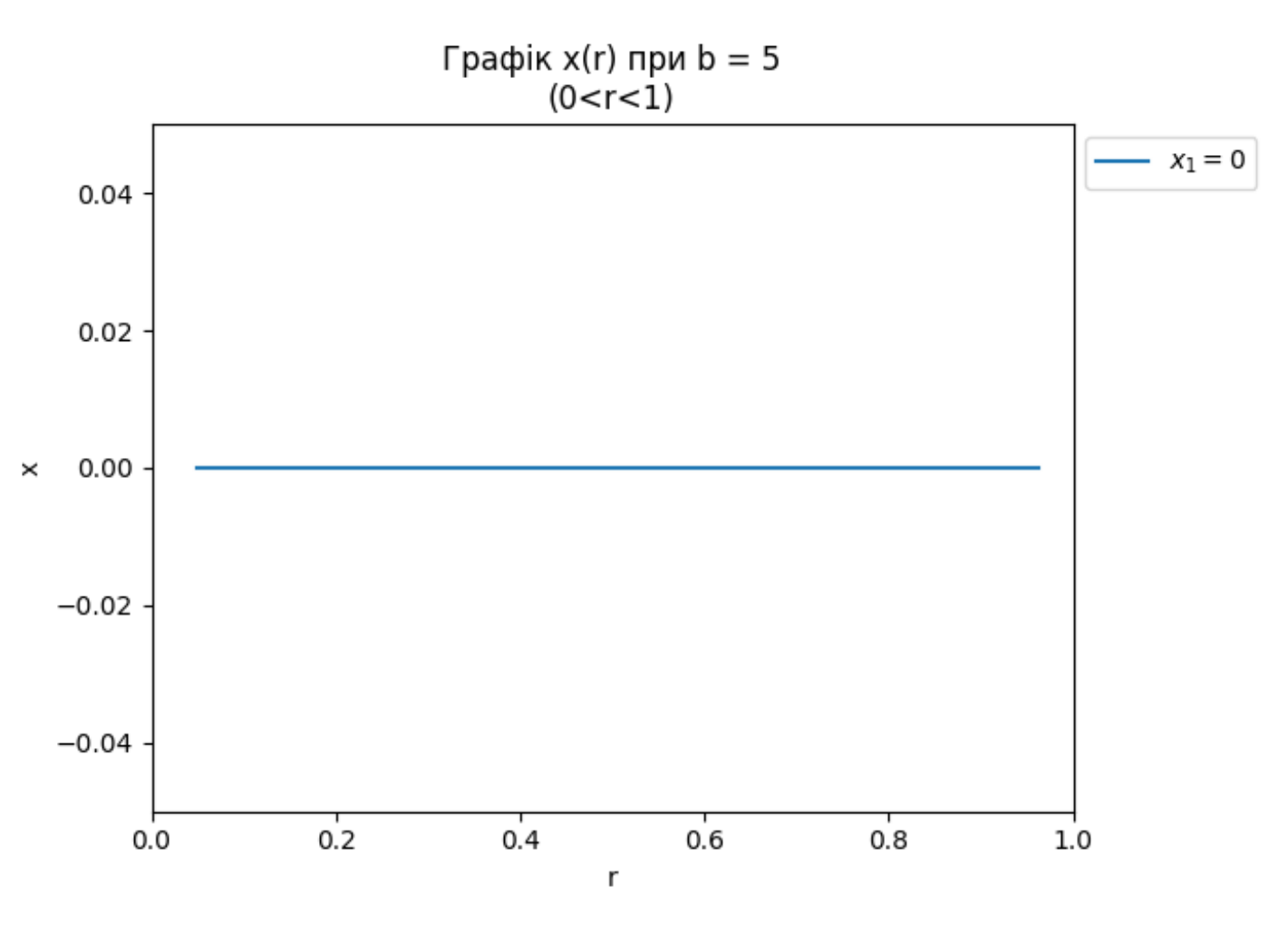
```
[0.05  0.088 0.126 0.164 0.202 0.24  0.278 0.316 0.354 0.392 0.43  0.468
 0.506 0.544 0.582 0.62  0.658 0.696 0.734 0.772 0.81  0.848 0.886 0.924
 0.962]
(25,) (25,)
```

In [4]:
```
print("  r11  x11  ")
list(zip(r11[:4], x11[:4]))
```

Out[4]:
```
  r11  x11
[(0.05, 0), (0.088, 0), (0.126, 0), (0.164, 0)]
```

In [5]:
```
plt.plot(r11, x11, label = "$x_1 = 0$")
#plt.plot(r, x2, label = "$x_2 =  \sqrt{b(r-1)}$")
#plt.plot(r, x3, label = "$x_3 = -\sqrt{b(r-1)}$")
#plt.text(r[0], x2[0], " $r_c$  ")
plt.title("Графік x(r) при b = " f'{b}' "\n(0<r<1)")
plt.legend(bbox_to_anchor=(1,1), loc="upper left")
plt.xlabel("r")
plt.ylabel("x")
ax=[0, 1, -0.05, 0.05]
plt.axis(ax)
```

Out[5]:
```
(0.0, 1.0, -0.05, 0.05)
```



- $r > 1:$

In [6]:
```
r = np.linspace(1, 20, 50, endpoint=False)
#print(r)
r = [round(ri,3) for ri in r]

x1 = [0 for i in range(len(r))]
x2 = [round((b*(ri-1))**(0.5), 3) for ri in r]
x3 = [round(-((b*(ri-1))**(0.5)), 3) for ri in r]
```

In [7]:
```
print(np.shape(r), np.shape(x1), np.shape(x2), np.shape(x3))
```

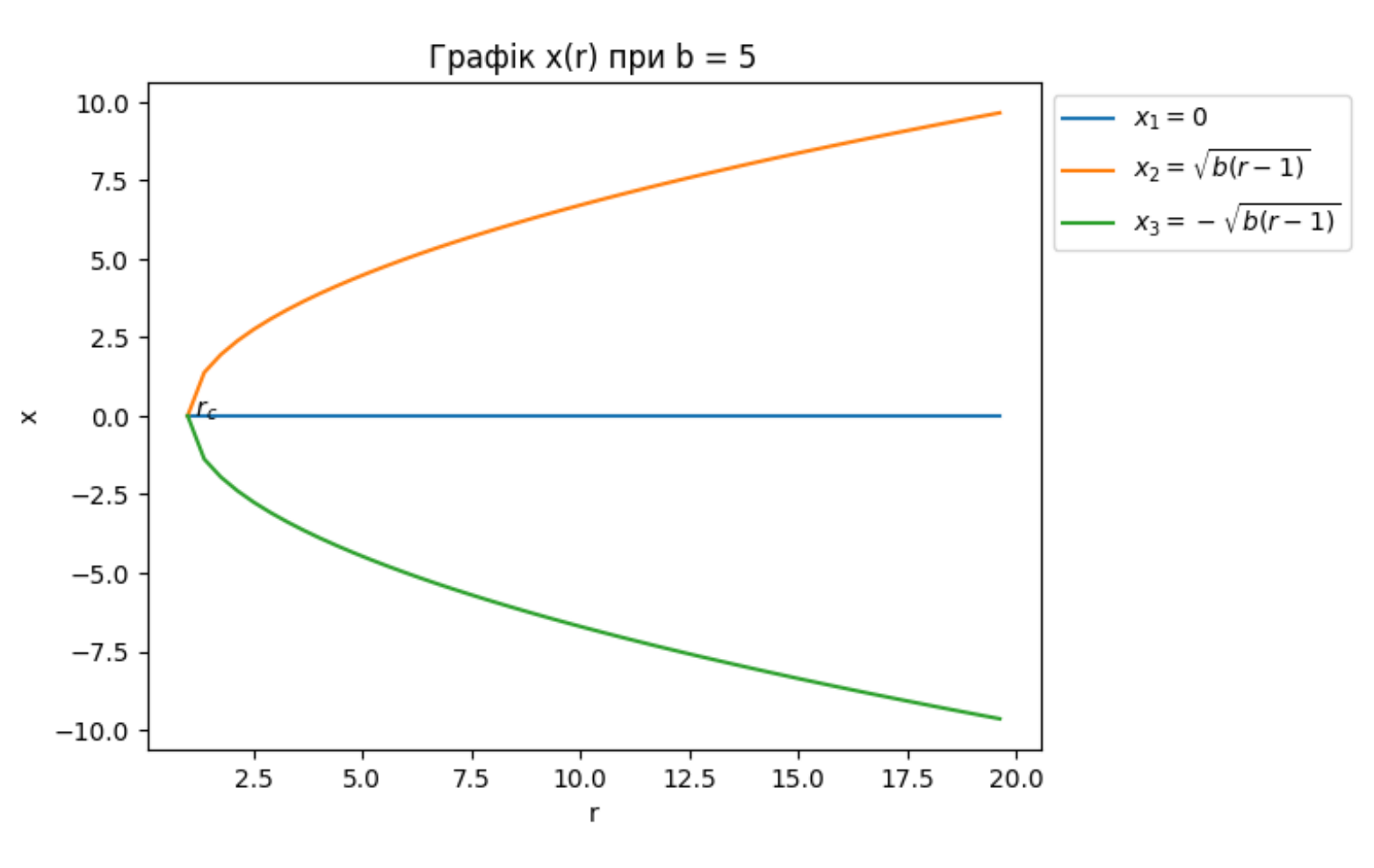
```
(50,) (50,) (50,) (50,)
```

In [8]:
```
print("  r  x1  x2   x3")
list(zip(r[:5], x1[:5], x2[:5], x3[:5]))
```

Out[8]:
```
  r  x1  x2   x3
[(1.0, 0, 0.0, -0.0),
 (1.38, 0, 1.378, -1.378),
 (1.76, 0, 1.949, -1.949),
 (2.14, 0, 2.387, -2.387),
 (2.52, 0, 2.757, -2.757)]
```

In [9]:
```
plt.plot(r, x1, label = "$x_1 = 0$")
plt.plot(r, x2, label = "$x_2 =  \sqrt{b(r-1)}$")
plt.plot(r, x3, label = "$x_3 = -\sqrt{b(r-1)}$")
plt.text(r[0], x2[0], " $r_c$  ")
plt.title("Графік x(r) при b = " f'{b}')
plt.legend(bbox_to_anchor=(1,1), loc="upper left")
plt.xlabel("r")
plt.ylabel("x")
```

Out[9]:
```
Text(0, 0.5, 'x')
```



Розглянемо стійкість стаціонарних станів.

$$\lambda = \frac{df}{dx}|_{x=x_i}$$

$$f(x,r,b) = \frac{x(br-b-x^2)}{b+x^2} = \frac{bxr-xb-x^3}{b+x^2}$$

$$\frac{df}{dx} = \frac{(br-b-3x^2)(b+x^2)-2x(bxr-xb-x^3)}{(b+x^2)^2} = \frac{b^2r-b^2-2bx^2-brx^2-x^4}{(b+x^2)^2} = \frac{(b^2r-brx^2)-(b^2+2bx^2+(x^2)^2)}{(b+x^2)^2} = \frac{br(b-x^2)-(b+x^2)^2}{(b+x^2)^2} = \frac{br(b-x^2)}{(b+x^2)^2} - 1$$

Отже,

$$\frac{df}{dx} = \frac{br(b-x^2)}{(b+x^2)^2} - 1$$

$$\lambda = \frac{df}{dx}|_{x=x_i}$$

Розглянемо різні значення параметру r.

- $0 < r < 1:$

  $x_1 = 0$

$$\lambda_{11} = \frac{br(b-0)}{(b+0)^2} - 1 = r - 1$$

  Так як $r < 1$, то $\lambda_{11} < 0$, стан стійкий.

- $r > 1:$

  $x_1 = 0$

$$\lambda_{21} = \frac{br(b-0)}{(b+0)^2} - 1 = r - 1$$

  Так як $r > 1$, то $\lambda_{21} > 0$, стан нестійкий.

  $x_{2,3} = \pm\sqrt{b(r-1)} \to x_2^2 = x_3^2 = b(r-1)$

$$\lambda_{22} = \frac{br(b-b(r-1))}{(b+b(r-1))^2} - 1 = \frac{2b-br}{br} - 1 = \frac{2}{r} - 2$$

  Так як $r > 1$, то $\lambda_{22} < 0$, стан стійкий.

Завантажимо необхідні бібліотеки для перевірки формул похідних.

In [10]:
```
!pip install sympy
```

```
Requirement already satisfied: sympy in c:\users\admin\anaconda3\lib\site-packages (1.11.1)
Requirement already satisfied: mpmath>=0.19 in c:\users\admin\anaconda3\lib\site-packages (from sympy) (1.2.1)
```

In [11]:
```
from sympy import diff, symbols
```

In [12]:
```
x, r, b = symbols('x r b')
df = diff((b*r*x - b*x - x**3)/(b+x**2), x)
df
```

Out[12]:

$$-\frac{2x\left(brx-bx-x^3\right)}{\left(b+x^2\right)^2} + \frac{br-b-3x^2}{b+x^2}$$

Якщо спростити цей вираз, то буде ж теж саме що ми знайшли раніше:

$$\frac{df}{dx} = \frac{br(b-x^2)}{(b+x^2)^2} - 1$$

In [13]:
```
x = 0
dif1 = (b*r*(b-x**2))/((b+x**2)**2)-1
dif1
```

Out[13]: $r - 1$

$$\lambda_1 = r - 1$$

In [14]:
```
x_2 = b*r-b
dif2 = (b*r*(b-x_2))/((b+x_2)**2)-1
dif2
```

Out[14]: $-1 + \frac{-br+2b}{br}$

Якщо спростити цей вираз, то буде ж теж саме що ми знайшли раніше:

$$\lambda_2 = \frac{2}{r} - 2$$

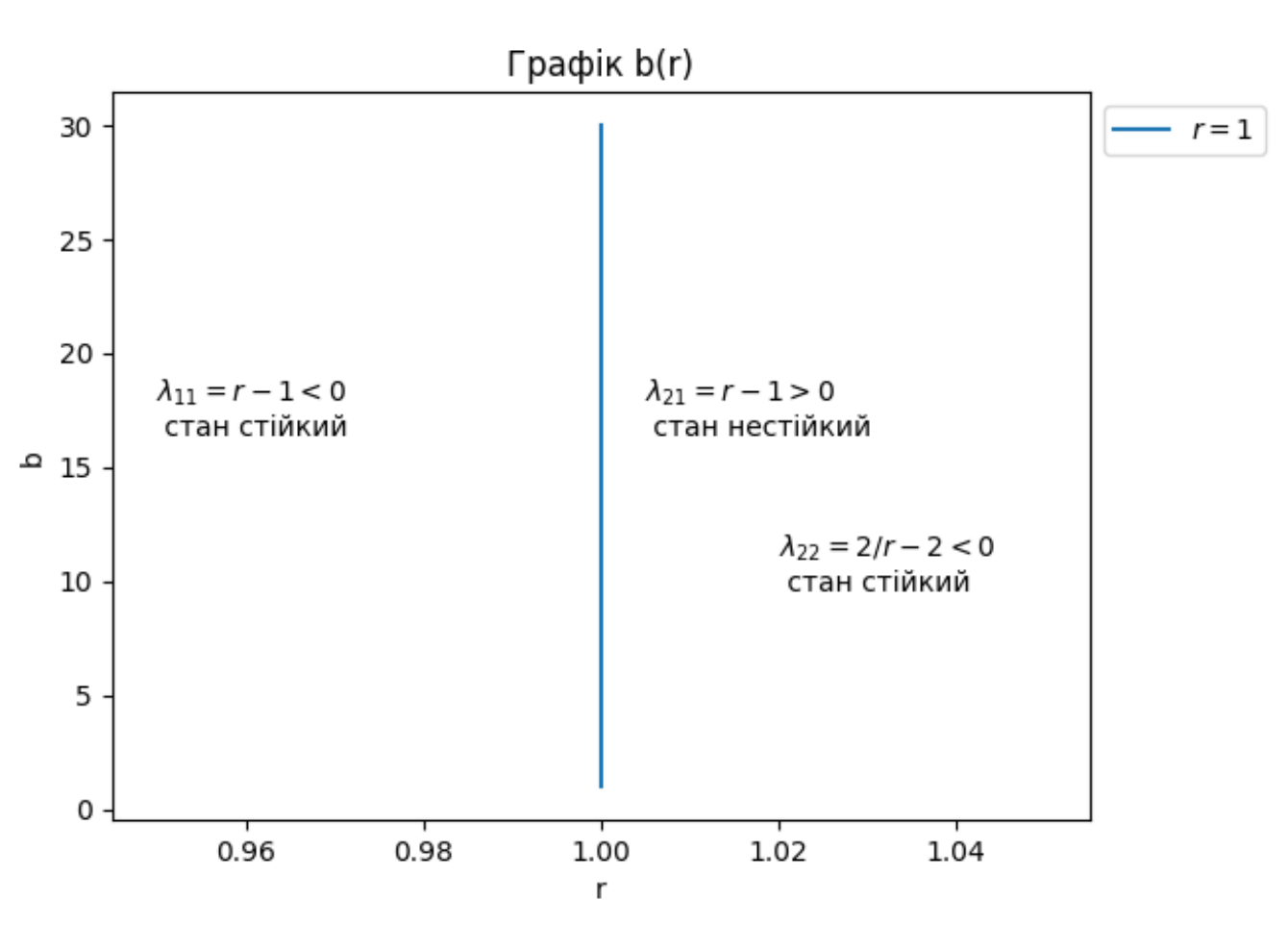
Тепер треба побудувати залежність одного параметра від іншого.

In [15]:
```
b_1 = np.linspace(1, 30, 35)
print("b_1", b_1[:10], "\nlen(b_1) =", len(b_1), "\n")

r_1 = [1 for i in range(35)]
print("r_1", r_1[:10], "\nlen(r_1) =", len(r_1))
```

```
b_1 [1.         1.85294118 2.70588235 3.55882353 4.41176471 5.26470588
 6.11764706 6.97058824 7.82352941 8.67647059]
len(b_1) = 35

r_1 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
len(r_1) = 35
```

In [16]:
```
plt.plot(r_1, b_1, label = "$r=1$")
plt.text(0.95, b_1[18], "$ \lambda_{11}=r-1 < 0 $ \n стан стійкий")
plt.text(1.005, b_1[18], "$ \lambda_{21}=r-1 > 0 $ \n стан нестійкий")
plt.text(1.02, b_1[10], "$ \lambda_{22}=2/r-2 < 0 $ \n стан стійкий")
plt.title("Графік b(r)")
plt.legend(bbox_to_anchor=(1,1), loc="upper left")
plt.xlabel("r")
plt.ylabel("b")
```

Out[16]:
```
Text(0, 0.5, 'b')
```



Далі чисельно розв'яжемо рівняння

$$\frac{dx}{dt} = \frac{x(br-b-x^2)}{b+x^2} = f(x,r,b)$$

In [17]:
```
from sympy import integrate
import math
```

In [18]:
```
x, r, b = symbols('x r b')
# dx/dt = f(x,r,b):
f = x*(b*r - b - x**2)/(b + x**2)
ans1 = integrate(f,x)
print("t =",ans1)
ans1
```

```
t = b*r*log(b + x**2)/2 - x**2/2
```

Out[18]:

$$\frac{br\log\left(b+x^2\right)}{2} - \frac{x^2}{2}$$

Покажемо, як веде себе функція $x = x(t)$ при різних початкових умовах.

Розглянемо різні значення параметру r.

$$t = \frac{br\,log(b+x^2)}{2} - \frac{x^2}{2}$$

- $0 < r < 1:$

  $x_1 = 0$

$$\lambda_{11} = r - 1 < 0 \text{ (стан стійкий)}$$

Задамо масиви необхідних змінних та параметрів.

In [19]:
```
print("(b>0 завжди)\n r<1 \n  x_1=0\n")
x = 0
print("x", x)

r = np.linspace(0.1, 1, 9, endpoint=False)
r = [round(ri,3) for ri in r]
print("r", r, "\nlen(r) =", len(r), "\n")

b = [i*2 for i in range(1,7)]
b = [round(bi,3) for bi in b]
print("b", b, "\nlen(b) =", len(b))
```

```
(b>0 завжди)
 r<1
  x_1=0

x 0
r [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
len(r) = 9

b [2, 4, 6, 8, 10, 12]
len(b) = 6
```

Обчислимо значення t за формулою отриманою раніше з інтегрування динамічного рівняння. На виході отримаємо складний список, що складається з значень x, r, b, t.

In [20]:
```
def fun_calcD_r1(r, b):
    #      x,  r,  b,  t
    D = [ [], [], [], [] ]
    for ri in r:
        for bi in b:
            xi = 0
            t_ = bi*ri*math.log(bi + xi**2)/2 - (xi**2)/2
            t_ = round(t_,3)
            D[0].append(xi)
            D[1].append(ri)
            D[2].append(bi)
            D[3].append(t_)
    print("len(D[3]) =", len(D[3]))
    return D

D = fun_calcD_r1(r, b)
print("D = array", np.shape(D))
print("D = [ [x],  [r],  [b],  [t] ]")
```

```
len(D[3]) = 54
D = array (4, 54)
D = [ [x],  [r],  [b],  [t] ]
```

Ось деякі значення відповідно параметрам.

In [21]:
```
#from scipy.special import logsumexp
```

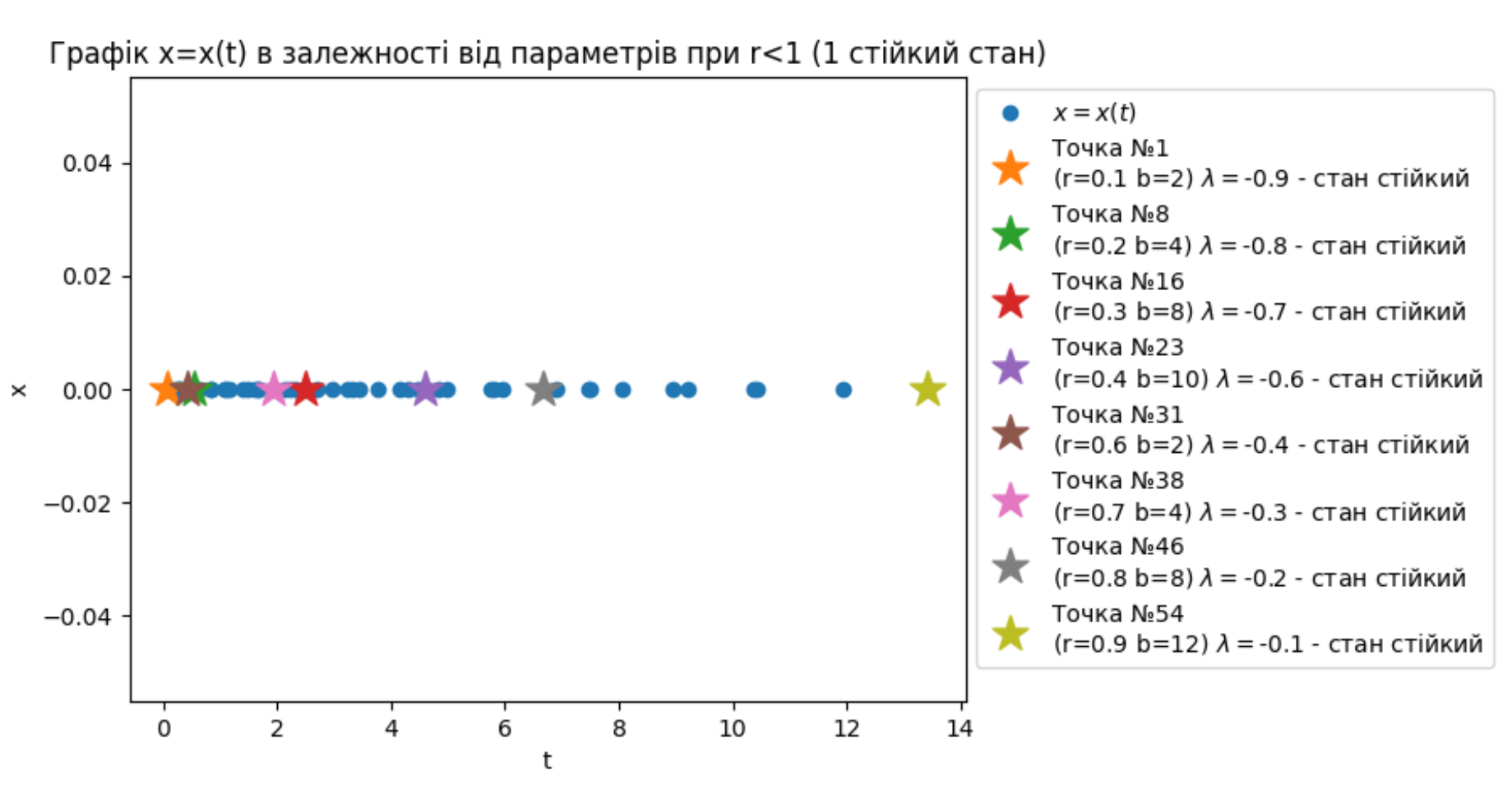
In [22]:
```
def fun_printExample(x,r,b,D):
    print(" x", x, "\n", "r", r,"\n", "b", b,"\n")
    print("  x    r   b   t")
    k_=[1, len(b), len(b)+1, 2*len(b), len(b)*len(r)]
    for k in k_:
        kk=list(zip(D[0][k-1:k], D[1][k-1:k], D[2][k-1:k], D[3][k-1:k]))
        print(kk)

fun_printExample(x,r,b,D)
```

```
 x 0
 r [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
 b [2, 4, 6, 8, 10, 12]

  x    r   b   t
[(0, 0.1, 2, 0.069)]
[(0, 0.1, 12, 1.491)]
[(0, 0.2, 2, 0.139)]
[(0, 0.2, 12, 2.982)]
[(0, 0.9, 12, 13.418)]
```

Зобразимо графік x=x(t) в залежності від параметрів при r<1.

In [23]:
```
def fun_plot_xt(D, graph="scatter"):
    # D = [  x,  r,  b,  t ]
    #        0   1   2   3
    axis_t = D[3]
    axis_x = D[0]
    r_ = D[1]
    b_ = D[2]

    if (graph=="plot"):
        plt.plot(axis_t, axis_x, c=[0.2, 0.5, 0.5], label = "$x=x(t)$")
    if (graph=="scatter"):
        plt.scatter(axis_t, axis_x, label = "$x=x(t)$")

    points_x = np.linspace(0, len(axis_x)-1, 8)
    for k in range(len(points_x)):
        key = int(points_x[k])
        lambda_ = round(r_[key]-1, 3)
        str1 = ""
        if (lambda_ < 0):
            str1 = " - стан стійкий"
        if (lambda_ > 0):
            str1 = " - стан нестійкий"
        plt.scatter(axis_t[key], axis_x[key], label="Точка №"f'{key+1}' " \n(r=" f'{r_[key]}'
                    " b=" f'{b_[key]}'") $\lambda =$" f'{lambda_}' f'{str1}',
        marker="*", s=250)

    plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    plt.xlabel("t")
    plt.ylabel("x")

fun_plot_xt(D)
plt.title("Графік x=x(t) в залежності від параметрів при r<1 (1 стійкий стан)")
#plt.show()
#fun_plot_xt(D, "plot")
```

Out[23]:
```
Text(0.5, 1.0, 'Графік x=x(t) в залежності від параметрів при r<1 (1 стійкий стан)')
```



- $r > 1:$

  $x_1 = 0$

$$\lambda_{21} = r - 1 > 0 \text{ (стан нестійкий)}$$

  $x_{2,3} = \pm\sqrt{b(r-1)} \to x_2^2 = x_3^2 = b(r-1)$

$$\lambda_{22} = \frac{2}{r} - 2 < 0 \text{ (стан стійкий)}$$

Задамо масиви необхідних змінних та параметрів.

In [24]:
```
print("(b>0 завжди)\n r>1 \n  x_1=0 \n  x_2,3 = +-sqrt(b(r-1))\n")
r2 = np.linspace(1.1, 15.1, 10)
r2 = [round(ri,3) for ri in r2]
print("r2", r2, "\nlen(r2) =", len(r2), "\n")

b2 = [i*2 for i in range(1,7)]
b2 = [round(bi,3) for bi in b2]
print("b2", b2, "\nlen(b2) =", len(b2))
```

```
(b>0 завжди)
 r>1
  x_1=0
  x_2,3 = +-sqrt(b(r-1))

r2 [1.1, 2.656, 4.211, 5.767, 7.322, 8.878, 10.433, 11.989, 13.544, 15.1]
len(r2) = 10

b2 [2, 4, 6, 8, 10, 12]
len(b2) = 6
```

Обчислимо значення t за формулою отриманою раніше з інтегрування динамічного рівняння. На виході отримаємо складний список, що складається з значень x_1, x_2, x_3, r, b, t.

In [25]:
```
def fun_calcD_r2(r, b):
    #     x1, x2, x3, r,  b,  t1, t2, t3
    D = [ [], [], [], [], [], [], [], [] ]
    #      0  1   2   3   4   5   6   7
    #print("D = ", D)
    #print("x1i x2i x3i ri bi t_1, t_2, t_3")
    for ri in r:
        for bi in b:
            xi_1 = 0
            t_1 = bi*ri*math.log(bi + xi_1**2)/2 - (xi_1**2)/2
            t_1 = round(t_1,3)

            xi_2 = (bi*(ri-1))**(0.5)
            xi_2 = round(xi_2, 3)
            t_2 = bi*ri*math.log(bi + xi_2**2)/2 - (xi_2**2)/2
            t_2 = round(t_2,3)

            xi_3 = -xi_2
            t_3 = bi*ri*math.log(bi + xi_3**2)/2 - (xi_3**2)/2
            t_3 = round(t_3,3)

            D[0].append(xi_1)
            D[1].append(xi_2)
            D[2].append(xi_3)
            D[3].append(ri)
            D[4].append(bi)
            D[5].append(t_1)
            D[6].append(t_2)
            D[7].append(t_3)

    print("len(D[5]) =", len(D[5]))
    return D

D2 = fun_calcD_r2(r2, b2)
print("D2 = array", np.shape(D2))
print("D2 = [ [x1], [x2], [x3], [r], [b], [t1], [t2], [t3] ]")
```

```
len(D[5]) = 60
D2 = array (8, 60)
D2 = [ [x1], [x2], [x3], [r], [b], [t1], [t2], [t3] ]
```

Ось деякі значення відповідно параметрам.

In [26]:
```
def fun_printExample2(r,b,D):
    print("r", r, "\nb", b)
    print(" x1  x2     x3      r    b  t1     t2     t3")
    k_=[1, len(b), len(b)+1, 2*len(b), len(b)*len(r)]
    for k in k_:
        kk=list(zip(D[0][k-1:k], D[1][k-1:k], D[2][k-1:k], D[3][k-1:k], D[4][k-1:k], D[5][k-1:k], D[6][k-1:k], D[7][k-1:k]))
        print(kk)

fun_printExample2(r2,b2,D2)
```

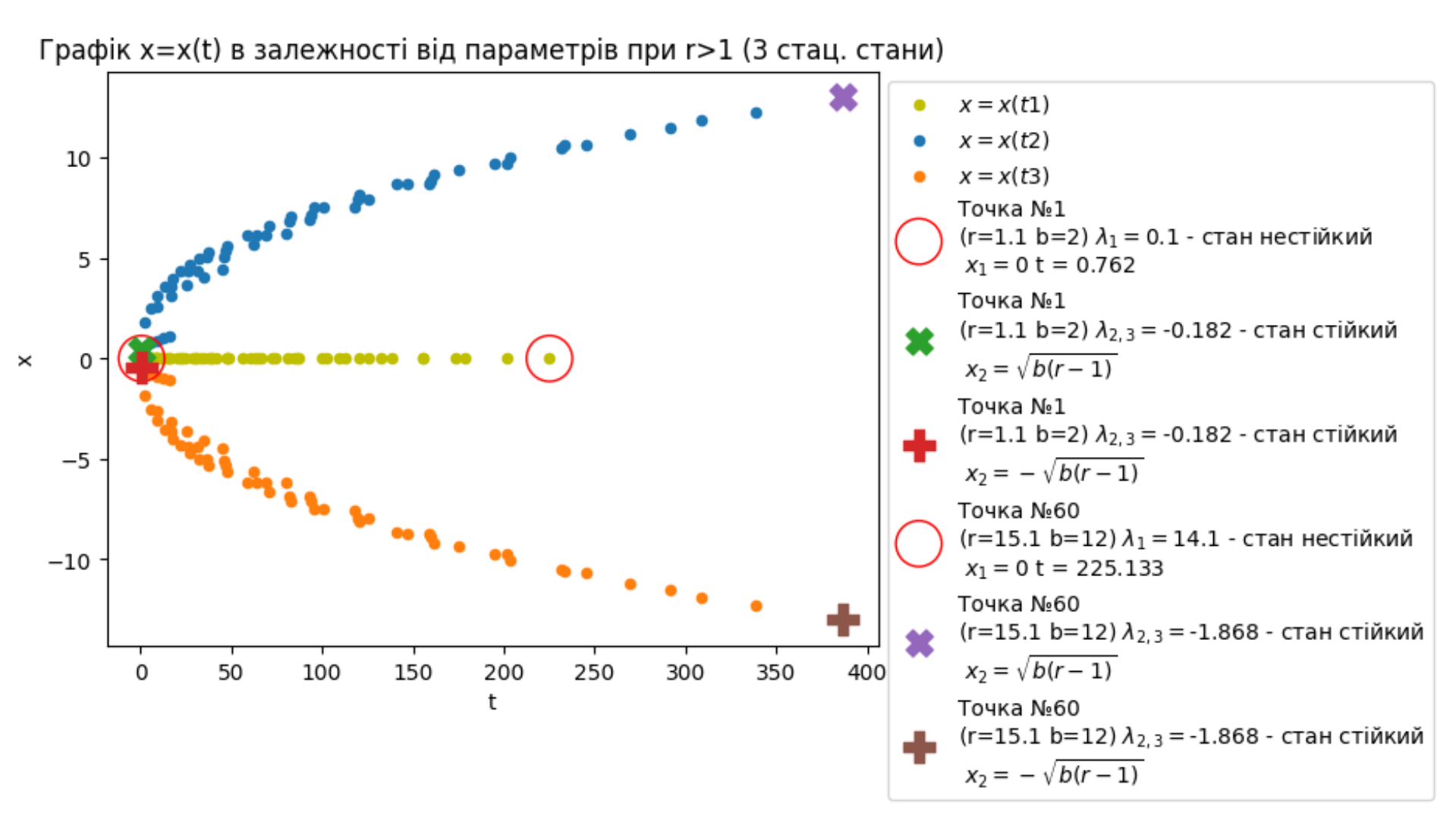
```
r [1.1, 2.656, 4.211, 5.767, 7.322, 8.878, 10.433, 11.989, 13.544, 15.1]
b [2, 4, 6, 8, 10, 12]
 x1  x2     x3      r    b  t1     t2     t3
[(0, 0.447, -0.447, 1.1, 2, 0.762, 0.767, 0.767)]
[(0, 1.095, -1.095, 1.1, 12, 16.4, 16.429, 16.429)]
[(0, 1.82, -1.82, 2.656, 2, 1.841, 2.779, 2.779)]
[(0, 4.458, -4.458, 2.656, 12, 39.599, 45.23, 45.23)]
[(0, 13.008, -13.008, 15.1, 12, 225.133, 386.484, 386.484)]
```

Зобразимо графік x=x(t) в залежності від параметрів при r>1.

In [27]:
```
#help ('matplotlib.markers')  # викор в plt.scatter
# '*' - зірочка, 'o' - кружечок, 'P' / 'p' - плюс, 'X'/'x' - крестик
# edgecolor="r" (колір границі фігури), c="none" (колір фігури), s - розмір
```

In [28]:
```
def fun_plot_xt2(D, graph="scatter"):
    #       x1, x2, x3, r,  b,  t1, t2, t3
    # D = [ [], [], [], [], [], [], [], [] ]
    #       0  1   2   3   4   5   6   7
    axis_t1 = D[5]
    axis_x1 = D[0]

    axis_t2 = D[6]
    axis_x2 = D[1]

    axis_t3 = D[7]
    axis_x3 = D[2]

    r_ = D[3]
    b_ = D[4]

    if (graph=="scatter"):
        plt.scatter(axis_t1, axis_x1, label = "$x=x(t1)$", s=20, c='y')
        plt.scatter(axis_t2, axis_x2, label = "$x=x(t2)$", s=20)
        plt.scatter(axis_t3, axis_x3, label = "$x=x(t3)$", s=20)

    points_x1 = np.linspace(0, len(axis_x1)-1, 2)
    for k in range(len(points_x1)):
        key = int(points_x1[k])
        lambda_1 = round(r_[key]-1, 3)
        lambda_23 = round(2/r_[key]-2, 3)

        str1, str2 = "", ""
        #condition_if_true if condition else condition_if_false
        str1 = " - стан стійкий" if lambda_1  < 0 else " - стан нестійкий"
        str2 = " - стан стійкий" if lambda_23 < 0 else " - стан нестійкий"

        plt.scatter(axis_t1[key], axis_x1[key], label="Точка №"f'{key+1}' " \n(r=" f'{r_[key]}'
                    " b=" f'{b_[key]}'") $\lambda_1 =$" f'{lambda_1}' f'{str1}' "\n $x_1=0$ t = " f'{axis_t1[key]}'" ",
                    marker='o', edgecolor="r", c="none", s=450)
        plt.scatter(axis_t2[key], axis_x2[key], label="Точка №"f'{key+1}' " \n(r=" f'{r_[key]}'
                    " b=" f'{b_[key]}'") $\lambda_{2,3} =$" f'{lambda_23}' f'{str2}' "\n $x_2=\sqrt{b(r-1)}$",
                    marker="X", s=150)
        plt.scatter(axis_t3[key], axis_x3[key], label="Точка №"f'{key+1}' " \n(r=" f'{r_[key]}'
                    " b=" f'{b_[key]}'") $\lambda_{2,3} =$" f'{lambda_23}' f'{str2}' "\n $x_2=-\sqrt{b(r-1)}$",
                    marker="P", s=200)
    plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    plt.xlabel("t")
    plt.ylabel("x")

fun_plot_xt2(D2)
plt.title("Графік x=x(t) в залежності від параметрів при r>1 (3 стац. стани)")
```

Out[28]:
```
Text(0.5, 1.0, 'Графік x=x(t) в залежності від параметрів при r>1 (3 стац. стани)')
```



In [ ]:

Комп'ютерне моделювання задач прикладної математики

Динамічні системи.

Лабораторна робота 3

Студентка Пороскун Олена. Група ПМ.м-21

Варіант 8

Task 2 Системи нелінійних рівнянь

Теорія

Система динамічних рівнянь

$$\begin{cases} \frac{dy}{dt} = f(x,y,b,r) \\ \frac{dx}{dt} = g(x,y,b,r) \end{cases}$$

Стаціонарні стани

$$f(x,y)=0 \;\; g(x,y)=0$$

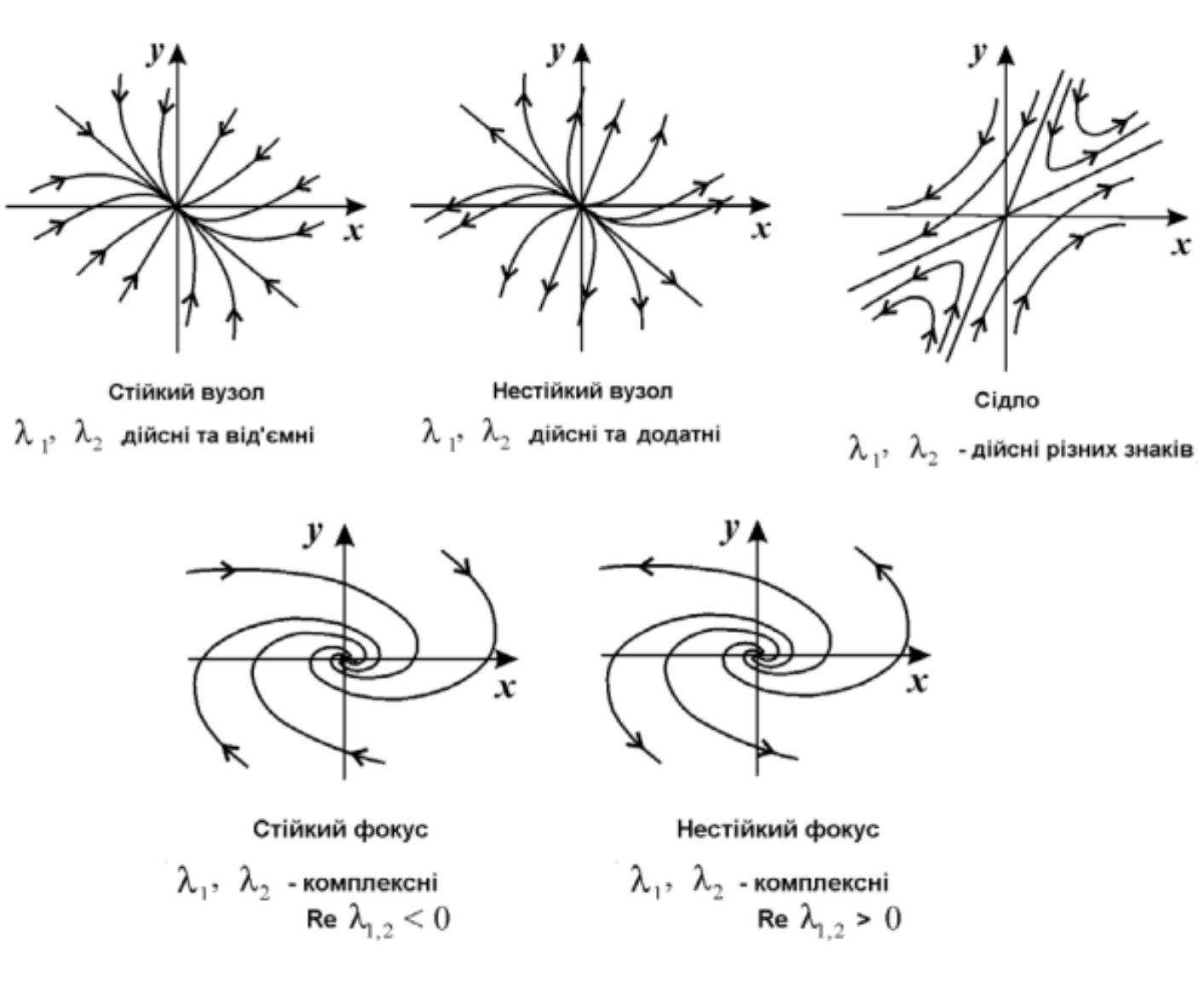
Стійкість стаціонарних станів

Матриця Якобі

$$M = \begin{pmatrix} \frac{df(x,y)}{dx}|_{x_0,y_0} - \lambda & \frac{df(x,y)}{dy}|_{x_0,y_0} \\ \frac{dg(x,y)}{dx}|_{x_0,y_0} & \frac{dg(x,y)}{dy}|_{x_0,y_0} - \lambda \end{pmatrix}$$
$$det(M) = 0 \to \lambda$$

Характеристичне рівняння і типи стаціонарних точок

$$A\lambda^2 + B\lambda + C = 0$$



Умова

5-8: система 1) при $\tau_z = 0$, $sigma(\sigma) = 1$

1)

$$\begin{cases} \tau_x \frac{dx}{dt} & = & \sigma(y-x) \\ \tau_y \frac{dy}{dt} & = & x(r-z)-y \\ \tau_z \frac{dz}{dt} & = & xy-bz \end{cases}$$

Р-ня

Знайдемо динамічні рівняння $\frac{dy}{dt}$ і $\frac{dx}{dt}$.

- $\tau_z = 0:$

  $\tau_z \frac{dz}{dt} = xy - bz = 0$
  $z = \frac{xy}{b}$

- $\tau_y = 1:$

  $\tau_y \frac{dy}{dt} = x(r-z) - y = x(r - (\frac{xy}{b})) - y = xr - \frac{x^2 y}{b} - y = f(x,y,b,r)$

- $\tau_x = 1, \sigma = 1:$

  $\tau_x \frac{dx}{dt} = \sigma(y-x) = y - x = g(x,y,b,r)$

  Отримуємо систему динам. рівняння:

$$\begin{cases} \frac{dy}{dt} = xr - \frac{x^2 y}{b} - y = f(x,y,b,r) \\ \frac{dx}{dt} = y - x = g(x,y,b,r) \end{cases}$$

Знайдемо стаціонарні стани.

$$f(x,y)=0 \;\; g(x,y)=0$$

$\begin{cases} xr - \frac{x^2 y}{b} - y = 0 \\ y - x = 0 \end{cases} \to \begin{cases} xr - \frac{x^2 (x)}{b} - (x) = 0 \\ y = x \end{cases}$

$xr - \frac{x^2(x)}{b} - (x) = 0$
$x(r - \frac{x^2}{b} - 1) = 0$
$x_1 = 0$
$r - 1 - \frac{x^2(x)}{b} = 0 \to x_{2,3} = \pm\sqrt{b(r-1)}$
$y = x \to y_{1,2,3} = x_{1,2,3}(\text{відповідно})$

Отже, маємо 3 стаціонарні стани $x_1 = y_1 = 0, x_{2,3} = y_{2,3} = \pm\sqrt{b(r-1)}$

Розглянемо різні значення параметру r. Параметр $b > 0$.

- $0 < r < 1:$
  $x_1 = y_1 = 0$
  Маємо тільки 1 стаціонарний стан, бо розглядаємо тільки дійсні розв'язки $(b(r-1)$ має бути $> 0)$.
- $r > 1:$
  $x_1 = y_1 = 0$
  $x_{2,3} = y_{2,3} = \pm\sqrt{b(r-1)}$
  Маємо 3 стаціонарні стани.

Далі побудуємо графік залежності динамічної величини x від параметра r. При цьому параметр $b = const$. Будувати графік аналогічний для y не будемо, тому що $y = x$. Вони однакові.

```
In [1]: from matplotlib import pyplot as plt
        import numpy as np
```

```
In [2]: b = 8
```

```
In [3]: r11 = np.linspace(0.05, 1, 10, endpoint=False)
        r11 = [round(ri,3) for ri in r11]
        x11 = [0 for i in range(len(r11))]
        print("r11", r11, "\nlen(r11) =", len(r11))
        print("x11", x11, "\nlen(x11) =", len(x11))
```

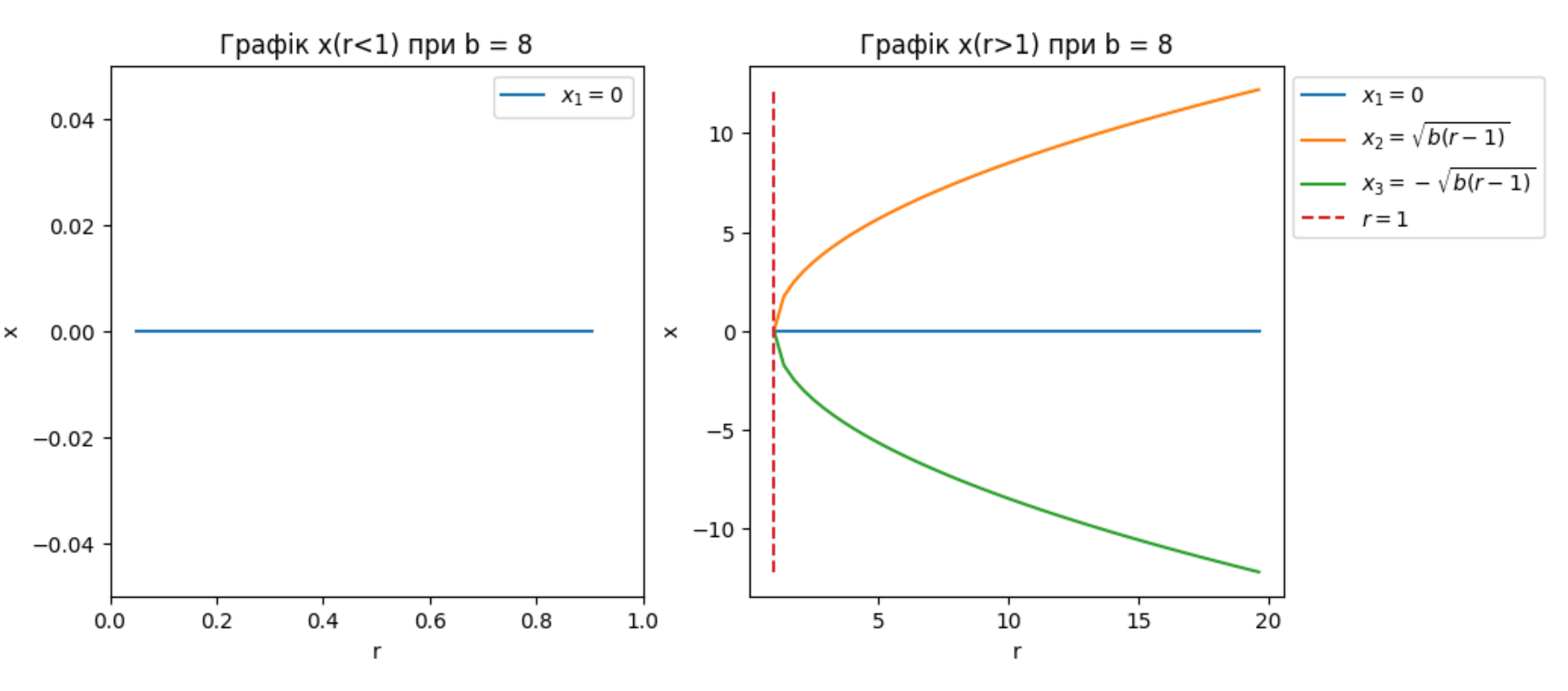
```
r11 [0.05, 0.145, 0.24, 0.335, 0.43, 0.525, 0.62, 0.715, 0.81, 0.905]
len(r11) = 10
x11 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
len(x11) = 10
```

```
In [4]: r12 = np.linspace(1, 20, 50, endpoint=False)
        r12 = [round(ri,3) for ri in r12]
        #print("r12", r12, "\nlen(r12) =", len(r12))

        x1 = [0 for i in range(len(r12))]
        x2 = [round((b*(ri-1))**(0.5), 3) for ri in r12]
        x3 = [round(-((b*(ri-1))**(0.5)), 3) for ri in r12]
        print("  r   x1   x2   x3")
        list(zip(r12[:5], x1[:5], x2[:5], x3[:5]))
```

```
          r   x1   x2   x3
Out[4]: [(1.0, 0, 0.0, -0.0),
         (1.38, 0, 1.744, -1.744),
         (1.76, 0, 2.466, -2.466),
         (2.14, 0, 3.02, -3.02),
         (2.52, 0, 3.487, -3.487)]
```

```
In [5]: plt.figure(figsize=(10,10))
        # plt.subplot(nrows, ncols, index)
        plt.subplot(2, 2, 1)
        plt.title("Графік x(r<1) при b = " f'{b}')
        plt.plot(r11, x11, label = "$x_1 = 0$")
        plt.xlabel("r")
        plt.ylabel("x")
        plt.legend(bbox_to_anchor=(1,1), loc= 'best')
        ax=[0, 1, -0.05, 0.05]
        plt.axis(ax)

        plt.subplot(2, 2, 2)
        plt.plot(r12, x1, label = "$x_1 = 0$")
        plt.plot(r12, x2, label = "$x_2 =  \sqrt{b(r-1)}$")
        plt.plot(r12, x3, label = "$x_3 = -\sqrt{b(r-1)}$")

        x0 = np.linspace(min(x3),max(x2),5)
        r0 = [1 for i in range(len(x0))]
        plt.title("Графік x(r>1) при b = " f'{b}')
        plt.plot(r0, x0, label = "$r = 1$", linestyle = 'dashed')
        plt.xlabel("r")
        plt.ylabel("x")
        plt.legend(bbox_to_anchor=(1,1), loc="upper left")
```

```
Out[5]: <matplotlib.legend.Legend at 0x15b9b539760>
```



*Тепер складемо **матрицю Якобі**, щоб проаналізувати стійкість стаціонарних станів.*

$$M = \begin{pmatrix} \frac{df(x,y)}{dx}|_{x_0,y_0} - \lambda & \frac{df(x,y)}{dy}|_{x_0,y_0} \\ \frac{dg(x,y)}{dx}|_{x_0,y_0} & \frac{dg(x,y)}{dy}|_{x_0,y_0} - \lambda \end{pmatrix}$$
$$det(M) = 0 \to \lambda$$

$A_{11} = \frac{df(x,y)}{dx}|_{x_0,y_0} - \lambda = (r - \frac{2y}{b}x)|_{x_0,y_0} - \lambda$
$A_{12} = \frac{df(x,y)}{dy}|_{x_0,y_0} = (-\frac{x^2}{b} - 1)|_{x_0,y_0}$
$A_{21} = \frac{dg(x,y)}{dx}|_{x_0,y_0} = -1$
$A_{22} = \frac{dg(x,y)}{dy}|_{x_0,y_0} = 1 - \lambda$

$det(M) = \begin{vmatrix} (r - \frac{2y}{b}x)|_{x_0,y_0} - \lambda & (-\frac{x^2}{b} - 1)|_{x_0,y_0} \\ -1 & 1-\lambda \end{vmatrix} \to [\text{Т. як } y = x] \to \begin{vmatrix} (r - \frac{2x^2}{b})|_{x_0,y_0} - \lambda & (-\frac{x^2}{b} - 1)|_{x_0,y_0} \\ -1 & 1-\lambda \end{vmatrix}$

$$\Delta = det(M) = \begin{vmatrix} (r - \frac{2x^2}{b})|_{x_0,y_0} - \lambda & (-\frac{x^2}{b} - 1)|_{x_0,y_0} \\ -1 & 1-\lambda \end{vmatrix}$$

Розглянемо різні стаціонарні стани.

- $\boldsymbol{y_1 = x_1 = 0}$

  $\Delta = det(M) = \begin{vmatrix} (r - \frac{2x^2}{b})|_{x_0,y_0} - \lambda & (-\frac{x^2}{b} - 1)|_{x_0,y_0} \\ -1 & 1-\lambda \end{vmatrix} = \begin{vmatrix} r-\lambda & -1 \\ -1 & 1-\lambda \end{vmatrix} = (r-\lambda)(1-\lambda) - (-1)(-1) = r - r\lambda - \lambda + \lambda^2 - 1 =$
  $= \lambda^2 - (r+1)\lambda + (r-1)$

  $\Delta = 0:$
  $\lambda^2 - (r+1)\lambda + (r-1) = 0$
  $D = b^2 - 4ac = (-(r+1))^2 - 4(r-1) = r^2 - 2r + 5$
  $y = r^2 - 2r + 5 - \text{парабола}$
  $r^2 - 2r + 5 = 0$
  $D_r = -16 < 0$
  $\text{Парабола не перетинає вісь X. Вітки направлені вгору, тому маємо :}$
  $D \neq 0 \text{ (при будь − яких значеннях параметру } r)$
  $D \geq 0 \text{ (також завжди тому немає обмежень для } r)$

  $\text{Т. як } D = r^2 - 2r + 5, \text{ то } \lambda = \frac{-b \pm \sqrt{D}}{2a}$

$$\lambda_{11,12} = \frac{r + 1 \pm \sqrt{r^2 - 2r + 5}}{2} = \frac{1}{2}(r + 1 \pm \sqrt{r^2 - 2r + 5})$$

- $\boldsymbol{y_{2,3} = x_{2,3} = \pm\sqrt{b(r-1)}} \to x_{2,3}^2 = b(r-1)$

$\Delta = det(M) = \begin{vmatrix} (r - \frac{2x^2}{b})|_{x_0,y_0} - \lambda & (-\frac{x^2}{b} - 1)|_{x_0,y_0} \\ -1 & 1-\lambda \end{vmatrix} = \begin{vmatrix} r - \frac{2b(r-1)}{b} - \lambda & -\frac{b(r-1)}{b} - 1 \\ -1 & 1-\lambda \end{vmatrix} = \begin{vmatrix} -r+2-\lambda & -r \\ -2 & 1-\lambda \end{vmatrix} =$
$= (2 - r - \lambda)(1-\lambda) - (-1)(-r) = \lambda^2 + (r-3)\lambda + (2 - 2r)$

$\Delta = 0:$
$\lambda^2 + (r-3)\lambda + (2-2r) = 0$
$D = (r-3)^2 - 4(2-2r) = r^2 + 2r + 1 = (r+1)^2 \geq 0 \to r \, \epsilon \, (-\infty, \infty)$
$\text{Т. як } D = (r+1)^2, \text{ то } \lambda = \frac{-b \pm \sqrt{D}}{2a}$
$\lambda_{21} = \frac{1}{2}(-(r-3) + \sqrt{(r+1)^2}) = 2$
$\lambda_{22} = \frac{1}{2}(-(r-3) - \sqrt{(r+1)^2}) = 1 - r$

Розглянемо різні значення параметру r. Параметр $b > 0$.

- $0 < r < 1:$
  $x_1 = y_1 = 0$
  Маємо тільки 1 стаціонарний стан, бо розглядаємо тільки дійсні розв'язки $(b(r-1)$ має бути $> 0)$

$$\lambda_{11,12} = \frac{1}{2}(r + 1 \pm \sqrt{r^2 - 2r + 5})$$

```
In [6]: import math
```

```
In [7]: arr_r = np.linspace(0.1, 1, 10, endpoint=False)
        arr_r = [round(ri,3) for ri in arr_r]
        print(" arr_r \n", arr_r, "\n")

        lambda_11 = [0.5*(r+1 + math.sqrt(r**2-2*r+5)) for r in arr_r]
        lambda_11 = [round(i,3) for i in lambda_11]
        print(" lambda_11 \n", lambda_11)

        lambda_12 = [0.5*(r+1 - math.sqrt(r**2-2*r+5)) for r in arr_r]
        lambda_12 = [round(i,3) for i in lambda_12]
        print(" lambda_12 \n", lambda_12)
```

```
 arr_r
 [0.1, 0.19, 0.28, 0.37, 0.46, 0.55, 0.64, 0.73, 0.82, 0.91]

 lambda_11
 [1.647, 1.674, 1.703, 1.733, 1.766, 1.8, 1.836, 1.874, 1.914, 1.956]
 lambda_12
 [-0.547, -0.484, -0.423, -0.363, -0.306, -0.25, -0.196, -0.144, -0.094, -0.046]
```

Як бачимо при $r < 1$ маємо коефіцієнти $\lambda_{11} > 0, \; \lambda_{12} < 0$ - дійсні корені різних знаків.

Тому маємо тип стаціонарної точки **сідло**.

- $r > 1:$
  $x_1 = y_1 = 0$

$$\lambda_{21,22} = \frac{1}{2}(r + 1 \pm \sqrt{r^2 - 2r + 5})$$

  $x_{2,3} = y_{2,3} = \pm\sqrt{b(r-1)}$

$$\lambda_{31} = 2$$
$$\lambda_{32} = 1 - r$$

Маємо 3 стаціонарні стани.

```
In [8]: arr_r2 = np.linspace(1.1, 20, 15)
        arr_r2 = [round(ri,3) for ri in arr_r2]
        print(" arr_r2 \n", arr_r2, "\n")

        lambda_21 = [0.5*(r+1 + math.sqrt(r**2-2*r+5)) for r in arr_r2]
        lambda_21 = [round(i,3) for i in lambda_21]
        print(" lambda_21 \n", lambda_21)

        lambda_22 = [0.5*(r+1 - math.sqrt(r**2-2*r+5)) for r in arr_r2]
        lambda_22 = [round(i,3) for i in lambda_22]
        print(" lambda_22 \n", lambda_22)
```

```
 arr_r2
 [1.1, 2.45, 3.8, 5.15, 6.5, 7.85, 9.2, 10.55, 11.9, 13.25, 14.6, 15.95, 17.3, 18.65, 20.0]

 lambda_21
 [2.051, 2.96, 4.12, 5.378, 6.676, 7.993, 9.32, 10.654, 11.991, 13.331, 14.673, 16.017, 17.361, 18.706, 20.052]
 lambda_22
 [0.049, 0.49, 0.68, 0.772, 0.824, 0.857, 0.88, 0.896, 0.909, 0.919, 0.927, 0.933, 0.939, 0.944, 0.948]
```

Як бачимо при $r > 1$ $(x_1 = y_1 = 0)$ маємо коефіцієнти $\lambda_{21} > 0, \; \lambda_{22} > 0$ - дійсні додатні корені.

Тому маємо тип стаціонарної точки **нестійкий вузол**.

```
In [9]: lambda_31 = [2 for r in arr_r2]
        print(" lambda_31 \n", lambda_31)

        lambda_32 = [1-r for r in arr_r2]
        lambda_32 = [round(i,3) for i in lambda_32]
        print(" lambda_32 \n", lambda_32)
```

```
 lambda_31
 [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
 lambda_32
 [-0.1, -1.45, -2.8, -4.15, -5.5, -6.85, -8.2, -9.55, -10.9, -12.25, -13.6, -14.95, -16.3, -17.65, -19.0]
```

Як бачимо при $r > 1$ $(x_{2,3} = y_{2,3} = \pm\sqrt{b(r-1)})$ маємо коефіцієнти $\lambda_{31} > 0, \; \lambda_{32} < 0$ - дійсні корені різних знаків.

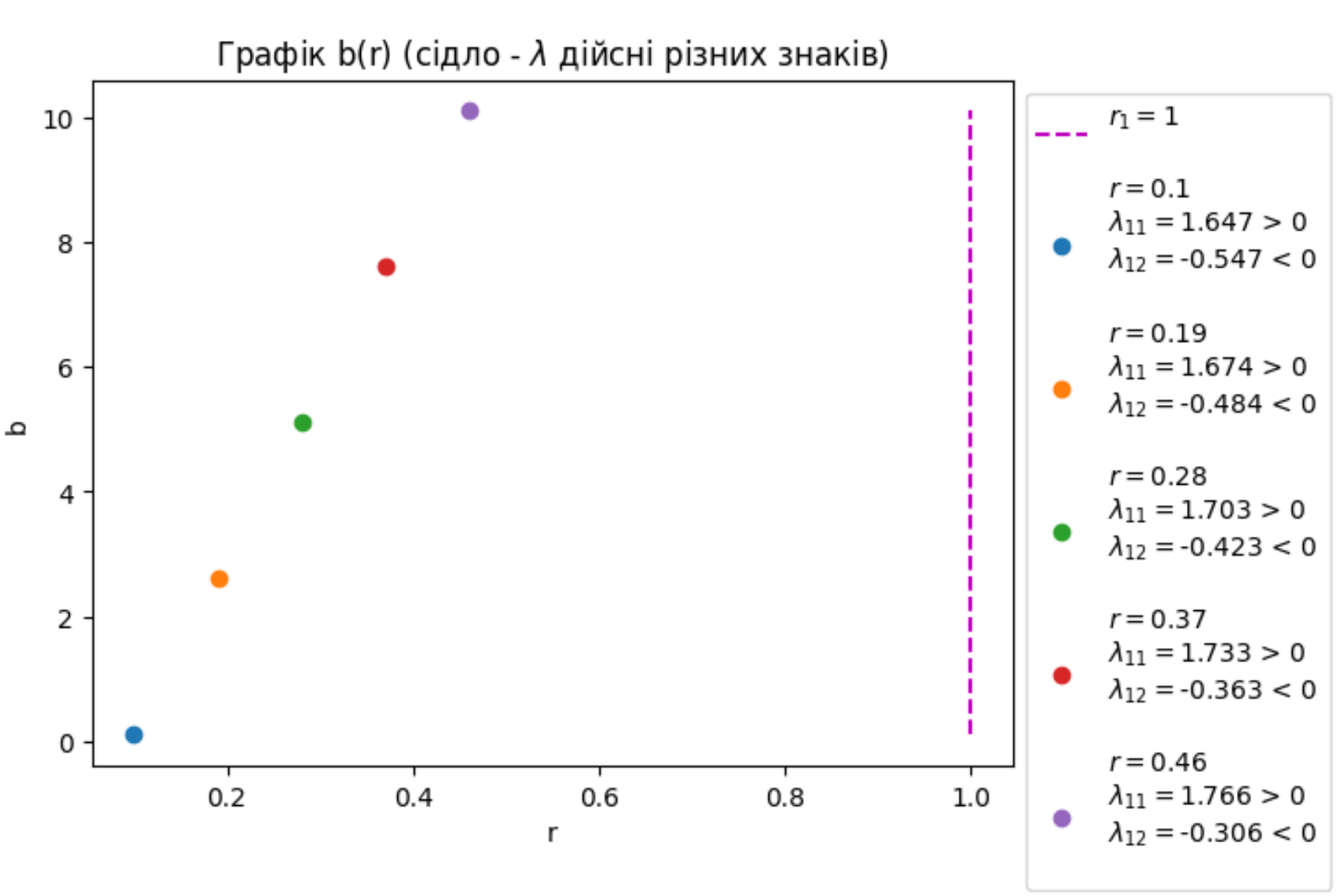
Тому маємо тип стаціонарної точки **сідло**.

Діаграма та динаміка системи

```
In [10]: b1 = np.linspace(0.1, 10.1, 5)
         r1 = [1 for i in range(len(b1))]
         #print("b1", b1[:10], "\nlen(b1) =", len(b1))
         #print("r1", r1[:10], "\nlen(r1) =", len(r1))
```

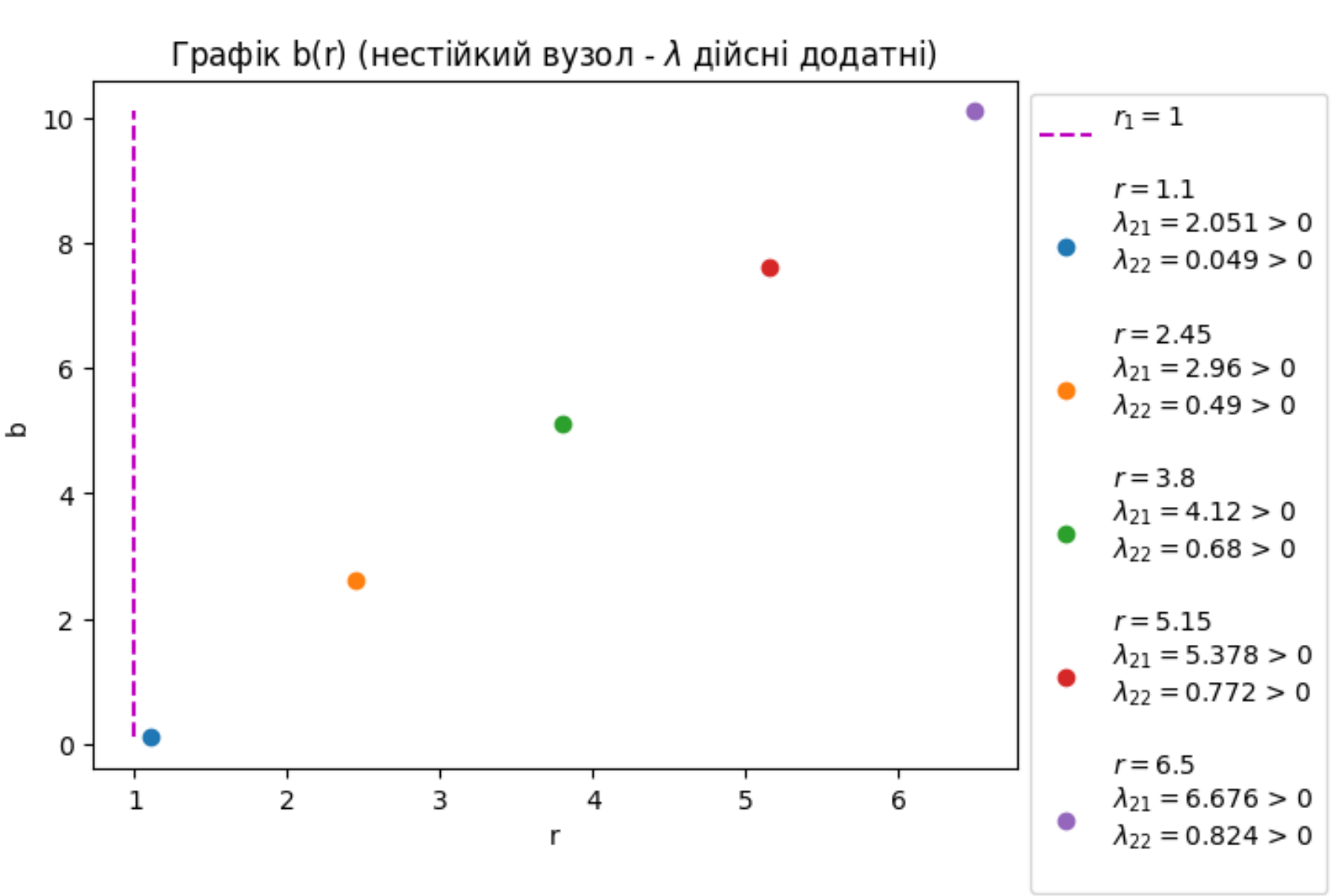
```
In [11]: plt.plot(r1, b1,'m--', label = "$r_1=1$\n")
         for i in range(0, len(b1)):
             plt.scatter(arr_r[i], b1[i], label = "$r=$" f'{arr_r[i]}' "\n$\lambda_{11}=$" f'{lambda_11[i]}' " > 0"
                         "\n$\lambda_{12}=$" f'{lambda_12[i]}' " < 0 \n")
             #plt.text(r_new[i], b1[i], "text1")

         plt.title("Графік b(r) (сідло - $\lambda$ дійсні різних знаків)")
         plt.legend(bbox_to_anchor=(1,1), loc="upper left")
         plt.xlabel("r")
         plt.ylabel("b")
```

```
Out[11]: Text(0, 0.5, 'b')
```



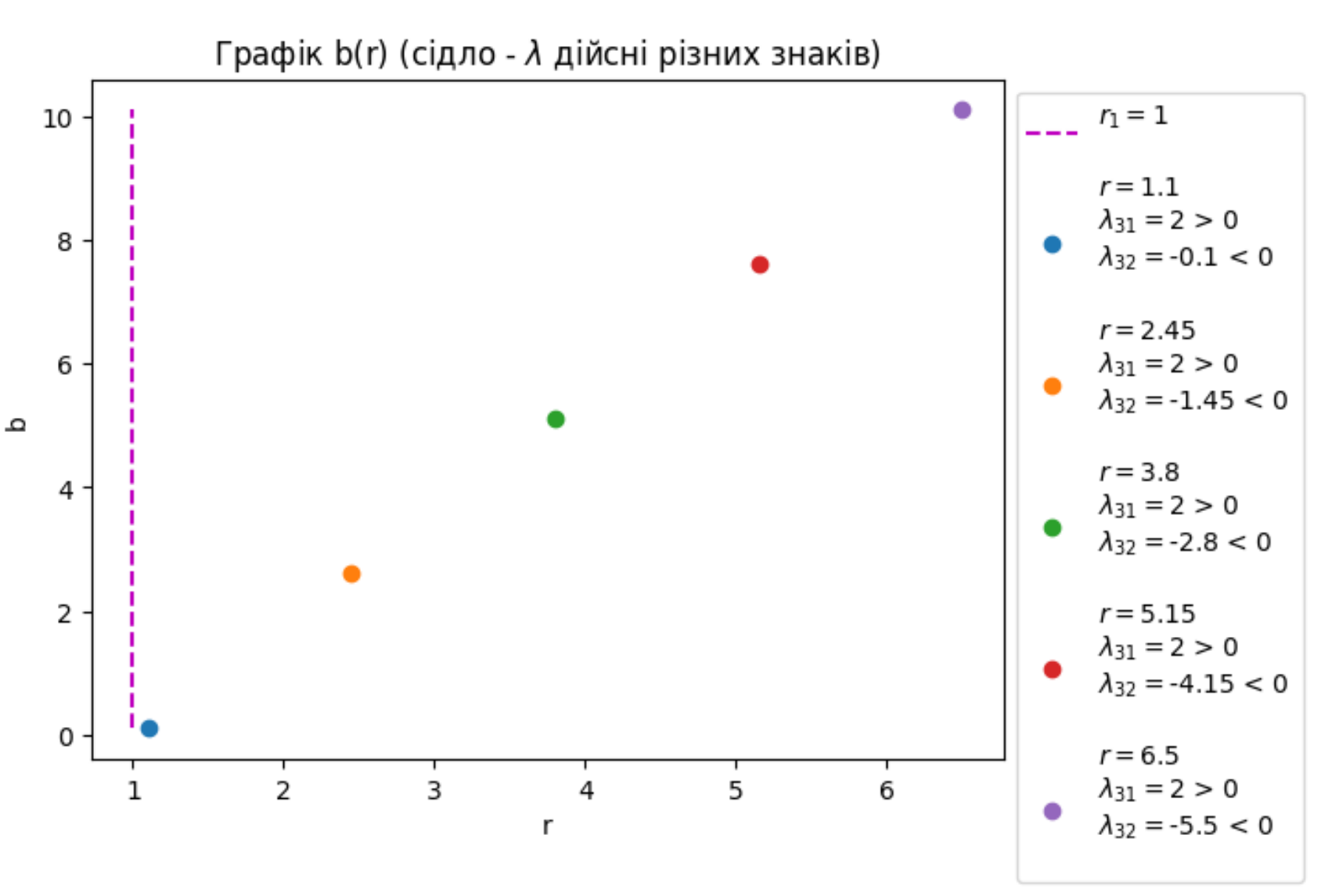
```
In [12]: plt.plot(r1, b1, 'm--', label = "$r_1=1$\n")

         for i in range(0, len(b1)):
             plt.scatter(arr_r2[i], b1[i], label = "$r=$" f'{arr_r2[i]}' "\n$\lambda_{21}=$" f'{lambda_21[i]}' " > 0"
                         "\n$\lambda_{22}=$" f'{lambda_22[i]}' " > 0 \n")
             #plt.text(r_new[i], b1[i], "text1")

         plt.title("Графік b(r) (нестійкий вузол - $\lambda$ дійсні додатні)")
         plt.legend(bbox_to_anchor=(1,1), loc="upper left")
         plt.xlabel("r")
         plt.ylabel("b")
```

```
Out[12]: Text(0, 0.5, 'b')
```



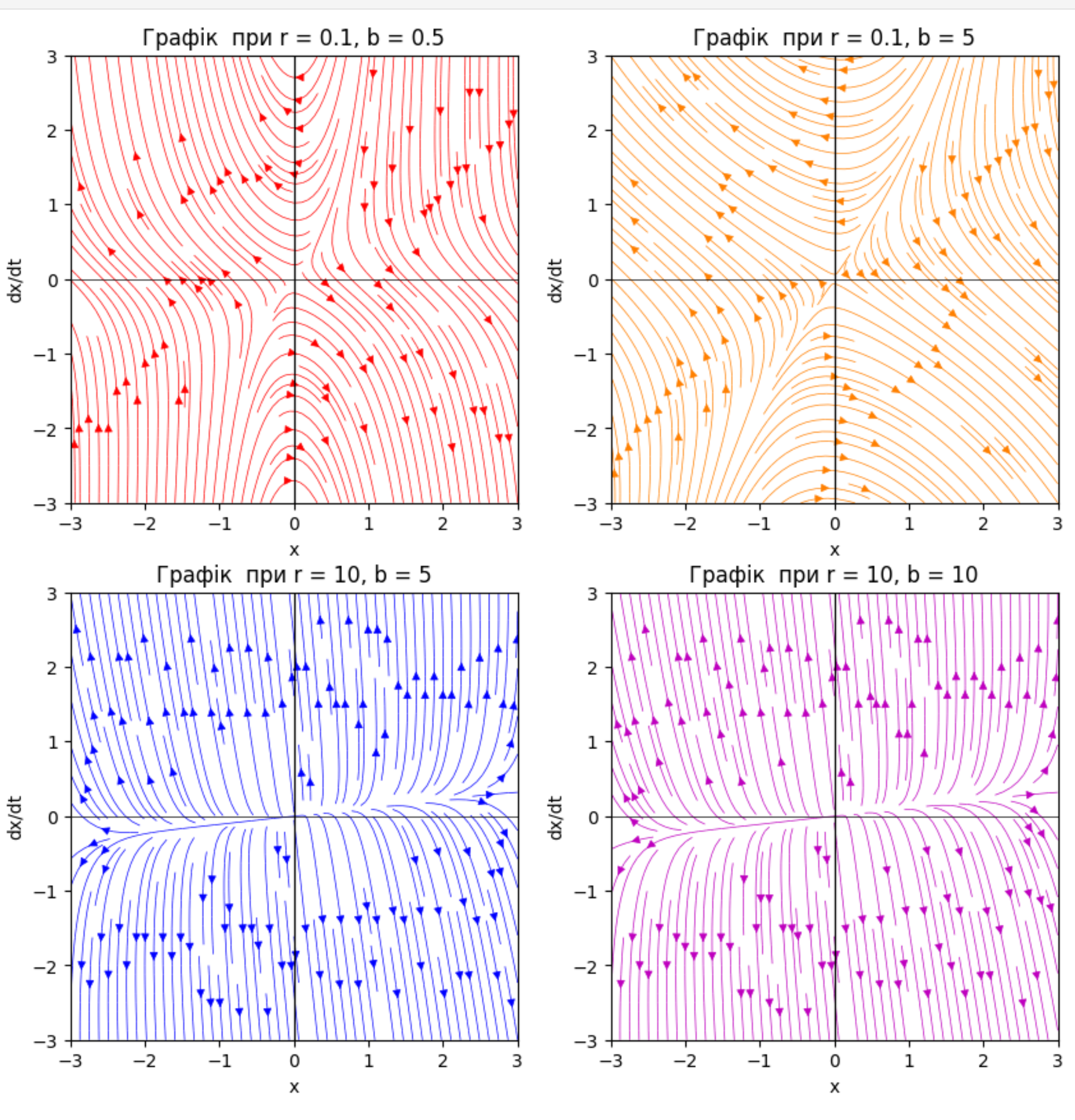
```
In [13]: plt.plot(r1, b1, 'm--', label = "$r_1=1$\n")

         for i in range(0, len(b1)):
             plt.scatter(arr_r2[i], b1[i], label = "$r=$" f'{arr_r2[i]}' "\n$\lambda_{31}=$" f'{lambda_31[i]}' " > 0"
                         "\n$\lambda_{32}=$" f'{lambda_32[i]}' " < 0 \n")
             #plt.text(r_new[i], b1[i], "text1")

         plt.title("Графік b(r) (сідло - $\lambda$ дійсні різних знаків)")
         plt.legend(bbox_to_anchor=(1,1), loc="upper left")
         plt.xlabel("r")
         plt.ylabel("b")
```

```
Out[13]: Text(0, 0.5, 'b')
```



Далі побудуємо фазові портрети на основі системи динамічних рівнянь:
$\begin{cases} \frac{dy}{dt} = xr - \frac{x^2 y}{b} - y = f(x,y,b,r) \\ \frac{dx}{dt} = y - x = g(x,y,b,r) \end{cases}$

```
In [14]: def dF1(y, x, r, b):
             return x*r-(x**2 * y)/b - y,  y-x

         def fun_plotPhasePortrait(r, b, colors='m'):
             #print("b ",b, "\nr ", r, "\n")
             Y, X = np.meshgrid(np.linspace(-3.0, 3.0, 30), np.linspace(-3.0, 3.0, 30))
             U, V = dF1(Y, X, r, b)
             #plt.streamplot(Y, X, U, V, color=colors, linewidth=0.5, density=1.6)
             plt.streamplot(Y, X, V, U, color=colors, linewidth=0.5, density=1.6)
             j=3
             plt.hlines(0, -j, j, 'k', linewidth=0.5) # label='y=0'
             plt.vlines(0, -j, j, 'k', linewidth=0.9) # label='x=0'
             #plt.legend(bbox_to_anchor=(1,1), loc="lower left") #loc="upper left"
             plt.title("Графік  при r = " f'{r}' ", b = " f'{b}')
             plt.axis('square')
             plt.axis([-j, j, -j, j])
             plt.xlabel("x")
             plt.ylabel("dx/dt")
             #plt.grid()
             #plt.show()

         plt.figure(figsize=(10,10))
         plt.subplot(2,2,1)
         fun_plotPhasePortrait(r=0.1, b=0.5, colors='r')       # color = red
         plt.subplot(2,2,2)
         fun_plotPhasePortrait(r=0.1, b=5, colors=[1, 0.5, 0]) # color = orange
         plt.subplot(2,2,3)
         fun_plotPhasePortrait(r=10, b=5, colors='b')          # color = blue
         plt.subplot(2,2,4)
         fun_plotPhasePortrait(r=10, b=10)
```



Отримали при $r < 1$ сідло, а $r > 1$ - нестійкий вузол.

Комп'ютерне моделювання задач прикладної математики

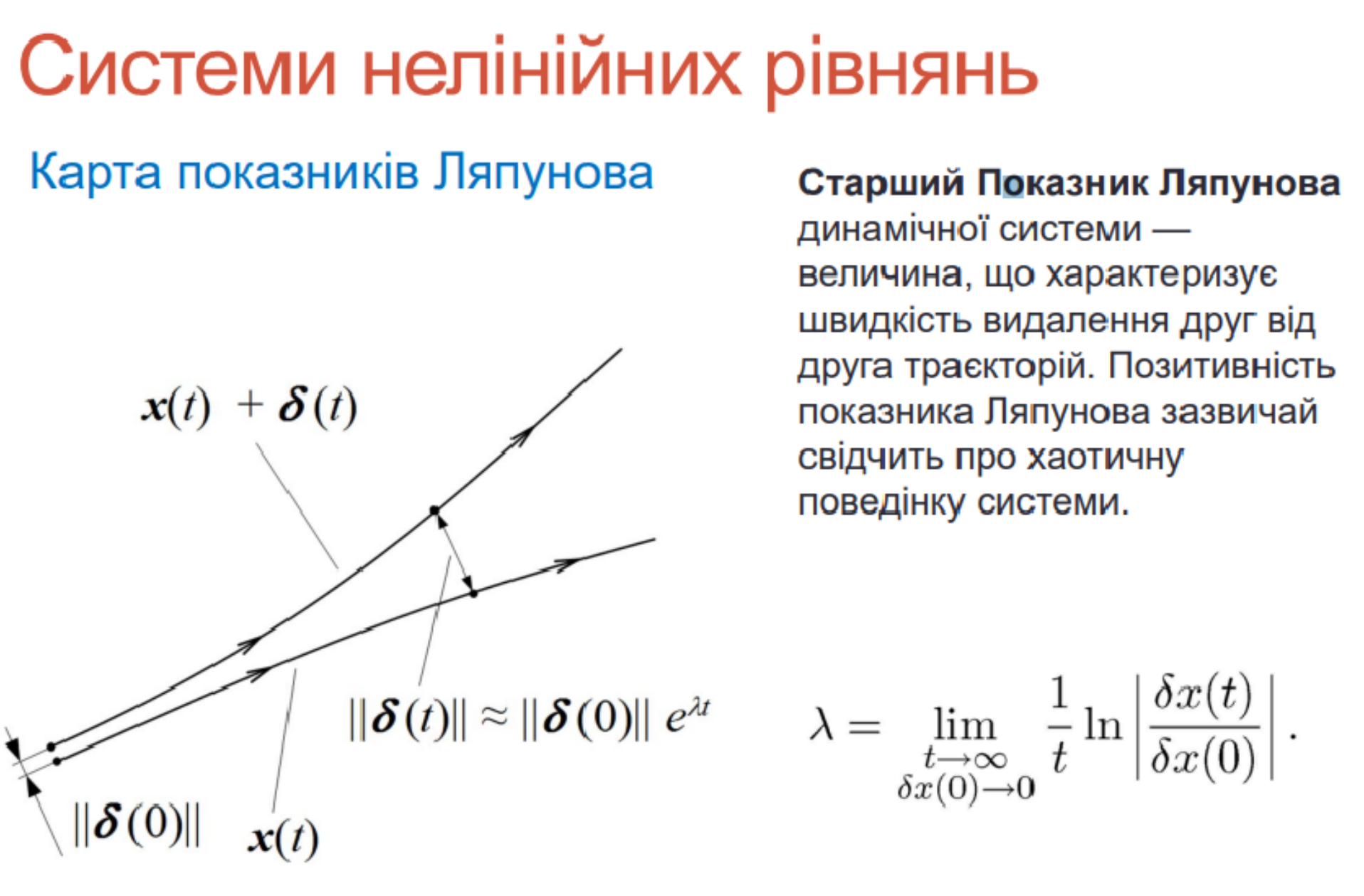
Динамічні системи.

Лабораторна робота 3

Студентка Пороскун Олена. Група ПМ.м-21

Варіант 8

Теорія

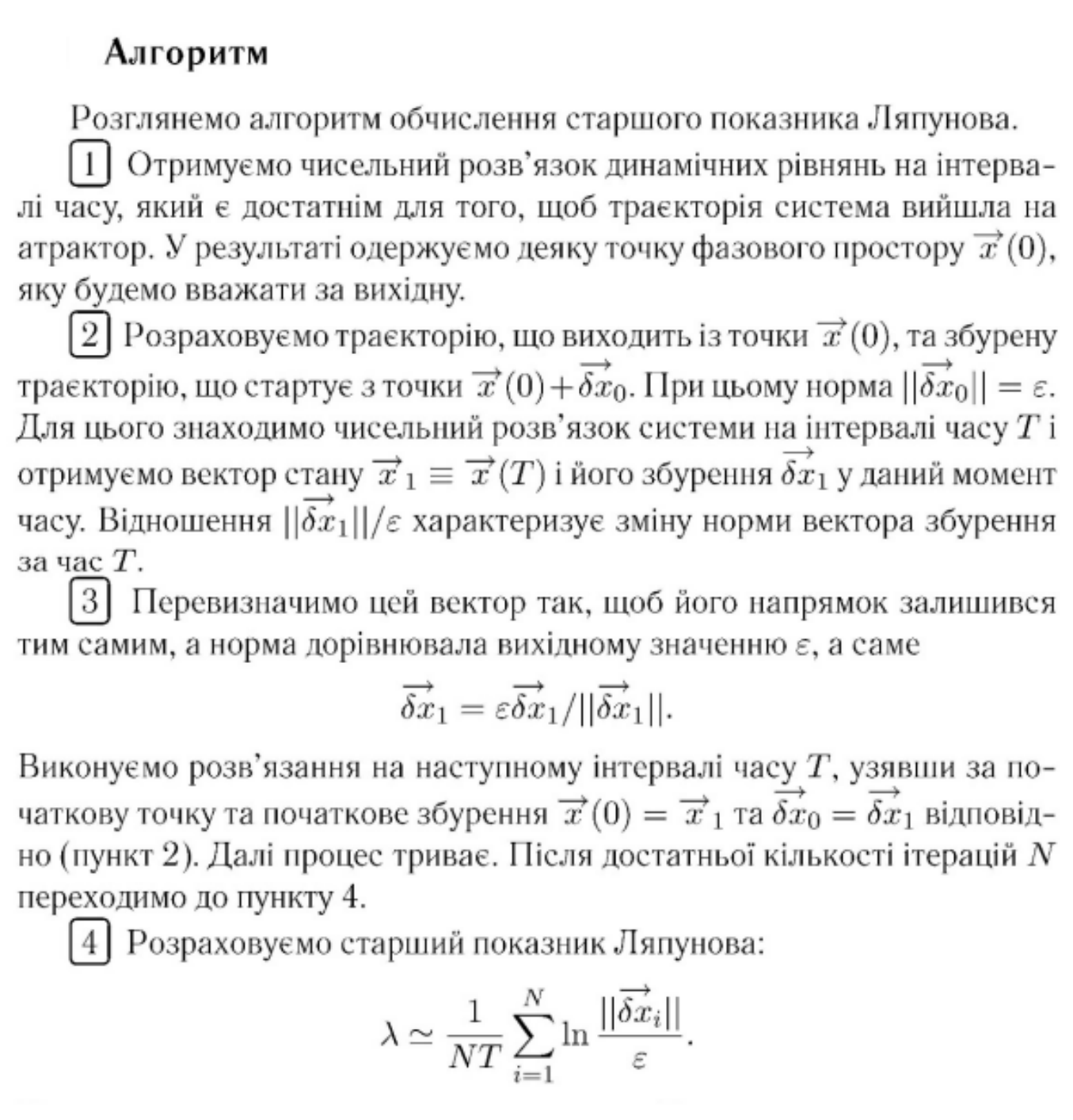


Системи нелінійних рівнянь

Карта показників Ляпунова

Старший Показник Ляпунова динамічної системи — величина, що характеризує швидкість видалення друг від друга траєкторій. Позитивність показника Ляпунова зазвичай свідчить про хаотичну поведінку системи.

$$\lambda = \lim_{\substack{t\to\infty \\ \delta x(0)\to 0}} \frac{1}{t} \ln \left| \frac{\delta x(t)}{\delta x(0)} \right| .$$



Алгоритм

Розглянемо алгоритм обчислення старшого показника Ляпунова.

$\boxed{1}$ Отримуємо чисельний розв'язок динамічних рівнянь на інтервалі часу, який є достатнім для того, щоб траєкторія система вийшла на атрактор. У результаті одержуємо деяку точку фазового простору $\vec{x}(0)$, яку будемо вважати за вихідну.

$\boxed{2}$ Розраховуємо траєкторію, що виходить із точки $\vec{x}(0)$, та збурену траєкторію, що стартує з точки $\vec{x}(0)+\vec{\delta x}_0$. При цьому норма $||\vec{\delta x}_0|| = \varepsilon$. Для цього знаходимо чисельний розв'язок системи на інтервалі часу T і отримуємо вектор стану $\vec{x}_1 \equiv \vec{x}(T)$ і його збурення $\vec{\delta x}_1$ у даний момент часу. Відношення $||\vec{\delta x}_1||/\varepsilon$ характеризує зміну норми вектора збурення за час T.

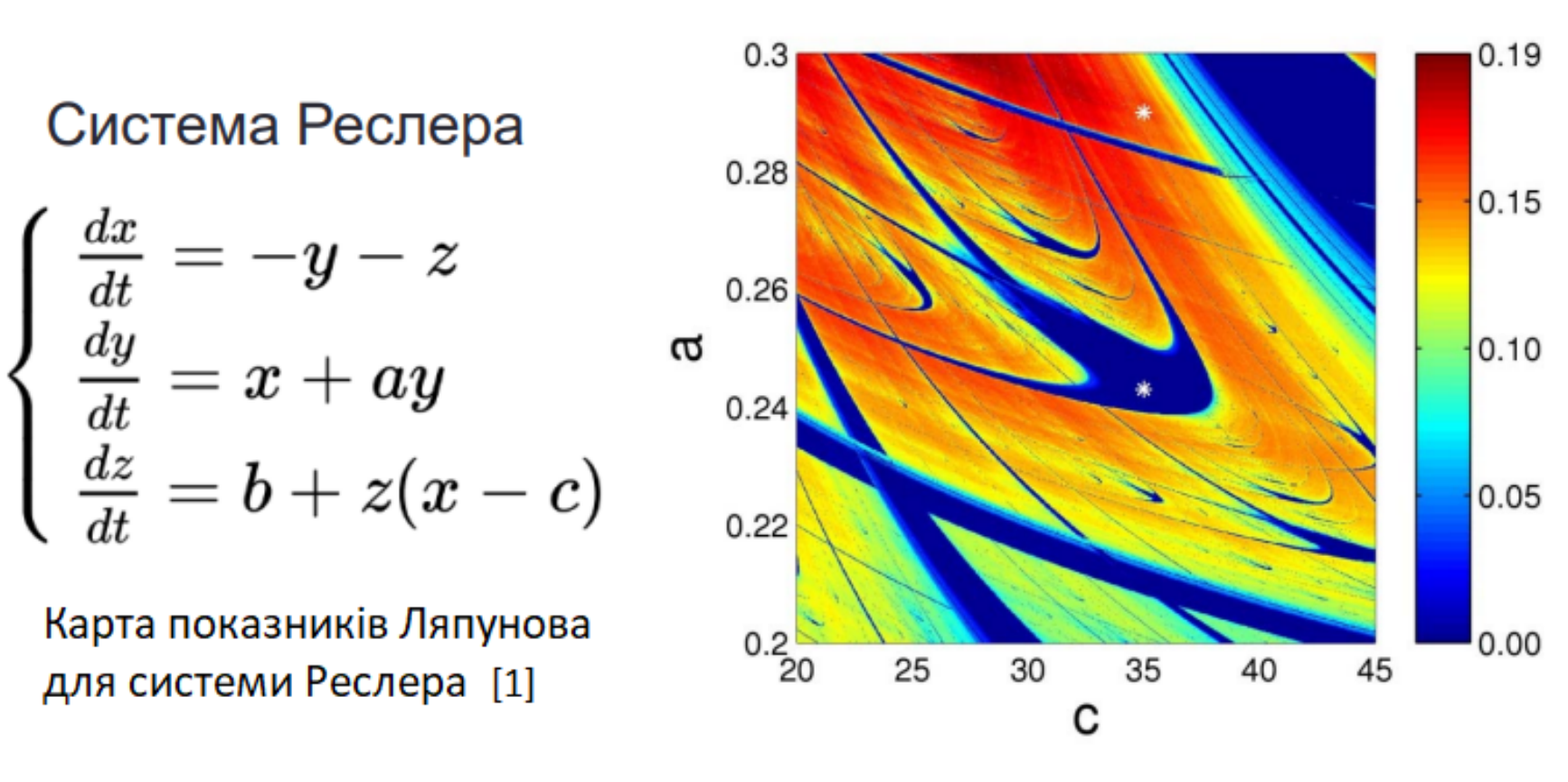
$\boxed{3}$ Перевизначимо цей вектор так, щоб його напрямок залишився тим самим, а норма дорівнювала вихідному значенню ε, а саме

$$\vec{\delta x}_1 = \varepsilon \vec{\delta x}_1 / ||\vec{\delta x}_1||.$$

Виконуємо розв'язання на наступному інтервалі часу T, узявши за початкову точку та початкове збурення $\vec{x}(0) = \vec{x}_1$ та $\vec{\delta x}_0 = \vec{\delta x}_1$ відповідно (пункт 2). Далі процес триває. Після достатньої кількості ітерацій N переходимо до пункту 4.

$\boxed{4}$ Розраховуємо старший показник Ляпунова:

$$\lambda \simeq \frac{1}{NT} \sum_{i=1}^{N} \ln \frac{||\vec{\delta x}_i||}{\varepsilon}.$$



Система Реслера

$$\begin{cases} \dfrac{dx}{dt} = -y - z \\ \dfrac{dy}{dt} = x + ay \\ \dfrac{dz}{dt} = b + z(x - c) \end{cases}$$

Карта показників Ляпунова для системи Реслера [1]

[1] Zou, Yong & Donner, Reik & Donges, Jonathan & Marwan, Norbert & Kurths, Juergen. (2010). Identifying complex periodic windows in continuous-time dynamical systems using recurrence-based methods. Chaos (Woodbury, N.Y.). 20. 043130. 10.1063/1.3523304.

Умова

6-10: система 2)

2)

$$\begin{cases} \dot{x} &= -z \\ \dot{y} &= x - y \\ \dot{z} &= \alpha x + y^2 + \beta z \end{cases}$$

Р-ня

In [1]:

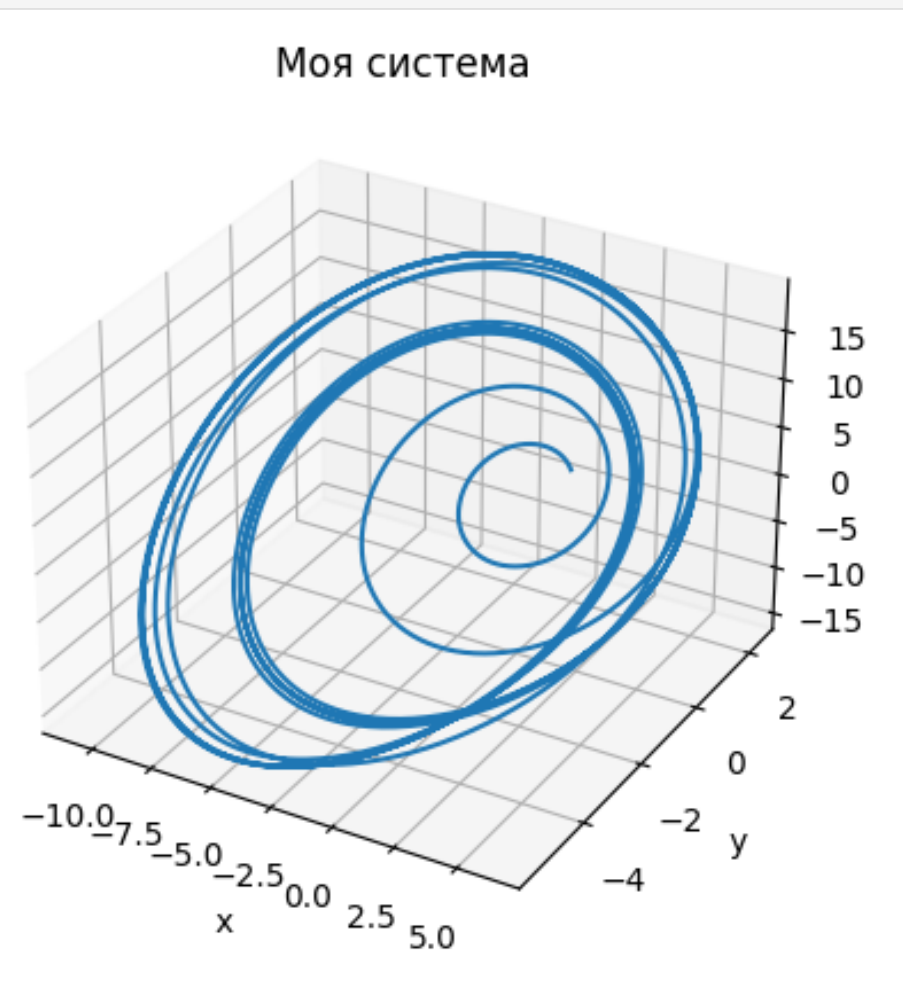
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from mpl_toolkits.mplot3d import Axes3D
from numpy.linalg import norm
```

1 пункт

Потрібно отримати чисельний розв'зок динамічних рівнянь на деякому інтервалі часу(траєкторія має вийти на атрактор). Для цього створимо функцію $fun1(alpha, beta, strplot)$, де $alpha, beta$ - параметри системи, $strplot$ - змінна, що показує чи треба будувати графік атрактора чи ні.("plot" і для того щоб графік не будувався достатньо ввести будь-яку іншу змінну). Ця функція на виході дає $states$ - дані кривої атрактора.

In [2]:

```python
def fun1(alpha = 5, beta = 0.6, str_plot = "plot"):
    # alpha = 5  # від 5 нормально (навіть 100)
    # beta  = 0.6 # до 0.6 нормально, потім ні
    #print('fun1')

    def f(state, t):
        x, y, z = state                          # Unpack the state vector
        return -z, x-y, alpha*x + y*y + beta*z   # Derivatives

    state0 = [1.0, 1.0, 1.0]
    t = np.arange(0.0, 40.0, 0.01)

    states = odeint(f, state0, t)

    if (str_plot == "plot"):
        fig = plt.figure()
        ax = fig.add_subplot(projection='3d')
        ax.plot(states[:, 0], states[:, 1], states[:, 2])
        plt.title("Моя система")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.draw()
        plt.show()

    return states

vector_0 = fun1()
```



Далі, щоб отримати деяку точку фазового простору $\vec{x}(0)$, скористаємося функцією $fun\ last\ el(array)$ - на виході дає останні значення стовпчиків масиву.

In [3]:

```python
def fun_last_el(array):
    #print('fun_last_el')
    end = len(array)
    return [array[end-1, i] for i in range(3)]
```

2 пункт

Тепер розраховуємо траєкторію з точки $\vec{x}(0)$ та збурену траєкторію з точки $\vec{x}(0) + \vec{\delta x_0}$. Знайдемо норму $\|\vec{\delta x_0}\| = \varepsilon$. Далі беремо період часу T = 100 і отримуємо вектор $\vec{x}_1 \equiv \vec{x}(T)$ і його збурення $\vec{\delta x_1}$ у даний момент часу.

Функція $fun2(alpha, beta, state0, T)$ поверне одразу останню точку траєкторії.

Її ж використовуємо і для функції $fun\ norm\ deltx1(vect0, vect\_\ deltx0, N, T, eps)$, що повертає масив норм $\vec{\delta x_1}$. Уже в цій функції ми переходимо до пункту 3.

3 пункт

Перевизначеємо вектор $\vec{\delta x_1} = \varepsilon\ \vec{\delta x_1} / \|\vec{\delta x_1}\|$.

Далі ми знову інтегруємо на наступному інтервалі T, взявши $\vec{\delta x(0)} = \vec{\delta x_1}$ та $\vec{\delta x_0} = \vec{\delta x_1}$ (тобто знову переходимо до ***пункту 2***, але з новими вхідними даними). Це відбувається у циклі з N ітерацій.

Тобто, на виході функції $fun\ norm\ deltx1$ ми маємо масив норм $\|\vec{\delta x_i}\|$, що необхідні для розрахунку **старшого показника Ляпунова**.

In [4]:

```python
def fun2(alpha = 5, beta = 0.6, state0 = [1.0, 1.0, 1.0], T=100):
    #print('fun2')
    def f(state, t):
        x, y, z = state                          # Unpack the state vector
        return -z, x-y, alpha*x + y*y + beta*z   # Derivatives
    t = np.arange(0.0, T, 0.01)
    states = odeint(f, state0, t)
    return fun_last_el(states)
```

In [5]:

```python
def fun_norm_deltx1(vect0, vect_deltx0, N=50, T=100, eps=0.001):
    #print('fun_norm_deltx1')
    array_norm_deltx1 = []
    for n in range(1,N+1):
        # проінтегруємо з проміжком часу T=100
        vect1       = fun2(state0 = vect0,       T=T)
        vect_deltx1 = fun2(state0 = vect_deltx0, T=T)
        # знайдемо збурення deltx1
        deltx1 = [vect_deltx1[i] - vect1[i] for i in range(3)]
        deltx1_no_ovveride = deltx1

        # додамо до масиву норм дельт значення ще НЕ перевизначеного вектору
        array_norm_deltx1.append(norm(deltx1_no_ovveride))

        # друк проміжних результатів
        '''
        print('n', n)
        print('vect1       ', vect1)
        print('vect_deltx1 ', vect_deltx1)
        '''
        # Перевизначимо вектор  за формулою
        # deltx1 = [eps*deltx1 / norm(deltx1)
        deltx1 = [eps*deltx1[i] / norm(deltx1) for i in range(3)]
        '''
        print('deltx1(до перевизначення) ', deltx1_no_ovveride)
        print(' norm(deltx1) ', norm(deltx1))
        print('deltx1(після перевизначення) ', deltx1)
        '''
        vect_deltx11 = [vect1[i] + deltx1[i] for i in range(3)]

        # знайдемо норму вектора deltx1 (перевизначеного)
        #norm_deltx1   = norm(deltx1)
        #array_norm_deltx1.append(norm_deltx1)

        # запишемо у початкові дані тепер знайдені величини (повернення до пункту 2)
        vect0       = vect1
        vect_deltx0 = vect_deltx11


    return array_norm_deltx1
```

4 пункт

Розраховуємо старший показник Ляпунова:

$$\lambda \simeq \frac{1}{NT} \sum_{i=1}^{N} \ln \frac{\|\vec{\delta x_i}\|}{\varepsilon}$$

In [6]:

```python
def fun_lyapunov_exponent(norm_deltx1=[0], T=100, N=50, eps=0.001):
    #print('fun_lyapunov_exponent')
    #print('T =', T, ', N =', N)
    len_ = len(norm_deltx1)
    ln = [np.log(norm_deltx1[i]) / eps  for i in range(len_)]
    sum_ln = np.sum(ln)
    lyapunov = sum_ln / (N*T)
    #print('lyapunov =', lyapunov)
    return lyapunov
```

Отже, тепер використаємо функції описані вище і обчислимо значення старшого показника Ляпунова для кожного зі значень в масиві $alpha, beta$ за допомогою функції $fun\ params\ and\ lyapunov(T, N, arr\ alpha, arr\_\ beta)$.

In [7]:

```python
def fun_params_and_lyapunov(T=100, N=10, arr_alpha=[5], arr_beta=[0.6]):
    #print('fun_ans_params_and_lyapunov')
    #array_ans = ['alpha', 'beta', 'lyap']
    array_ans = [[], [], []]
    for alpha_ in arr_alpha:
        for beta_ in arr_beta:
            '''
            print('\n початок ')
            print('alpha_ ', alpha_)
            print('beta_  ', beta_)
            '''
            vector_0 = fun1(alpha = alpha_, beta = beta_, str_plot = "no_plot")
            vect0 = fun_last_el(vector_0)
            deltx0 = [0.001 for i in vect0]
            vect_deltx0 = [i + deltx0[0] for i in vect0]
            eps = norm(deltx0)
            '''
            print('vect0 ', vect0)
            print('deltx0 ', deltx0)
            print('vect_deltx0 ', vect_deltx0)
            print("eps = norm(delta) =", eps)
            print()
            '''
            array_norm_deltx1 = fun_norm_deltx1(vect0, vect_deltx0, N=N, T=T, eps=eps)
            lyapunov = fun_lyapunov_exponent(array_norm_deltx1, T=T, N=N, eps=eps)

            array_ans[0].append(alpha_)
            array_ans[1].append(beta_)
            array_ans[2].append(lyapunov)
    return array_ans
```

In [8]:

```python
import time
start_time = time.time()

#alphas = np.linspace(5, 9, 10)
#betas  = np.linspace(0.1, 0.3, 10)

count_points = 25

alphas = np.linspace(5, 14, count_points)
betas  = np.linspace(0.15, 0.45, count_points)

alphas = np.round(alphas, 6)
betas  = np.round(betas, 6)

array_lyapunov = fun_params_and_lyapunov(T=100, N=10, arr_alpha=alphas, arr_beta=betas)
print("--- %s seconds ---" % (time.time() - start_time))
```

```
C:\Users\Admin\anaconda3\lib\site-packages\scipy\integrate\_odepack_py.py:247: ODEintWarning: Excess work done on this call (perhaps wrong Dfun type). Run with full_output = 1 to get quanti
tative information.
  warnings.warn(warning_msg, ODEintWarning)
C:\Users\Admin\AppData\Local\Temp\ipykernel_4884\999960437.py:23: RuntimeWarning: invalid value encountered in double_scalars
  deltx1 = [eps*deltx1[i] / norm(deltx1) for i in range(3)]
C:\Users\Admin\anaconda3\lib\site-packages\scipy\integrate\_odepack_py.py:247: ODEintWarning: Illegal input detected (internal error). Run with full_output = 1 to get quantitative informati
on.
  warnings.warn(warning_msg, ODEintWarning)
C:\Users\Admin\AppData\Local\Temp\ipykernel_4884\546411691.py:5: RuntimeWarning: divide by zero encountered in log
  ln = [np.log(norm_deltx1[i]) / eps  for i in range(len_)]
C:\Users\Admin\anaconda3\lib\site-packages\scipy\integrate\_odepack_py.py:247: ODEintWarning: Excess accuracy requested (tolerances too small). Run with full_output = 1 to get quantitative
information.
  warnings.warn(warning_msg, ODEintWarning)
--- 105.23503088951111 seconds ---
```

Далі побудуємо теплову карту з показниками Ляпунова. Вісь X - параметри $alpha$, Y - параметри $beta$.

In [9]:

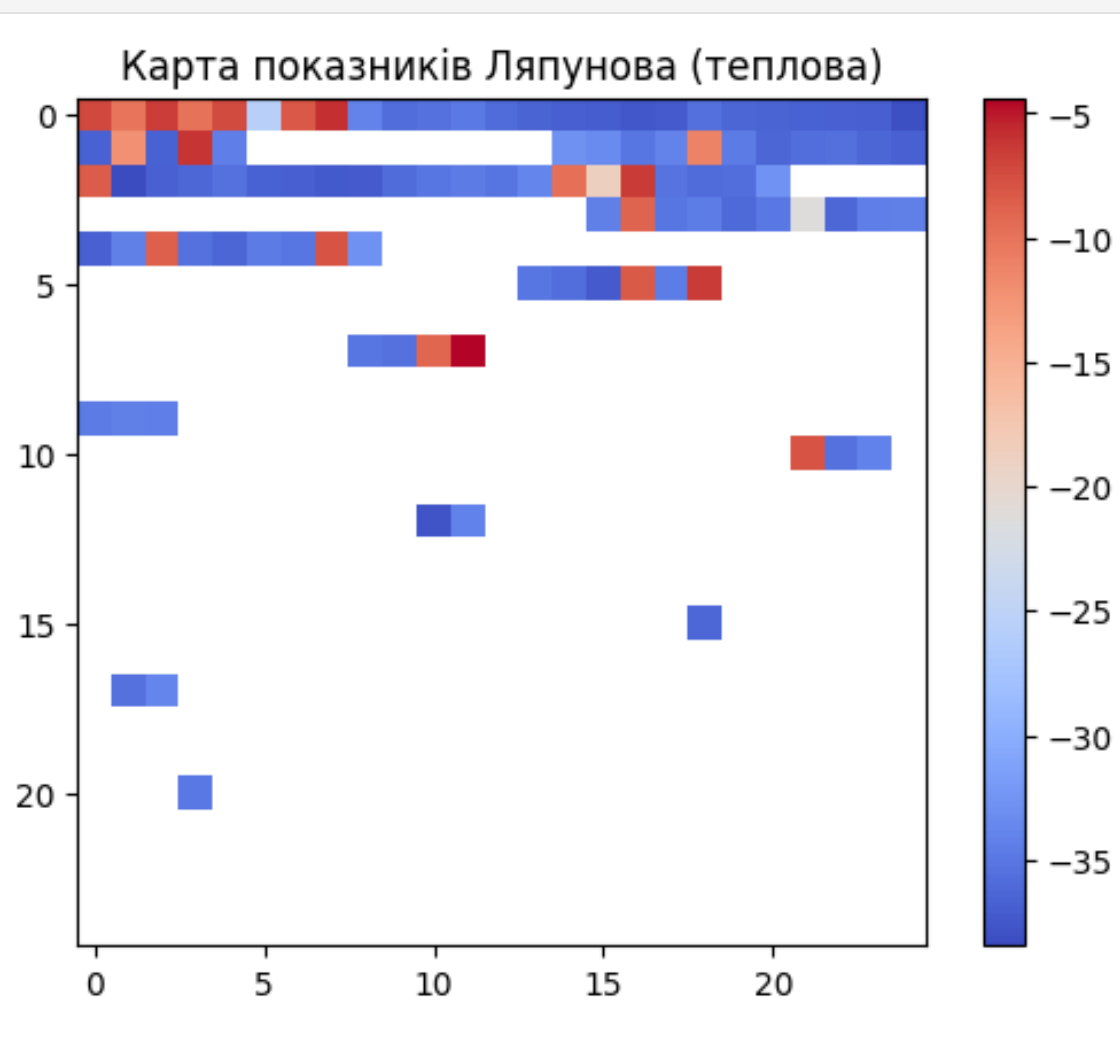
```python
lyap = array_lyapunov[:][2]
np.shape(lyap)
```

Out[9]:

```
(625,)
```

In [10]:

```python
from matplotlib import cm

# Make plot with vertical (default) colorbar
fig, ax = plt.subplots()

lyap = array_lyapunov[2][:]
#lyap = np.linspace(-20, 20, 100)
data = np.resize(lyap, (count_points,count_points))

cax = ax.imshow(data, interpolation='nearest', cmap=cm.coolwarm)
ax.set_title('Карта показників Ляпунова (теплова)') # with vertical colorbar

# Add colorbar, make sure to specify tick locations to match desired ticklabels
cbar = fig.colorbar(cax)
#cbar = fig.colorbar(cax, ticks=[-1, 0, 1])
#cbar.ax.set_yticklabels(['< -1', '0', '> 1'])  # vertically oriented colorbar

plt.show()
```



In [11]:

```python
np.shape(array_lyapunov)
```

Out[11]:

```
(3, 625)
```

In [12]:

```python
lyap[:30]
```

Out[12]:

```
[-7.23921152929229,
 -10.175264943087033,
 -6.546225924778387,
 -9.952618107919623,
 -7.432475171722499,
 -25.612178913188917,
 -8.241367149256446,
 -5.8559079540493695,
 -33.97546566762948,
 -35.86412686603707,
 -35.425451633436936,
 -34.83035965875268,
 -35.96576200051633,
 -36.56103010975358,
 -36.93500596435278,
 -37.08183352471553,
 -37.42650164939994,
 -37.17843291255186,
 -35.59290662058288,
 -36.37749843147974,
 -36.54767271851105,
 -36.64738451043416,
 -36.747633749105276,
 -36.94490447618703,
 -38.13510977987174,
 -36.623385051513885,
 -12.18429496454322,
 -36.652350335419264,
 -6.235595376392586,
 -34.37169224023317]
```

In [13]:

```python
import seaborn as sns
import pandas as pd
```

Створимо датафрейм для виводу через бібліотеку **seaborn**.

In [14]:

```python
'''
data_frame = pd.DataFrame(np.array([[lyapunov_11, lyapunov_12, ...],
                                    [lyapunov_21, lyapunov_22, ...]]),
                          columns = ['beta1', 'beta2','beta3...],
                          index   = ['alpha1', 'alpha2', ...])
data_frame
'''
```

Out[14]:

```
"\ndata_frame = pd.DataFrame(np.array([[lyapunov_11, lyapunov_12, ...], \n                                    [lyapunov_21, lyapunov_22, ...]]),  \n                          columns = ['bet
a1', 'beta2','beta3...],\n                          index   = ['alpha1', 'alpha2', ...])\ndata_frame\n"
```

In [15]:

```python
#print(array_lyapunov[0][:11])
#print(array_lyapunov[1][:11])
#print(array_lyapunov[2][:11])
print('alpha ', 'beta  ', 'lyapunov exponent')
list(zip(array_lyapunov[0][:11], array_lyapunov[1][:11], array_lyapunov[2][:11]))
```

```
alpha  beta   lyapunov exponent
```

Out[15]:

```
[(5.0, 0.15, -7.23921152929229),
 (5.0, 0.1625, -10.175264943087033),
 (5.0, 0.175, -6.546225924778387),
 (5.0, 0.1875, -9.952618107919623),
 (5.0, 0.2, -7.432475171722499),
 (5.0, 0.2125, -25.612178913188917),
 (5.0, 0.225, -8.241367149256446),
 (5.0, 0.2375, -5.8559079540493695),
 (5.0, 0.25, -33.97546566762948),
 (5.0, 0.2625, -35.86412686603707),
 (5.0, 0.275, -35.425451633436936)]
```

In [16]:

```python
lyap     = array_lyapunov[2][:]
lyap_arr = np.array(lyap)
lyap_arr = np.reshape(lyap_arr, (count_points,count_points))

data_frame = pd.DataFrame(lyap_arr,
                   columns = betas,
                   index = alphas)
print('alpha ⇊,  beta ⇉')
data_frame
```

```
alpha ⇊,  beta ⇉
```

Out[16]:

	0.1500	0.1625	0.1750	0.1875	0.2000	0.2125	0.2250	0.2375	0.2500	0.2625	...	0.3375	0.3500	0.3625	0.3750	0.3875	0.4000	0.4125	0.4250
5.000	-7.239212	-10.175265	-6.546226	-9.952618	-7.432475	-25.612179	-8.241367	-5.855908	-33.975466	-35.864127	...	-37.081834	-37.426502	-37.178433	-35.592907	-36.377498	-36.547673	-36.647385	-36.747634
5.375	-36.623385	-12.184295	-36.652350	-6.235595	-34.371692	-inf	-inf	-inf	-inf	-inf	...	-33.294376	-35.072550	-34.004767	-11.037711	-34.611303	-36.455470	-35.751233	-35.543946
5.750	-8.384936	-38.457792	-36.730357	-36.273210	-35.498038	-36.650122	-36.980420	-37.261547	-37.163129	-36.028526	...	-18.808550	-6.466134	-35.360385	-36.020738	-35.758604	-32.645893	-inf	-inf
6.125	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-34.294559	-8.984536	-35.067559	-34.533758	-36.185816	-34.980715	-21.311501	-36.276736
6.500	-36.821751	-34.310899	-8.674252	-35.439815	-36.391922	-34.659719	-35.040395	-7.853160	-32.813243	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
6.875	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-37.195955	-8.292771	-34.514685	-6.441659	-inf	-inf	-inf	-inf
7.250	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
7.625	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-35.012979	-35.444187	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
8.000	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
8.375	-34.725118	-34.215516	-34.418322	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
8.750	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-7.959990	-35.433141
9.125	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
9.500	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
9.875	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
10.250	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
10.625	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-36.259774	-inf	-inf	-inf	-inf
11.000	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
11.375	-inf	-35.457254	-33.907184	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
11.750	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
12.125	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
12.500	-inf	-inf	-inf	-34.979810	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
12.875	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
13.250	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
13.625	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf
14.000	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf	...	-inf	-inf	-inf	-inf	-inf	-inf	-inf	-inf

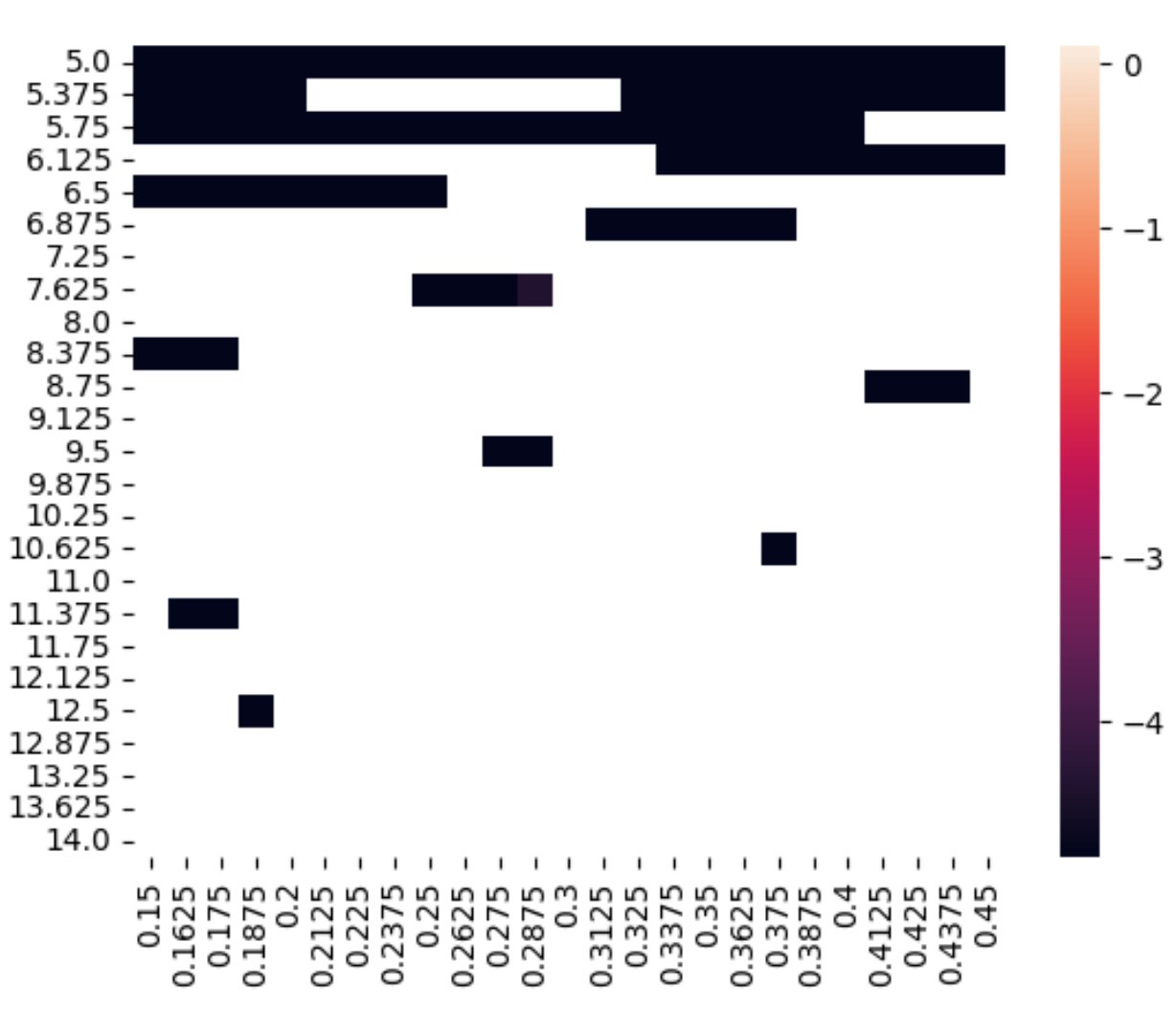
25 rows × 25 columns

In [17]:

```python
sns.heatmap(data_frame, annot = False, xticklabels='auto', yticklabels='auto', fmt=".1f")
# значеняня 'nan' виводиться як біле без числа
```

Out[17]:

```
<AxesSubplot: >
```



Ще один спосіб вивести

In [18]:

```python
'''
start_time = time.time()

alphas = np.linspace(5, 10, 10)
betas  = np.linspace(0.1, 0.6, 10)

Alphas, Betas = np.meshgrid(alphas, betas)

array_lyapunov2 = fun_array_lyapunov(T=100, N=10, arr_alpha=alphas, arr_beta=betas)

x = array_lyapunov2[:][0]
y = array_lyapunov2[:][1]
z = array_lyapunov2[:][2]

#X, Y = np.meshgrid(x, y)

cs = plt.contourf(x, y, z)
plt.xlabel("x")
plt.ylabel("y")
cbar = plt.colorbar(cs)

print("--- %s seconds ---" % (time.time() - start_time))
'''
```

Out[18]:

```
'\nstart_time = time.time()\n\nalphas = np.linspace(5, 10, 10)\nbetas  = np.linspace(0.1, 0.6, 10)\n\nAlphas, Betas = np.meshgrid(alphas, betas)\n\narray_lyapunov2 = fun_array_lyapunov(T=10
0, N=10, arr_alpha=alphas, arr_beta=betas)\n\nx = array_lyapunov2[:][0]\ny = array_lyapunov2[:][1]\nz = array_lyapunov2[:][2]\n\n#X, Y = np.meshgrid(x, y)\n\ncs = plt.contourf(x, y, z)\npl
t.xlabel("x")\nplt.ylabel("y")\ncbar = plt.colorbar(cs)\n\nprint("--- %s seconds ---" % (time.time() - start_time))\n'
```

Комп'ютерне моделювання задач прикладної математики

Динамічні системи.

Лабораторна робота 4 Реакційно-дифузійні системи та їх застосування

__Студентка Пороскун Олена. Група ПМ.м-21__

Варіант 8

Task 1 Розподіл температури по пластині.

Постановка задачі

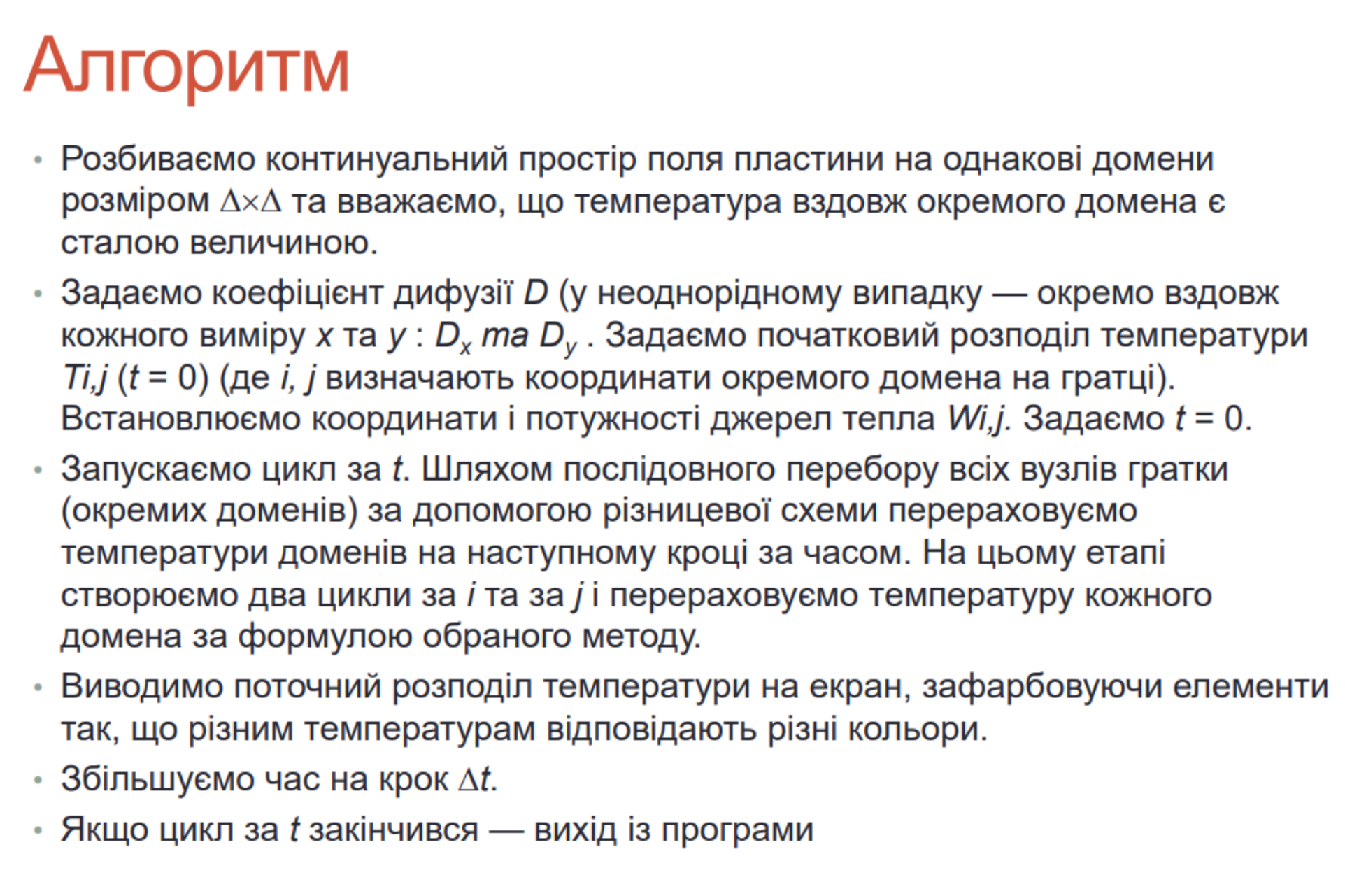
Дано

- Прямокутна пластина зі сталим коефіцієнтом температуропровідності D.
- Задано початковий розподіл температури T = T (x, y), потужність Wi та координати (xi, yi)

джерел тепла.

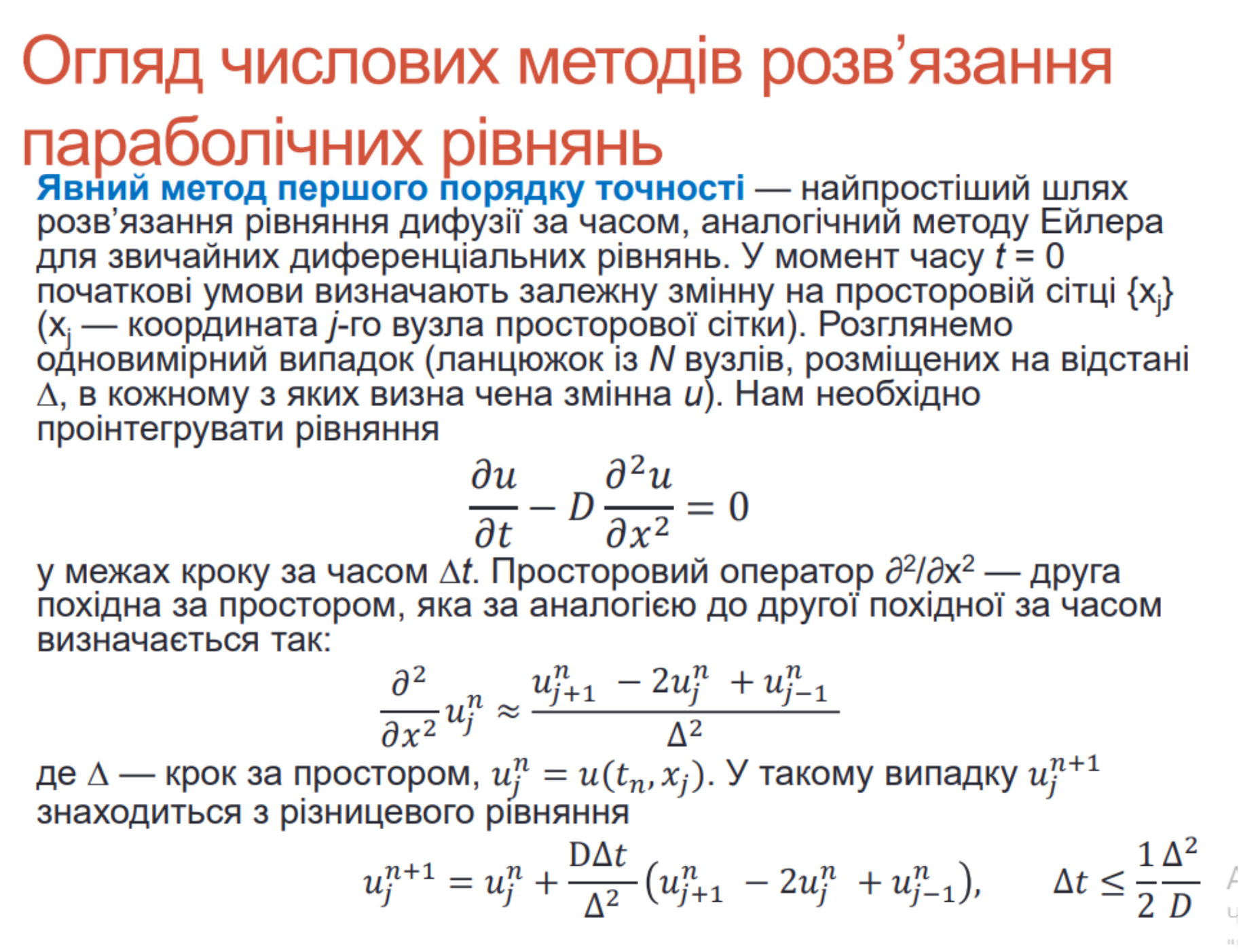
Задача

Проаналізувати зміну температури різних точок пластини з часом



Алгоритм

- Розбиваємо континуальний простір поля пластини на однакові домени розміром Δ×Δ та вважаємо, що температура вздовж окремого домена є сталою величиною.
- Задаємо коефіцієнт дифузії *D* (у неоднорідному випадку — окремо вздовж кожного виміру *x* та *y* : D_x *та* D_y . Задаємо початковий розподіл температури *Ti,j* (*t* = 0) (де *i, j* визначають координати окремого домена на гратці). Встановлюємо координати і потужності джерел тепла *Wi,j*. Задаємо *t* = 0.
- Запускаємо цикл за *t*. Шляхом послідовного перебору всіх вузлів гратки (окремих доменів) за допомогою різницевої схеми перераховуємо температури доменів на наступному кроці за часом. На цьому етапі створюємо два цикли за *i* та за *j* і перераховуємо температуру кожного домена за формулою обраного методу.
- Виводимо поточний розподіл температури на екран, зафарбовуючи елементи так, що різним температурам відповідають різні кольори.
- Збільшуємо час на крок Δ*t*.
- Якщо цикл за *t* закінчився — вихід із програми



Огляд числових методів розв'язання параболічних рівнянь

Явний метод першого порядку точності — найпростіший шлях розв'язання рівняння дифузії за часом, аналогічний методу Ейлера для звичайних диференціальних рівнянь. У момент часу *t* = 0 початкові умови визначають залежну змінну на просторовій сітці {x_j} (x_j — координата *j*-го вузла просторової сітки). Розглянемо одновимірний випадок (ланцюжок із *N* вузлів, розміщених на відстані Δ, в кожному з яких визна чена змінна *u*). Нам необхідно проінтегрувати рівняння

$$\frac{\partial u}{\partial t} - D\frac{\partial^2 u}{\partial x^2} = 0$$

у межах кроку за часом Δ*t*. Просторовий оператор $\partial^2/\partial x^2$ — друга похідна за простором, яка за аналогією до другої похідної за часом визначається так:

$$\frac{\partial^2}{\partial x^2}u_j^n \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta^2}$$

де Δ — крок за простором, $u_j^n = u(t_n, x_j)$. У такому випадку u_j^{n+1} знаходиться з різницевого рівняння

$$u_j^{n+1} = u_j^n + \frac{\mathrm{D}\Delta t}{\Delta^2}\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right), \qquad \Delta t \le \frac{1}{2}\frac{\Delta^2}{D}$$

Умова

$D -$ сталий коефіцієнт температуропровідності (коефіцієнт дифузії)

$T = T(x,y) -$ початковий розподіл температури

$Wi -$ потужність

$\Delta x \Delta -$ розміри пластини

Р-ня

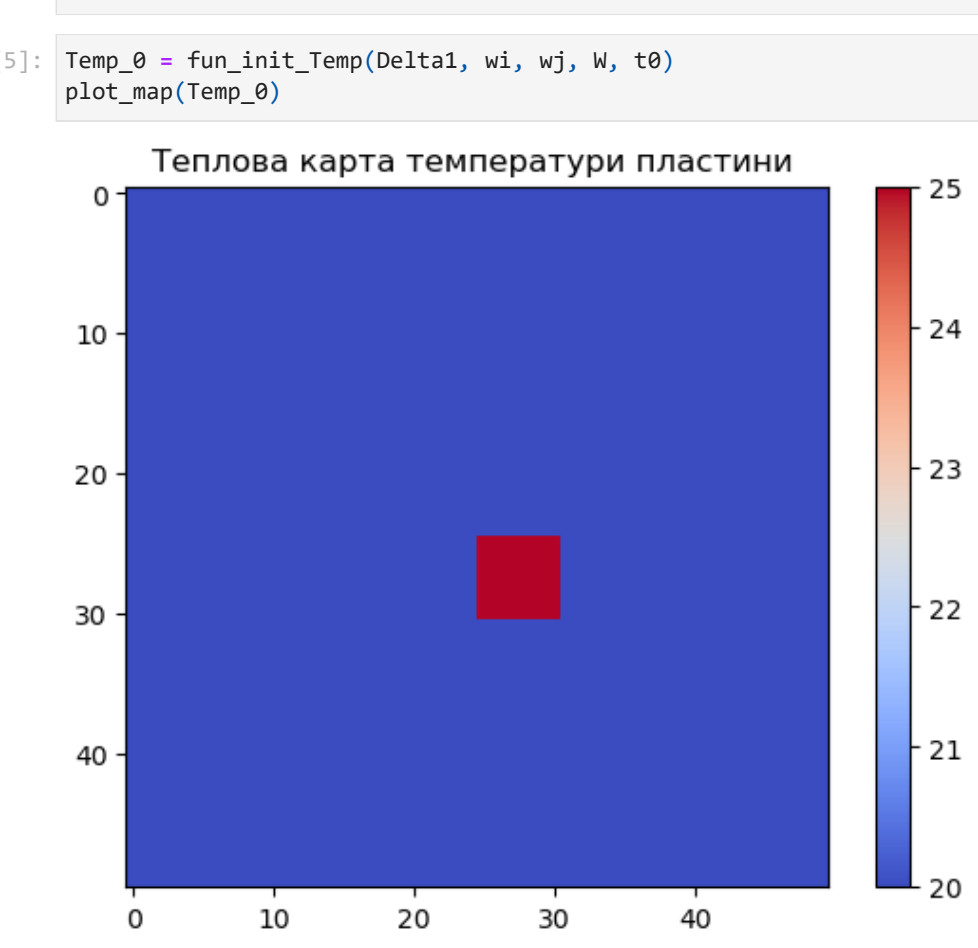
In [1]:
```python
import numpy as np
from matplotlib import pyplot as plt
from matplotlib import cm
```

In [2]:
```python
#D = 2.8*(10**(-6))
D = 0.17 # 70
Delta1 = 50
W = 25
wi = [25,26,27,28,29,30] # номери координат з потужністю W
wj = wi
t0 = 20
```

In [3]:
```python
def fun_init_Temp(Delta, wi, wj, W, t0=20):
    T_starts = np.zeros((Delta, Delta))
    for i in range(Delta):
        for j in range(Delta):
            T_starts[i,j] = t0
            if (i in wi) & (j in wj):
                T_starts[i,j] = W
    return T_starts
```

In [4]:
```python
def plot_map(Temp):
    # Make plot with vertical (default) colorbar
    fig, ax = plt.subplots()

    data = Temp

    cax = ax.imshow(data, interpolation='nearest', cmap=cm.coolwarm)
    ax.set_title('Теплова карта температури пластини ') # with vertical colorbar
    cbar = fig.colorbar(cax)
    #plt.show()
```

In [5]:
```python
Temp_0 = fun_init_Temp(Delta1, wi, wj, W, t0)
plot_map(Temp_0)
```



In [6]:
```python
times = np.linspace(0, 40, 1000)
times = np.round(times,4)
delta_t = times[1]-times[0]
delta_t
```

Out[6]:
```
0.04
```

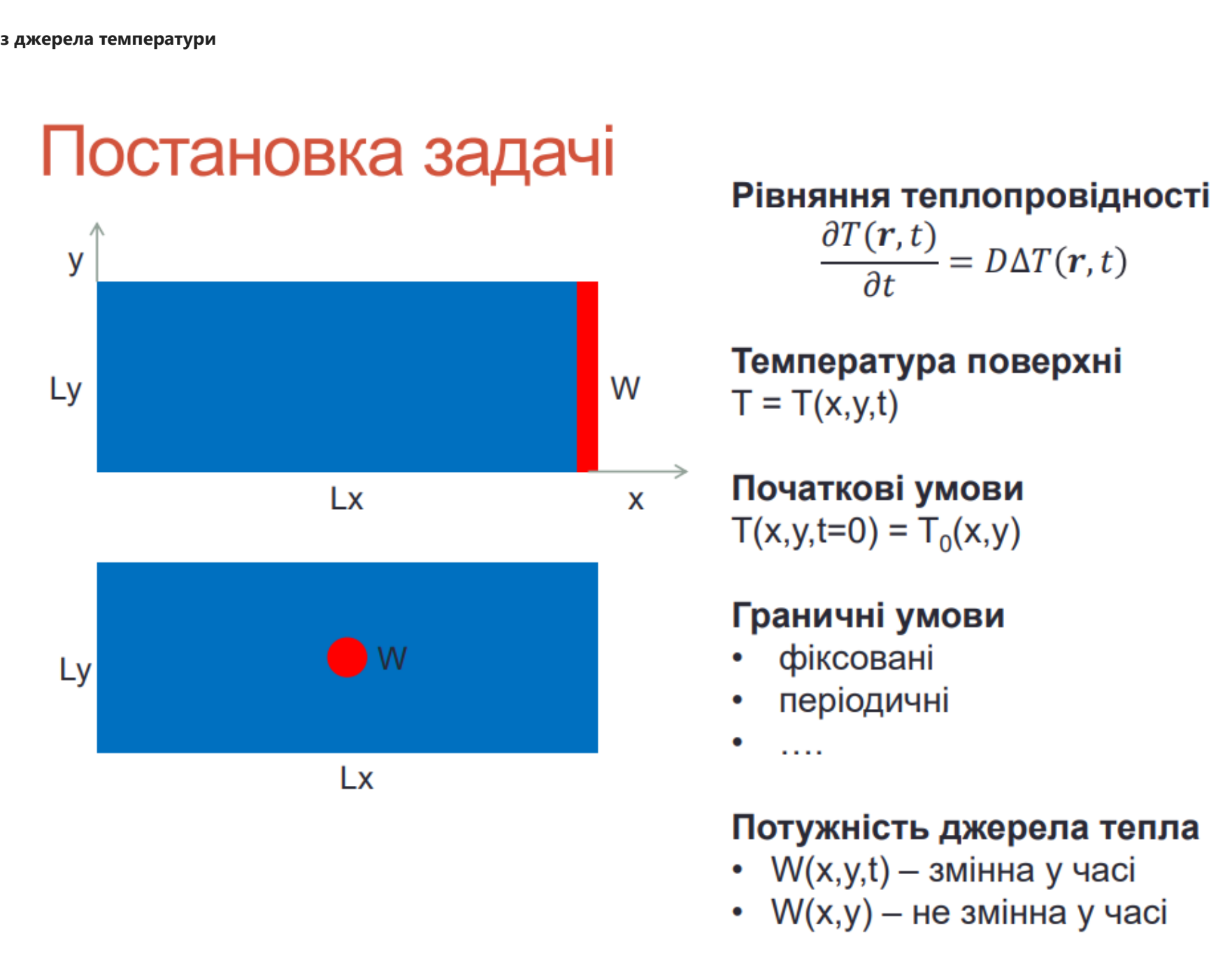
In [7]:
```python
0.5*(Delta1**2 / D)
```

Out[7]:
```
7352.941176470587
```

In [8]:
```python
delta_t < (0.5*(Delta1**2 / D))
```

Out[8]:
```
True
```

- **Без джерела температури**



Постановка задачі

Рівняння теплопровідності

$$\frac{\partial T(\boldsymbol{r},t)}{\partial t} = D\Delta T(\boldsymbol{r},t)$$

Температура поверхні

T = T(x,y,t)

Початкові умови

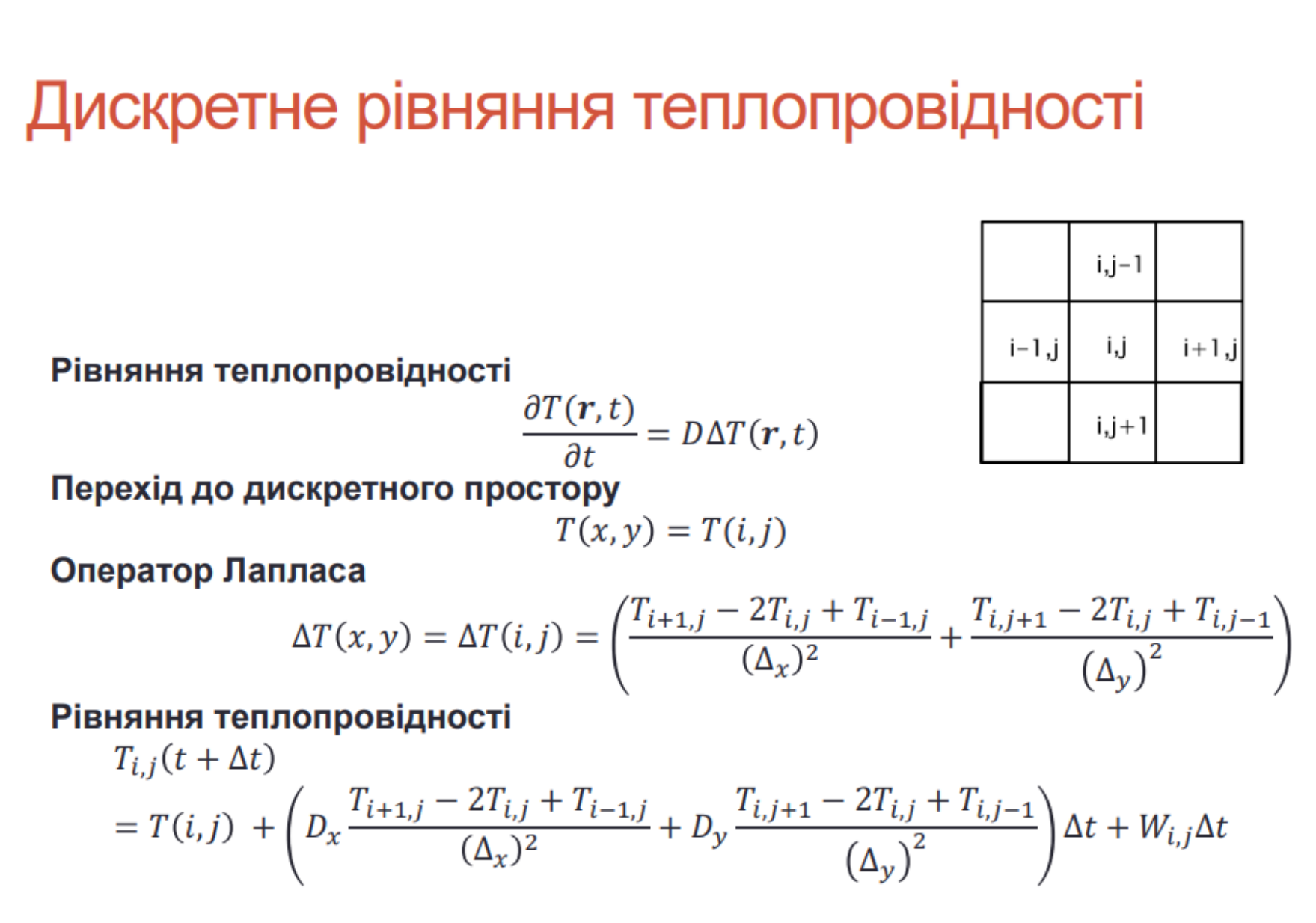
T(x,y,t=0) = T_0(x,y)

Граничні умови

- фіксовані
- періодичні
- ….

Потужність джерела тепла

- W(x,y,t) – змінна у часі
- W(x,y) – не змінна у часі



Дискретне рівняння теплопровідності

Рівняння теплопровідності

$$\frac{\partial T(\boldsymbol{r},t)}{\partial t} = D\Delta T(\boldsymbol{r},t)$$

Перехід до дискретного простору

$$T(x,y) = T(i,j)$$

Оператор Лапласа

$$\Delta T(x,y) = \Delta T(i,j) = \left(\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta_x)^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta_y)^2}\right)$$

Рівняння теплопровідності

$$T_{i,j}(t+\Delta t) = T(i,j) + \left(D_x\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta_x)^2} + D_y\frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta_y)^2}\right)\Delta t + W_{i,j}\Delta t$$

Візьмемо формулу з рівняння теплопровідності.

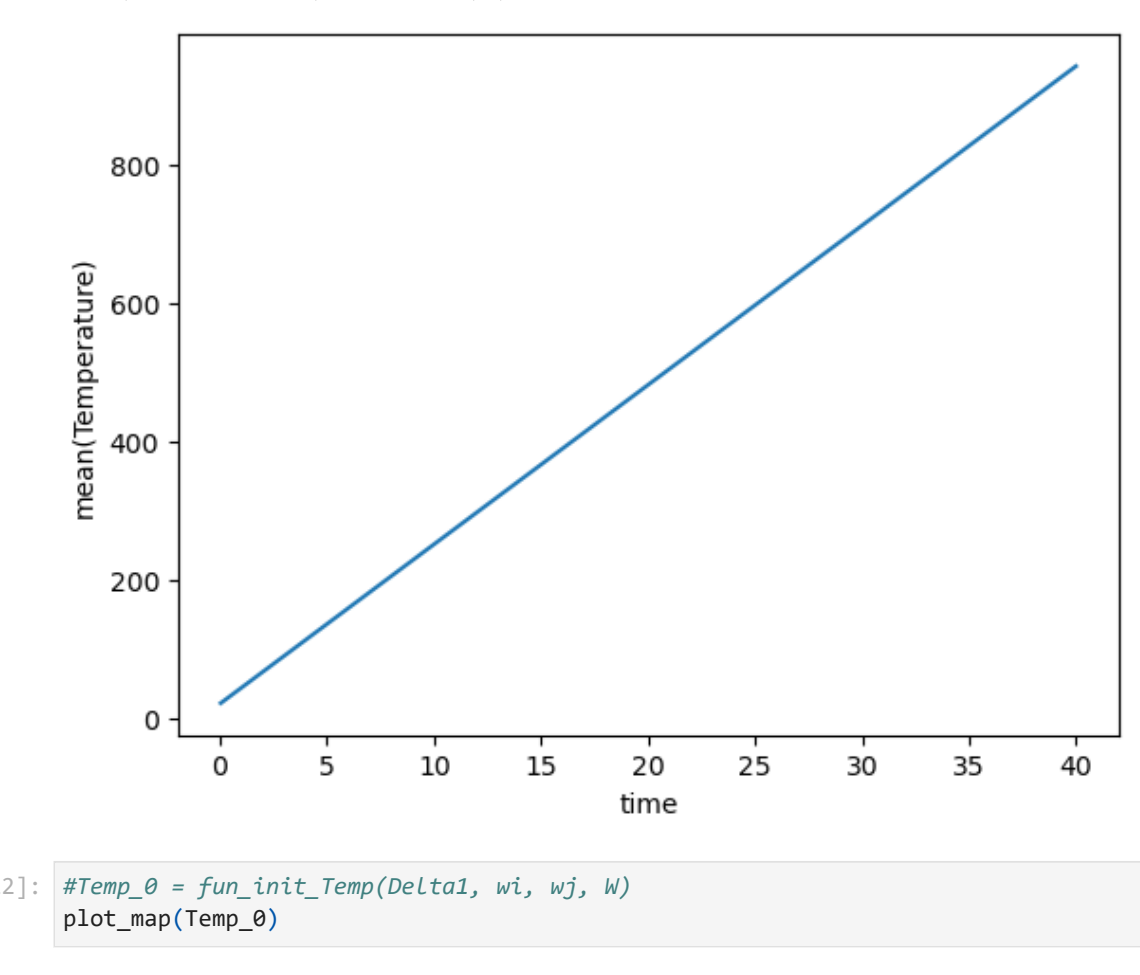
In [9]:
```python
#'''
def fun_temp_change(D, Delta, T_start, delta_t, W):
    print('t0 =', t0, '\t - початкова температура пластини крім нагрітої частини')
    print('D =', D, '\t  - коефіцієнт дифузії')
    print('Delta =', Delta, '\t  - пластина розміром Delta x Delta')
    print('delta_t =', delta_t, '\t  - крок по часу')
    print('W =', W, '\t  - потужність(для частини пластини, де нагріваємо)')
    t_ = 0
    time = []
    T_mean = []
    T = np.empty((Delta, Delta))
    while(t_ < 40):
        if (t_ == 0):
            T =  T_start.copy()
        # граничні умови (фіксовані):
        # тому рамка з 1 стовпчика, 1 рядка, останього стовпчика та рядка - не змінюється
        for i in range(1, Delta-1):
            for j in range(1, Delta-1):
                #T[i,j] = T[i,j] + (D*delta_t/(Delta**2))*(T[i, j+1] - 2*T[i,j] + T[i, j-1])
                # р-ня теплоємності
                T[i,j] = T[i,j] + (D/(Delta**2)) * ((T[i+1,j] - 2* T[i,j] + T[i-1,j]) + \
                         (T[i,j+1]-2*T[i,j]+T[i,j-1])) * delta_t + W*delta_t
        t_ += delta_t
        t_ = round(t_, 5)
        time.append(t_)
        T_mean.append(np.mean(T))
    #print(t_)
    return time, T, T_mean
#'''
```

In [10]:
```python
#time1 = []
#T1 = []
#T_mean1 = []
time1, T1, T_mean1 = fun_temp_change(D, Delta1, Temp_0, delta_t, W)
```

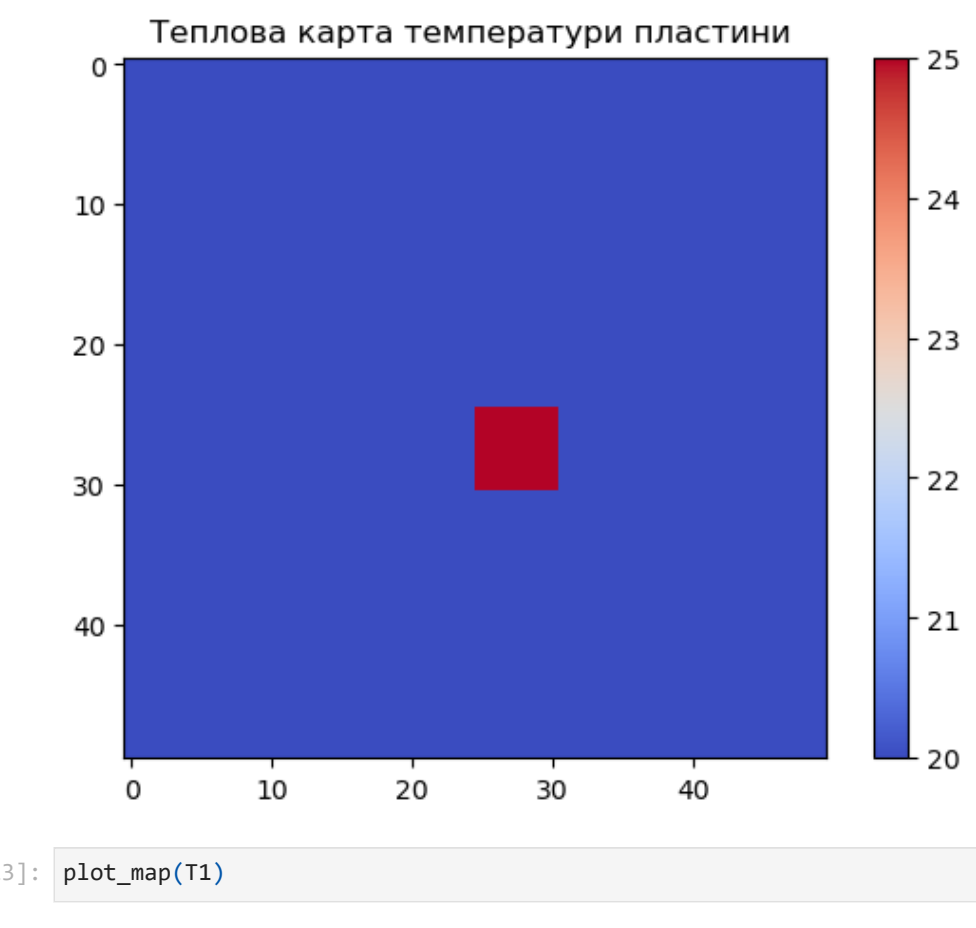
```
t0 = 20          - початкова температура пластини крім нагрітої частини
D = 0.17          - коефіцієнт дифузії
Delta = 50        - пластина розміром Delta x Delta
delta_t = 0.04    - крок по часу
W = 25    - потужність(для частини пластини, де нагріваємо)
```

In [11]:
```python
plt.plot(time1, T_mean1)
plt.xlabel('time')
plt.ylabel('mean(Temperature)')
```

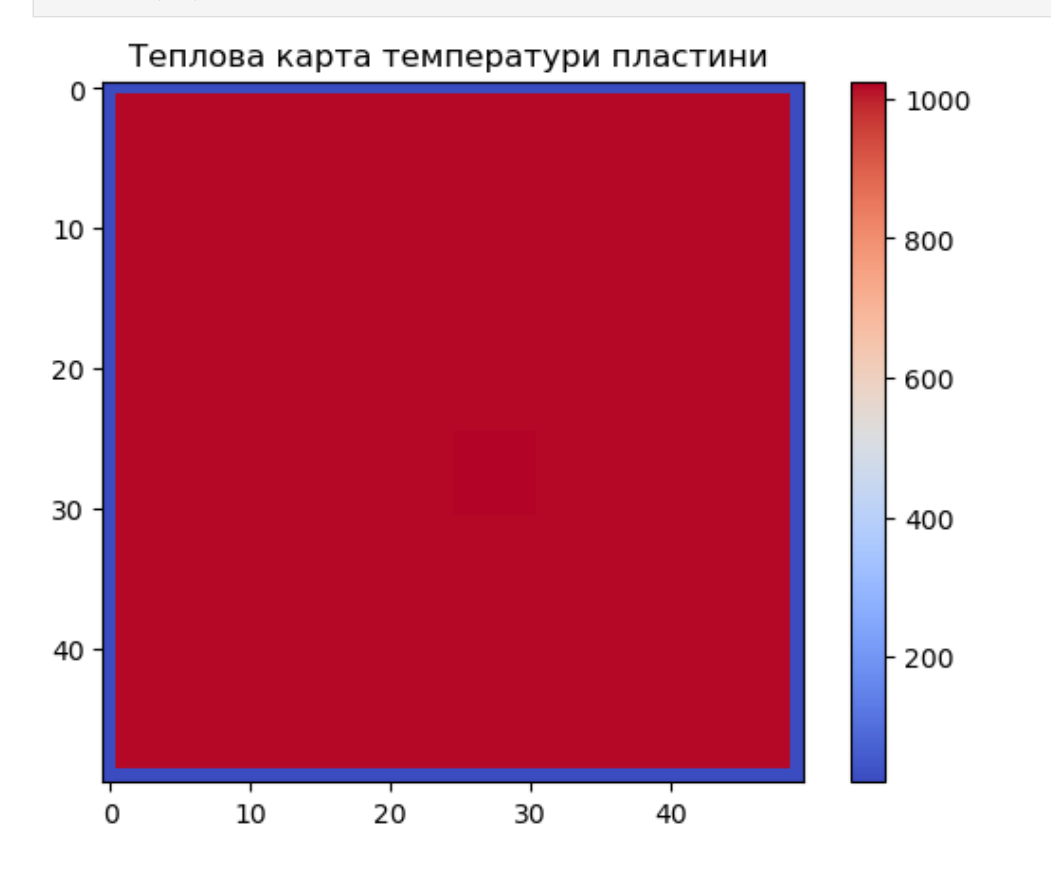
Out[11]:
```
Text(0, 0.5, 'mean(Temperature)')
```



In [12]:
```python
#Temp_0 = fun_init_Temp(Delta1, wi, wj, W)
plot_map(Temp_0)
```



In [13]:
```python
plot_map(T1)
```



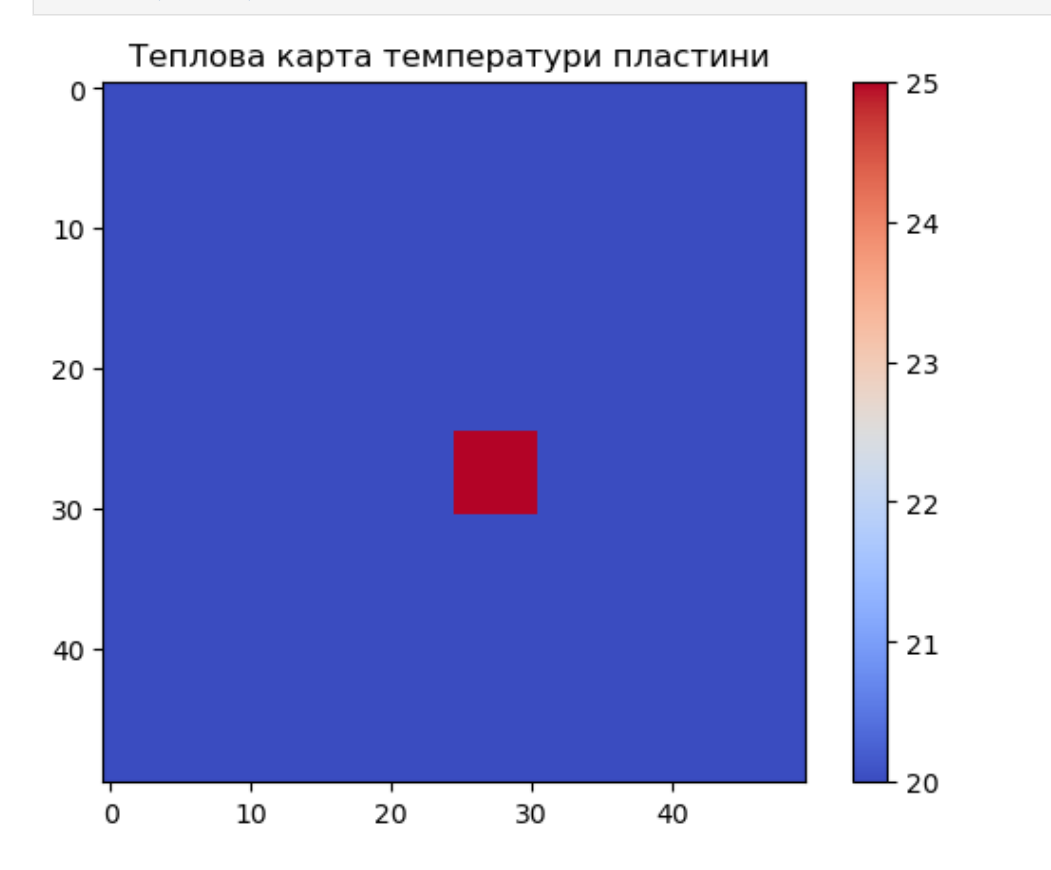
Візьмемо формулу для комірки, що дорівнює середньому по сусідам.

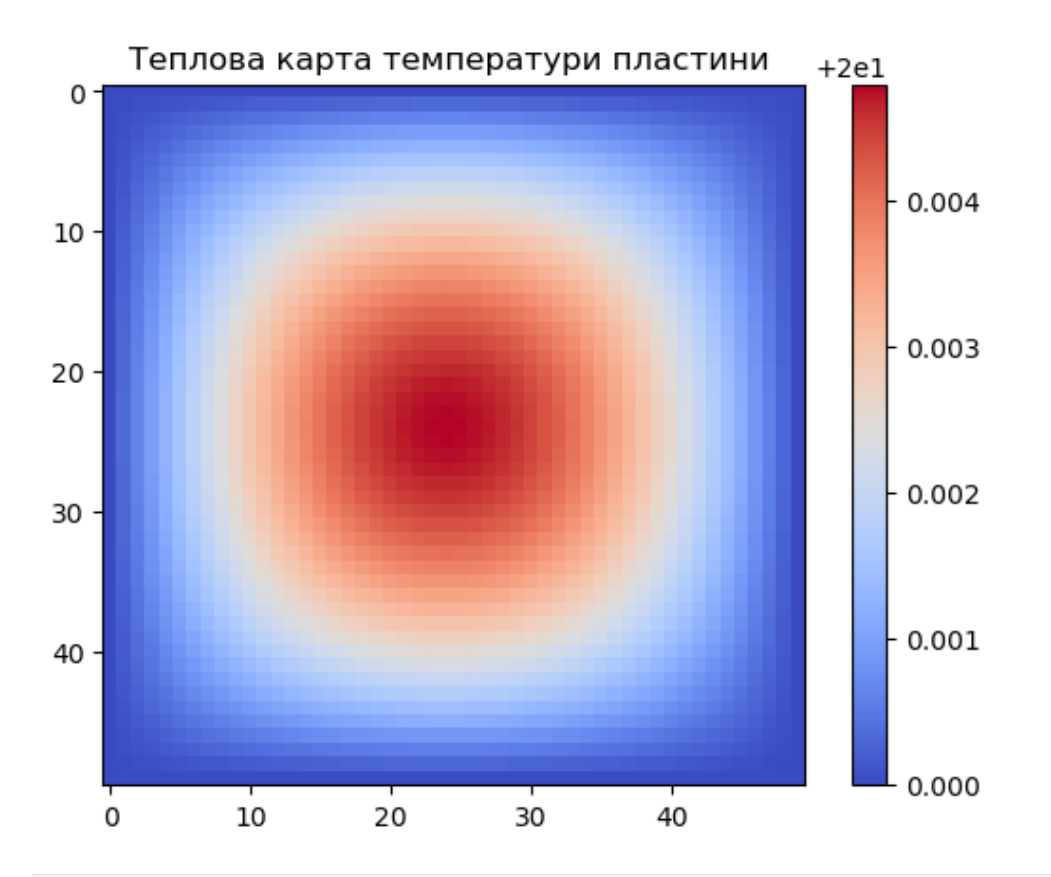
In [14]:
```python
#'''
def fun_temp_change_avr(D, Delta, T_start, delta_t, W):
    print('t0 =', t0, '\t - початкова температура пластини крім нагрітої частини')
    print('D =', D, '\t  - коефіцієнт дифузії')
    print('Delta =', Delta, '\t  - пластина розміром Delta x Delta')
    print('delta_t =', delta_t, '\t  - крок по часу')
    print('W =', W, '\t  - потужність(для частини пластини, де нагріваємо)')
    t_ = 0
    time = []
    T_mean = []
    T = np.empty((Delta, Delta))
    while(t_ < 40):
        if (t_ == 0):
            T =  T_start.copy()
        # граничні умови (фіксовані):
        # тому рамка з 1 стовпчика, 1 рядка, останього стовпчика та рядка - не змінюється
        for i in range(1, Delta-1):
            for j in range(1, Delta-1):
                #T[i,j] = T[i,j] + (D*delta_t/(Delta**2))*(T[i, j+1] - 2*T[i,j] + T[i, j-1])
                # р-ня теплоємності
                #T[i,j] = T[i,j] + (D/(Delta**2)) * ((T[i+1,j] - 2* T[i,j] + T[i-1,j]) + \
                #          (T[i,j+1]-2*T[i,j]+T[i,j-1])) * delta_t + W*delta_t
                # просто середнє по сусідам комірки
                T[i,j] = (T[i-1,j] + T[i+1,j] + T[i,j-1] + T[i,j+1])/4
        t_ += delta_t
        t_ = round(t_, 5)
        time.append(t_)
        T_mean.append(np.mean(T))
    #print(t_)
    return time, T, T_mean
#'''
```

In [15]:
```python
time1_avr, T1_avr, T_mean1_avr = fun_temp_change_avr(D, Delta1, Temp_0, delta_t, W)
```

```
t0 = 20          - початкова температура пластини крім нагрітої частини
D = 0.17          - коефіцієнт дифузії
Delta = 50        - пластина розміром Delta x Delta
delta_t = 0.04    - крок по часу
W = 25    - потужність(для частини пластини, де нагріваємо)
```

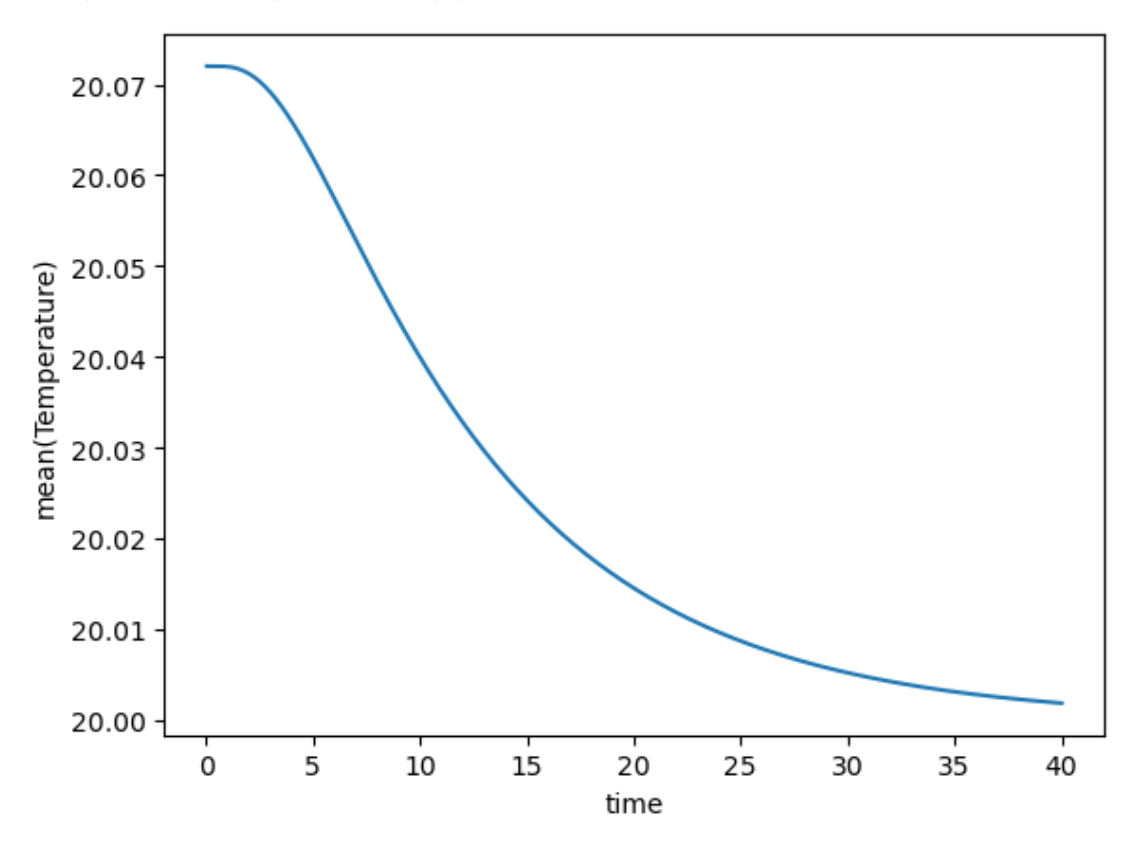
In [16]:
```python
plot_map(Temp_0)
plot_map(T1_avr)
```





In [17]:
```python
plt.plot(time1_avr, T_mean1_avr)
plt.xlabel('time')
plt.ylabel('mean(Temperature)')
```

Out[17]:
```
Text(0, 0.5, 'mean(Temperature)')
```



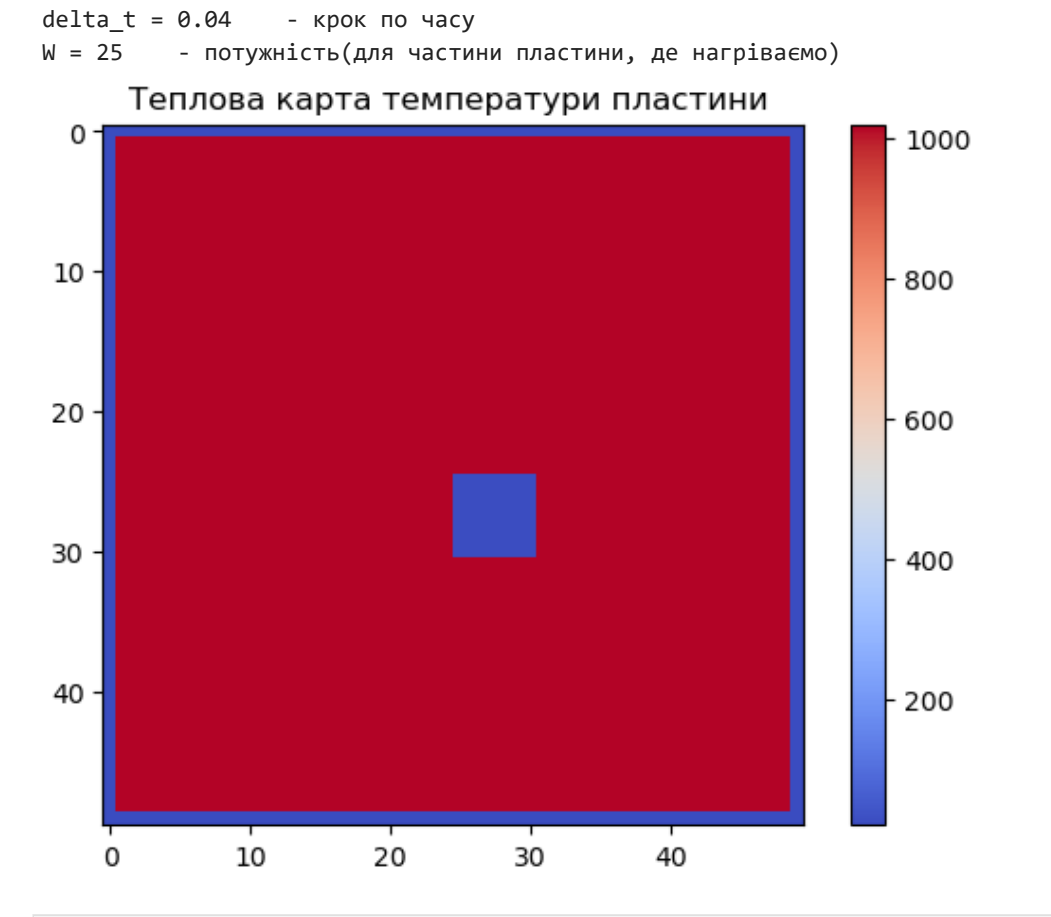
- **З постійним джерелом температури**

Візьмемо формулу з рівняння теплопровідності.

In [18]:
```python
def fun_temp_change_W(D = 5, Delta=10, T_start=[0], delta_t=0.01, W=5, wi=5):
    print('t0 =', t0, '\t - початкова температура пластини крім нагрітої частини')
    print('D =', D, '\t  - коефіцієнт дифузії')
    print('Delta =', Delta, '\t  - пластина розміром Delta x Delta')
    print('delta_t =', delta_t, '\t  - крок по часу')
    print('W =', W, '\t  - потужність(для частини пластини, де нагріваємо)')

    t_ = 0
    time = []
    T_mean = []
    T = np.empty((Delta, Delta))

    T = T_start.copy()
    t_ += delta_t

    wi = wj
    while(t_ <= 40):
        # граничні умови (фіксовані):
        # тому рамка з 1 стовпчика, 1 рядка, останього стовпчика та рядка - не змінюється
        for i in range(1, Delta-1):
            for j in range(1, Delta-1):
                if (i in wi) & (j in wj):
                    T[i,j] = W
                else:
                    #T[i,j] = T[i,j] + (D*delta_t/(Delta**2))*(T[i, j+1] - 2*T[i,j] + T[i, j-1])
                    # р-ня теплоємності
                    T[i,j] = T[i,j] + (D/(Delta**2)) * ((T[i+1,j] - 2* T[i,j] + T[i-1,j]) + \
                          (T[i,j+1]-2*T[i,j]+T[i,j-1])) * delta_t + W*delta_t
                    # просто середнє по сусідам комірки
                    #T[i,j] = (T[i-1,j] + T[i+1,j] + T[i,j-1] + T[i,j+1])/4

        t_ += delta_t
        t_ = np.round(t_,5)
        time.append(t_)
        T_mean.append(np.mean(T))
    return time, T, T_mean
```

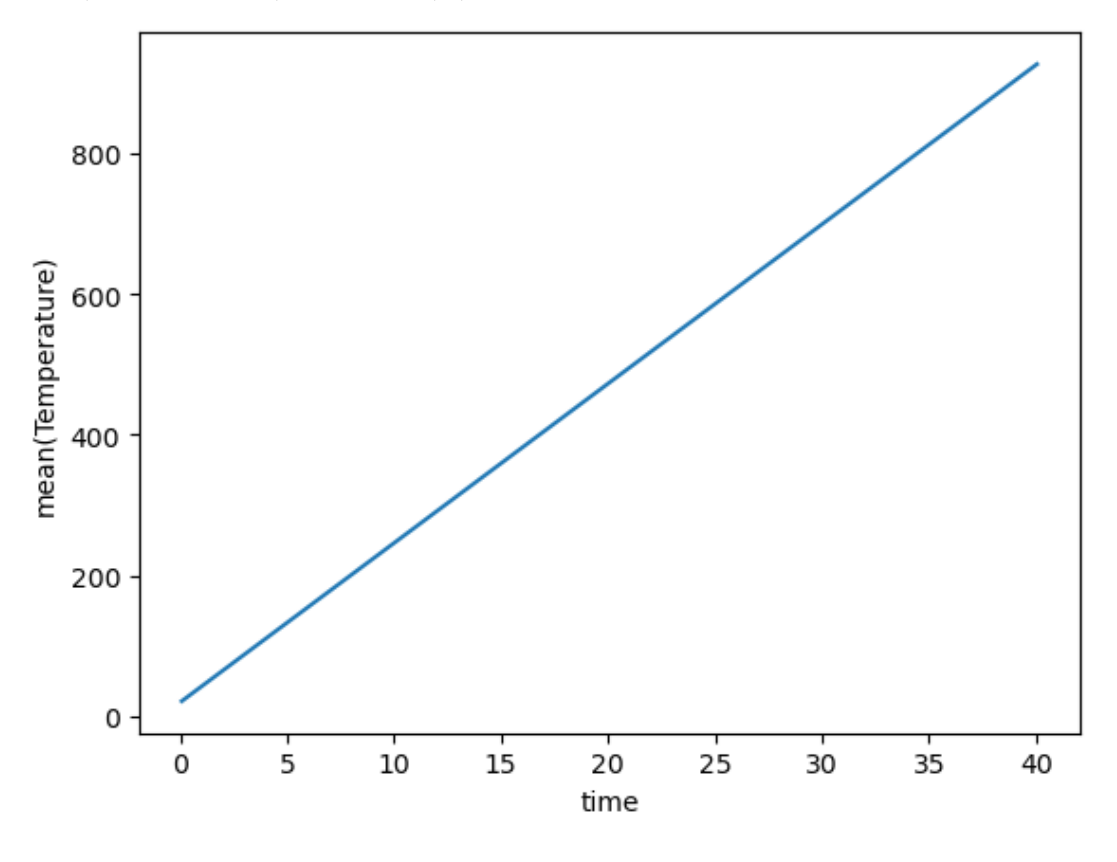
In [19]:
```python
time_w, T_w, T_mean_w = fun_temp_change_W(D, Delta1, Temp_0, delta_t, W, wi)
plot_map(T_w)
```

```
t0 = 20          - початкова температура пластини крім нагрітої частини
D = 0.17          - коефіцієнт дифузії
Delta = 50        - пластина розміром Delta x Delta
delta_t = 0.04    - крок по часу
W = 25    - потужність(для частини пластини, де нагріваємо)
```



In [20]:
```python
plt.plot(time_w, T_mean_w)
plt.xlabel('time')
plt.ylabel('mean(Temperature)')
```

Out[20]:
```
Text(0, 0.5, 'mean(Temperature)')
```

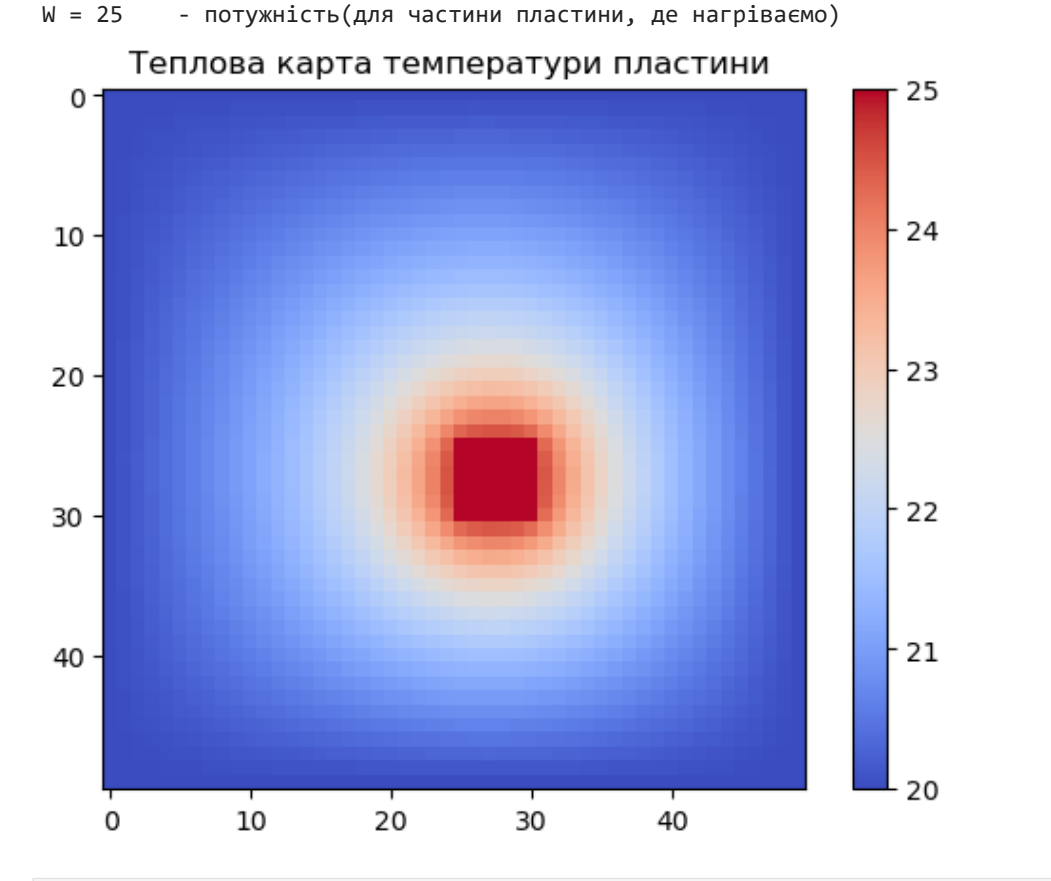


Візьмемо формулу для комірки, що дорівнює середньому по сусідам.

In [21]:
```python
def fun_temp_change_W_avr(D = 5, Delta=10, T_start=[0], delta_t=0.01, W=5, wi=5):
    print('t0 =', t0, '\t - початкова температура пластини крім нагрітої частини')
    print('D =', D, '\t  - коефіцієнт дифузії')
    print('Delta =', Delta, '\t  - пластина розміром Delta x Delta')
    print('delta_t =', delta_t, '\t  - крок по часу')
    print('W =', W, '\t  - потужність(для частини пластини, де нагріваємо)')

    t_ = 0
    time = []
    T_mean = []
    T = np.empty((Delta, Delta))

    T = T_start.copy()
    t_ += delta_t

    wi = wj
    while(t_ <= 40):
        # граничні умови (фіксовані):
        # тому рамка з 1 стовпчика, 1 рядка, останього стовпчика та рядка - не змінюється
        for i in range(1, Delta-1):
            for j in range(1, Delta-1):
                if (i in wi) & (j in wj):
                    T[i,j] = W
                else:
                    #T[i,j] = T[i,j] + (D*delta_t/(Delta**2))*(T[i, j+1] - 2*T[i,j] + T[i, j-1])
                    # р-ня теплоємності
                    #T[i,j] = T[i,j] + (D/(Delta**2)) * ((T[i+1,j] - 2* T[i,j] + T[i-1,j]) + \
                    #      (T[i,j+1]-2*T[i,j]+T[i,j-1])) * delta_t + W*delta_t
                    # просто середнє по сусідам комірки
                    T[i,j] = (T[i-1,j] + T[i+1,j] + T[i,j-1] + T[i,j+1])/4

        t_ += delta_t
        t_ = np.round(t_,5)
        time.append(t_)
        T_mean.append(np.mean(T))
    return time, T, T_mean
```

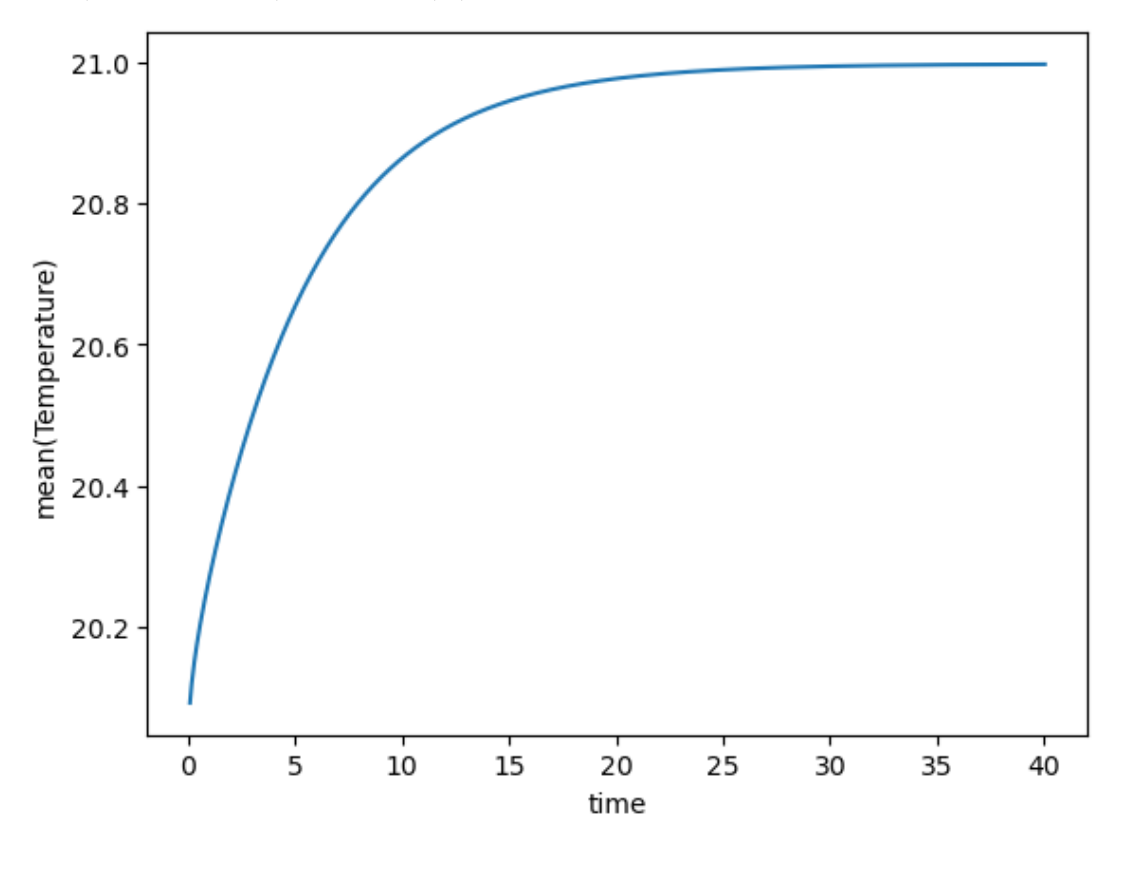
In [22]:
```python
time_w_avr, T_w_avr, T_mean_w_avr = fun_temp_change_W_avr(D, Delta1, Temp_0, delta_t, W, wi)
plot_map(T_w_avr)
```

```
t0 = 20          - початкова температура пластини крім нагрітої частини
D = 0.17          - коефіцієнт дифузії
Delta = 50        - пластина розміром Delta x Delta
delta_t = 0.04    - крок по часу
W = 25    - потужність(для частини пластини, де нагріваємо)
```



In [23]:
```python
plt.plot(time_w_avr, T_mean_w_avr)
plt.xlabel('time')
plt.ylabel('mean(Temperature)')
```

Out[23]:
```
Text(0, 0.5, 'mean(Temperature)')
```



Комп'ютерне моделювання задач прикладної математики

Динамічні системи.

Лабораторна робота 4 Реакційно-дифузійні системи та їх застосування

Студентка Пороскун Олена. Група ПМ.м-21

Варіант 8

Task 2 Фазове розшарування бінарних систем.

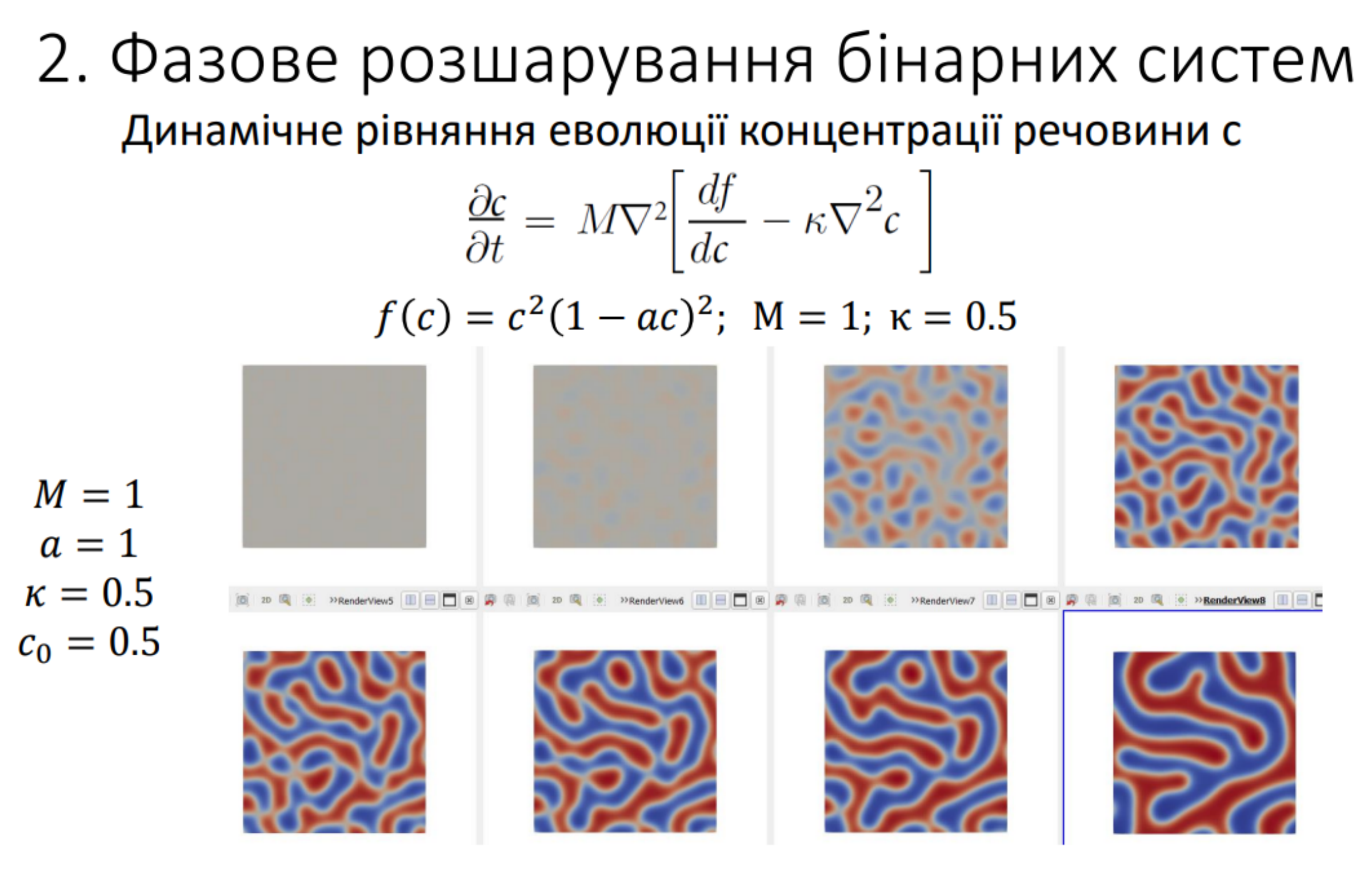
Постановка задачі

Дано

- Нехай існує два типи речовин A та B.
- Резервуар розміром L x L заповнюють речовинами A та B з концентраціями cA та cB однорідно.

Задача

- Необхідно провести числове моделювання динаміки фазового розшарування однорідного розчину A-B на фази, збагачені на речовину A та речовину B.


2. Фазове розшарування бінарних систем

Динамічне рівняння еволюції концентрації речовини c

$$\frac{\partial c}{\partial t} = M\nabla^2\left[\frac{df}{dc} - \kappa\nabla^2 c\right]$$

$$f(c) = c^2(1 - ac)^2;\ \ \mathrm{M} = 1;\ \kappa = 0.5$$

$M = 1$
$a = 1$
$\kappa = 0.5$
$c_0 = 0.5$

Умова

$M = 1$
$a = 1$
$k = 0.5$
$c0 = 0.5$

Р-ня

In [1]:
```
import numpy as np
from matplotlib import pyplot as plt
from matplotlib import cm
```

In [2]:
```
!pip install sympy
```

```
Requirement already satisfied: sympy in c:\users\admin\anaconda3\lib\site-packages (1.11.1)
Requirement already satisfied: mpmath>=0.19 in c:\users\admin\anaconda3\lib\site-packages (from sympy) (1.2.1)
```

In [3]:
```
from sympy import diff, symbols
```

In [4]:
```
a, c = symbols('a c')
f = (c**2)*((1-a*c)**2)
print('Функція f(c,a):')
f
```

```
Функція f(c,a):
```

Out[4]: $c^2(-ac + 1)^2$

Константи

In [5]:
```
L = 20 #L = 50
M = 1
k = 0.5
c0 = 0.5
a = 1
print('L =',L,' M =', M,'\t k =',k,'\t c0 =',c0,'\t a =', a)
```

```
L = 20  M = 1    k = 0.5         c0 = 0.5        a = 1
```

In [6]:
```
c = symbols('c')
f = (c**2)*((1-a*c)**2)
df1_symbol = diff(f, c) # 1 похідна
print("Перша похідна від f(c,a=" f'{a}' ") по c -> df1(c,a):")
df1_symbol
```

```
Перша похідна від f(c,a=1) по c -> df1(c,a):
```

Out[6]: $c^2 \cdot (2c - 2) + 2c(1 - c)^2$

In [7]:
```
print("Друга похідна від f(c,a) по c -> df2(c,a=" f'{a}' ") (оператор Лапласа):" )
op_laplas_symbol = diff(f, c, 2) # 2 похідна = оператор Лапласа
op_laplas_symbol
```

```
Друга похідна від f(c,a) по c -> df2(c,a=1) (оператор Лапласа):
```

Out[7]: $2\left(c^2 + 4c(c - 1) + (c - 1)^2\right)$

#df1 = c**2*(2*c-2) + 2*c*(1-c)**2 #op_laplas = 2*c**2 - 8*c*(1-c) + 2*(1-c)**2 # спрощена форма від op_laplas_symbol

Функція ініціалізації даних початкової концентрації значеннями близькими до c_0.

In [8]:
```
def fun_init_data(L, c0):
    print('Резервуар розміром LxL, де L =', L)
    print('Початкова концентрація близька до значення c0 =', c0)

    data = np.linspace(c0-0.05, c0+0.05, L*L)
    len_data = len(data)
    new_size = int(np.sqrt(len_data))
    data_resize = np.resize(data, (new_size, new_size))
    '''
    C_starts = np.zeros((L, L))
    for i in range(L):
        for j in range(L):
            C_starts[i,j] = c0
    return C_starts
    '''
    return data_resize
```

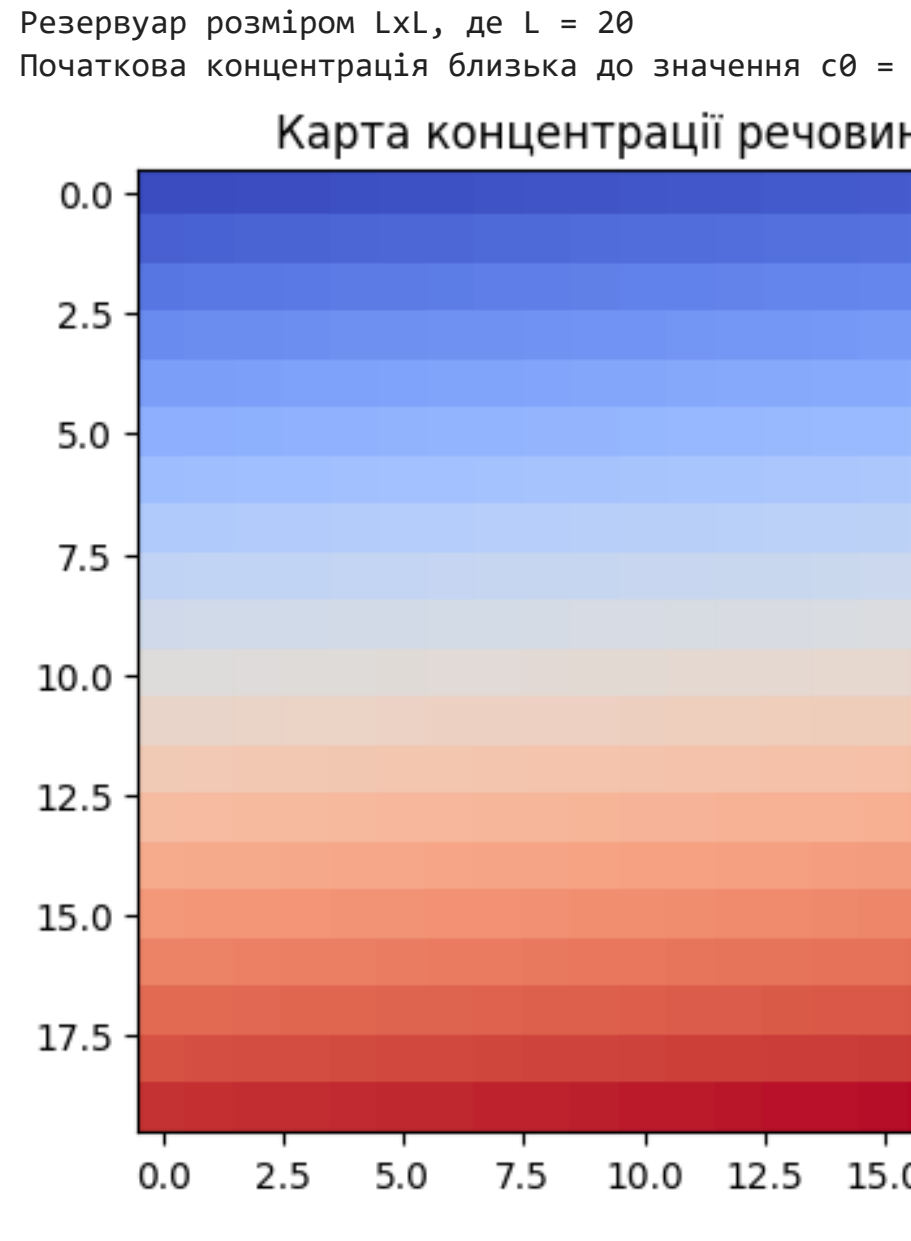
Побудова графіку двумірних даних концентрації.

In [9]:
```
def plot_map(data):
    # графік з вертикальною (за замовчуванням) панеллю кольорів
    fig, ax = plt.subplots()
    cax = ax.imshow(data, interpolation='nearest', cmap=cm.coolwarm)
    ax.set_title('Карта концентрації речовини ')
    cbar = fig.colorbar(cax)
    plt.show()
```

Отже, ініціалізуємо початкові дані та зобразимо їх.

In [10]:
```
c_init_2D = fun_init_data(L, c0)
plot_map(c_init_2D)
```

```
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
```



In [11]:
```
from scipy.integrate import odeint
```

Функція, що змінює розмірність даних (робить двумірні дані з одномірних даних, наприклад був масив 100x1 стане 10x10):

In [12]:
```
def plot_resize_data(data):
    len_data = len(data)
    new_size = int(np.sqrt(len_data))
    newdata = np.resize(data, (new_size, new_size))
    plot_map(newdata)
    return newdata
```

Функція, що повертає останній елемент масиву:

In [13]:
```
def fun_last_el(array):
    #print('fun_last_el')
    end = len(array)
    return array[end-1]
```

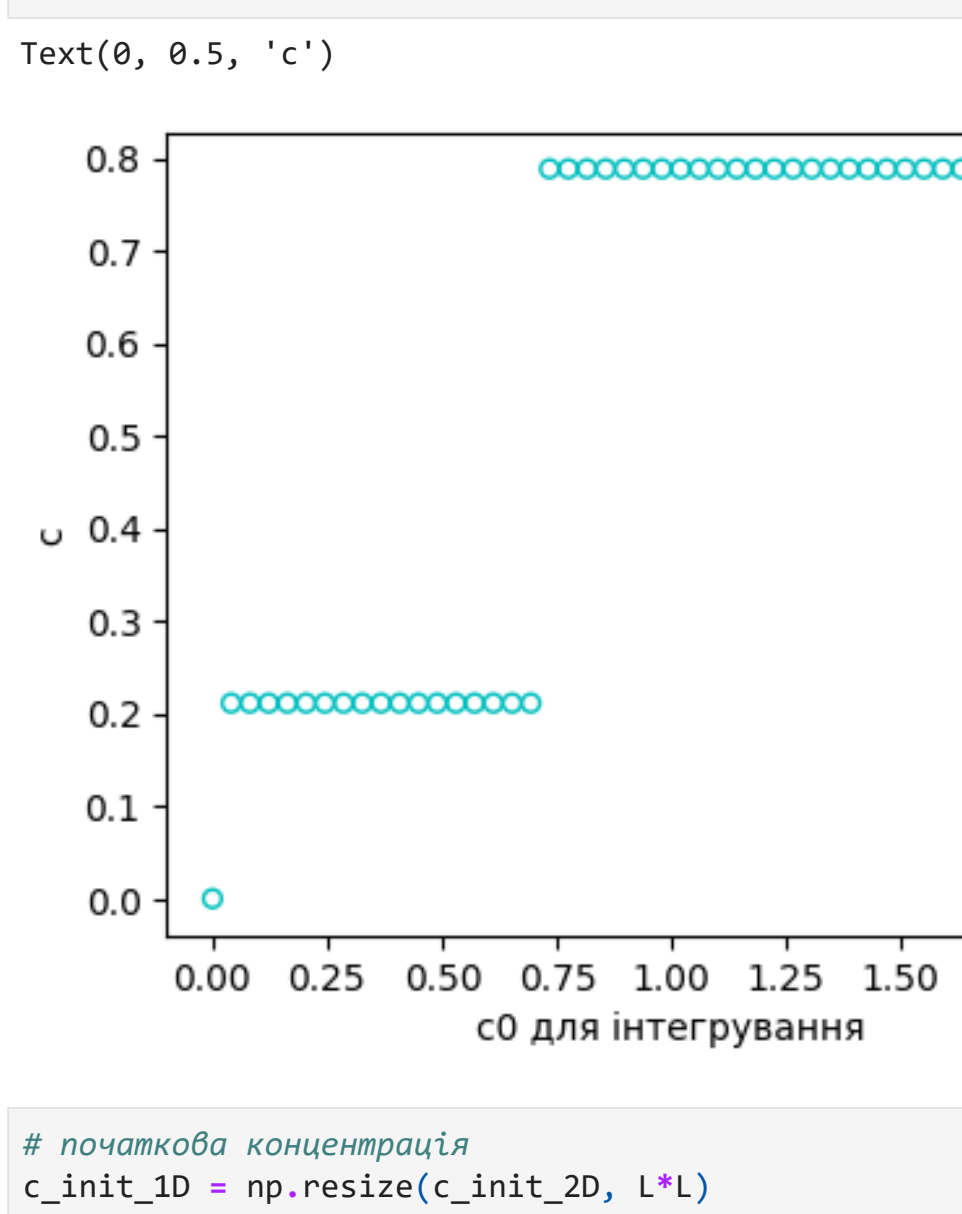
Функція, що здійснює інтегрування з використанням початкової точки $state0$ за формулою:

$$\frac{dc}{dt} = M\nabla^2[\frac{df}{dc} - k\nabla^2 c],$$
$$\text{де } \nabla^2 - \text{оператор Лапласа (друга похідна від} f(c))$$

In [14]:
```
def fun_1(state0=0.5):
    # процес інтегрування f(c) = c**2 * (1-a*c) за формулою
    # dc/dt = M*op_laplas*((df/dc) - k*op_laplas*c)
    def dcdt(state, t):
        c = state                                        # Розпаковка вектору
        return M*op_laplas(c)*(df1(c) - k*op_laplas(c)*c) # Похідні

    def df1(c):
        #df1 = df/dc
        return c**2*(2*c-2) + 2*c*(1-c)**2

    def op_laplas(c):
        #df1 = df/dc
        #op_laplas = d(df1)/dc - оператор Лапласа
        return 2*c**2 - 8*c*(1-c) + 2*(1-c)**2

    t = np.arange(0.0, 40.0, 0.1)
    states = odeint(dcdt, state0, t)
    states = np.round(states, 4)

    last_states = fun_last_el(states)

    #print('Процес інтегрування')
    #print('Останній елемент масиву =', last_states)
    #print('Розмір масиву =', np.shape(states))
    #print('Перші 10 елементів масиву: \n', states[:10])

    return last_states
```

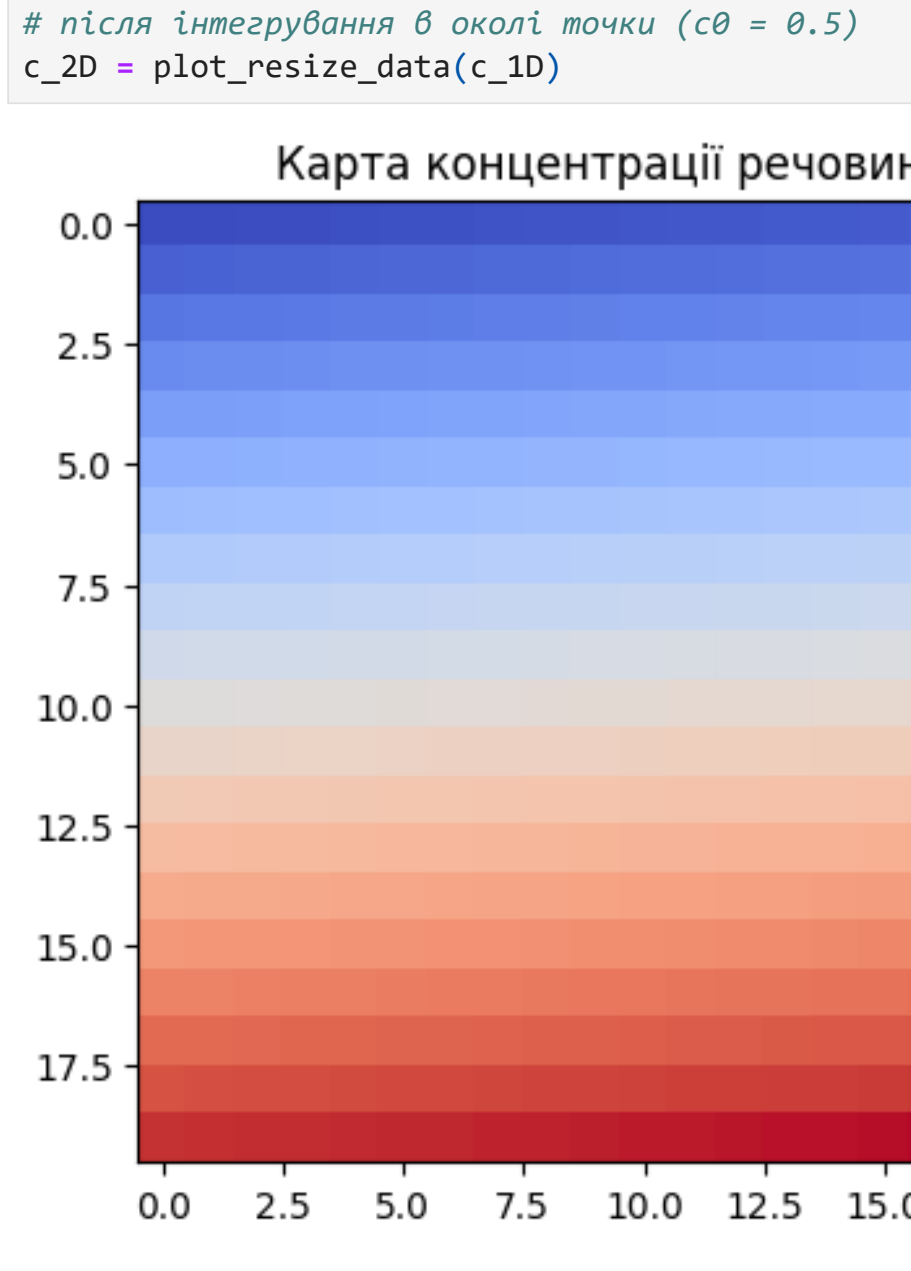
In [15]:
```
#Результат інтегрування в залежності від початкової точки
x1 = np.linspace(0, 2, 50)
y1 = [ fun_1(state0=i) for i in x1 ]

plt.figure(figsize =(5,4))
plt.scatter(x1, y1, s=25, marker ='o', c='none', edgecolors ='c')
plt.xlabel('c0 для інтегрування')
plt.ylabel('c')
```

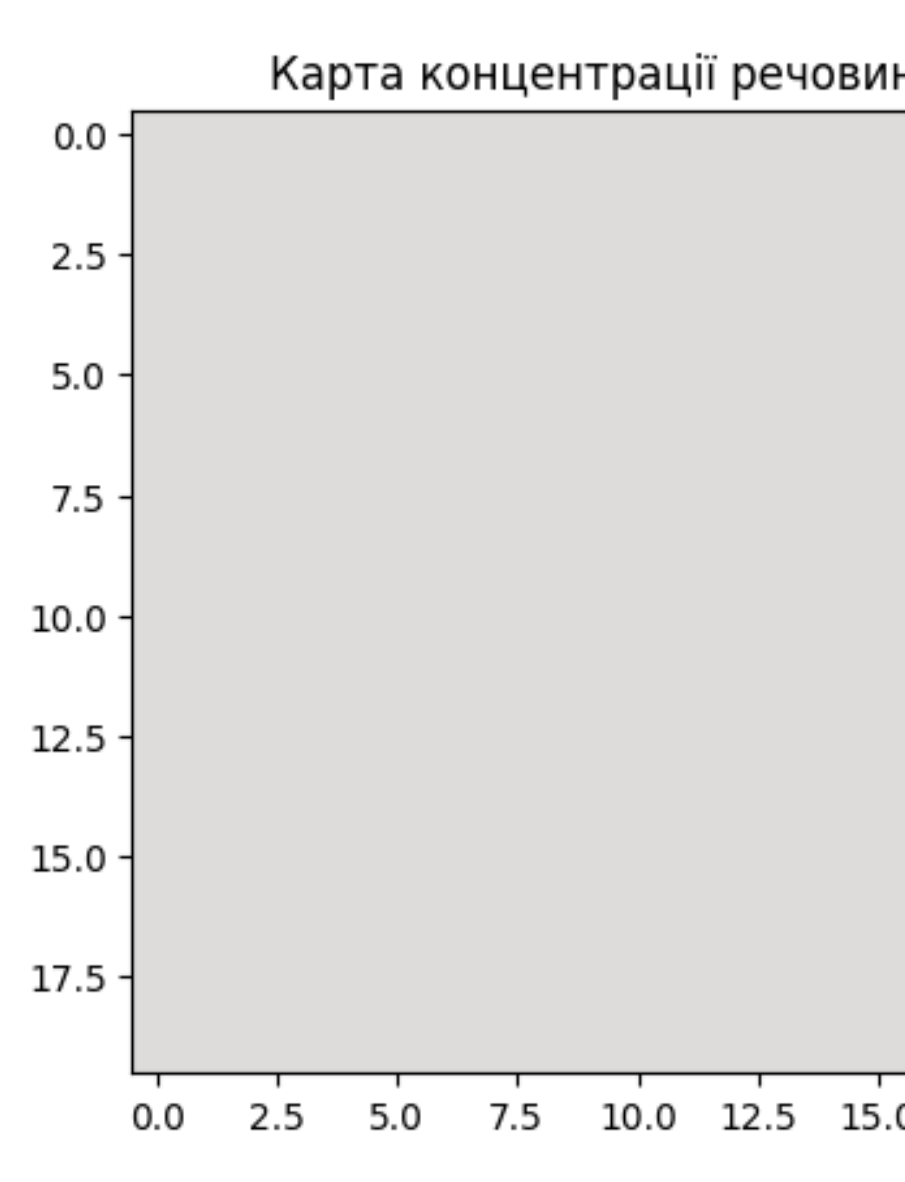
Out[15]:
```
Text(0, 0.5, 'c')
```

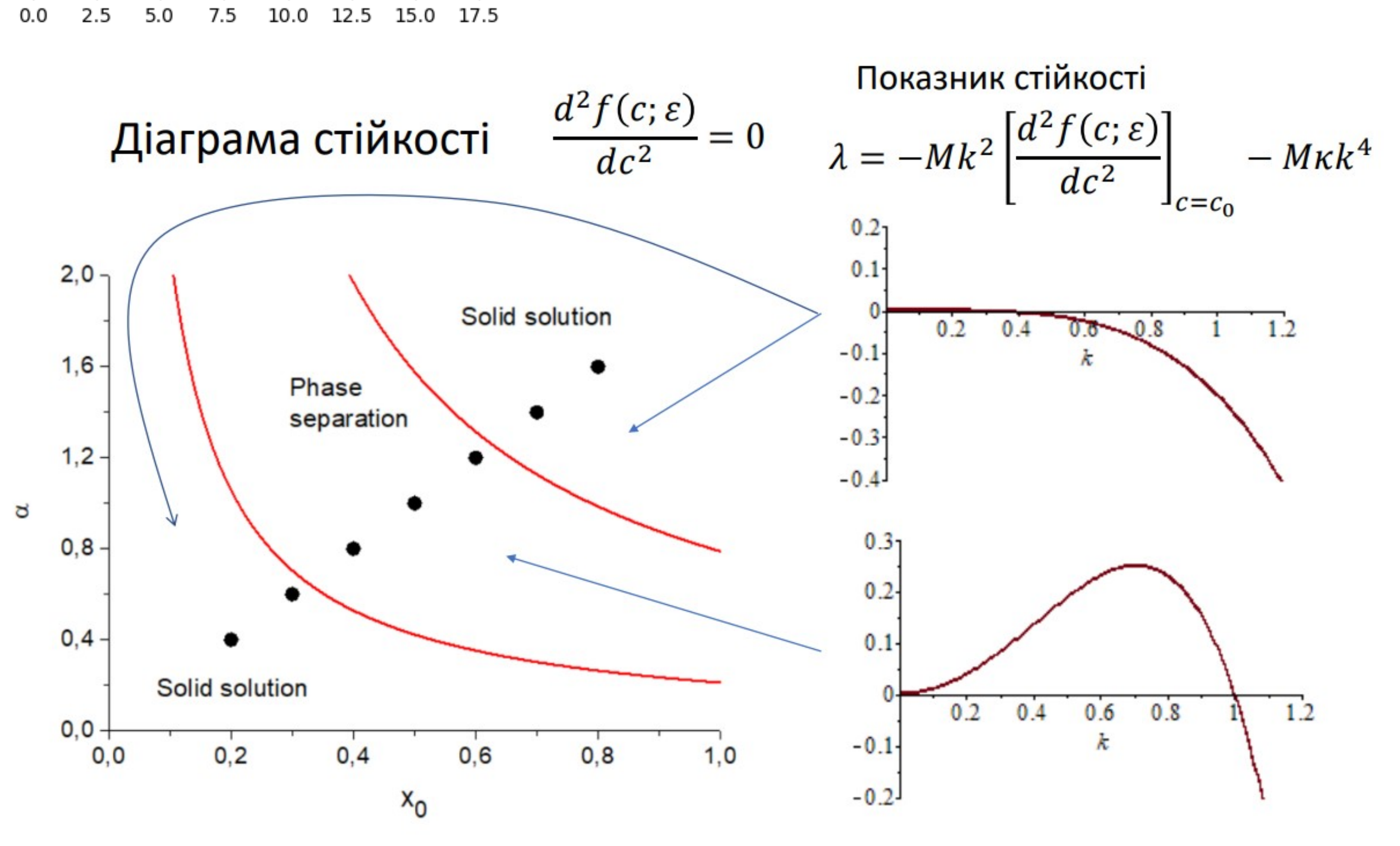


In [16]:
```
# початкова концентрація
c_init_1D = np.resize(c_init_2D, L*L)
# після інтегрування в околі точки (c0 = 0.5)
c_1D = [ fun_1(state0=c0i) for c0i in c_init_1D ]
```

In [17]:
```
# початкова концентрація
c_init_2D = plot_resize_data(c_init_1D)
# після інтегрування в околі точки (c0 = 0.5)
c_2D = plot_resize_data(c_1D)
```






Діаграма стійкості

$$\frac{d^2 f(c;\varepsilon)}{dc^2} = 0$$

Показник стійкості

$$\lambda = -Mk^2\left[\frac{d^2 f(c;\varepsilon)}{dc^2}\right]_{c=c_0} - M\kappa k^4$$

In [18]:
```
#a, c = symbols('a c')
#f = (c**2)*((1-a*c)**2)
print('Функція f(c, a):')
f
```

```
Функція f(c, a):
```

Out[18]: $c^2(1 - c)^2$

In [19]:
```
a, c = symbols('a c')
f = (c**2)*((1-a*c)**2)
df2_symbol = diff(f, c, 2) # 2 похідна
print('Друга похідна від f(c, a) по c -> df2(c,a):')
df2_symbol
```

```
Друга похідна від f(c, a) по c -> df2(c,a):
```

Out[19]: $2\left(a^2c^2 + 4ac(ac - 1) + (ac - 1)^2\right)$

In [20]:
```
def df2(c, a):
    df2_ = 2*(a**2 * c**2 + 4*a*c*(a*c-1) + (a*c-1)**2)
    # 2 похідна від f(c) = c**2 * (1-a*c) по c (або оператор Лапласа)
    return df2_
```

Функція, що розраховує показник стійкості (lambda):

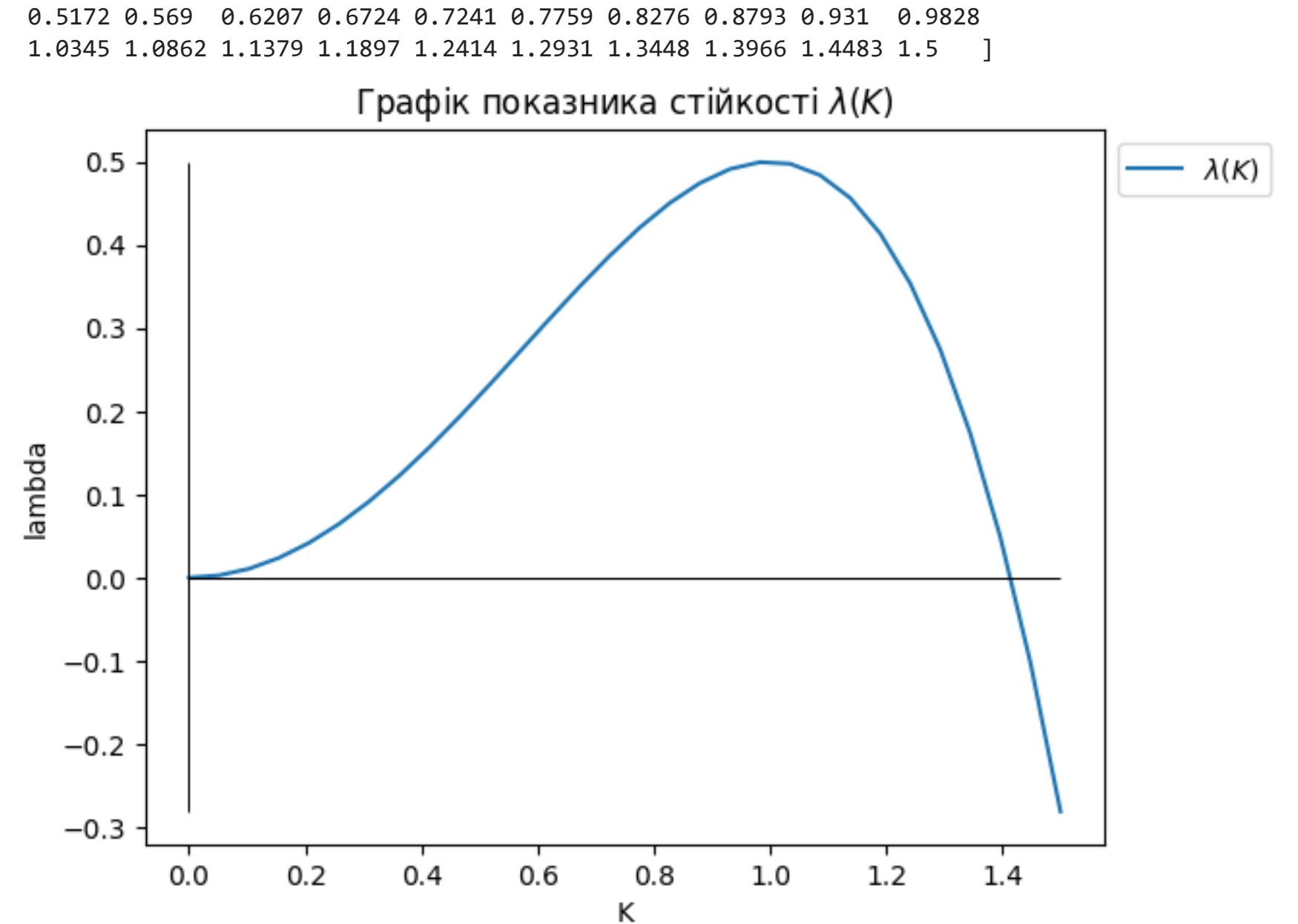
In [21]:
```
def stability_indicator(M=1, k=0.5, c0=0.5, a=1, K=[0]):
    print('Розраховуємо показник стійкості (lambda)')
    print(' M =', M, '\t k =', k, '\t c0 =', c0, '\n a =', a, '\n K =', K)
    #arr_lambda = [-M*(Ki**2)*(df2(c0, a)) - M*k*(Ki**4) for Ki in K]
    arr_lambda = [-M*(K[i]**2)*(df2(c0, a[i])) - M*k*(K[i]**4) for i in range(len(K))]
    arr_lambda = np.round(arr_lambda, 5)
    #print('\n lambda: \n', arr_lambda)
    return arr_lambda
```

Функція, що будує графік показника стійкості $lambda$ в залежності від значення параметрів a, K:

In [22]:
```
def plot_lambda(K, lambda_):
    plt.plot(K, lambda_, label = '$\lambda (K)$')
    plt.hlines(0, min(K), max(K), color='k', linewidth = .7)
    plt.vlines(0, min(lambda_), max(lambda_), color='k', linewidth = .7)
    plt.title('Графік показника стійкості $\lambda (K)$')
    plt.legend(bbox_to_anchor=(1,1), loc="upper left")
    plt.xlabel('K')
    plt.ylabel('lambda')
    #plt.grid()
    plt.show()
```

In [23]:
```
K1 = np.linspace(0, 1.5, 30)
K1 = np.round(K1, 4)
a1 = [1 for i in range(len(K1))]
lambda1 = stability_indicator(M, k, c0, a=a1, K=K1)
plot_lambda(K1, lambda1)
```

```
Розраховуємо показник стійкості (lambda)
 M = 1   k = 0.5         c0 = 0.5
 a = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
 K = [0.     0.0517 0.1034 0.1552 0.2069 0.2586 0.3103 0.3621 0.4138 0.4655
 0.5172 0.569  0.6207 0.6724 0.7241 0.7759 0.8276 0.8793 0.931  0.9828
 1.0345 1.0862 1.1379 1.1897 1.2414 1.2931 1.3448 1.3966 1.4483 1.5   ]
```



Аналіз стійкості однорідних станів до неоднорідних збурень

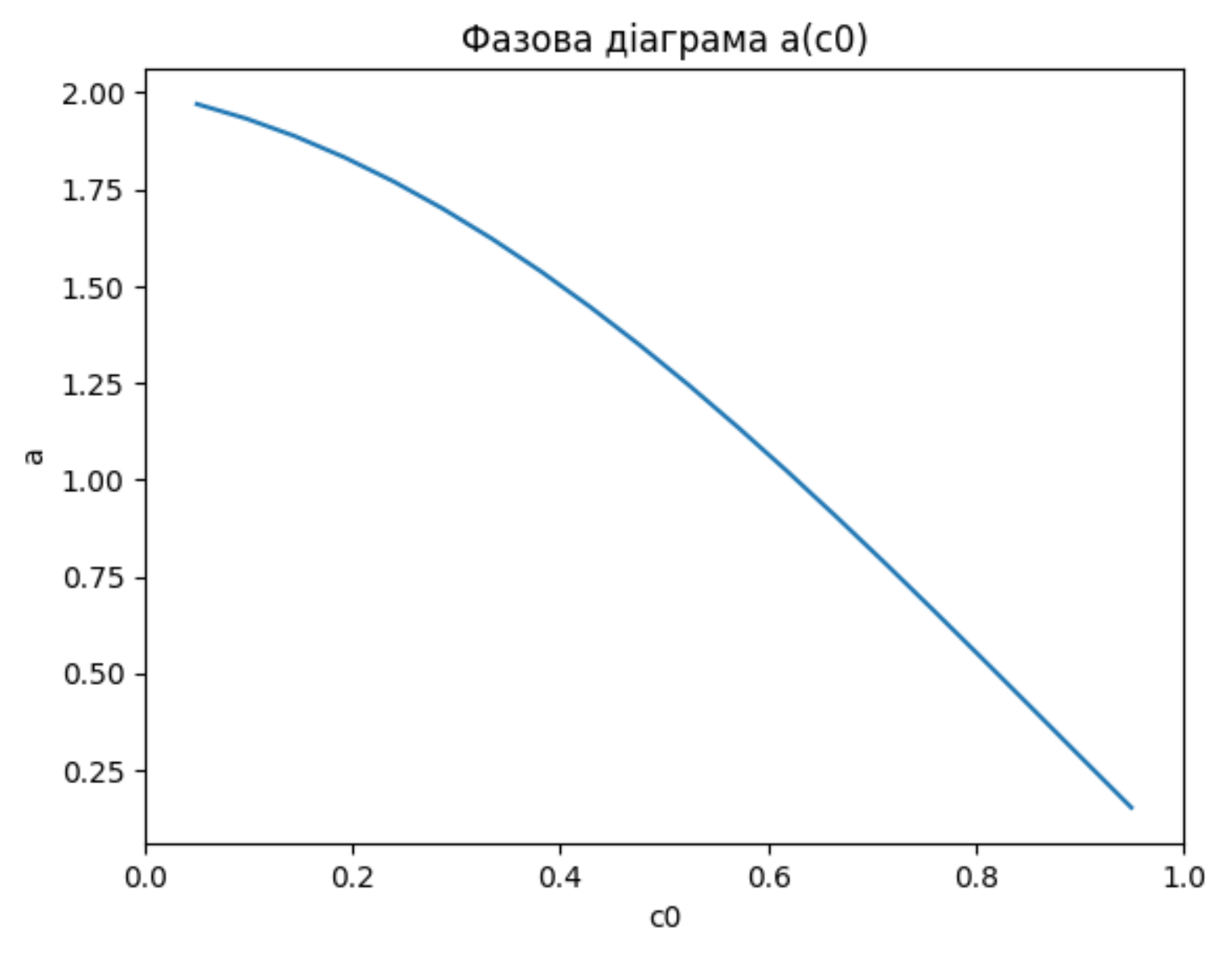
Фазова діаграма

$$a(c_0) = \frac{d^2 f(c;a)}{dc^2}|_{c=c_0}$$

In [24]:
```
def fun_phase_diaghram(arr_a):
    arr_c0 = np.linspace(0.05, 0.95, 20)
    arr_a_c0 = [df2(arr_c0[i], arr_a[i]) for i in range(len(arr_c0))]

    arr_c02  = np.round(arr_c0, 4)
    arr_a    = np.round(arr_a , 4)
    arr_a_c0 = np.round(arr_a_c0 , 4)

    print('arr_c0: \n',   arr_c0, '\n')
    print('arr_a: \n',    arr_a, '\n')
    print('arr_a_c0: \n', arr_a_c0)

    plt.plot(arr_c0, arr_a_c0)
    plt.xlabel('c0')
    plt.ylabel('a')
    plt.title('Фазова діаграма a(c0)')
    plt.xlim((0,1))
    return arr_c0, arr_a, arr_a_c0
```

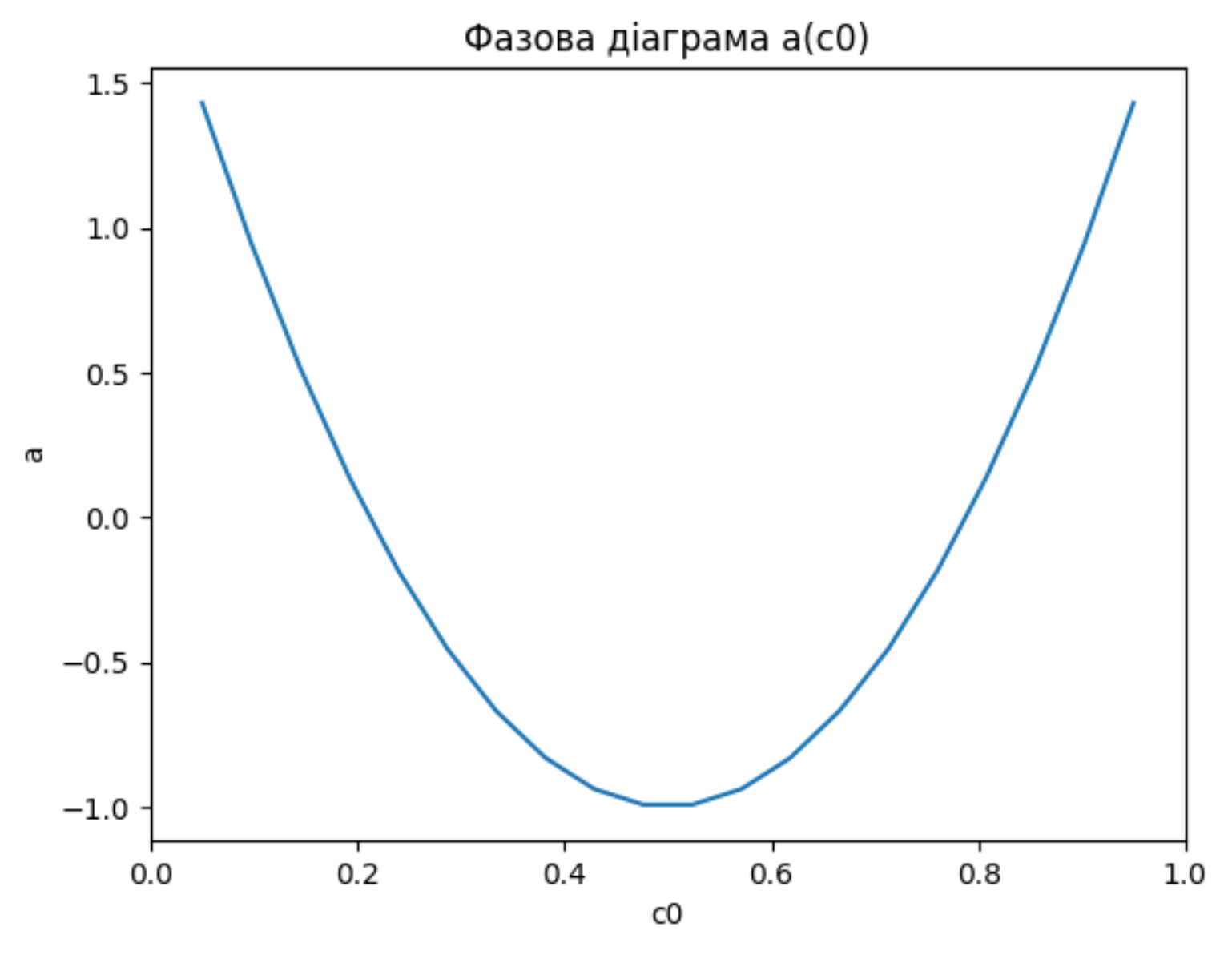
In [25]:
```
a_1 = np.linspace(0.05, 0.20, 20)
ans_1 = fun_phase_diaghram(a_1)
print('\nРозмірність масиву ans_1 =', np.shape(ans_1)) #(3, 20)
```

```
arr_c0:
 [0.05       0.09736842 0.14473684 0.19210526 0.23947368 0.28684211
 0.33421053 0.38157895 0.42894737 0.47631579 0.52368421 0.57105263
 0.61842105 0.66578947 0.71315789 0.76052632 0.80789474 0.85526316
 0.90263158 0.95      ]

arr_a:
 [0.05   0.0579 0.0658 0.0737 0.0816 0.0895 0.0974 0.1053 0.1132 0.1211
 0.1289 0.1368 0.1447 0.1526 0.1605 0.1684 0.1763 0.1842 0.1921 0.2   ]

arr_a_c0:
 [1.9701 1.9327 1.8868 1.8325 1.7701 1.6999 1.6222 1.5374 1.4458 1.348
 1.2444 1.1355 1.022  0.9045 0.7835 0.6598 0.5342 0.4073 0.28   0.1532]

Розмірність масиву ans_1 = (3, 20)
```



In [26]:
```
a_2 = [1 for i in range(20)]
ans_2 = fun_phase_diaghram(a_2)
print('\nРозмірність масиву ans_2 =', np.shape(ans_2)) #(3, 20)
```

```
arr_c0:
 [0.05       0.09736842 0.14473684 0.19210526 0.23947368 0.28684211
 0.33421053 0.38157895 0.42894737 0.47631579 0.52368421 0.57105263
 0.61842105 0.66578947 0.71315789 0.76052632 0.80789474 0.85526316
 0.90263158 0.95      ]

arr_a:
 [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]

arr_a_c0:
 [ 1.43    0.9453  0.5145  0.1376 -0.1855 -0.4548 -0.6702 -0.8317 -0.9394
 -0.9933 -0.9933 -0.9394 -0.8317 -0.6702 -0.4548 -0.1855  0.1376  0.5145
  0.9453  1.43  ]

Розмірність масиву ans_2 = (3, 20)
```



Комп'ютерне моделювання задач прикладної математики

Динамічні системи.

Лабораторна робота 4 Реакційно-дифузійні системи та їх застосування

Студентка Пороскун Олена. Група ПМ.м-21

Варіант 8

Task 3 Вплив параметра та початкової концентрації.

Постановка задачі

Дано

- Нехай існує два типи речовин A та B.
- Резервуар розміром L x L заповнюють речовинами A та B з концентраціями cA та cB однорідно.

Задача

- Необхідно провести числове моделювання динаміки фазового розшарування однорідного розчину A-B на фази, збагачені на речовину A та речовину B.

Умова

$M = 1, \ \ k = 0.5$

- $a = 0.5, \ \ c_0 = 0.5$
- $a = 1, \ \ c_0 = 0.3$
- $a = 1, \ \ c_0 = 0.5$
- $a = 1.5, \ \ c_0 = 0.5$
- $a = 1, \ \ c_0 = 0.7$

Р-ня

In [1]:

```
import numpy as np
from matplotlib import pyplot as plt
from matplotlib import cm
```

In [2]:

```
!pip install sympy
```

```
Requirement already satisfied: sympy in c:\users\admin\anaconda3\lib\site-packages (1.11.1)
Requirement already satisfied: mpmath>=0.19 in c:\users\admin\anaconda3\lib\site-packages (from sympy) (1.2.1)
```

In [3]:

```
from sympy import diff, symbols
```

In [4]:

```
import random
```

Константи та параметри

In [5]:

```
L = 20
M = 1
k = 0.5
arr_a  = [0.5, 1.0, 1.0, 1.5, 1.0]
arr_c0 = [0.5, 0.3, 0.5, 0.5, 0.7]
print(' L =', L,' M =', M,'\t k =', k, '\n a  =', arr_a, '\n c0 =',arr_c0)
```

```
 L = 20  M = 1   k = 0.5
 a  = [0.5, 1.0, 1.0, 1.5, 1.0]
 c0 = [0.5, 0.3, 0.5, 0.5, 0.7]
```

In [6]:

```
a, c = symbols('a c')
f = (c**2)*((1-a*c)**2)
print('Функція f(c,a):')
f
```

```
Функція f(c,a):
```

Out[6]: $c^2(-ac+1)^2$

In [7]:

```
#c = symbols('c')
f = (c**2)*((1-a*c)**2)
df1_symbol = diff(f, c) # 1 похідна
print("Перша похідна від f(c,a) по c -> df1(c,a):")
df1_symbol
```

```
Перша похідна від f(c,a) по c -> df1(c,a):
```

Out[7]: $-2ac^2(-ac+1) + 2c(-ac+1)^2$

In [8]:

```
print("Друга похідна від f(c,a) по c -> df2(c,a) (оператор Лапласа):" )
op_laplas_symbol = diff(f, c, 2) # 2 похідна = оператор Лапласа
op_laplas_symbol
```

```
Друга похідна від f(c,a) по c -> df2(c,a) (оператор Лапласа):
```

Out[8]: $2\left(a^2c^2 + 4ac(ac-1) + (ac-1)^2\right)$

In [9]:

```
# df1 = -2*a*(c**2)*(-a*c+1) + 2*c*(-a*c+1)**2       # 1 похідна
# df2 = 2*(a**2 * c**2 + 4*a*c*(a*c-1) + (a*c-1)**2) # 2 похідна
# op_laplas = df2                                    # 2 похідна = оператор Лапласа
```

Функції ініціалізації даних початкової концентрації значеннями близькими до c_0.

In [10]:

```
def fun_init_data(L, c0):
    print('Резервуар розміром LxL, де L =', L)
    print('Початкова концентрація близька до значення c0 =', c0)
    data = np.linspace(c0-0.05, c0+0.05, L*L)
    len_data = len(data)
    new_size = int(np.sqrt(len_data))
    data_resize = np.resize(data, (new_size, new_size))
    return data_resize


def fun_init_data2(L, c0):
    print('Резервуар розміром LxL, де L =', L)
    print('Початкова концентрація близька до значення c0 =', c0)
    data = []
    a1, b1 = c0-0.05, c0+0.05
    for i in range(L*L):
        ab0 = random.random()
        ab1 = ab0*(b1-a1) + a1
        data.append(ab1)
    len_data = len(data)
    new_size = int(np.sqrt(len_data))
    data_resize = np.resize(data, (new_size, new_size))
    return data_resize
```

Побудова графіку двумірних даних концентрації.

In [11]:

```
def plot_map(data):
    # графік з вертикальною (за замовчуванням) панеллю кольорів
    fig, ax = plt.subplots()
    cax = ax.imshow(data, interpolation='nearest', cmap=cm.coolwarm)
    ax.set_title('Карта концентрації речовини ')
    cbar = fig.colorbar(cax)
    plt.show()
```

Отже, ініціалізуємо початкові дані та зобразимо їх.

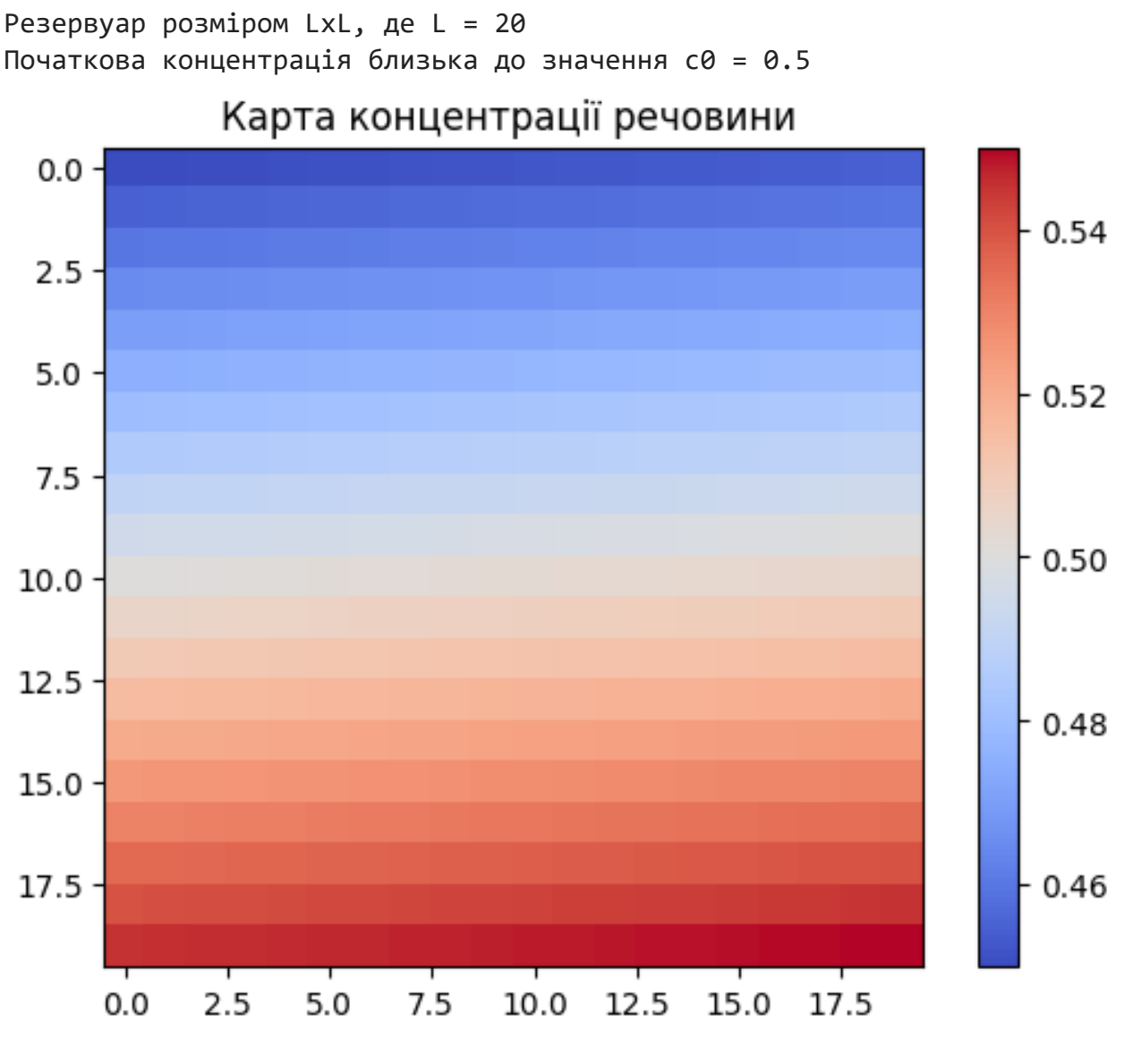
In [12]:

```
c0 = arr_c0[0]
```

In [13]:

```
c_init_2D = fun_init_data(L, c0)
plot_map(c_init_2D)
```

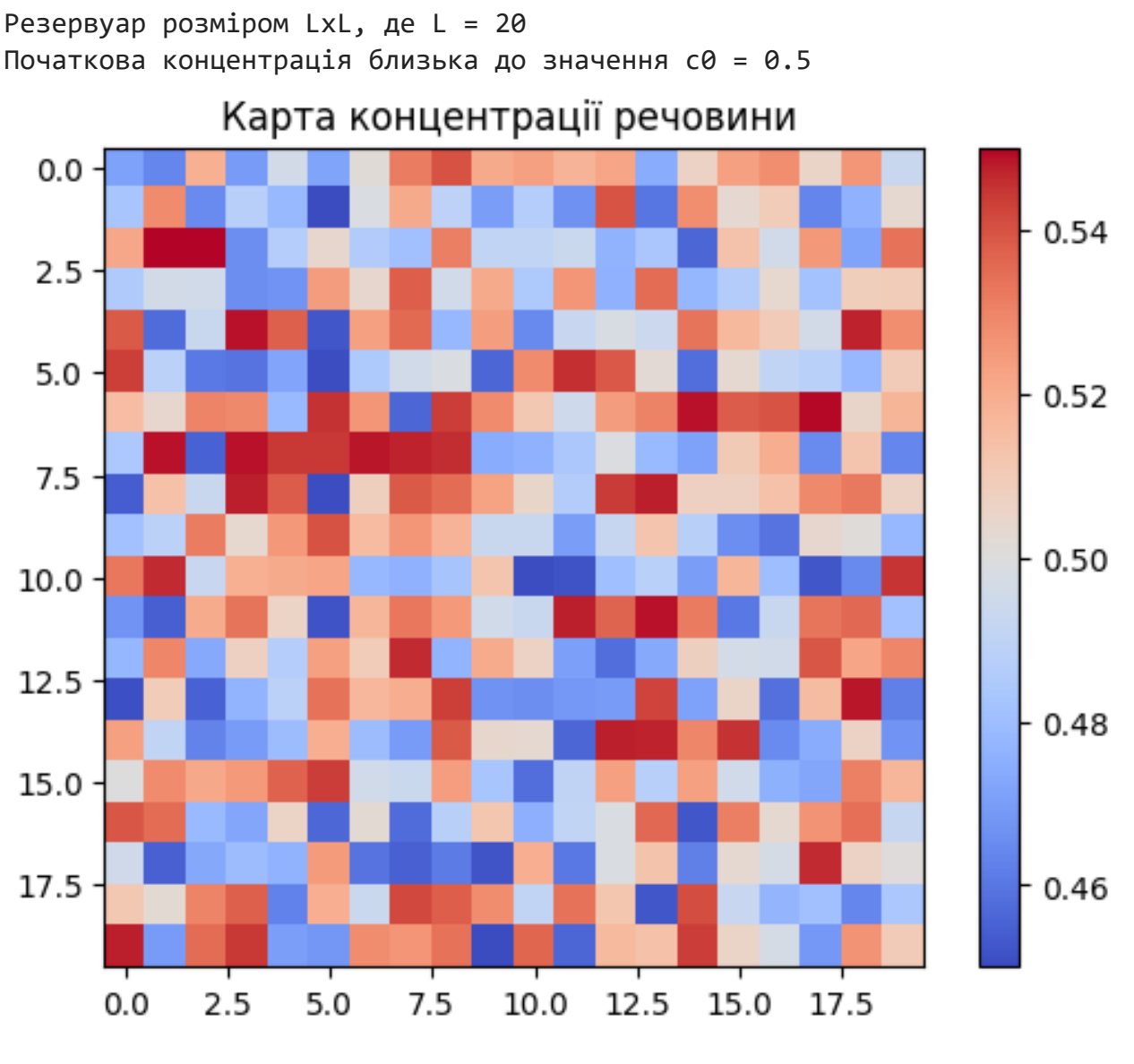
```
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
```



In [14]:

```
c_init_2D_ = fun_init_data2(L, c0)
plot_map(c_init_2D_)
```

```
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
```



In [15]:

```
from scipy.integrate import odeint
```

Функція, що змінює розмірність даних (робить двумірні дані з одномірних даних, наприклад був масив 100x1 стане 10x10):

In [16]:

```
def plot_resize_data(data):
    len_data = len(data)
    new_size = int(np.sqrt(len_data))
    newdata = np.resize(data, (new_size, new_size))
    plot_map(newdata)
    return newdata
```

Функція, що повертає останній елемент масиву:

In [17]:

```
def fun_last_el(array):
    #print('fun_last_el')
    end = len(array)
    return array[end-1]
```

Функція, що здійснює інтегрування з використанням початкової точки $state0$ за формулою:

$$\frac{dc}{dt} = M\nabla^2[\frac{df}{dc} - k\nabla^2 c],$$
$$\text{де } \nabla^2 - \text{оператор Лапласа (друга похідна від} f(c))$$

In [18]:

```
# df1 = -2*a*(c**2)*(-a*c+1) + 2*c*(-a*c+1)**2       # 1 похідна
# df2 = 2*(a**2 * c**2 + 4*a*c*(a*c-1) + (a*c-1)**2) # 2 похідна
# op_laplas = df2                                    # 2 похідна = оператор Лапласа
```

In [19]:

```
def fun_31(a = 0.5, state0=0.5):
    # процес інтегрування f(c) = c**2 * (1-a*c) за формулою
    # dc/dt = M*op_laplas*((df/dc) - k*op_laplas*c)
    def dcdt(state, t):
        c = state                                             # Розпаковка вектору
        return M*op_laplas(c)*(df1(c) - k*op_laplas(c)*c) # Похідні

    def df1(c):
        #df1 = df/dc
        ans1 = -2*a*(c**2)*(-a*c+1) + 2*c*(-a*c+1)**2
        # c**2*(2*c-2) + 2*c*(1-c)**2
        return ans1

    def op_laplas(c):
        #df1 = df/dc
        #op_laplas = d(df1)/dc - оператор Лапласа
        ans2 = 2*(a**2 * c**2 + 4*a*c*(a*c-1) + (a*c-1)**2)
        #2*c**2 - 8*c*(1-c) + 2*(1-c)**2
        return ans2

    t = np.arange(0.0, 40.0, 0.1)
    states = odeint(dcdt, state0, t)
    states = np.round(states, 4)

    last_states = fun_last_el(states)

    #print('a =', a, '\t c0 =', state0, '\t c =', last_states)
    #print('Процес інтегрування')

    return last_states
```

In [20]:

```
def fun_initial_array(L, arr_c0):
    print('1 спосіб задання початквоих значень')
    c_init_2D = []
    for i in range(len(arr_c0)):
        c_init_2D.append(fun_init_data(L, c0=arr_c0[i]))
        #plot_map(c_init_2D[i])
    print(np.shape(c_init_2D))
    return c_init_2D

def fun_initial_array2(L, arr_c0):
    print('\n 2 спосіб задання початквоих значень')
    c_init_2D = []
    for i in range(len(arr_c0)):
        c_init_2D.append(fun_init_data2(L, c0=arr_c0[i]))
        #plot_map(c_init_2D[i])
    print(np.shape(c_init_2D))
    return c_init_2D
```

In [21]:

```
c1_init_2D = fun_initial_array(L, arr_c0)
c2_init_2D = fun_initial_array2(L, arr_c0)
```

```
1 спосіб задання початквоих значень
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.3
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.7
(5, 20, 20)

 2 спосіб задання початквоих значень
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.3
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.5
Резервуар розміром LxL, де L = 20
Початкова концентрація близька до значення c0 = 0.7
(5, 20, 20)
```

Функція, яка показує початкову концентрацію та кінцеву (для масивів):

In [22]:

```
def integr_and_plot(L=20, arr_a=[0.5], arr_c0=[0.5]):
    c_init_2D = []
    c_init_2D = fun_initial_array2(L, arr_c0)

    c_1D = []
    for n in range(len(arr_c0)):
        # початкова концентрація
        c_init_1D = np.resize(c_init_2D[n], L*L)
        print('окіл точки c0 =',arr_c0[n], '\t ( a =',arr_a[n],')')
        # після інтегрування в околі точки c0
        cc =[ fun_31(a = arr_a[n], state0 = ci0) for ci0 in c_init_1D ]
        c_1D.append(cc)

    for n in range(len(arr_c0)):
        # початкова концентрація
        plot_map(c_init_2D[n])

        # після інтегрування в околі точки c0
        c1_2D = plot_resize_data(c_1D[n])

    return c_1D
```

In [23]:

```
#c1_1D = integr_and_plot(L, arr_a, arr_c0)
```

Функція, яка показує початкову концентрацію та кінцеву (для одного ел. з масиву):

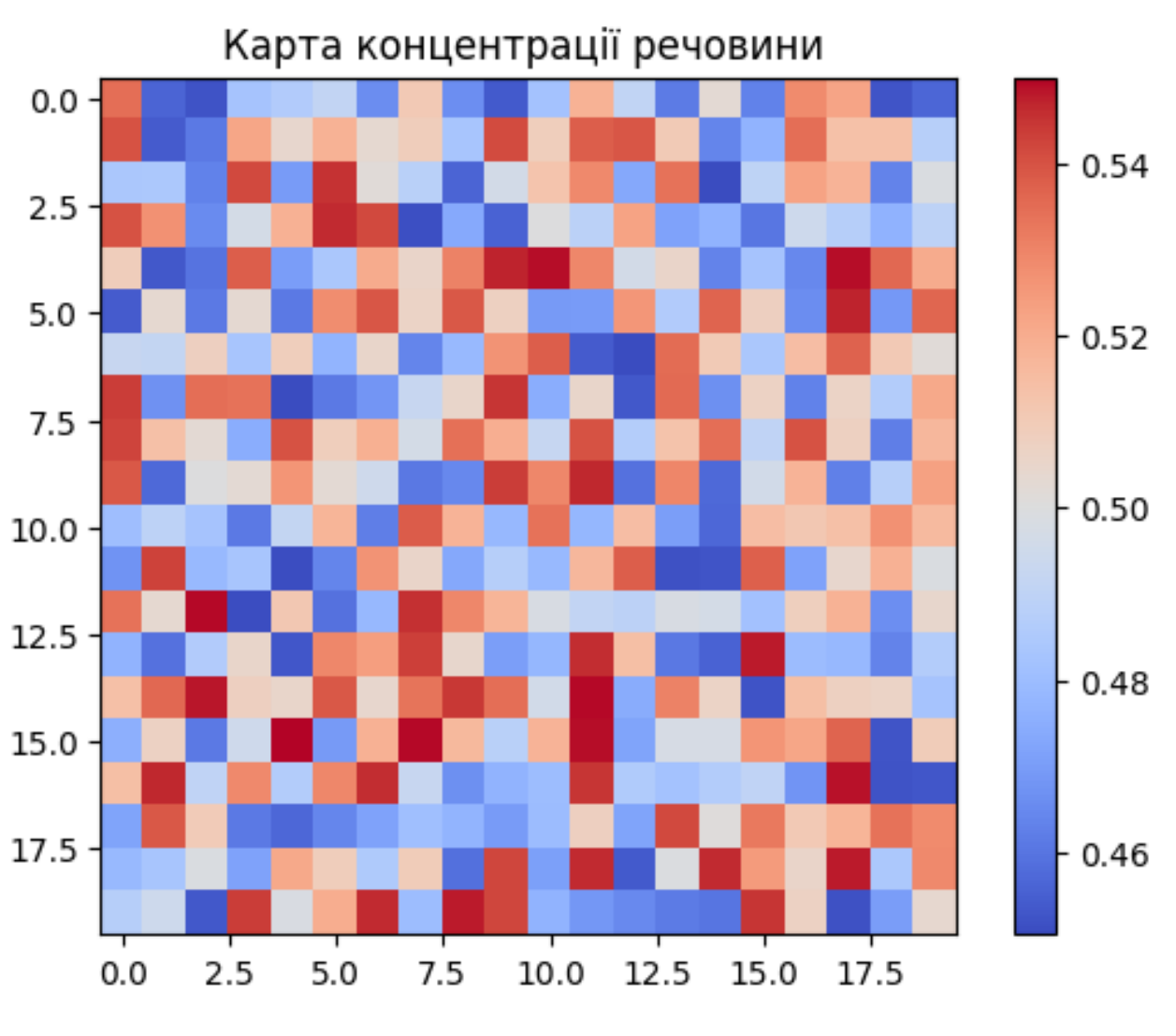
In [24]:

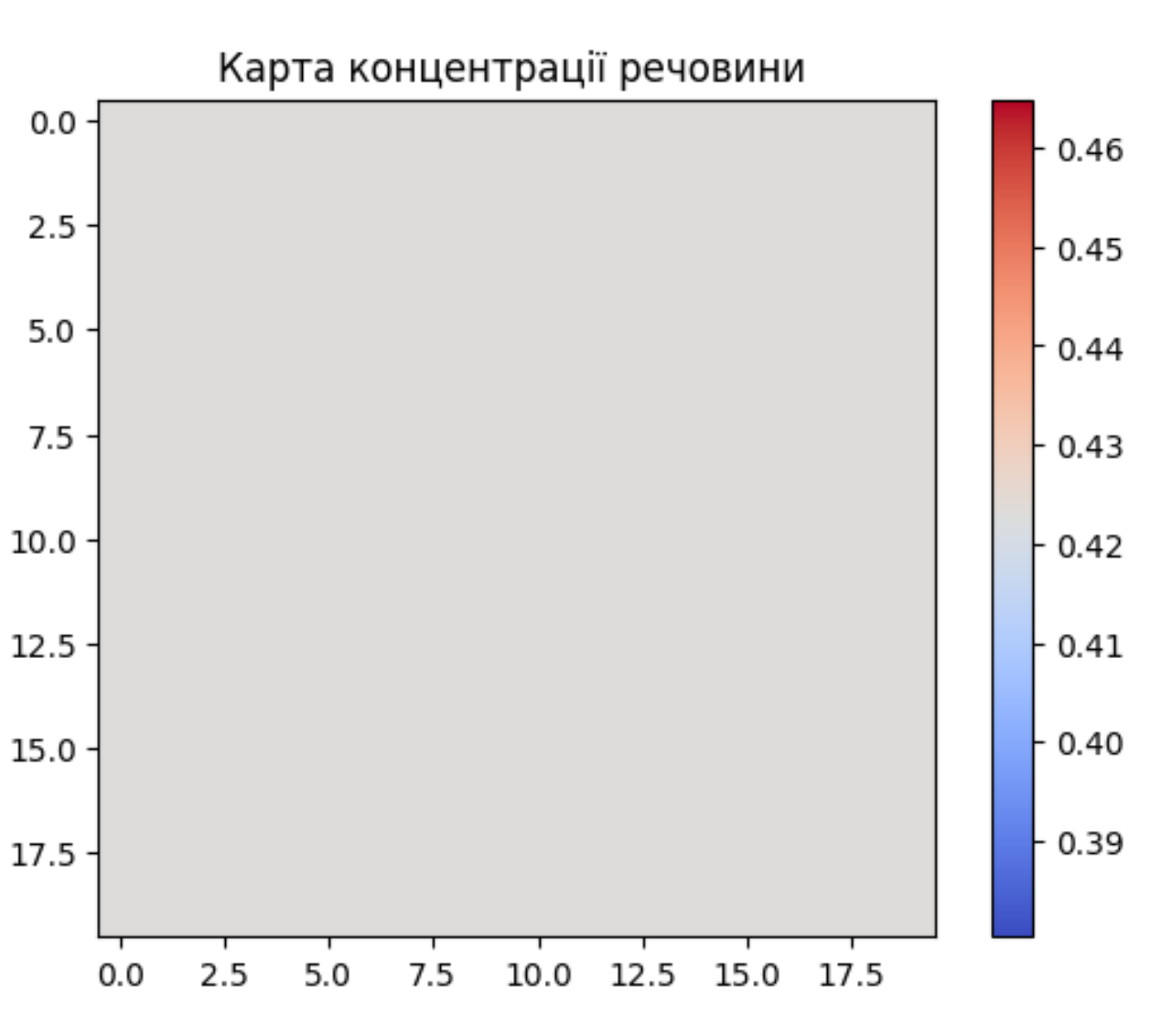
```
def integr_and_plot_new(L=20, arr_a=[0.5], arr_c0=[0.5], c_init_2D=[]):
    c_1D = []
    for n in range(len(arr_c0)):
        # початкова концентрація
        c_init_1D = np.resize(c_init_2D[n], L*L)
        print('окіл точки c0 =',arr_c0[n], '\t ( a =',arr_a[n],')')
        # після інтегрування в околі точки c0
        cc =[ fun_31(a = arr_a[n], state0 = ci0) for ci0 in c_init_1D ]
        c_1D.append(cc)

    # початкова концентрація
    plot_map(c_init_2D[0])

    # після інтегрування в околі точки c0
    c1_2D = plot_resize_data(c_1D[0])

    return c_1D
```

In [25]:

```
k=0
c_1D_new = integr_and_plot_new(L, [arr_a[k]], [arr_c0[k]], [c2_init_2D[k]])
```

```
окіл точки c0 = 0.5      ( a = 0.5 )
```

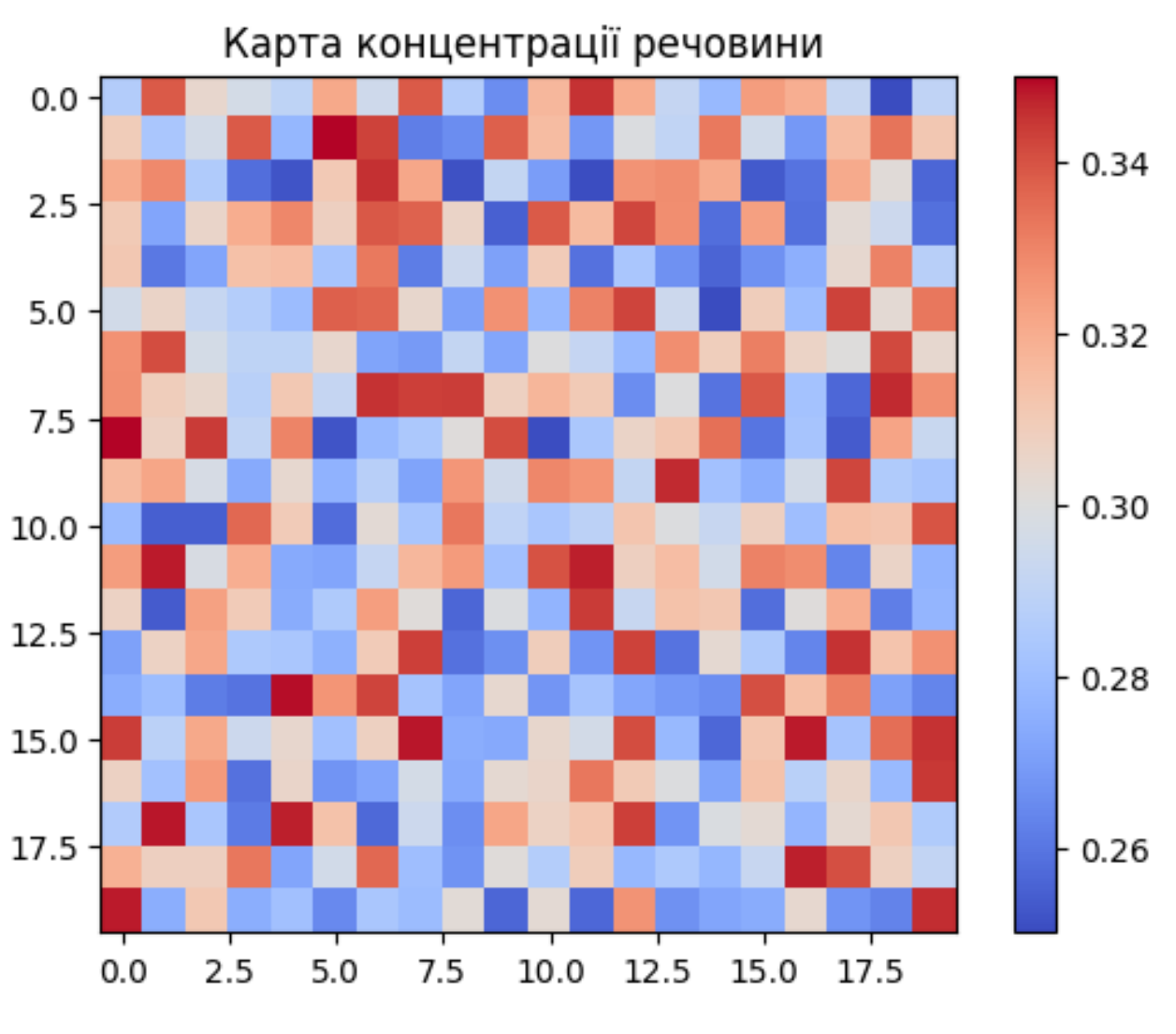


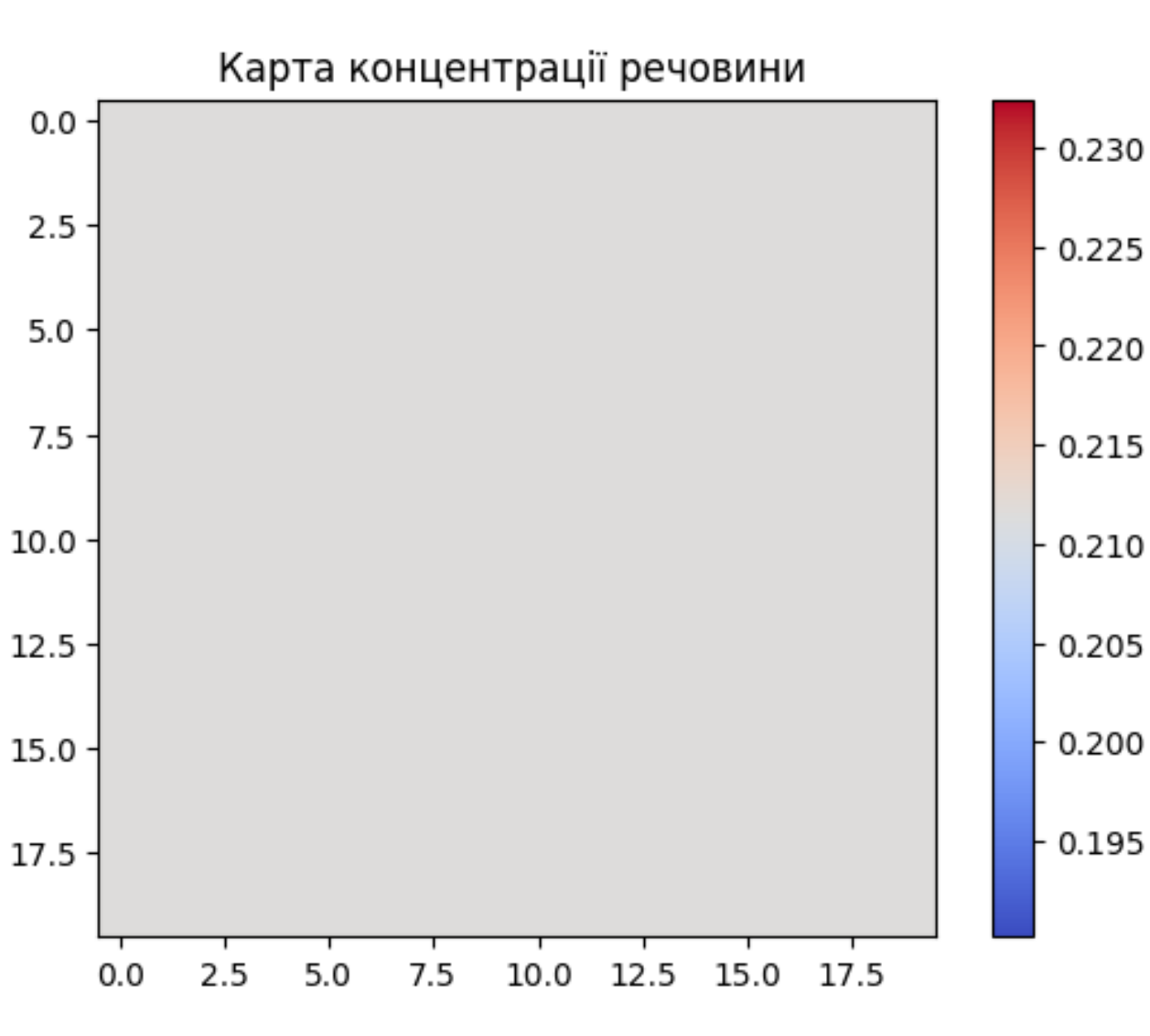


In [26]:

```
k=1
c_1D_new = integr_and_plot_new(L, [arr_a[k]], [arr_c0[k]], [c2_init_2D[k]])
```

```
окіл точки c0 = 0.3      ( a = 1.0 )
```

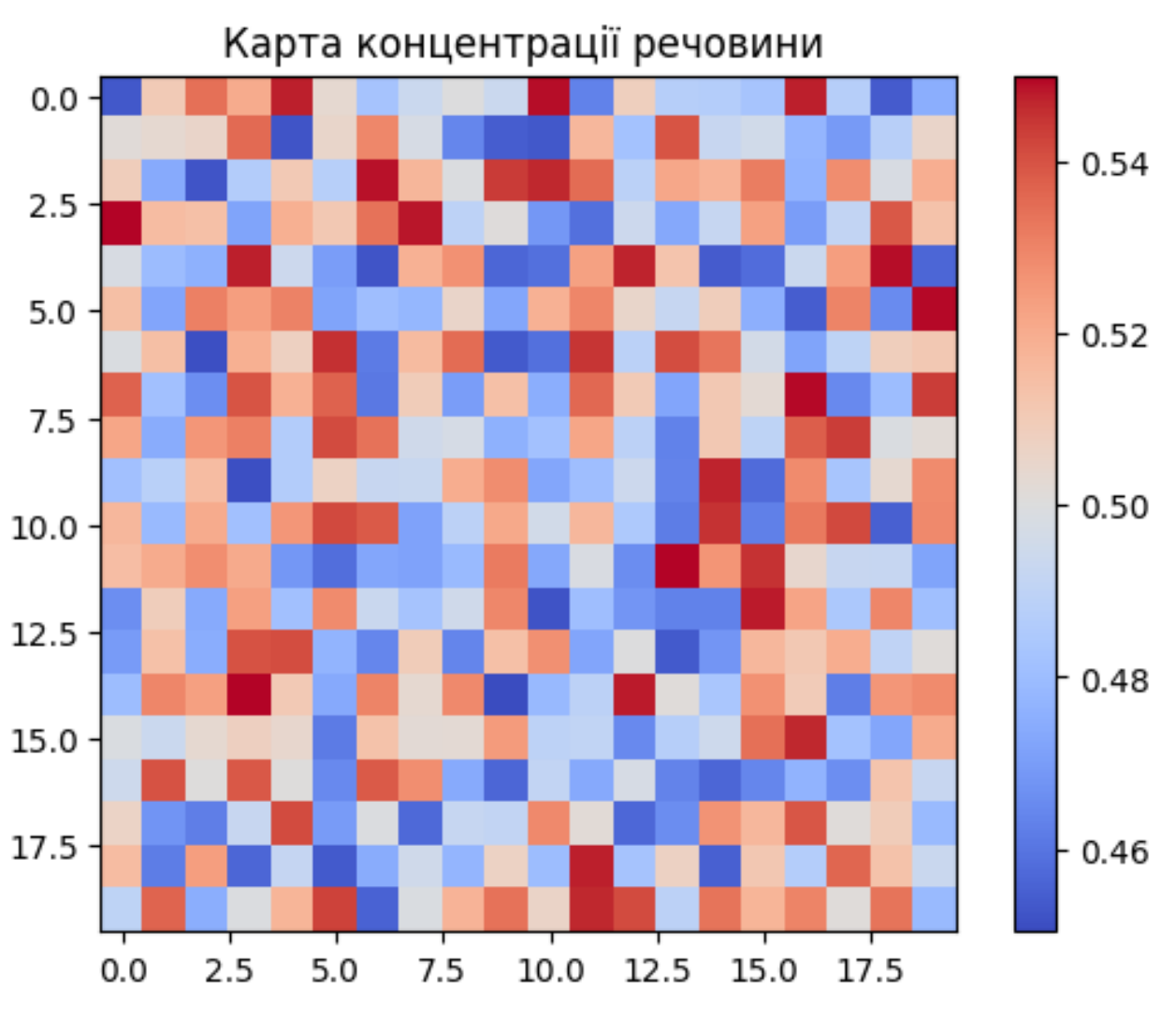


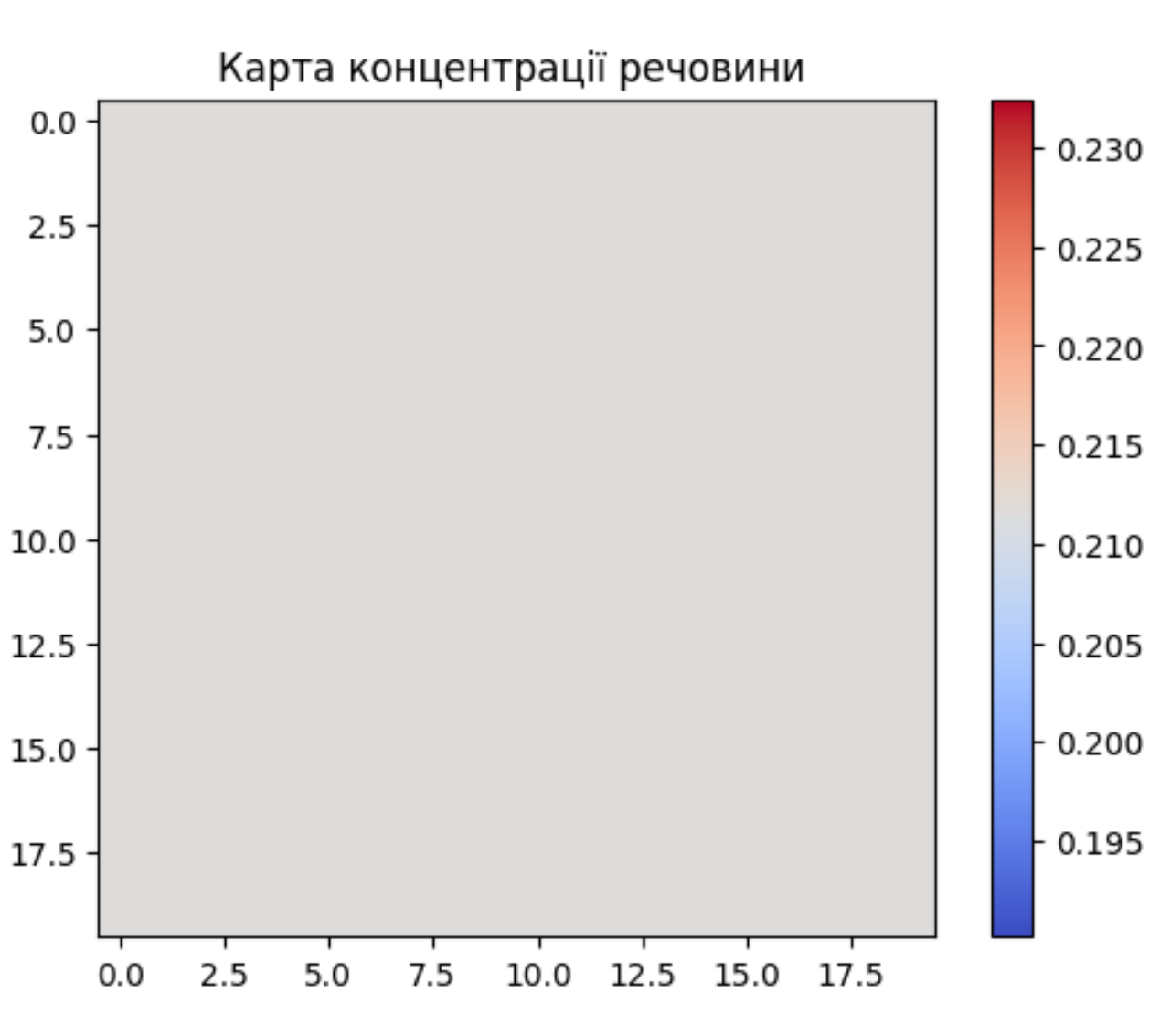


In [27]:

```
k=2
c_1D_new = integr_and_plot_new(L, [arr_a[k]], [arr_c0[k]], [c2_init_2D[k]])
```

```
окіл точки c0 = 0.5      ( a = 1.0 )
```

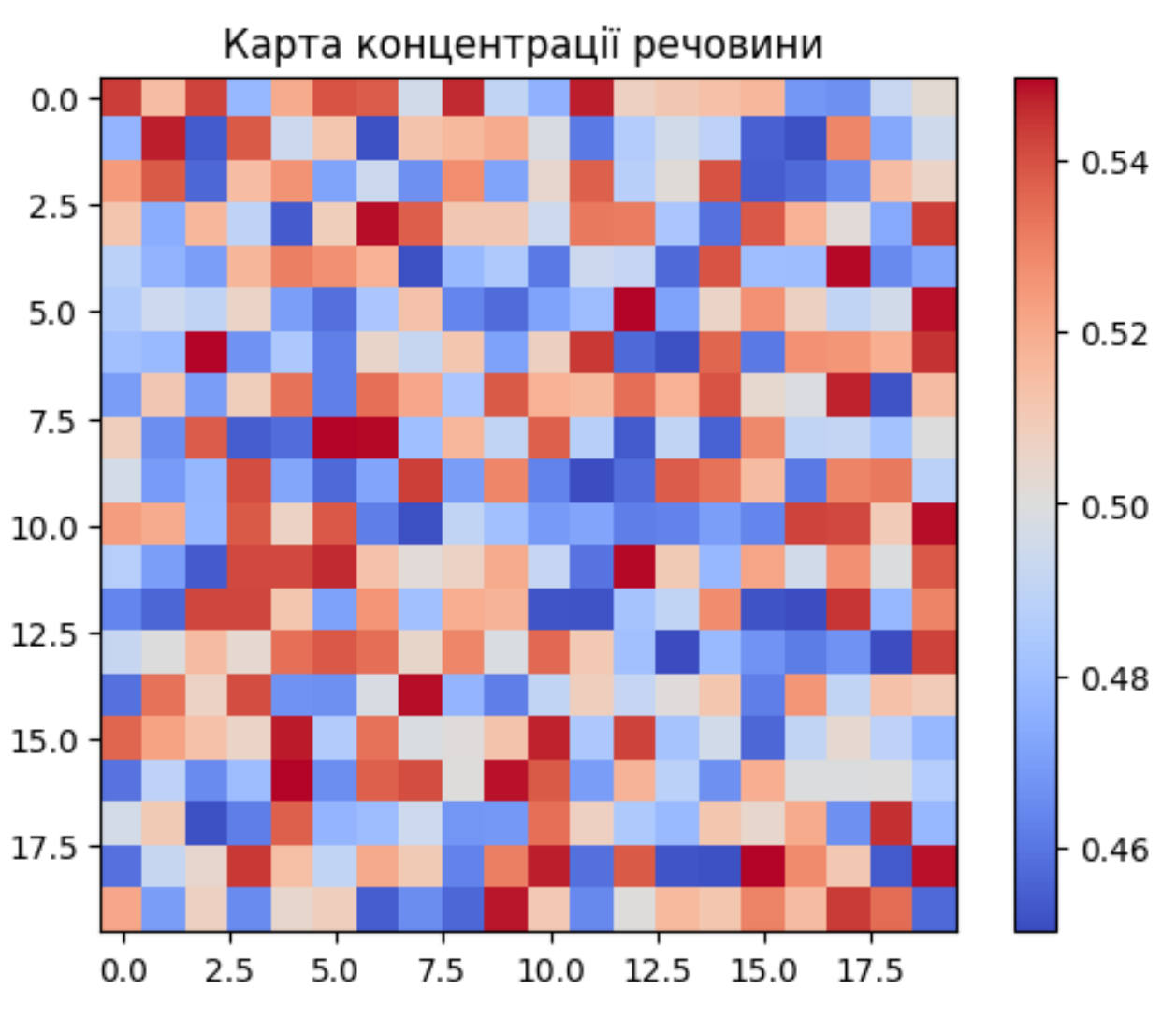


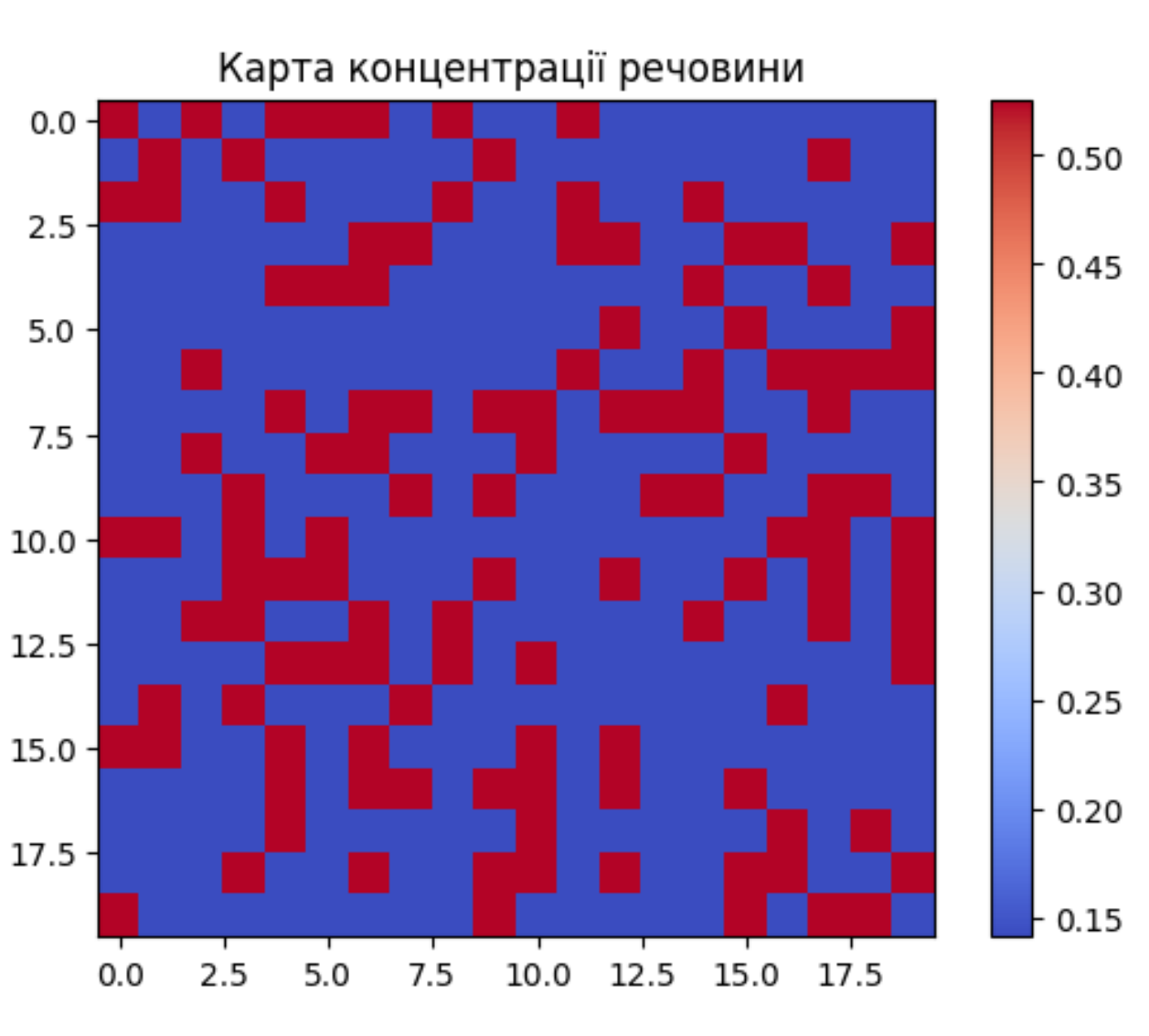


In [31]:

```
k=3
c_1D_new = integr_and_plot_new(L, [arr_a[k]], [arr_c0[k]], [c2_init_2D[k]])
```

```
окіл точки c0 = 0.5      ( a = 1.5 )
```





In [32]:

```
k=4
c_1D_new = integr_and_plot_new(L, [arr_a[k]], [arr_c0[k]], [c2_init_2D[k]])
```

```
окіл точки c0 = 0.7      ( a = 1.0 )
```